Deep Reinforcement Learning

슬라이드별 학습 가이드

순차 의사결정부터 모델 기반 강화학습까지

2026년 2학기

Contents

1 01-1. 강화학습의 문제와 강좌 개요 **13**

1.1 슬라이드 01 · Deep Reinforcement Learning . . . 13
1.2 슬라이드 02 · 강좌 안내 . . . 13
1.3 슬라이드 03 · 오늘의 흐름 . . . 13
1.4 슬라이드 04 · What is reinforcement learning? . . . 13
1.5 슬라이드 05 · 강화학습의 목표 . . . 13
1.6 슬라이드 06 · 에이전트와 환경 . . . 13
1.7 슬라이드 07 · 순차적 의사결정 문제 . . . 13
1.8 슬라이드 08 · 강좌의 초점 . . . 13
1.9 슬라이드 09 · 다룰 주제 . . . 14
1.10 슬라이드 10 · 강좌 일정 . . . 14
1.11 슬라이드 11 · 선수 지식 . . . 14
1.12 슬라이드 12 · 평가 구성 . . . 14
1.13 슬라이드 13 · 과제 구성 . . . 14
1.14 슬라이드 14 · 과제 진행 안내 . . . 14
1.15 슬라이드 15 · 시험 안내 . . . 14
1.16 슬라이드 16 · 출석 안내 . . . 14
1.17 슬라이드 17 · 수업 참여 . . . 14
1.18 슬라이드 18 · 소통과 자료 . . . 15
1.19 슬라이드 19 · 참고 자료 . . . 15
1.20 슬라이드 20 · Why study reinforcement learning? . . . 15
1.21 슬라이드 21 · 인공지능의 발전 . . . 15
1.22 슬라이드 22 · 주요 발전 사례 . . . 15
1.23 슬라이드 23 · 생성 모델과 AI 도구 . . . 15
1.24 슬라이드 24 · 지도학습만으로 충분한가? . . . 15
1.25 슬라이드 25 · 칩 설계의 예 . . . 15
1.26 슬라이드 26 · 언어모델의 인간 피드백 . . . 15
1.27 슬라이드 27 · 이미지 생성과 보상 . . . 16
1.28 슬라이드 28 · 코드 에이전트 . . . 16
1.29 슬라이드 29 · 로봇 보행 . . . 16
1.30 슬라이드 30 · 로봇 동작 . . . 16
1.31 슬라이드 31 · 강화학습이 필요한 이유 . . . 16
1.32 슬라이드 32 · 다음 주제: 모방학습 . . . 16

2 01-2. 모방학습과 로봇 정책 **17**

2.1 슬라이드 01 · Imitation Learning . . . 17

2.2 슬라이드 02 · 강화학습과 모방학습의 위치 17
2.3 슬라이드 03 · 에이전트–환경 고리 다시 보기 17
2.4 슬라이드 04 · 상태, 행동, 정책 17
2.5 슬라이드 05 · 궤적과 전이의 표기 17
2.6 슬라이드 06 · 정답 행동과 보상 18
2.7 슬라이드 07 · 이 장의 질문 18
2.8 슬라이드 08 · Behavioral Cloning의 목적함수 18
2.9 슬라이드 09 · 시연의 품질과 범위 19
2.10 슬라이드 10 · 시연을 수집하는 방식 19
2.11 슬라이드 11 · 실패 원인으로 넘어가기 19
2.12 슬라이드 12 · 실패를 분해하는 틀 19
2.13 슬라이드 13 · 첫째 실패: 오류의 누적 19
2.14 슬라이드 14 · 오류 누적을 줄이는 두 경로 20
2.15 슬라이드 15 · 둘째 실패: 여러 정답 행동 20
2.16 슬라이드 16 · 셋째 실패: 전문가와 에이전트의 관측 차이 21
2.17 슬라이드 17 · 세 실패의 진단 순서 21
2.18 슬라이드 18 · 온라인 교정으로 넘어가기 21
2.19 슬라이드 19 · DAgger의 네 단계 21
2.20 슬라이드 20 · 사람이 개입하는 DAgger 22
2.21 슬라이드 21 · 로봇 사례를 읽는 틀 22
2.22 슬라이드 22 · ALOHA와 ACT의 결합 23
2.23 슬라이드 23 · 저비용 하드웨어가 바꾸는 것 23
2.24 슬라이드 24 · ACT가 마주한 두 문제 23
2.25 슬라이드 25 · 행동을 묶어 예측하는 이유 23
2.26 슬라이드 26 · 절대 관절 목표와 잠재변수 23
2.27 슬라이드 27 · ACT 구조를 입력과 출력으로 읽기 24
2.28 슬라이드 28 · 50개 시연으로 만든 조작 정책 24
2.29 슬라이드 29 · 배터리 삽입의 정밀도 24
2.30 슬라이드 30 · Mobile ALOHA로 확장하기 25
2.31 슬라이드 31 · 원격 조작 시연의 역할 25
2.32 슬라이드 32 · 자율 실행의 평가 25
2.33 슬라이드 33 · Diffusion Policy로 넘어가기 25
2.34 슬라이드 34 · 세 실패 원인을 다시 연결하기 25
2.35 슬라이드 35 · 노이즈에서 미래 행동열로 25
2.36 슬라이드 36 · 결과 영상의 해석 26
2.37 슬라이드 37 · 로봇 파운데이션 모델의 조건부 정책 26
2.38 슬라이드 38 · DROID와 환경 다양성 26

2.39 슬라이드 39 · Open X-Embodiment와 로봇 몸체의 차이 26
2.40 슬라이드 40 · π_0의 자율 수행 장면 26
2.41 슬라이드 41 · 사전학습과 과업별 후속 학습 26
2.42 슬라이드 42 · VLM과 행동 전문가의 연결 26
2.43 슬라이드 43 · 로봇 데이터와 텍스트 데이터의 규모 비교 27
2.44 슬라이드 44 · 데이터 수집 규모의 현실 27
2.45 슬라이드 45 · 가정 환경의 긴 작업 27
2.46 슬라이드 46 · ACT-1이라는 이름을 해석할 때 27
2.47 슬라이드 47 · 다양성을 시각화한 데이터 지도 27
2.48 슬라이드 48 · 새로운 로봇 플랫폼에서의 조작 27
2.49 슬라이드 49 · 모방학습의 도달 범위 27
2.50 슬라이드 50 · 보상으로 배우는 단계 27

3 02-1. 강화학습의 수학적 기초 28
3.1 슬라이드 01 · 강화학습의 수학적 기초 28
3.2 슬라이드 02 · 복습에서 형식화로 28
3.3 슬라이드 03 · 평가 요소와 개념 학습 28
3.4 슬라이드 04 · 시험에서 MDP를 설명하는 법 28
3.5 슬라이드 05 · 출석과 연속적인 학습 28
3.6 슬라이드 06 · 질문으로 확인할 수 있는 가정 28
3.7 슬라이드 07 · 자료를 사용할 때의 순서 28
3.8 슬라이드 08 · 교재와 현대 알고리즘 사이 28
3.9 슬라이드 09 · 모방학습을 되짚는 이유 28
3.10 슬라이드 10 · 결정 문제의 최소 구성 29
3.11 슬라이드 11 · 행동 복제의 기준선 29
3.12 슬라이드 12 · 모방학습의 세 실패 29
3.13 슬라이드 13 · DAgger의 상태 분포 수정 29
3.14 슬라이드 14 · 사람의 개입과 보상의 차이 29
3.15 슬라이드 15 · RL 문제로 넘어가는 연결 29
3.16 슬라이드 16 · 누적 보상의 의미 29
3.17 슬라이드 17 · RL 문제를 확률로 쓰기 30
3.18 슬라이드 18 · MDP가 환경과 과업을 함께 담는 이유 30
3.19 슬라이드 19 · 확률 과정의 기초 30
3.20 슬라이드 20 · Markov 성질의 정확한 뜻 30
3.21 슬라이드 21 · Markov chain의 두 요소 30
3.22 슬라이드 22 · 공부와 과제의 Markov chain 31
3.23 슬라이드 23 · Markov Reward Process 31

3.24 슬라이드 24 · 즉시 비용과 최종 보상 . . . 32
3.25 슬라이드 25 · 선택을 더한 MDP . . . 32
3.26 슬라이드 26 · 공부 예시에 행동을 넣기 . . . 32
3.27 슬라이드 27 · 환경을 정의하는 식과 학습 목표 . . . 32
3.28 슬라이드 28 · 정책이 정의하는 행동 . . . 32
3.29 슬라이드 29 · 초기 상태 분포까지 포함하기 . . . 33
3.30 슬라이드 30 · 시작 상태가 여러 개일 때 . . . 33
3.31 슬라이드 31 · 상태와 행동은 과업마다 다르다 . . . 33
3.32 슬라이드 32 · 환경을 완전히 정의해도 모를 수 있다 . . . 33
3.33 슬라이드 33 · 실제 던지기의 MDP . . . 34
3.34 슬라이드 34 · 환경 API와 수학 기호의 대응 . . . 34
3.35 슬라이드 35 · 완전 관측의 한계 . . . 34
3.36 슬라이드 36 · Ant 환경을 MDP로 쓰기 . . . 35
3.37 슬라이드 37 · 게임에서의 상태와 보상 . . . 35
3.38 슬라이드 38 · 실제 로봇에서의 상태와 보상 . . . 35
3.39 슬라이드 39 · 대화의 상태와 보상 . . . 35
3.40 슬라이드 40 · 용어를 한 궤적에 모으기 . . . 35
3.41 슬라이드 41 · 정책경사로의 연결 . . . 35

4 02-2. 정책경사와 REINFORCE 36

4.1 슬라이드 01 · 정책경사의 질문 . . . 36
4.2 슬라이드 02 · 반환과 기울기 . . . 36
4.3 슬라이드 03 · 한 궤적의 기호 . . . 36
4.4 슬라이드 04 · MDP와 정책의 역할 . . . 36
4.5 슬라이드 05 · 확률적 정책의 의미 . . . 36
4.6 슬라이드 06 · 지도학습과 다른 손실 . . . 37
4.7 슬라이드 07 · 정책 손실을 바로 쓸 수 없는 이유 . . . 37
4.8 슬라이드 08 · 궤적 확률의 곱 . . . 37
4.9 슬라이드 09 · 로그미분 항등식 . . . 37
4.10 슬라이드 10 · 환경 전이 항이 사라지는 이유 . . . 38
4.11 슬라이드 11 · Monte Carlo 정책경사 . . . 38
4.12 슬라이드 12 · REINFORCE의 반복 . . . 38
4.13 슬라이드 13 · 유도의 네 단계 복기 . . . 39
4.14 슬라이드 14 · 장점과 두 한계 . . . 39
4.15 슬라이드 15 · On-policy 데이터 조건 . . . 39
4.16 슬라이드 16 · 표본 기울기의 분산 . . . 39
4.17 슬라이드 17 · 분산을 줄이는 두 방향 . . . 40

4.18 슬라이드 18 · 표본 수만 늘리기 어려운 이유 40
4.19 슬라이드 19 · 더 나은 가중치의 계보 40
4.20 슬라이드 20 · Actor–Critic으로의 연결 40

5 03-1. 가치함수와 Actor–Critic 41

5.1 슬라이드 01 · Actor–Critic의 출발점 41
5.2 슬라이드 02 · 분산 감소의 세 단계 41
5.3 슬라이드 03 · 로그미분 유도 다시 보기 41
5.4 슬라이드 04 · 정책 로그확률만 남기기 41
5.5 슬라이드 05 · 표본 정책경사의 구조 41
5.6 슬라이드 06 · On-policy와 분산의 문제 42
5.7 슬라이드 07 · 가중치를 단계적으로 바꾸기 42
5.8 슬라이드 08 · 먼저 필요한 점수 함수의 성질 42
5.9 슬라이드 09 · 점수 함수의 기대값이 0인 이유 42
5.10 슬라이드 10 · 과거 보상은 현재 행동의 결과가 아니다 42
5.11 슬라이드 11 · 전체 반환을 시점별로 나누기 43
5.12 슬라이드 12 · 과거 보상 항의 기대값 43
5.13 슬라이드 13 · Reward-to-go가 남긴 것 43
5.14 슬라이드 14 · 인과성의 직관 43
5.15 슬라이드 15 · 가치함수가 필요한 순간 44
5.16 슬라이드 16 · V^π와 Q^π의 차이 44
5.17 슬라이드 17 · 가치와 advantage를 한 표기로 44
5.18 슬라이드 18 · 상태가치의 학습 목표 44
5.19 슬라이드 19 · 할인율의 두 역할 45
5.20 슬라이드 20 · Bellman 식에 할인율 넣기 45
5.21 슬라이드 21 · Reward-to-go를 더 안정적으로 추정하기 45
5.22 슬라이드 22 · Q가 행동의 공로를 평가한다 45
5.23 슬라이드 23 · 진짜 Q를 얻는 두 방법 46
5.24 슬라이드 24 · 상태 baseline을 생각하기 46
5.25 슬라이드 25 · Baseline을 빼도 기대값이 같은 증명 46
5.26 슬라이드 26 · 기대값 보존과 분산 감소는 별개 46
5.27 슬라이드 27 · 분산을 실제로 비교하는 식 47
5.28 슬라이드 28 · 왜 $V^\pi(s)$를 선택하는가 47
5.29 슬라이드 29 · 평균보다 좋은 행동 강화하기 47
5.30 슬라이드 30 · Advantage와 Actor–Critic 47
5.31 슬라이드 31 · 네 가지 가중치의 비교 48
5.32 슬라이드 32 · REINFORCE의 데이터 요구 48

5.33 슬라이드 33 · Actor와 critic의 반복 . . . 48
5.34 슬라이드 34 · 표본 효율의 정확한 비교 . . . 48
5.35 슬라이드 35 · 다음 단계: 길이와 재사용 . . . 49

6 03-2. GAE와 PPO 50

6.1 슬라이드 01 · PPO가 해결하려는 문제 . . . 50
6.2 슬라이드 02 · 두 개선 방향 . . . 50
6.3 슬라이드 03 · 정책경사 가중치의 계보 . . . 50
6.4 슬라이드 04 · REINFORCE가 기다리는 것 . . . 50
6.5 슬라이드 05 · Actor–Critic의 한 단계 . . . 50
6.6 슬라이드 06 · 자주 갱신할 수 있다는 뜻 . . . 51
6.7 슬라이드 07 · 남은 두 약점 . . . 51
6.8 슬라이드 08 · n단계 반환 . . . 51
6.9 슬라이드 09 · GAE의 가중 평균과 재귀 . . . 51
6.10 슬라이드 10 · Advantage 추정과 데이터 재사용은 별개 . . . 52
6.11 슬라이드 11 · 왜 현재 정책 데이터가 필요한가 . . . 52
6.12 슬라이드 12 · 중요도 샘플링의 기본식 . . . 53
6.13 슬라이드 13 · 궤적 비율에서 행동 비율로 . . . 53
6.14 슬라이드 14 · 여러 갱신에서 이전 정책 고정하기 . . . 53
6.15 슬라이드 15 · 큰 중요도 비율과 정책 변화 . . . 54
6.16 슬라이드 16 · PPO를 구성하는 네 조각 . . . 54
6.17 슬라이드 17 · Advantage 부호에 따른 clipping 직관 . . . 54
6.18 슬라이드 18 · 하나의 PPO-Clip 목적함수로 합치기 . . . 54
6.19 슬라이드 19 · PPO 학습 반복의 순서 . . . 55
6.20 슬라이드 20 · PPO가 널리 쓰이는 이유와 한계 . . . 55
6.21 슬라이드 21 · 접촉이 많은 로봇 조립 . . . 55
6.22 슬라이드 22 · SAPG와 병렬 학습 . . . 55
6.23 슬라이드 23 · 사족 보행의 실제 로봇 시연 . . . 56
6.24 슬라이드 24 · 보행을 넘어서는 전신 제어 . . . 56
6.25 슬라이드 25 · 조작 정책 시연을 읽는 법 . . . 56
6.26 슬라이드 26 · 사람 피드백과 언어모델 . . . 56
6.27 슬라이드 27 · 왜 다음은 off-policy인가 . . . 56

7 04-1 Q-learning과 DQN 57

7.1 슬라이드 01 · Q-learning의 출발 . . . 57
7.2 슬라이드 02 · 지금까지의 정책경사 . . . 57
7.3 슬라이드 03 · 정책 반복에서 가치 반복으로 . . . 57
7.4 슬라이드 04 · V와 Q의 관계 . . . 57

7.5 슬라이드 05 · 모델을 요구하는 fitted value iteration . . . 57
7.6 슬라이드 06 · 최적 Bellman 방정식 . . . 58
7.7 슬라이드 07 · Fitted Q-iteration . . . 58
7.8 슬라이드 08 · 세 가지 반복을 구분하기 . . . 58
7.9 슬라이드 09 · Q-learning의 장점과 위험 . . . 59
7.10 슬라이드 10 · Deep Q-Network . . . 59
7.11 슬라이드 11 · DQN을 지탱하는 세 장치 . . . 59
7.12 슬라이드 12 · 연속 transition의 상관 . . . 59
7.13 슬라이드 13 · Replay buffer의 역할 . . . 59
7.14 슬라이드 14 · 움직이는 목표 문제 . . . 59
7.15 슬라이드 15 · 자기 자신을 쫓는 회귀 . . . 60
7.16 슬라이드 16 · Target network . . . 60
7.17 슬라이드 17 · 탐색이 남는다 . . . 60
7.18 슬라이드 18 · ϵ-greedy의 확률 . . . 60
7.19 슬라이드 19 · Frame skip과 frame stack . . . 60
7.20 슬라이드 20 · 시각 자료: Atari 화면 . . . 61
7.21 슬라이드 21 · DQN 구조를 다시 묶기 . . . 61
7.22 슬라이드 22 · DQN을 개선하는 질문 . . . 61
7.23 슬라이드 23 · 네 가지 개선 축 . . . 61
7.24 슬라이드 24 · 큰 Q와 큰 실제 반환은 같은가 . . . 61
7.25 슬라이드 25 · 과대평가가 드러나는 경우 . . . 61
7.26 슬라이드 26 · 최대값 편향의 간단한 유도 . . . 61
7.27 슬라이드 27 · Double DQN의 분리 . . . 62
7.28 슬라이드 28 · 다음 개선: 목표의 지평 . . . 62
7.29 슬라이드 29 · n단계 Q 목표 . . . 62
7.30 슬라이드 30 · n단계의 off-policy 주의점 . . . 62
7.31 슬라이드 31 · 자료를 뽑는 방법도 중요하다 . . . 63
7.32 슬라이드 32 · Prioritized experience replay . . . 63
7.33 슬라이드 33 · 평균 가치만으로 충분한가 . . . 63
7.34 슬라이드 34 · 반환 분포와 평균의 차이 . . . 64
7.35 슬라이드 35 · 불확실성의 출처 . . . 64
7.36 슬라이드 36 · 분포적 Bellman 관계 . . . 64
7.37 슬라이드 37 · C51의 고정 지지점 . . . 64
7.38 슬라이드 38 · C51 목표 분포의 투영 . . . 65
7.39 슬라이드 39 · 분포적 학습이 바꾸는 것 . . . 65
7.40 슬라이드 40 · 개선들의 상호작용 . . . 65
7.41 슬라이드 41 · Rainbow . . . 66

7.42 슬라이드 42 · 연속 행동으로 넘어가기 66

8 05-1 Off-policy Actor–Critic 67

8.1 슬라이드 01 · 연속 행동을 향하여 67
8.2 슬라이드 02 · 다룰 알고리즘의 관계 67
8.3 슬라이드 03 · 정책 개선의 두 경로 67
8.4 슬라이드 04 · Bellman 식의 재확인 67
8.5 슬라이드 05 · DQN의 안전장치 67
8.6 슬라이드 06 · DQN 한 사이클 67
8.7 슬라이드 07 · DQN 개선의 역할 68
8.8 슬라이드 08 · 최댓값은 오차를 고른다 68
8.9 슬라이드 09 · Double DQN의 기준 68
8.10 슬라이드 10 · 개선책의 한계도 이어진다 68
8.11 슬라이드 11 · Q-learning의 정책은 암묵적이다 69
8.12 슬라이드 12 · DDPG로 넘어가는 단계 69
8.13 슬라이드 13 · 연속 행동에서 전수 비교가 막힌다 69
8.14 슬라이드 14 · 결정적 actor로 최대화를 근사 69
8.15 슬라이드 15 · 결정적 정책경사의 연쇄법칙 69
8.16 슬라이드 16 · 두 target 망의 소프트 갱신 70
8.17 슬라이드 17 · DDPG의 전체 업데이트 70
8.18 슬라이드 18 · DQN에서 DDPG로 바뀐 자리 70
8.19 슬라이드 19 · DDPG의 실용적 한계 70
8.20 슬라이드 20 · TD3가 해결할 세 문제 70
8.21 슬라이드 21 · TD3의 세 안정화 장치 70
8.22 슬라이드 22 · 두 critic이 주는 보수성 71
8.23 슬라이드 23 · TD3 학습 흐름 71
8.24 슬라이드 24 · 성능 그래프를 해석할 때 71
8.25 슬라이드 25 · TD3 요약 71
8.26 슬라이드 26 · 확률적 actor라는 다른 선택 71
8.27 슬라이드 27 · 평균과 분산을 모두 배우기 72
8.28 슬라이드 28 · 최대 엔트로피 목적 72
8.29 슬라이드 29 · 무작위성의 효용과 비용 73
8.30 슬라이드 30 · Soft Bellman 식 73
8.31 슬라이드 31 · SAC의 critic과 actor 목표 73
8.32 슬라이드 32 · 온도 α를 자동 조절 74
8.33 슬라이드 33 · SAC와 TD3의 공통 기반 74
8.34 슬라이드 34 · SAC가 유용한 조건 74

8.35 슬라이드 35 · TD3와 SAC 실험 비교 . . . 74
8.36 슬라이드 36 · 지금까지의 학습 지형 . . . 74
8.37 슬라이드 37 · 벤치마크로 이어지는 질문 . . . 75

9 05-2 강화학습 벤치마크 76
9.1 슬라이드 01 · 벤치마크를 읽는 관점 . . . 76
9.2 슬라이드 02 · 최종 성능과 표본 효율 . . . 76
9.3 슬라이드 03 · 벤치마크의 지도 . . . 76
9.4 슬라이드 04 · Gymnasium MuJoCo와 DMC . . . 76
9.5 슬라이드 05 · Atari의 관측과 예산 . . . 76
9.6 슬라이드 06 · 로봇 벤치마크의 폭 . . . 76
9.7 슬라이드 07 · LIBERO와 평생학습 . . . 76
9.8 슬라이드 08 · RoboTwin 2.0의 양팔 과제 . . . 76
9.9 슬라이드 09 · SIMPLER: 현실 정책을 시뮬레이션에서 평가 . . . 76
9.10 슬라이드 10 · RoboArena의 분산형 실기 평가 . . . 77
9.11 슬라이드 11 · FurnitureBench의 재현 가능한 실물 조립 . . . 77
9.12 슬라이드 12 · HumanoidBench의 전신 제어 . . . 77
9.13 슬라이드 13 · 과제 그림에서 평가 설계로 . . . 77

10 06 오프라인 강화학습 78
10.1 슬라이드 01 · 데이터가 고정되면 달라지는 문제 . . . 78
10.2 슬라이드 02 · 이 장의 방법 지도 . . . 78
10.3 슬라이드 03 · Offline과 online의 자료 흐름 . . . 78
10.4 슬라이드 04 · 행동 정책과 목표 정책 . . . 78
10.5 슬라이드 05 · BC와 궤적 연결 . . . 78
10.6 슬라이드 06 · 보지 못한 행동의 함정 . . . 78
10.7 슬라이드 07 · 언제 필요한가 . . . 79
10.8 슬라이드 08 · 보통 off-policy Q-learning을 그대로 쓰면 . . . 79
10.9 슬라이드 09 · 예측 Q와 실제 반환의 분리 . . . 79
10.10슬라이드 10 · Offline RL의 핵심 긴장 . . . 79
10.11슬라이드 11 · 두 가지 주요 통제 위치 . . . 79
10.12슬라이드 12 · TD3+BC의 목적 . . . 79
10.13슬라이드 13 · 정책 제약과 가치 보수화 . . . 80
10.14슬라이드 14 · 데이터 밖 Q를 낮추는 이유 . . . 80
10.15슬라이드 15 · CQL의 첫 목적식 . . . 80
10.16슬라이드 16 · 데이터 행동과의 상대적 보수성 . . . 81
10.17슬라이드 17 · Log-sum-exp가 나오는 이유 . . . 81
10.18슬라이드 18 · CQL 학습과 조절 계수 . . . 81

10.19슬라이드 19 · Offline RL의 평가는 왜 까다로운가 . . . 82
10.20슬라이드 20 · D4RL 데이터셋 . . . 82
10.21슬라이드 21 · 데이터의 질과 다양성 . . . 82
10.22슬라이드 22 · OGBench와 목표 조건부 학습 . . . 82
10.23슬라이드 23 · TD3+BC와 CQL을 비교하기 . . . 83
10.24슬라이드 24 · 다른 개선 경로의 질문 . . . 83
10.25슬라이드 25 · OOD 평가가 남는 자리 . . . 83
10.26슬라이드 26 · 보상을 쓰는 행동복제 . . . 83
10.27슬라이드 27 · Filtered BC . . . 83
10.28슬라이드 28 · AWR의 연속적 가중치 . . . 83
10.29슬라이드 29 · KL 제약에서 지수 가중치가 나오는 직관 . . . 84
10.30슬라이드 30 · AWR의 장점과 경계 . . . 84
10.31슬라이드 31 · SARSA식 데이터 안 부트스트랩 . . . 85
10.32슬라이드 32 · 데이터 행동 중 좋은 쪽을 고르기 . . . 85
10.33슬라이드 33 · Expectile 가치의 의미 . . . 85
10.34슬라이드 34 · 비대칭 제곱손실 예 . . . 86
10.35슬라이드 35 · IQL의 세 단계 . . . 86
10.36슬라이드 36 · 방법들의 공통 기준 . . . 86
10.37슬라이드 37 · 고정 자료 학습의 응용 . . . 86
10.38슬라이드 38 · 로봇에서 offline 사전학습과 online 미세조정 . . . 86
10.39슬라이드 39 · 핵융합 플라스마 제어 . . . 86
10.40슬라이드 40 · 건물 냉각 제어 . . . 86
10.41슬라이드 41 · 칩 회로 설계와 PrefixRL . . . 87
10.42슬라이드 42 · 전체 정리 . . . 87

11 07-1. Model-based RL의 기초 88

11.1 슬라이드 01 · Model-based RL . . . 88
11.2 슬라이드 02 · 세 가지 질문 . . . 88
11.3 슬라이드 03 · 환경 모델은 무엇인가 . . . 88
11.4 슬라이드 04 · 모델을 모르는 환경 . . . 88
11.5 슬라이드 05 · 모델을 안다면 가능한 일 . . . 88
11.6 슬라이드 06 · 던지기의 반사실적 비교 . . . 89
11.7 슬라이드 07 · 모델의 두 사용법 . . . 89
11.8 슬라이드 08 · Planning으로 전환 . . . 89
11.9 슬라이드 09 · 행동열의 목적함수 . . . 89
11.10슬라이드 10 · 전수 검색의 비용 . . . 89
11.11슬라이드 11 · 미분 가능한 모델에서 행동을 개선하기 . . . 89

11.12슬라이드 12 · 샘플링으로 행동열 비교하기 . . . 90
11.13슬라이드 13 · 기울기와 샘플링의 절충 . . . 90
11.14슬라이드 14 · Cross-Entropy Method . . . 90
11.15슬라이드 15 · CEM을 어디에 쓰는가 . . . 90
11.16슬라이드 16 · 정확한 모델이라는 가정 . . . 91
11.17슬라이드 17 · 전이 모델을 데이터로 학습하기 . . . 91
11.18슬라이드 18 · 모델 오류가 계획에 들어오는 경로 . . . 91
11.19슬라이드 19 · 계획으로 수집한 데이터를 다시 넣기 . . . 91
11.20슬라이드 20 · 절벽 사례의 교정 . . . 92
11.21슬라이드 21 · Open-loop와 closed-loop . . . 92
11.22슬라이드 22 · Model-Predictive Control . . . 92
11.23슬라이드 23 · Model-based planning의 경계 . . . 92
11.24슬라이드 24 · 정책 학습으로 전환 . . . 92
11.25슬라이드 25 · 계획 결과를 정책으로 압축하기 . . . 93
11.26슬라이드 26 · 짧은 가상 rollout을 어디서 시작할까 . . . 93
11.27슬라이드 27 · 환경 자료와 모델 자료의 순환 . . . 93
11.28슬라이드 28 · 언제 모델이 유리한가 . . . 93
11.29슬라이드 29 · 두 경로의 정리 . . . 93
11.30슬라이드 30 · 다음 단계 . . . 94

12 07-2. Dreamer와 TD-MPC 95

12.1 슬라이드 01 · Advanced model-based RL . . . 95
12.2 슬라이드 02 · 두 알고리즘을 읽는 순서 . . . 95
12.3 슬라이드 03 · Model-based policy optimization 복습 . . . 95
12.4 슬라이드 04 · 어디에서 가상 궤적을 시작하는가 . . . 95
12.5 슬라이드 05 · 가상 자료를 쓰는 학습 고리 . . . 95
12.6 슬라이드 06 · Dreamer . . . 95
12.7 슬라이드 07 · Dyna의 실제 경험과 모의 경험 . . . 95
12.8 슬라이드 08 · Dyna를 더 일반적으로 읽기 . . . 96
12.9 슬라이드 09 · 이미지에서 작동하는 Dreamer . . . 96
12.10슬라이드 10 · World model의 잠재 상태 . . . 96
12.11슬라이드 11 · 긴 영상 예측을 어떻게 읽는가 . . . 97
12.12슬라이드 12 · 잠재 상태 위의 actor와 critic . . . 97
12.13슬라이드 13 · 잠재 상상에서 반환 만들기 . . . 97
12.14슬라이드 14 · 보상 기울기가 초기 행동까지 가는 길 . . . 97
12.15슬라이드 15 · Dreamer의 네 단계 고리 . . . 97
12.16슬라이드 16 · DreamerV1의 과업 범위 . . . 98

12.17슬라이드 17 · DreamerV1의 비교 그래프 . . . 98
12.18슬라이드 18 · DreamerV2와 이산 세계모델 . . . 98
12.19슬라이드 19 · DreamerV3의 범용성 . . . 98
12.20슬라이드 20 · DayDreamer의 실제 로봇 . . . 98
12.21슬라이드 21 · Dreamer에서 얻을 설계 원칙 . . . 98
12.22슬라이드 22 · TD-MPC로 전환 . . . 98
12.23슬라이드 23 · MPC의 짧은 지평 복습 . . . 98
12.24슬라이드 24 · 먼 미래를 다루는 두 길 . . . 99
12.25슬라이드 25 · 계획 끝에 가치함수 붙이기 . . . 99
12.26슬라이드 26 · TD-MPC의 계획 점수 . . . 99
12.27슬라이드 27 · 과업 지향 잠재 동역학 . . . 99
12.28슬라이드 28 · 느리게 움직이는 target encoder . . . 100
12.29슬라이드 29 · TD-MPC의 세 손실과 정책 손실 . . . 100
12.30슬라이드 30 · MPC에 정책을 함께 배우는 두 이유 . . . 100
12.31슬라이드 31 · TD-MPC의 실행 절차 . . . 101
12.32슬라이드 32 · 수집·학습·계획의 순환 . . . 101
12.33슬라이드 33 · 고차원 연속 제어 사례 . . . 101
12.34슬라이드 34 · 움직임 영상으로 보는 비교 . . . 101
12.35슬라이드 35 · 복잡한 보행에서의 차이 . . . 101
12.36슬라이드 36 · 학습 곡선 읽기 . . . 101
12.37슬라이드 37 · 계획 예산을 늘릴 때 . . . 101
12.38슬라이드 38 · TD-MPC2로 확장하기 . . . 101
12.39슬라이드 39 · Dreamer와 TD-MPC의 선택 . . . 102
12.40슬라이드 40 · 더 큰 탐색 문제로 . . . 102

13 과제 가이드 **103**

13.1 과제 1 · 모방학습 . . . 103
13.2 과제 2 · 정책경사와 PPO . . . 107
13.3 과제 3 · DQN과 Double DQN . . . 112

1 01-1. 강화학습의 문제와 강좌 개요

강화학습은 행동이 다음 상황을 바꾸는 문제를 다룬다. 먼저 무엇을 학습하는지 정의하고, 이어서 어떤 문제에서 이 접근이 필요한지 살펴본다.

1.1 슬라이드 01 · Deep Reinforcement Learning

순차적 의사결정과 그 정책을 신경망으로 학습하는 문제를 다룬다.

1.2 슬라이드 02 · 강좌 안내

강좌 운영 방식과 과제 일정에 관한 안내다. 구체적인 제출·평가 기준은 강좌 공지에서 확인한다.

1.3 슬라이드 03 · 오늘의 흐름

강좌 소개와 운영 안내를 거쳐 강화학습의 정의와 활용 사례로 들어간다.

1.4 슬라이드 04 · What is reinforcement learning?

핵심 질문은 **현재 행동이 미래의 상황과 성과까지 바꿀 때 무엇을 선택해야 하는가**다.

1.5 슬라이드 05 · 강화학습의 목표

정책(policy) π는 상태 s에서 행동 a를 고르는 규칙이다. 확률적 정책은 $\pi(a \mid s)$로 쓴다. 강화학습은 매 순간의 보상만이 아니라 여러 시점에 걸쳐 얻는 **누적 보상**이 커지도록 정책을 학습한다. 예를 들어 로봇이 물체에 곧장 손을 뻗으면 당장은 가까워져도, 충돌 때문에 결국 집지 못할 수 있다.

1.6 슬라이드 06 · 에이전트와 환경

시점 t에서 에이전트는 상태 s_t를 보고 행동 a_t를 고른다. 환경은 보상 r_t와 다음 상태 s_{t+1}을 돌려준다. 이 고리에서는 **행동이 다음 입력을 바꾼다**. 따라서 이미 주어진 입력마다 정답을 예측하는 문제와 달리, 정책을 실행했을 때 어떤 상태를 만나게 되는지도 중요하다.

1.7 슬라이드 07 · 순차적 의사결정 문제

게임의 수, 로봇의 움직임, 대화 에이전트의 도구 사용에는 공통점이 있다. 지금의 행동이 이후 선택지를 바꾼다는 점이다. 강화학습은 이런 문제를 표현하는 틀이며, 가치 추정·정책 최적화 등은 그 안에서 좋은 행동을 찾는 방법이다.

1.8 슬라이드 08 · 강좌의 초점

정책과 가치함수를 신경망으로 나타내는 deep RL을 중심으로, 모방학습에서 출발해 보상 기반 학습과 현대의 응용으로 이어진다.

1.9 슬라이드 09 · 다룰 주제

강좌는 모방학습, Markov decision process, 가치 기반 방법, policy gradient, actor–critic과 다양한 응용을 연결한다. 각 방법은 **어떤 데이터와 학습 신호를 쓰는가**로 구분하면 흐름을 잡기 쉽다.

1.10 슬라이드 10 · 강좌 일정

주차별 강의와 과제의 전체 순서를 보여 준다. 모방학습으로 정책 학습의 출발점을 익힌 뒤, 보상으로 장기 성과를 개선하는 방법을 배운다.

1.11 슬라이드 11 · 선수 지식

미분·확률·선형대수는 손실함수와 기대값을 읽는 데, 딥러닝과 PyTorch는 정책·가치함수를 구현하는 데 쓰인다. 뒤의 수식에서는 새 표기가 처음 나올 때 의미를 함께 설명한다.

1.12 슬라이드 12 · 평가 구성

과제 35%, 중간시험 30%, 기말시험 30%, 출석 3%, 참여 2%로 구성된다. 일정과 세부 기준은 강좌 공지를 기준으로 확인한다.

1.13 슬라이드 13 · 과제 구성

과제는 이론을 코드와 실험으로 확인하도록 배치되어 있다. 앞 과제의 정책·환경 표기가 뒤 과제에서도 이어지므로 실행 결과뿐 아니라 가정과 평가 지표를 기록해 두면 좋다.

1.14 슬라이드 14 · 과제 진행 안내

환경 설정과 학습 실행에 시간이 걸릴 수 있다. 과제를 시작할 때 실행 환경, 제공 코드의 입력·출력, 제출 형식을 먼저 확인한다.

1.15 슬라이드 15 · 시험 안내

시험의 범위·형식·시간에 관한 운영 안내다. 최신 세부 사항은 강좌 공지에서 확인한다.

1.16 슬라이드 16 · 출석 안내

출석 인정 방식과 예외 처리에 관한 안내다. 실제 적용 기준은 강좌 공지를 따른다.

1.17 슬라이드 17 · 수업 참여

질문과 토론 등 수업 참여에 관한 안내다. 개념을 설명할 때는 결론뿐 아니라 사용한 가정과 근거를 함께 제시하면 좋다.

1.18 슬라이드 18 · 소통과 자료

강의 자료, 질의응답, 이메일 및 면담 채널을 안내한다. 과제와 시험 관련 공지는 강좌 공식 채널에서 확인한다.

1.19 슬라이드 19 · 참고 자료

Sutton·Barto의 *Reinforcement Learning: An Introduction*은 기본 개념의 기준서다. Stanford CS224R과 UC Berkeley CS285는 딥러닝 기반 방법과 응용을 더 살펴볼 때 유용하다.

1.20 슬라이드 20 · Why study reinforcement learning?

이제 강좌 운영에서 벗어나, 지도학습만으로 해결하기 어려운 문제를 살펴본다.

1.21 슬라이드 21 · 인공지능의 발전

인공지능의 발전 사례를 돌아보며, 다음 성과를 위해 어떤 학습 신호가 필요한지 묻는다.

1.22 슬라이드 22 · 주요 발전 사례

이미지는 분류, 생성, 게임 분야의 대표적 성과를 나란히 보여 준다. AlphaGo는 **선택의 결과가 여러 수 뒤에 나타나는** 문제의 예다.

1.23 슬라이드 23 · 생성 모델과 AI 도구

이미지는 생성형 AI와 코드 도구의 사례다. 뛰어난 생성 능력과, 여러 행동을 선택해 목표를 끝까지 달성하는 능력은 구분해서 평가해야 한다.

1.24 슬라이드 24 · 지도학습만으로 충분한가?

AlphaGo의 수 선택처럼 좋은 행동의 의미가 **나중에 드러나는 결과**에 달려 있다면, 매 상태의 정답 행동만 모으기 어렵다. 지도학습은 주어진 정답을 모방할 수 있지만, 강화학습은 실제 결과를 통해 행동의 장기 가치를 평가하고 개선한다.

1.25 슬라이드 25 · 칩 설계의 예

회로의 배치·배선은 앞선 선택이 뒤의 설계 가능성과 성능을 바꾼다. 그림은 PrefixRL을 전자설계 자동화(EDA)와 비교하는 사례로, 순차적 설계 문제에 RL을 적용한 맥락을 보여 준다.

1.26 슬라이드 26 · 언어모델의 인간 피드백

그림은 시범 답변으로 모델을 지도학습한 뒤, 사람의 선호로 보상모델을 만들고, 그 보상을 사용해 정책을 개선하는 RLHF 흐름을 보여 준다. 선호는 매 토큰의 정답을 직접 주기보다 **완성된 응답의 품질**을 평가한다.

1.27 슬라이드 27 · 이미지 생성과 보상

DDPO 사례에서는 생성 과정을 일련의 선택으로 보고, 최종 이미지에 대한 보상으로 생성 정책을 개선한다. 그림은 서로 다른 보상 기준에 따라 결과가 달라질 수 있음을 보여 준다.

1.28 슬라이드 28 · 코드 에이전트

코드 에이전트는 파일을 읽고 수정하며 도구를 여러 차례 호출한다. 이미지는 이런 순차적 작업의 사례이며, 특정 도구의 내부 학습 방식까지 뜻하지는 않는다.

1.29 슬라이드 29 · 로봇 보행

네 발 로봇의 보행 사례다. 한 번의 관절 명령은 다음 자세와 균형에 영향을 주므로, 매 순간의 움직임보다 안정적인 이동 전체를 평가해야 한다.

1.30 슬라이드 30 · 로봇 동작

로봇 동작 영상의 사례다. 영상은 결과를 보여 주지만, 사용된 정책과 학습 방법은 별도의 근거가 있어야 판단할 수 있다.

1.31 슬라이드 31 · 강화학습이 필요한 이유

정답 행동을 매 순간 제시하기 어렵고, 행동의 좋고 나쁨이 장기 결과로 드러나는 문제가 많다. 강화학습은 환경의 피드백으로 선택을 개선할 수 있다. 반면 시연이 있다면 먼저 모방학습으로 출발하는 편이 효율적일 수 있다.

1.32 슬라이드 32 · 다음 주제: 모방학습

전문가의 상태–행동 시연으로 정책을 만드는 방법부터 시작한다. 이어서 시연을 잘 따라 했는데도 실제 실행에서 실패하는 이유를 살펴본다.

2 01-2. 모방학습과 로봇 정책

전문가가 이미 좋은 행동을 보여 주었다면 먼저 그 행동을 배울 수 있다. 모방학습은 정책을 시작하는 가장 직접적인 방법이지만, 실제로 움직일 때는 훈련 때 보지 못한 상태를 만들고 여러 유효한 행동 중 하나를 골라야 한다. 이 장은 행동 복제의 목적함수에서 출발해 분포 이동, DAgger, 현대 로봇 정책까지 연결한다.

2.1 슬라이드 01 · Imitation Learning

모방학습은 전문가가 보여 준 상태–행동 쌍에서 정책을 배운다. 이 장에서는 행동 복제(BC)의 손실 함수, 실행 중 생기는 분포 이동, 온라인 교정, 로봇 정책 사례를 순서대로 살펴본다.

2.2 슬라이드 02 · 강화학습과 모방학습의 위치

강화학습은 행동의 **결과**인 보상으로 정책을 개선한다. 모방학습은 전문가의 **행동**을 학습 신호로 쓴다. 시연이 있다면 먼저 기본 정책을 만드는 데 유용하다.

2.3 슬라이드 03 · 에이전트–환경 고리 다시 보기

에이전트가 현재 정보 s_t를 보고 행동 a_t를 내면 환경은 보상 r_t와 다음 상태 s_{t+1}을 반환한다. 전문가의 시연도 이 고리에서 만들어진 궤적이므로, 훈련 상태는 전문가의 **방문 분포**에 집중된다. 예를 들어 전문가가 늘 컵의 왼쪽에서 접근했다면 오른쪽으로 틀어진 팔 자세의 영상은 데이터에 드물다.

학습 정책이 오른쪽으로 틀어지는 순간 다음 입력도 달라진다. 따라서 한 시점의 행동 정답률과 별도로, 정책을 실행한 궤적에서 만나는 상태와 최종 성공률을 평가한다. 이 분포 차이가 뒤의 오류 누적 분석으로 이어진다.

2.4 슬라이드 04 · 상태, 행동, 정책

정책은 입력 상태 s에서 행동 a를 고르는 규칙이다. 확률적 정책은 $\pi_\theta(a \mid s)$, 결정론적 정책은 $a = \mu_\theta(s)$로 쓴다. 로봇의 상태에는 카메라 영상, 관절 각도, 그리퍼 상태가 포함될 수 있고 행동은 목표 관절 위치나 모터 토크일 수 있다.

"최적 행동열을 찾는다"는 말은 매 시점의 행동을 따로 맞히는 것보다 어렵다. 물체를 집기 전의 접근 방향이 이후의 잡기 가능성을 바꾸기 때문이다. 여기서 정책의 입력에 필요한 정보가 빠지면 관측만으로 좋은 행동을 결정할 수 없게 된다.

행동의 단위도 명시해야 한다. 같은 숫자 0.2가 관절의 절대 목표 각도인지, 현재 각도에서 더할 변화량인지, 모터 토크인지에 따라 결과가 전혀 다르다. 정책을 비교할 때 관측과 행동의 차원뿐 아니라 좌표계, 단위, 제어 주기까지 맞춰야 하는 이유다.

2.5 슬라이드 05 · 궤적과 전이의 표기

시간 $t = 0, 1, \ldots$에서 상태 s_t, 행동 a_t, 전이 $P(s_{t+1} \mid s_t, a_t)$를 구분하자. 관측 o_t는 센서가 실제로 보여 주는 값이고, 상태 s_t는 미래를 예측하기에 충분하다고 가정하는 정보다. 둘이 같을 수도 있지만 카메라 한 장에 속도나 가려진 물체 위치가 없으면 다르다.

정책을 환경에서 실행해 얻은 $\tau = (s_0, a_0, s_1, a_1, \ldots, s_T)$를 rollout 또는 episode라고 한다. 시연 하나도 하나의 궤적이고, 그 안의 상태–행동 쌍을 잘라 행동 복제의 훈련 데이터로 사용할 수 있다.

궤적을 쌍으로 자를 때 (s_t, a_t)의 시각이 정확히 맞아야 한다. 카메라 프레임이 100ms 늦었다면 데이터에는 행동의 결과 영상과 이전 행동이 잘못 짝지어질 수 있다. 정책은 "컵이 이미 왼쪽으로 움직인 뒤에 오른쪽으로 밀라" 같은 뒤늦은 규칙을 배울 수 있다.

episode와 rollout은 항상 같은 길이일까? episode는 보통 환경의 시작부터 종료까지를 뜻한다. rollout은 학습에 필요한 고정 길이만 수집해 중간에 끊을 수도 있다. 시간 제한으로 끊긴 표본을 진짜 실패 종료와 혼동하면 가치 학습의 목표도 달라진다.

2.6 슬라이드 06 · 정답 행동과 보상

지도학습은 입력마다 따라야 할 정답을 알고 있다고 가정한다. 행동 복제에서는 전문가 행동 a^*가 그 정답 역할을 한다. 강화학습에서는 정답 행동 없이 선택의 결과를 나타내는 보상만 받을 수 있다. 로봇이 컵을 떨어뜨렸다는 피드백은 "정확히 어느 관절 각도를 골랐어야 하는지"를 알려 주지 않는다.

두 방식은 서로 대체 관계로만 볼 필요가 없다. 시연으로 초기 정책을 만든 뒤, 실제 성공 보상으로 세부 행동을 개선할 수 있다. 다만 보상을 얻기 위한 실제 상호작용이 비싸거나 위험하면 시연의 가치가 커진다.

예를 들어 장난감 환경에서 성공하면 마지막에 1점만 받는다면, 실패한 로봇에게는 어느 관절 명령이 잘못되었는지 알 길이 없다. 전문가 시연의 각 시점 행동은 그 공백을 메운다. 반대로 시연에서 손목이 매번 조금 흔들린다면 지도학습은 그 흔들림도 목표로 보지만, 보상 기반 평가는 최종 성공에 무해한 차이를 구분할 수 있다.

2.7 슬라이드 07 · 이 장의 질문

먼저 시연으로 정책을 학습하고, 시연을 수집하는 방법과 실행 시의 실패 원인을 살펴본다.

2.8 슬라이드 08 · Behavioral Cloning의 목적함수

전문가 데이터 $\mathcal{D} = \{(s_i, a_i^*)\}_{i=1}^{N}$는 N개의 상태 s_i와 전문가 행동 a_i^*의 쌍이다. θ를 정책 신경망의 파라미터라 하자. $\pi_\theta(a_i^* \mid s_i)$는 그 상태에서 전문가 행동에 부여한 **확률질량**이며, 연속 행동이면 해당 점의 **확률밀도**다. 행동 복제(BC)는 다음 평균 음의 로그우도 $\mathcal{L}_{\mathrm{BC}}$를 최소화한다.

$$\mathcal{L}_{\mathrm{BC}}(\theta) = -\frac{1}{N}\sum_{i=1}^{N} \log \pi_\theta(a_i^* \mid s_i).$$

행동이 범주형이면 이 식은 정답 클래스의 교차 엔트로피가 된다. 연속 행동에 고정 분산 Gaussian 정책을 가정하면 음의 로그우도는 상수항을 제외하고 평균 행동의 제곱오차에 비례한다. 그래서 같은 "BC"라도 선택한 행동 분포에 따라 학습 손실이 다르게 보인다. 초기 자율주행 연구인 ALVINN은 카메라 입력에서 조향을 배우는 고전적인 예다.

연속 행동에서 L2 손실과 최대우도 학습이 언제 같은가? 정책을 평균 $\mu_\theta(s)$와 고정 분산 $\sigma^2 I$를 가진 Gaussian으로 두면 $-\log \pi_\theta(a \mid s) = \|a - \mu_\theta(s)\|^2/(2\sigma^2) + C$다. 분산까지 학습하거나 다봉 분포를 쓰면 단순 L2 회귀와 같지 않다.

이산 행동에서 전문가 행동에 부여한 확률이 0.8이면 한 표본 손실은 $-\log 0.8 \approx 0.22$, 0.2이면 $-\log 0.2 \approx 1.61$이다. 연속 행동의 밀도는 1보다 클 수도 있어 이 숫자 예와 구별해야 한다. 어느 경우든 이 손실은 전문가가 방문한 상태에서의 행동 예측을 측정한다. 배포 시 정책 스스로 방문하는 상태의 성공률은 rollout으로 평가한다.

2.9 슬라이드 09 · 시연의 품질과 범위

다음 질문은 전문가의 상태–행동 쌍을 실제로 어디서 얻는가다.

2.10 슬라이드 10 · 시연을 수집하는 방식

텍스트나 운전 기록처럼 사람이 평소 하는 행동을 로그로 남길 수도 있다. 로봇에서는 팔을 직접 움직이는 kinesthetic teaching, 원격 조종기, 사람의 조작을 로봇의 다른 팔에 전달하는 puppeteering을 쓴다. 각각 장비 비용, 조작 편의, 영상에 사람이 가리는 정도, 행동의 정밀도가 다르다.

사람이 그대로 시연할 수 없는 행동도 있다. 네 발로 걷는 로봇의 각 관절 토크를 사람이 매 순간 정답으로 제시하기는 어렵다. 이 경우 시뮬레이션이나 보상 기반 RL이 필요할 수 있다.

시연을 많이 모으면 어떤 데이터 문제가 남을까? 관측과 행동의 시간 지연, 서로 다른 사람의 행동 방식, 센서 캘리브레이션, 실패 시연의 혼입이 남는다. 양을 늘리기 전에 시연이 실제 배포 정책이 볼 상태와 같은 표현으로 저장되는지 확인해야 한다.

시연 장치가 다르면 학습 가능한 행동의 정밀도도 바뀐다. 원격 조작자가 큰 버튼으로 그리퍼를 여닫는다면 세밀한 손가락 힘 조절을 라벨로 얻을 수 없다. 반대로 고정밀 장치를 쓰더라도 조작자의 지연이 크면 빠른 접촉 순간의 타이밍이 흐려질 수 있다. 따라서 데이터 수집 인터페이스 자체가 정책의 성능 상한에 영향을 준다.

2.11 슬라이드 11 · 실패 원인으로 넘어가기

이제 시연 데이터를 잘 맞힌 정책이 실제 실행에서 실패하는 이유로 넘어간다.

2.12 슬라이드 12 · 실패를 분해하는 틀

모방학습의 대표적 난점은 오류 누적, 같은 상태에서 가능한 여러 행동, 전문가와 정책 사이의 관측 정보 차이다.

2.13 슬라이드 13 · 첫째 실패: 오류의 누적

조향 정책이 한 번 차선 가장자리로 틀어지면 다음 영상은 전문가 시연에서 드문 상태가 된다. 정책이 만든 상태를 다시 입력받는 이 구조에서, 전문가의 방문분포와 학습 정책의 방문분포가 달라지는 현상을 **정책 유발 분포 이동**이라고 한다.

오류 누적을 수량화해 보자. π^*는 전문가 정책, π는 학습 정책, T는 행동 횟수다. $d^t_{\pi^*}$와 d^t_{π}는 각각 시점 t에 두 정책이 방문하는 상태의 확률분포다. 분석을 간단히 하기 위해 두 정책은 결정론적이라고 두고, 전문가 상태에서의 시점별 행동 오류율 e_t와 그 평균 ϵ을 정의한다.

$$e_t = \Pr_{s\sim d^t_{\pi^*}}\big[\pi(s) \neq \pi^*(s)\big], \qquad \epsilon = \frac{1}{T}\sum_{t=1}^{T} e_t.$$

두 정책을 같은 초기 상태와 환경의 무작위 결과에 놓으면, 첫 행동 오류 전까지는 같은 상태를 지나간다. D_t를 시점 t까지 두 경로가 갈라진 사건이라고 하자. $i \le t$에서 첫 오류가 일어날 사건의 확률은 전문가 경로에서 시점 i에 오류가 날 확률 e_i 이하이므로, 사건들의 합집합에 대한 **union bound**로 $\Pr(D_t) \le \sum_{i=1}^{t} e_i$를 얻는다. 단계별 오류가 독립일 필요는 없다.

이제 $c_t = c(s_t, a_t) \in [0,1]$을 시점 t의 비용으로 놓고, $J(\pi) = E_\pi[\sum_{t=1}^{T} c_t]$를 정책의 총기대비용이라 하자. E_π는 초기 상태와 환경의 무작위성까지 포함해 π가 만든 궤적에 대해 평균낸다는 뜻이다. 같은

상태·행동과 같은 환경 결과를 결합한 두 경로는 갈라지기 전 비용도 같다. 갈라진 뒤에는 한 시점의 비용 차이가 최대 1이므로, 기대 비용 차이는 각 시점에서 이미 갈라졌을 확률의 합 이하이다.

$$\begin{aligned} J(\pi) - J(\pi^*) &\le \sum_{t=1}^{T} \Pr(D_t) \\ &\le \sum_{t=1}^{T} \sum_{i=1}^{t} e_i = \sum_{i=1}^{T} (T - i + 1) e_i \\ &\le T \sum_{i=1}^{T} e_i = T^2 \epsilon. \end{aligned}$$

i번째 오류가 남은 최대 $T - i + 1$단계에 영향을 줄 수 있어 $T^2\epsilon$이 나타난다. 각 시점의 오류율이 동일한 $e_i = \epsilon$인 예에서는 위 식의 중간 합이 $\epsilon T(T+1)/2$다. $T = 20$, $\epsilon = 0.01$이면 추가 비용의 상한은 2.1이다. 실제 비용은 복구 가능성에 따라 달라진다. DAgger 원논문

시연 데이터에서 행동 정확도가 높은데 실행은 왜 실패할까? 시연 기반 시험은 $d^t_{\pi^*}$에서 오류율을 재지만, 실제 실행은 d^t_π의 상태를 만든다. 한 번 이탈한 뒤의 상태가 시연에 거의 없다면 시험 정확도는 복구 능력을 보여 주지 않는다. 정책을 실행한 rollout에서 방문 상태와 전체 과제 성공률을 함께 확인해야 한다.

2.14 슬라이드 14 · 오류 누적을 줄이는 두 경로

하나는 매우 넓은 시연 데이터를 모아 배포 중 만날 상태를 미리 덮는 것이다. 그러나 가능한 상태 공간이 커서 모든 이탈 상황을 수동으로 수집하기 어렵다. 다른 하나는 정책이 실제로 만든 실수 상태에서 **어떻게 복구할지** 전문가의 교정 행동을 받는 것이다.

예를 들어 조향 정책이 차선 왼쪽으로 치우친 영상을 만든다면, 중앙에서 직진하는 시연을 더 모으기보다 왼쪽으로 치우친 상태에서 오른쪽으로 복구하는 행동을 추가해야 한다. 다음 DAgger가 바로 이 원리를 반복적인 데이터 수집 절차로 만든다.

교정 데이터에는 잘못 조향한 후의 영상과 **그 상태에서 취할 복구 행동**이 함께 있어야 한다. 실패 영상만 있으면 BC 손실의 목표 행동을 만들 수 없다. DAgger는 전문가에게 이 행동을 묻고, RL은 실행 결과의 보상으로 정책을 평가한다.

2.15 슬라이드 15 · 둘째 실패: 여러 정답 행동

같은 물체를 왼쪽으로 돌아 잡거나 오른쪽으로 돌아 잡는 두 경로가 모두 성공할 수 있다. 이때 제곱 오차를 최소화하는 결정론적 예측은 조건부 평균 $E[a \mid s]$다. 두 행동의 평균이 물체를 정면으로 미는 방향이라면, 데이터에 없던 실패 행동을 출력한다.

이때 $p(a \mid s)$는 여러 봉우리를 가진 **다봉(multimodal) 분포**다. 범주형 분포, Gaussian mixture, 잠재변수를 가진 VAE, 확산 정책 등으로 여러 행동 모드를 표현할 수 있다. 실행 중 매 순간 모드를 바꾸면 경로가 지그재그로 흔들릴 수 있으므로 한 경로를 일관되게 따르는지도 평가한다.

L1 손실로 바꾸면 다봉 문제가 사라질까? L1의 최적 예측은 조건부 중앙값이므로 L2의 평균과는 다르다. 하지만 두 유효 행동 사이의 비유효 행동을 고를 가능성은 남는다. 여러 행동 경로를 분포로 표현하고 일관된 하나를 샘플링하는 접근이 더 직접적이다.

수치 예를 들면 같은 상태에서 유효 행동이 $a = -1$과 $a = +1$이고 두 행동이 동일한 빈도로 관측되었다고 하자. 제곱오차 $E[(a - \hat{a})^2]$를 최소화하는 예측은 $\hat{a} = E[a] = 0$이다. 그런데 0이 장애물로 직진하는 행동이라면 회귀 오차의 최적해가 과업에서는 실패다. 여러 모드를 표현하고 실행 중 한 모드를 유지하는 정책이 필요한 이유다.

평균이 최적이라는 주장은 미분으로 확인할 수 있다. 상태 s를 고정하고 예측 m에 대한 조건부 위험을 $L_s(m) = E[(a-m)^2 \mid s]$라고 두면

$$\frac{dL_s}{dm} = E[2(m-a) \mid s] = 2\{m - E[a \mid s]\}, \qquad \frac{d^2L_s}{dm^2} = 2 > 0.$$

따라서 유일한 최소점은 $m^* = E[a \mid s]$다. 이 결과는 회귀 손실에 대해서만 최적이라는 뜻이다. 행동의 성공 여부가 비선형이면 m^*가 성공 행동 집합 밖에 있을 수 있다. 다차원 관절 명령에도 각 좌표의 평균이 같은 방식으로 나오므로, 두 유효 동작의 좌표별 평균이 물체와 충돌할 수 있다.

2.16 슬라이드 16 · 셋째 실패: 전문가와 에이전트의 관측 차이

전문가는 과거 대화에서 상대가 농구 경기를 했다는 정보를 보고 답했는데, 데이터셋에는 현재의 "잘 지내?"만 남았다고 하자. 에이전트가 볼 수 없는 과거를 이용해 전문가의 답변을 맞히라고 하면 같은 입력에 서로 다른 답이 붙는다. 모델이 크더라도 숨겨진 정보 자체를 복원할 근거가 없다.

해결책은 에이전트에도 필요한 대화 이력을 주거나, 전문가가 에이전트와 같은 정보만 보고 시연하게 하는 것이다. 로봇에서도 전문가가 손끝의 힘을 느끼는데 정책에는 영상만 주면 같은 문제가 생긴다. 이 경우 촉각 센서를 추가하거나 시연의 정보 조건을 다시 맞춰야 한다.

관측 불일치는 데이터 양만 늘려도 풀리지 않는다. 같은 현재 영상 o에 대해 전문가가 과거 접촉 정보 h를 보고 왼쪽 또는 오른쪽을 택했다면, o만 받는 정책은 그 차이를 구별할 수 없다. 시간에 걸친 관측을 입력하거나 촉각 신호를 추가하는 등 **결정에 필요한 정보**를 정책에 전달해야 한다.

2.17 슬라이드 17 · 세 실패의 진단 순서

세 실패 원인을 구분해야 해결책도 달라진다. 오류 누적에는 실행 중 교정 데이터가, 여러 유효 행동에는 표현력 있는 행동 분포가, 관측 불일치에는 필요한 정보의 제공이 우선이다.

2.18 슬라이드 18 · 온라인 교정으로 넘어가기

정책이 실제로 방문한 상태에서 전문가가 행동을 알려 주는 온라인 교정으로 넘어간다.

2.19 슬라이드 19 · DAgger의 네 단계

DAgger(Data Aggregation)는 현재 정책을 실행해 방문 상태를 모으고, 그 상태의 전문가 행동을 받아 기존 데이터에 합친 뒤 정책을 다시 학습한다. 반복 번호를 k, 그때의 정책을 π_k, 모은 데이터를 $\mathcal{D}_k$라고 쓰자. d_{π_k}는 $d^t_{\pi_k}$를 시간에 걸쳐 평균한 방문 상태분포로, 여기서 $s \sim d_{\pi_k}$는 실행 궤적에서 시점을 고른 뒤 그 상태를 뽑는다는 약기다. $\ell(\pi(s), a^*)$는 예측 행동과 전문가 행동 $a^* = \pi^*(s)$의 차이를 재는 감독 손실이다.

$$\mathcal{D}_{k+1} = \mathcal{D}_k \cup \{(s, \pi^*(s)) : s \sim d_{\pi_k}\}, \qquad \pi_{k+1} = \arg\min_{\pi} E_{(s,a^*)\sim\mathcal{D}_{k+1}} \ell(\pi(s), a^*).$$

중요한 변화는 학습 상태 분포가 실제 정책의 방문 상태를 포함한다는 것이다. 정책이 차선 밖으로 향하는 순간의 상태에 복구 행동을 붙일 수 있다. 한계는 에이전트가 제어하는 동안에도 전문가가 상태를 보고 답할 수 있어야 한다는 점이다. DAgger 논문

DAgger는 보상으로 학습하므로 강화학습인가? 표준 DAgger는 정책이 방문한 상태를 수집하지만 그 상태의 학습 신호는 여전히 전문가 행동 라벨이다. 환경과 상호작용한다는 점만으로 보상 기반 RL이 되는 것은 아니다.

첫 반복의 정책 π_0가 차선 중앙만 알고 있다가 왼쪽으로 치우친 상태를 만들었다고 하자. 전문가가 그 상태에서 오른쪽 조향을 라벨로 붙이면 $\mathcal{D}_1$에 복구 사례가 추가된다. 다음 정책은 그 상태를 만났을 때 복구할 근거가 생긴다. 이 예에서 환경의 보상은 새 라벨이 아니며, **전문가가 같은 상태를 보고 답한다**는 조건이 중요하다.

앞의 제곱 지평은 전문가 분포에서만 측정한 ϵ의 한계였다. $\ell(s,\pi)=\mathbf{1}\{\pi(s)\neq\pi^*(s)\}$를 행동 불일치 지표로 두고, 정책 자체의 방문분포에서 측정한 오류율을 다음처럼 정의하자.

$$\epsilon_\pi = \frac{1}{T}\sum_{t=1}^{T} E_{s\sim d_\pi^t}[\ell(s,\pi)].$$

이 값을 작게 만들면 다른 경계를 얻는다. $\pi^{(t)}$가 처음 t번 행동은 학습 정책 π, 그 뒤는 전문가 π^*를 따르는 hybrid 정책이라고 하자. 따라서 $\pi^{(0)}=\pi^*$이고 $\pi^{(T)}=\pi$다. $\pi^{(t)}$와 $\pi^{(t-1)}$는 시점 t의 행동만 다르므로 그 직전 상태분포는 둘 다 d_π^t다. 그 상태에서 학습 정책의 행동 하나를 택한 뒤 전문가를 따를 때의 **추가 기대 비용**이 언제나 u 이하라고 가정하면, 행동이 일치할 때 차이는 0이고 불일치할 때 최대 u다. 전체 차이를 각 hybrid 사이의 차이로 분해하면

$$\begin{aligned} J(\pi)-J(\pi^*) &= \sum_{t=1}^{T}\big[J(\pi^{(t)})-J(\pi^{(t-1)})\big] \\ &\le u\sum_{t=1}^{T} E_{s\sim d_\pi^t}\ell(s,\pi) = uT\epsilon_\pi. \end{aligned}$$

이 경계의 $T\epsilon_\pi$ 항은 실제 정책이 방문할 상태에서의 오류를 뜻한다. DAgger가 그런 상태에 전문가 라벨을 붙이는 이유가 여기에 있다. 라벨이 일관되고 학습기가 집계 데이터를 잘 맞힐수록 ϵ_π를 낮출 수 있다. DAgger의 성능 분석

2.20 슬라이드 20 · 사람이 개입하는 DAgger

사람이 로봇의 모든 상태에 일일이 정답 행동을 입력하기 어렵다면, 로봇을 관찰하다가 실패 조짐이 보일 때 제어권을 가져와 복구 시연을 제공할 수 있다. 이런 human-gated 방식은 부분 시연을 데이터에 더한다. 사람의 개입 순간이 곧 정책의 취약한 상태를 알려 준다.

다만 빠른 시스템에서는 개입을 결정하고 실제로 조작할 때까지 지연이 생긴다. 고속 조향이나 접촉 직전의 순간에는 실수를 놓칠 수 있다. 안전 장치, 개입 지연, 어떤 상태에서 사람이 관여했는지를 결과와 함께 기록해야 한다.

개입 데이터에는 선택 편향이 있다. 사람은 쉬운 상태에서는 개입하지 않고 실패가 가까운 상태에서 주로 조종한다. 따라서 개입 구간의 행동만 무작정 섞으면 데이터의 상태 비율이 크게 달라진다. 개입 기준, 개입 전후의 관측, 기존 전문가 시연과의 혼합 비율을 함께 기록해야 결과를 해석할 수 있다.

사람이 개입하지 않은 구간은 정답 행동이 없는가? 그 구간에는 정책이 실행한 행동 기록이 있지만, 그것이 전문가의 정답이라는 뜻은 아니다. 사람의 개입이 없었다는 사실을 성공 라벨로 자동 해석하면 조용히 실패하는 행동까지 시연으로 취급할 수 있다.

2.21 슬라이드 21 · 로봇 사례를 읽는 틀

앞의 문제와 해결책이 실제 로봇 조작에서 어떻게 나타나는지 사례로 살펴본다.

2.22 슬라이드 22 · ALOHA와 ACT의 결합

ALOHA는 저비용 양팔 로봇으로 시연을 수집하는 시스템이고, ACT는 그 시연에서 행동열을 학습하는 방법이다. 하드웨어와 학습 알고리즘의 역할을 구분해 보자. ACT 연구

2.23 슬라이드 23 · 저비용 하드웨어가 바꾸는 것

사람이 두 로봇 팔을 원격 조작하며 양팔 조작 시연을 모으는 장면이다. 저비용 장치로도 정교한 시연 데이터를 모을 수 있다는 사례다.

2.24 슬라이드 24 · ACT가 마주한 두 문제

첫 문제는 높은 제어 주기에서 반복되는 작은 오류다. 50 Hz라면 1초에 50번 행동을 고르므로 한 단계 예측이 조금만 흔들려도 접촉 작업이 망가질 수 있다. 둘째 문제는 사람 시연이 같은 작업을 서로 다른 경로로 수행한다는 다봉성이다.

선행 방법의 성공률 표는 긴 조작의 세부 단계가 병목이 될 수 있음을 보여 준다. 비교할 때는 과업·장치·시연 수를 맞추고 어느 단계에서 성공률이 떨어졌는지 읽는다.

50 Hz에서 60단계 행동열의 예측 지평은 1.2초다. 재관측과 재계획을 더 자주 하면 실제로 관측 없이 실행하는 구간은 이보다 짧아진다. 두 길이를 구분해야 반응 속도를 알 수 있다.

2.25 슬라이드 25 · 행동을 묶어 예측하는 이유

ACT는 한 장면에서 한 행동 대신 약 60개의 미래 행동을 예측한다. 그러면 손이 물체로 접근하고 잡는 동안 매 프레임 새 결정을 내려 모드가 흔들리는 문제를 줄일 수 있다. 기본 action chunking은 묶음을 실행한 뒤 다시 관측하지만, ACT의 temporal ensembling은 매 시점 새 chunk를 예측하고 이전 chunk와 겹치는 현재 행동을 합칠 수도 있다.

행동 묶음을 오래 열린 고리로 실행하면 교정 기회가 줄고, 짧게 실행하면 한 단계 정책의 흔들림이 되살아난다. **행동열의 예측 길이와 재관측 주기**는 따로 정하는 설계 변수다.

행동 묶음은 오류 누적 문제를 완전히 없애나? 아니다. 묶음 안의 예측이 틀리면 실행 중 바로 수정하지 못할 수 있다. 행동 묶음은 고주파의 작은 흔들림을 줄이지만, 긴 열린 고리 구간과 환경 변화에 대한 취약성은 별도로 관리해야 한다.

접촉이 거의 없는 팔 이동은 긴 묶음으로 부드럽게 이어 갈 수 있다. 플러그가 구멍 입구에 닿는 순간에는 예상 밖의 힘이 생기므로 짧은 실행 구간과 빠른 재관측이 유리하다. 과업의 접촉 변화가 두 주기를 정하는 근거가 된다.

ACT의 temporal ensembling에서는 서로 다른 시점에 예측한 chunk가 **현재 시점의 행동**을 여러 개 제안한다. 시간 가중 평균으로 이 제안들을 합쳐 chunk 경계의 급격한 명령 변화를 줄인다. 예측 지평 H, 재관측 간격, 겹친 예측의 가중치는 각각 조정할 수 있다. ALOHA · ACT 원논문

2.26 슬라이드 26 · 절대 관절 목표와 잠재변수

관절 각도의 작은 상대 변화량만 예측하면 한 번의 오차가 다음 목표에 더해져 드리프트가 커질 수 있다. 절대 목표 관절 위치를 출력하면 각 시점의 목표를 직접 지정해 이 누적을 줄일 수 있다. 제어기의 추종 오차는 남으므로 정확한 상태 측정과 안전 범위가 필요하다.

여러 유효 시연을 다루기 위해 ACT는 잠재변수 z를 둔 conditional VAE를 사용한다. 학습 때 시연 행동열 A와 관절 관측 $\bar{o}$를 인코딩한 $q_\phi(z \mid \bar{o}, A)$에서 z를 뽑고, 영상까지 포함한 관측 o와 함께 행동 묶음을 재구성한다. 인코더 분포가 표준정규 prior에서 너무 멀어지지 않도록 KL 항을 둔다. **원 논문**

의 평가 절차에서는 z를 prior 평균인 0으로 고정해 결정론적으로 디코딩한다. 잠재변수가 있다는 사실만으로 실행 때 여러 모드를 무작위로 샘플한다고 해석해서는 안 된다. ALOHA · ACT 원논문

여기서 ϕ는 시연 행동열을 잠재변수 z로 인코딩하는 분포 q_ϕ의 파라미터이고, θ는 관측 o와 z에서 행동열 $\widehat{A}_\theta(o, z)$를 내는 decoder의 파라미터다. $\mathcal{L}_{\text{rec}}$는 실제 행동열 A와 예측 행동열의 재구성 오차, D_{KL}은 두 확률분포의 차이를 재는 KL divergence다. $\mathcal{N}(0, I)$는 평균 0 · 단위 공분산의 정규분포이며, β는 KL 항의 가중치다. 이를 합친 학습 목표는

$$\mathcal{L}_{\text{ACT}} = E_{z\sim q_\phi(z|\bar{o},A)}[\mathcal{L}_{\text{rec}}(A, \widehat{A}_\theta(o, z))] + \beta D_{\text{KL}}(q_\phi(z \mid \bar{o}, A) \,\|\, \mathcal{N}(0, I)) \,.$$

첫 항은 시연 행동열 A와 예측 행동열 $\widehat{A}$를 맞춘다. 원 논문 본문은 정밀한 행동 예측을 위해 L1 재구성 손실을 사용했다고 설명한다. 둘째 항은 학습 때 본 행동열이 없는 실행 시점에도 쓸 수 있는 prior를 만든다. 실행 때 $z = 0$을 대입하는 선택과 학습 때 잠재변수로 시연 차이를 설명하려는 목적을 분리해 이해하자.

상대 명령의 드리프트를 수치로 보면 매 단계 0.01만큼 목표를 잘못 더하는 경우 50단계 뒤 누적 목표 오차는 0.5가 된다. 절대 목표는 매 단계 현재 시점의 목표를 새로 지정하므로 같은 방식으로 합산되지는 않는다. 다만 센서가 잘못된 관절 위치를 알려 주거나 하위 제어기가 목표를 못 따라가면 절대 목표도 실패할 수 있다.

잠재변수 z를 두면 실행 때 반드시 다양한 행동이 나올까? 아니다. 원 논문의 기본 평가에서는 $z = 0$으로 고정하므로 같은 관측에서의 출력은 결정론적이다. 다른 z를 사용하더라도 학습이 이를 무시하는 posterior collapse가 생겼다면 행동열이 비슷할 수 있다. 다양성을 주장하려면 실제 샘플별 행동열과 성공률을 평가해야 한다.

2.27 슬라이드 27 · ACT 구조를 입력과 출력으로 읽기

정책은 카메라 영상과 현재 관절 상태를 입력으로 받고, transformer가 이 정보를 결합해 미래의 관절 목표열을 출력한다. 영상은 물체와 공간의 배치를, 관절 상태는 현재 팔의 자세를 알려 준다. 둘 중 하나가 빠지면 "물체는 어디 있나" 또는 "팔은 지금 어디 있나"를 추측해야 한다.

출력은 한 숫자가 아니라 시간축이 있는 행동 행렬이다. 학습 손실을 읽을 때 어느 시간의 목표와 예측을 비교하는지, 패딩한 끝부분은 제외되는지, 양팔의 행동 차원이 어떻게 배열되는지 확인해야 한다. 이것이 구현에서 자주 생기는 조용한 오류다.

행동 텐서를 $B \times H \times d_a$로 쓰면 B는 batch 크기, H는 미래 길이, d_a는 한 시점의 행동 차원이다. 예를 들어 양팔이 각각 7개 관절 목표와 그리퍼 1개를 내면 $d_a = 16$이다. 행동 묶음 60개를 예측하면 한 예측의 출력은 60×16이다. 이 차원을 시연 기록과 비교하면 시간축과 관절축을 바꾸는 실수를 잡기 쉽다.

2.28 슬라이드 28 · 50개 시연으로 만든 조작 정책

ACT를 50개 시연으로 학습해 컵을 다루는 작업에서 성공률 84%를 보고한 시연 영상이다. 수치는 이 슬라이드의 실험 조건에서 읽어야 한다.

2.29 슬라이드 29 · 배터리 삽입의 정밀도

같은 연구의 배터리 삽입 영상이다. 50개 시연으로 성공률 96%를 보고하며, 정밀한 양팔 동작의 사례를 보여 준다.

2.30 슬라이드 30 · Mobile ALOHA로 확장하기

Mobile ALOHA는 양팔 조작에 이동 기능을 더한 시스템이다. 작업 공간까지 이동하는 행동과 접촉 조작이 한 궤적에 이어진다.

2.31 슬라이드 31 · 원격 조작 시연의 역할

이동형 로봇을 사람이 원격 조작해 시연을 수집하는 장면이다. 로봇이 학습할 입력과 실제 제어 명령을 함께 기록한다.

2.32 슬라이드 32 · 자율 실행의 평가

학습된 Mobile ALOHA 정책이 주방에서 음식을 조리하는 자율 실행 사례다. 이동과 양팔 조작을 함께 요구한다.

2.33 슬라이드 33 · Diffusion Policy로 넘어가기

Diffusion Policy는 한 상태에서 여러 행동열이 가능할 때 조건부 확산 모델로 그 분포를 표현한다. 뒤 슬라이드에서 생성 과정을 구체화한다.

2.34 슬라이드 34 · 세 실패 원인을 다시 연결하기

앞서 본 세 난점 중 확산 정책이 주로 겨냥하는 것은 **다봉 행동 분포**다. 행동열을 함께 생성해 시간적 일관성도 높인다. 실행 중 상태 분포 이동이나 관측 정보 부족이 자동으로 사라지는 것은 아니다.

2.35 슬라이드 35 · 노이즈에서 미래 행동열로

현재 관측 o_t에서 길이 H의 미래 행동열 $A_t = (a_t, \dots, a_{t+H-1})$을 생성한다. 시연의 깨끗한 행동열을 $A^{(0)}$이라 쓰고, $A^{(k)}$는 k번째 노이즈 수준으로 오염한 열이라 하자. K는 전체 노이즈 단계 수다. 수준마다 $\beta_k \in (0,1)$를 정하고 $\alpha_k = 1 - \beta_k$, $\bar{\alpha}_k = \prod_{j=1}^{k} \alpha_j$로 누적 신호 비율을 정의한다. I는 행동열 차원의 항등행렬이며, 표준정규 잡음 $\epsilon \sim \mathcal{N}(0, I)$를 뽑으면 정방향 오염 표본을 한 번에 만들 수 있다.

$$A^{(k)} = \sqrt{\bar{\alpha}_k}\, A^{(0)} + \sqrt{1 - \bar{\alpha}_k}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I).$$

k가 커질수록 원래 행동열의 비중 $\sqrt{\bar{\alpha}_k}$는 줄고 잡음의 비중은 커진다. Schedule을 $\bar{\alpha}_K \approx 0$이 되게 정하면 마지막 $A^{(K)}$의 분포는 표준정규에 가깝다. 이 조건이 **생성 때 $A^{(K)} \sim \mathcal{N}(0, I)$에서 시작하는 이유**다.

학습에서는 시연 데이터 $\mathcal{D}$에서 $(o_t, A^{(0)})$을 뽑고, k를 $\{1, \dots, K\}$에서 균등하게 뽑은 뒤 표준정규 잡음 ϵ을 샘플링해 위 식으로 $A^{(k)}$를 만든다. $\epsilon_\theta(A^{(k)}, o_t, k)$는 파라미터 θ인 신경망이 예측한 잡음이다. 아래 $E_{\mathcal{D},k,\epsilon}$는 이 세 번의 샘플링에 대한 평균이고, $\|\cdot\|_2^2$는 행동열의 각 성분 오차 제곱을 더한다. 학습 목표는

$$\mathcal{L}(\theta) = E_{\mathcal{D},k,\epsilon}\left[\left\|\epsilon - \epsilon_\theta(A^{(k)}, o_t, k)\right\|_2^2\right].$$

왜 노이즈를 맞히는지 조건부 기댓값으로 이해할 수 있다. 오염된 열 $x = A^{(k)}$와 관측 o_t, 수준 k를 고정하고 예측값을 f라 하면 $E[\|\epsilon - f\|_2^2 \mid x, o_t, k]$의 최소점은 $f^* = E[\epsilon \mid x, o_t, k]$다. 이는 제곱을 전개해 f로 미분하면 $2\{f - E[\epsilon \mid x, o_t, k]\} = 0$이 되기 때문이다. 예측 노이즈가 주어지면 위 정방향

식을 $A^{(0)}$에 대해 풀어 $\widehat{A}^{(0)} = (A^{(k)} - \sqrt{1-\bar{\alpha}_k}\,\epsilon_\theta)/\sqrt{\bar{\alpha}_k}$라는 깨끗한 열의 추정치를 얻는다. 실제 역방향 생성은 이 추정치를 사용해 k를 한 단계씩 낮춘다. Diffusion Policy 연구

전체 행동열을 함께 생성하므로 시간상 맞지 않는 움직임을 줄일 수 있고, 서로 다른 시연 모드도 확률분포로 유지할 수 있다. 여러 정제 단계의 계산 지연은 제어 주기와 함께 고려해야 한다.

실행에서는 $A^{(K)}$에서 시작해 k를 줄이며 예측 노이즈를 제거한다. 생성한 열의 첫 몇 행동을 실행하고 다시 관측해 새 열을 만들면 **receding horizon** 제어가 된다. 따라서 행동열의 전체 길이 H와 재관측까지 실제로 실행하는 행동 수는 서로 다를 수 있다.

무작위 노이즈에서 시작하는데 왜 행동이 무작위로 끝나지 않을까? 학습된 모델이 관측과 노이즈 수준에 맞춰 매 단계 행동열을 시연 분포 쪽으로 정제하기 때문이다. 시작 노이즈는 여러 가능한 경로를 샘플링하는 재료이며, 학습된 정제 규칙이 없다면 유효한 행동으로 바뀌지 않는다.

2.36 슬라이드 36 · 결과 영상의 해석

집기·붓기·밀기 과제의 실행 영상이다. 확산 정책이 생성한 행동열을 일부 실행하고 다시 관측해 다음 열을 만드는 방식의 결과를 보여 준다.

2.37 슬라이드 37 · 로봇 파운데이션 모델의 조건부 정책

로봇 파운데이션 모델의 정책은 관측 s, 과업 지시 z를 조건으로 행동 a를 고른다. $\pi(a \mid s, z)$라는 표기는 같은 장면에서도 "접기"와 "치우기"가 다른 행동을 요구함을 나타낸다.

2.38 슬라이드 38 · DROID와 환경 다양성

DROID는 다양한 실제 공간에서 수집한 로봇 조작 데이터셋이다. 같은 동작도 배경과 물체 배치가 달라질 수 있음을 보여 준다. DROID 연구

2.39 슬라이드 39 · Open X-Embodiment와 로봇 몸체의 차이

Open X-Embodiment는 서로 다른 로봇의 시연 데이터와 모델을 묶은 사례다. 로봇마다 관절 수와 행동 단위가 다르므로 공동 학습에는 행동 표현을 맞추는 과정이 필요하다. OXE 연구

2.40 슬라이드 40 · π_0의 자율 수행 장면

영상은 π_0 정책의 자율 로봇 조작 장면이다. 시각·언어 입력을 행동으로 연결하는 사례로, 학습 구조는 뒤 슬라이드에서 다룬다.

2.41 슬라이드 41 · 사전학습과 과업별 후속 학습

π_0는 다양한 로봇 시연으로 사전학습한 뒤, 특정 과업의 추가 시연으로 조정한다. 슬라이드의 10,000시간과 과업별 5–100시간은 각각 보고된 학습 데이터 규모다.

2.42 슬라이드 42 · VLM과 행동 전문가의 연결

그림은 시각·언어 정보를 처리하는 VLM과 행동열을 생성하는 action expert의 연결을 보여 준다. π_0는 행동 생성을 flow matching으로 학습한다. 모델 규모와 계산 자원 수치는 해당 연구의 설정이다. 원 논문

2.43 슬라이드 43 · 로봇 데이터와 텍스트 데이터의 규모 비교

로봇 시연의 시간과 언어모델의 텍스트 토큰 규모를 비교한 그림이다. 둘은 동일한 정보 단위가 아니므로 정확한 양적 비교보다 로봇 데이터 수집의 어려움을 보여 주는 직관으로 읽는다.

2.44 슬라이드 44 · 데이터 수집 규모의 현실

여러 대의 로봇과 작업자가 시연을 모으는 장면이다. 로봇 데이터에는 조작뿐 아니라 물체 재배치, 장비 관리, 실패 복구 비용이 든다.

2.45 슬라이드 45 · 가정 환경의 긴 작업

가정 환경에서 식탁을 정리하는 긴 작업의 예시다. 한 번의 집기와 달리 물체를 다루는 순서와 중간 상태가 다음 행동을 바꾼다.

2.46 슬라이드 46 · ACT-1이라는 이름을 해석할 때

Sunday Robotics의 ACT-1 사례다. 앞서 배운 Action Chunking with Transformers(ACT)와 이름이 비슷하지만 별개의 모델이다. 영상은 긴 작업과 손재주를 목표로 하는 응용을 보여 준다.

2.47 슬라이드 47 · 다양성을 시각화한 데이터 지도

서로 다른 로봇 행동 데이터를 2차원에 투영한 지도다. 데이터의 다양성을 시각화하지만, 그림의 두 점 사이 거리만으로 실제 행동 궤적의 거리를 판단할 수는 없다.

2.48 슬라이드 48 · 새로운 로봇 플랫폼에서의 조작

다른 로봇 몸체로 옮겨 조작하는 영상이다. 몸체가 달라지면 같은 행동 숫자도 다른 움직임을 뜻할 수 있어 행동 표현과 변환 규칙이 중요하다.

2.49 슬라이드 49 · 모방학습의 도달 범위

모방학습은 시연이 있을 때 강한 출발점이다. 그러나 전문 시연을 얻기 어렵거나, 시연 밖 상태에서 회복해야 하거나, 결과를 기준으로 더 좋은 전략을 찾아야 할 때는 보상 기반 학습이 필요하다. BC 자체는 결과 보상을 직접 최적화하지 않는다.

2.50 슬라이드 50 · 보상으로 배우는 단계

다음 장에서는 전문가의 행동 정답 대신 환경이 돌려주는 보상을 학습 신호로 삼는다.

3 02-1. 강화학습의 수학적 기초

모방학습은 전문가 행동이라는 목표값에서 출발했다. 이제 목표값 없이 환경의 결과를 평가할 수 있는 문제를 정의한다. **Markov decision process(MDP)**는 상태, 행동, 전이, 보상과 시간에 대한 선호를 하나의 공통 언어로 묶는다. 이후 정책경사와 Q-learning은 모두 같은 MDP를 다른 방식으로 푼다.

3.1 슬라이드 01 · 강화학습의 수학적 기초

강화학습 문제를 상태, 행동, 전이, 보상으로 형식화하는 장이다.

3.2 슬라이드 02 · 복습에서 형식화로

앞 강의의 모방학습을 간단히 복습한 뒤, 강화학습을 수학적 문제로 정의한다.

3.3 슬라이드 03 · 평가 요소와 개념 학습

과제와 평가 방식에 관한 운영 안내다. 최신 기준은 강좌 공지에서 확인한다.

3.4 슬라이드 04 · 시험에서 MDP를 설명하는 법

시험 형식과 시간에 관한 안내다. 세부 사항은 강좌 공지를 기준으로 확인한다.

3.5 슬라이드 05 · 출석과 연속적인 학습

출석 방식과 예외 처리에 관한 운영 안내다.

3.6 슬라이드 06 · 질문으로 확인할 수 있는 가정

수업 중 질문과 토론을 통한 참여 방식의 안내다.

3.7 슬라이드 07 · 자료를 사용할 때의 순서

강의 자료와 질의응답에 사용하는 공식 채널을 안내한다.

3.8 슬라이드 08 · 교재와 현대 알고리즘 사이

기본 개념은 Sutton·Barto의 교재, 딥러닝 기반 알고리즘은 관련 강좌 자료를 함께 참고할 수 있다.

3.9 슬라이드 09 · 모방학습을 되짚는 이유

행동 복제는 전문가의 상태–행동 쌍을 따른다. 강화학습에서는 행동의 결과인 보상을 학습 신호로 쓴다.

3.10 슬라이드 10 · 결정 문제의 최소 구성

정책은 현재 상태에서 행동을 고른다. 순차적 문제에서는 선택한 행동이 다음 상태를 바꾸므로, 한 시점의 예측 정확도만으로 정책을 평가할 수 없다.

3.11 슬라이드 11 · 행동 복제의 기준선

BC는 전문가 시연에서 정책이 전문가 행동에 주는 확률을 높인다. 이 장에서는 시연이 없는 경우의 학습 신호인 보상으로 넘어간다.

3.12 슬라이드 12 · 모방학습의 세 실패

BC의 세 난점은 오류 누적, 다봉 행동 분포, 관측 정보 차이다. 강화학습으로 넘어가도 상태와 관측을 올바르게 정해야 한다는 요구는 남는다.

3.13 슬라이드 13 · DAgger의 상태 분포 수정

DAgger는 학습 정책을 실행해 방문한 상태에서 전문가 행동을 받아 데이터에 추가한다. 전문가 상태와 정책 상태의 분포 차이를 줄이는 방법이다.

3.14 슬라이드 14 · 사람의 개입과 보상의 차이

사람이 오류 직전에 개입해 교정 시연을 제공하는 방식이다. 여기서 받은 신호는 여전히 **전문가 행동**이며, 결과의 좋고 나쁨을 나타내는 보상과는 다르다.

3.15 슬라이드 15 · RL 문제로 넘어가는 연결

이제 시연 데이터 없이 행동의 결과를 평가하는 강화학습 문제를 정의한다.

3.16 슬라이드 16 · 누적 보상의 의미

시점 t부터 길이 T의 episode가 끝날 때까지 받는 보상을 $r_t, r_{t+1}, \dots, r_{T-1}$이라 하자. 할인율 $\gamma \in [0,1]$는 한 단계 뒤 보상의 가중치를 정한다. **반환** $G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k}$는 현재와 미래 보상을 합친 정책의 평가량이다.

예를 들어 게임에서 당장 점수를 얻는 길이 곧 패배로 이어질 수 있다. 누적 보상이 목표라면 그 길을 피해야 한다. γ는 뒤 보상을 얼마나 반영할지 정하지만, 보상의 단위와 episode 길이도 결과에 영향을 준다.

수치 예를 들어 첫 단계에 3점, 다음 단계에 4점, 마지막에 5점을 받고 $\gamma = 0.5$라면 $G_0 = 3 + 0.5 \cdot 4 + 0.5^2 \cdot 5 = 6.25$다. 같은 세 보상도 $\gamma = 1$이면 12점이 된다. 할인율을 바꾸면 단순히 수식 모양만 바뀌는 것이 아니라 어느 시간의 결과를 정책이 더 중시하는지가 달라진다.

보상을 매 단계 2배로 하면 최적 정책이 바뀔까? 모든 보상을 같은 양의 양수로 곱하고 다른 제약이 없다면 각 정책의 반환도 같은 비율로 바뀌므로 최적 정책의 순서는 유지된다. 하지만 일부 보상 항만 키우거나 학습 알고리즘의 clip·정규화 설정을 그대로 두면 실제 학습 행동은 달라질 수 있다.

3.17 슬라이드 17 · RL 문제를 확률로 쓰기

길이 T의 궤적을 $\tau = (s_0, a_0, s_1, a_1, \dots, s_T)$라 하자. $p_0(s_0)$는 초기 상태 확률, $\pi(a_t \mid s_t)$는 정책이 시점 t의 상태에서 행동을 뽑을 확률, $P(s_{t+1} \mid s_t, a_t)$는 환경의 다음 상태 전이 확률이다. 이 조건부 확률들을 곱하면 정책 π가 궤적 τ를 만들 확률 $p_\pi(\tau)$가 된다.

$$p_\pi(\tau) = p_0(s_0) \prod_{t=0}^{T-1} \pi(a_t \mid s_t) P(s_{t+1} \mid s_t, a_t).$$

보상도 확률적이면 $p(r_t \mid s_t, a_t)$ 항을 곱해 넣을 수 있다. 목표는 정책이 만드는 궤적에서 반환의 기대값 $J(\pi) = E_{\tau \sim p_\pi}[G_0]$을 높이는 것이다. 뒤의 정책경사는 이 식을 θ에 대해 미분한다.

곱의 각 항은 한 단계의 조건부 확률에서 나온다. 먼저 $p_0(s_0)$로 시작 상태를 뽑고, 그 상태에서 $\pi(a_0 \mid s_0)$로 행동을 뽑고, $P(s_1 \mid s_0, a_0)$로 다음 상태를 뽑는다. 같은 과정을 T번 반복하면 chain rule에 따라 전체 궤적의 확률이 곱으로 표현된다. **전이가 Markov이고 정책이 현재 상태에만 의존한다**는 조건을 사용했기에 더 오래된 과거는 각 항에 나타나지 않는다.

결정론적 정책에서는 행동 확률이 사라질까? 이산 행동에서는 선택된 행동에 확률 1, 나머지에 0을 주는 특수한 정책으로 볼 수 있다. 연속 행동에서는 Dirac delta로 나타내거나 행동을 상태의 함수로 대입해 궤적의 확률을 쓴다. 정책경사의 로그확률 식을 그대로 쓰려면 보통 확률적 정책이 편리하다.

3.18 슬라이드 18 · MDP가 환경과 과업을 함께 담는 이유

MDP는 전이 P와 보상 r을 함께 포함해 **환경의 움직임과 과업의 목적**을 정의한다. 같은 로봇·방에서 속도를 보상하면 빨리 달리는 정책이, 에너지 절약을 보상하면 천천히 움직이는 정책이 유리할 수 있다. 전이는 같아도 최적 정책이 달라진다.

에이전트가 마찰과 접촉의 확률표를 알 필요는 없다. 실제 전이 (s, a, r, s')를 모아 정책이나 가치를 학습할 수 있다. 이때 보상은 사람이 원한 목표를 수치로 표현하므로, 성공·안전 지표와의 관계를 별도로 살핀다.

3.19 슬라이드 19 · 확률 과정의 기초

확률 과정은 시간에 따라 정렬된 확률변수의 모음이다. 강화학습에서는 상태 $S_0, S_1, \dots$가 전이의 확률성에 따라 달라진다.

3.20 슬라이드 20 · Markov 성질의 정확한 뜻

Markov 성질은 현재 상태를 알면 과거가 다음 상태의 분포에 추가 정보를 주지 않는다는 조건이다. 시간 t의 확률적 상태를 S_t라 쓰면

$$P(S_{t+1} \mid S_0, \dots, S_t) = P(S_{t+1} \mid S_t).$$

예를 들어 공의 위치만으로는 다음 위치를 예측하기 어렵지만, 속도까지 상태에 넣으면 같은 위치에서 반대로 움직이는 두 경우를 구분할 수 있다. Markov 성질은 무엇을 상태에 포함하느냐에 달려 있다.

3.21 슬라이드 21 · Markov chain의 두 요소

Markov 과정 또는 chain은 상태 집합 S와 전이 확률 $P(s' \mid s)$로 나타낼 수 있다. $P(s' \mid s)$는 현재 s에서 다음에 s'가 될 확률이며, 각 s에 대해 $\sum_{s'} P(s' \mid s) = 1$이어야 한다.

상태와 전이만 있으므로 아직 선택할 행동이 없다. 어떤 상태가 "좋다"는 보상도 없다. 이 단계에서 계산할 수 있는 것은 특정 상태에 도달할 확률이나 오래 머무를 확률 같은 확률적 성질이다.

두 상태 A, B의 예에서 $P(A \mid A) = 0.8$, $P(B \mid A) = 0.2$, $P(A \mid B) = 0.4$, $P(B \mid B) = 0.6$이라고 하자. 현재 A라면 두 단계 뒤 다시 A일 확률은 $0.8 \cdot 0.8 + 0.2 \cdot 0.4 = 0.72$다. 중간 상태 A와 B라는 두 경로의 확률을 더한 것이다. 이 계산이 전이 행렬 제곱 P^2의 한 원소와 같다.

3.22 슬라이드 22 · 공부와 과제의 Markov chain

공부, 숙제, 제출, 영상 시청을 상태로 생각하자. 화살표에 적힌 수는 현재 상태에서 다음 상태로 옮길 확률이다. 공부에서 숙제로 갈 수도, 영상을 보기 시작할 수도 있다. 한 번 영상 상태에 들어간 뒤 다시 공부로 돌아올 수도 있으므로 경로는 매번 다르다.

그림의 세 sample episode는 같은 전이 규칙에서 나온 서로 다른 실현이다. 화살표 하나는 다음 한 단계의 확률이고, 여러 단계 뒤 제출에 닿을 확률은 가능한 경로들의 확률을 더해 구한다. 제출을 흡수 상태로 두면 그 뒤 계속 제출에 머무르며, episode 종료로 두면 그 지점에서 경로를 멈춘다.

3.23 슬라이드 23 · Markov Reward Process

Markov reward process(MRP)는 Markov chain에 보상 함수 $r(s)$와 할인율 γ를 더한다. 대문자 S_t는 시점 t의 확률적인 상태, 소문자 s는 조건으로 고정한 상태다. $E[\cdot \mid S_t = s]$는 그 상태에서 출발한 전이 경로들의 평균을 뜻한다. 이제 상태에서 시작해 얻는 할인 반환 $G_t = \sum_{k=0}^{\infty} \gamma^k r(S_{t+k})$의 기댓값을 가치 $V(s)$로 정의한다.

$$V(s) = E\left[\sum_{k=0}^{\infty} \gamma^k r(S_{t+k}) \mid S_t = s\right].$$

$0 \le \gamma < 1$이면 보상이 유계인 계속 과업에서 합이 잘 정의된다. 유한 episode는 $\gamma = 1$을 써도 반환이 유한할 수 있다. γ는 미래 보상 선호뿐 아니라 무한 합을 안정적으로 다루는 역할도 한다.

가치함수의 재귀식을 직접 만들 수 있다. 반환을 첫 보상과 나머지로 분리하면 $G_t = r(S_t) + \gamma G_{t+1}$이다. $S_t = s$를 조건으로 기대값을 취하고 다음 상태 s'에 대한 확률을 풀어 쓰면

$$V(s) = r(s) + \gamma \sum_{s'} P(s' \mid s) V(s').$$

이것이 MRP의 Bellman equation이다. 지금 받는 보상과 다음 상태의 기대 가치를 더한 식이며, 다음 상태의 가치를 안다면 현재 상태의 가치를 계산할 수 있다는 직관과 연결된다.

상태가 유한하면 각 $V(s)$를 가치 벡터 v, 각 상태의 즉시 보상을 벡터 r, 상태 간 전이확률을 행렬 P에 모을 수 있다. I는 상태 수만큼의 항등행렬이다. 앞 식은 $v = r + \gamma P v$이므로

$$(I - \gamma P)v = r, \qquad v = (I - \gamma P)^{-1} r = \sum_{k=0}^{\infty} (\gamma P)^k r.$$

P의 각 행 합은 1이고 $\gamma < 1$이면 γP의 거듭제곱이 작아져 마지막 등식의 기하급수가 수렴한다. $P^k r$는 k단계 뒤의 보상 기대값이므로 행렬식은 처음의 반환 정의와 정확히 같은 내용을 담는다. 예를 들어 s_1에서 항상 s_2로 가고, s_2는 매 단계 보상 1을 주며 그곳에 머문다면 $\gamma = 0.9$에서 $V(s_2) = 1 + 0.9V(s_2) = 10$, $V(s_1) = 0 + 0.9V(s_2) = 9$다. 종료 상태에서 한 번만 1을 주는 과업과는 다른 설정이다.

3.24 슬라이드 24 · 즉시 비용과 최종 보상

공부와 과제 진행에는 작은 시간 비용, 제출에는 큰 보상이 있다고 하자. 보상을 받기 위해서는 중간의 음의 보상을 감수해야 한다. 예를 들어 지금 공부에 -1, 다음 과제에 -1, 그 뒤 제출에 100을 받고 $\gamma = 0.9$라면 반환은 $-1 + 0.9(-1) + 0.9^2(100) = 79.1$이다.

반면 영상 시청에서 계속 -0.1만 받고 끝나지 않는다면 할인 반환은 $-0.1/(1 - 0.9) = -1$이다. 이 간단한 계산은 즉시 보상이 더 큰 행동이 장기적으로 더 좋다는 뜻이 아님을 보여 준다. 다만 그림 자체의 확률을 사용한 **기대값**은 가능한 모든 경로를 전이 확률로 가중해 계산해야 한다.

앞 계산에서 제출 보상 100점은 두 단계 뒤에 오므로 0.9^2를 곱했다. 만약 제출로 갈 확률이 0.5이고 실패하면 보상을 받지 못한다면, 100점 항의 기대 기여는 $0.5 \cdot 0.9^2 \cdot 100 = 40.5$다. 보상을 받은 한 경로의 반환과 **그 경로가 일어날 확률까지 반영한 기대 가치**를 구분해야 한다.

3.25 슬라이드 25 · 선택을 더한 MDP

MDP는 S, A, P, r, γ의 묶음이다. 행동 a를 고른 뒤 전이 $P(s' \mid s, a)$가 다음 상태를 만들고 보상 $r(s, a)$ 또는 더 일반적으로 $r(s, a, s')$가 주어진다. Markov chain에서는 전이가 상태만의 함수였지만 MDP에서는 행동이 전이를 바꾼다.

어떤 책은 보상을 확률변수로 두어 $P(s', r \mid s, a)$로 합쳐 쓴다. 표기가 달라도 현재 상태와 행동을 조건으로 다음 상태와 보상의 분포가 정해진다는 뜻은 같다. 목표는 선택 가능한 정책 가운데 높은 기대 반환을 얻는 것이다.

아주 작은 MDP에서 같은 상태 s에 "공부"와 "영상 보기" 두 행동이 있다고 하자. 공부를 고르면 제출 상태로 갈 확률이 0.8, 영상을 고르면 0.1이라면 두 행동은 현재 상태가 같아도 미래 분포를 다르게 만든다. 행동이 없는 Markov chain은 이 차이를 표현할 수 없고, 행동별 전이 $P(s' \mid s, a)$가 필요한 이유다.

3.26 슬라이드 26 · 공부 예시에 행동을 넣기

MDP에서는 "숙제 중"처럼 현재 관측된 조건을 상태로, "제출한다"처럼 지금 선택할 수 있는 것을 행동으로 둔다. 공부 중에 영상을 고르면 영상 상태로, 과제를 시작하면 숙제 상태로 갈 확률이 달라진다. 정책은 상태별 행동 확률을, 보상은 선택 뒤 결과의 가치를 정한다.

같은 "공부"라는 단어도 문제 설정에 따라 상태나 행동이 될 수 있다. 이미 책을 읽고 있는 상황은 상태이고, 지금 읽기를 시작하는 결정은 행동이다. **선택 가능성과 시간 순서**로 구분한다.

3.27 슬라이드 27 · 환경을 정의하는 식과 학습 목표

MDP는 정책이 주어졌을 때 어떤 궤적이 나올지 확률적으로 정의한다. 학습자는 전이와 보상의 결과를 관측해 $J(\pi) = E_{\tau \sim p_\pi}[\sum_t \gamma^t r_t]$를 크게 만드는 정책을 찾는다.

과제 제출 버튼에만 $+10$을 주고 내용 품질은 평가하지 않으면, 에이전트는 빈 문서를 즉시 제출해 $+10$을 얻는다. 수학적 최적화는 주어진 보상에 충실하다. 따라서 과업 목표가 "잘 작성한 과제"라면 품질을 측정하는 지표나 제약도 설계에 넣어야 한다.

3.28 슬라이드 28 · 정책이 정의하는 행동

Markov 정책은 현재 상태 s에서 행동 a를 고를 확률 $\pi(a \mid s)$를 정한다. 각 상태에서 $\sum_a \pi(a \mid s) = 1$이다. 환경 전이가 $P(s' \mid s, a)$라면, 정책을 고정했을 때의 상태 전이는 전체확률법칙으로

$$P_\pi(s' \mid s) = \sum_a \pi(a \mid s) P(s' \mid s, a)$$

가 된다. 즉 행동을 평균내면 정책 아래의 상태 흐름이 하나의 Markov chain이 된다. 정책을 바꾸면 환경의 P는 그대로여도 P_π는 달라진다. 보상 $r(s,a)$도 $r_\pi(s) = \sum_a \pi(a \mid s) r(s,a)$로 평균내면 정책 가치의 Bellman 식 $V^\pi = r_\pi + \gamma P_\pi V^\pi$를 얻는다. 이 관계가 정책 평가의 출발점이다.

3.29 슬라이드 29 · 초기 상태 분포까지 포함하기

MDP의 정의에 $p_0(s_0)$를 넣으면 episode가 어디서 시작할지 명시된다. 출발 위치가 늘 같을 때와 무작위일 때는 같은 정책의 평균 성과가 달라진다. 환경 전이와 보상이 같아도 초기 분포가 바뀌면 학습 목표 $E_{s_0 \sim p_0}[V^\pi(s_0)]$가 달라진다.

실험에서 초기 상태를 무작위화하면 더 넓은 상태를 연습할 수 있다. 반면 평가할 때만 쉬운 시작 상태를 쓰면 성능을 과대평가한다. 학습과 평가의 초기 분포를 따로 기록해야 한다.

초기 상태가 쉬운 s_A와 어려운 s_B로 절반씩 나오고 정책의 성공 확률이 각각 1과 0이라면 평균 성공률은 0.5다. 같은 정책을 s_A에서만 평가하면 1로 보인다. 정책이나 환경 전이를 고치지 않아도 시작 분포만으로 수치가 크게 바뀐다. 따라서 논문의 성과를 비교할 때 초기화 규칙을 확인해야 한다.

3.30 슬라이드 30 · 시작 상태가 여러 개일 때

그림의 "여기서 시작" 표시는 과제를 시작하는 위치가 하나가 아님을 보여 준다. 같은 정책도 바로 숙제 상태에서 시작하면 제출까지 가는 길이 짧고, 영상 상태에서 시작하면 공부로 돌아오는 데 시간이 걸릴 수 있다.

초기 상태 분포는 단순한 구현 세부 사항이 아니다. 어떤 상황에서 좋은 정책을 원하느냐를 정의한다. p_0가 거의 방문하지 않는 어려운 상태에서의 정책 능력은 평균 반환에 작게 반영될 수 있으므로, 별도의 스트레스 평가가 필요하다.

평균 반환이 높은 정책이 어려운 출발점에서 약할 수 있다. 예컨대 시작의 95%가 쉬운 상태라면 드문 5%의 실패는 평균에서 작게 보인다. 서비스나 로봇의 실제 안전이 그 5%에 달려 있다면 시작 조건별 성공률을 별도로 보고해야 한다. 평균 성능과 최악 조건 성능은 다른 질문이다.

3.31 슬라이드 31 · 상태와 행동은 과업마다 다르다

보드게임의 상태는 바둑판의 돌 배치이고 행동은 다음 수의 위치다. 로봇의 상태는 관절 각도·속도·카메라·촉각 등이고 행동은 관절 토크나 그리퍼 명령이다. 같은 수학적 MDP라도 변수의 의미와 크기가 크게 다르다.

영상 한 장에는 가려진 물체, 접촉력, 속도가 빠질 수 있다. 바둑판에서도 돌의 배치 외에 현재 합법 수를 결정하는 규칙의 이력이 필요할 수 있다. 상태 표현은 입력의 크기보다 **다음 전이와 보상을 예측하는 데 필요한 정보를 담았는가**로 판단한다.

3.32 슬라이드 32 · 환경을 완전히 정의해도 모를 수 있다

수학적으로 MDP를 쓴다는 것은 전이와 보상이 **존재하고 일정한 규칙을 따른다**는 모델링이다. 에이전트가 그 확률표나 물리 방정식을 알고 있다는 뜻은 아니다. 실제 로봇의 접촉을 정확히 계산하지 못해도 행동을 실행해 표본 (s_t, a_t, r_t, s_{t+1})을 관측할 수 있다.

카드 게임의 전이 확률표를 몰라도 여러 번 플레이해 (s, a, r, s') 표본을 모을 수 있다. 모델 기반 방법은 이 표본으로 결과를 예측하는 모델을 만들고 계획한다. 모델 프리 방법은 전이 모델을 명시적으로 만들지 않고 정책이나 가치를 갱신한다. 두 접근 모두 자료의 범위와 품질에 영향을 받는다.

3.33 슬라이드 33 · 실제 던지기의 MDP

바나나를 바구니에 넣는 로봇을 생각하자. 상태는 로봇과 물체의 위치·속도, 행동은 관절 제어, 다음 상태는 실제 물리, 보상은 바구니에 들어갔는지의 신호일 수 있다. 성공 보상이 마지막에만 주어지면 어느 관절 동작이 성공에 기여했는지 바로 알기 어렵다.

물리 전이가 알려지지 않은 상태에서 여러 번 던져 얻은 표본으로 정책을 개선할 수 있다. 그러나 실패한 던지기의 비용과 안전 문제가 있으므로 실제 로봇에서는 시뮬레이션, 시연, 안전 제약과 병행하는 경우가 많다.

던지기 직전의 각도와 속도가 조금만 달라도 착지 위치는 달라진다. 성공 보상이 마지막에만 1이면 실패 궤적 대부분의 보상은 0이다. 거리 기반 중간 보상을 넣으면 신호가 풍부해지지만, 바구니 옆을 오래 맴도는 행동이 높은 점수를 받을 수도 있다. **보상 밀도와 목표 일치** 사이의 절충을 이해하기 좋은 사례다.

3.34 슬라이드 34 · 환경 API와 수학 기호의 대응

환경을 초기화하면 s_0 또는 초기 관측을 받고, 한 단계 진행하면 s_{t+1}, r_t, 종료 신호와 추가 정보를 받는다. 에이전트는 명령을 보낸 뒤 내부 물리 계산의 모든 세부를 몰라도 다음 전이 표본을 얻을 수 있다.

코드에서 중요한 것은 반환 형식과 종료 처리다. Gym 계열 API의 버전에 따라 초기화가 관측과 정보의 쌍을 주고, 진행 결과가 종료와 시간 제한을 따로 줄 수 있다. 시간 제한으로 잘린 episode를 진짜 종료와 혼동하면 다음 상태 가치의 부트스트랩 처리가 틀릴 수 있다.

환경의 전이를 모르는데 Bellman 방정식을 사용할 수 있나? 가능하다. Bellman 방정식은 기대값의 관계를 정의하고, 구현에서는 환경에서 얻은 표본으로 그 기대값을 근사한다. 전이 확률표를 직접 알아야 하는 동적 계획법과 표본 기반 TD 학습을 구분하면 된다.

구체적으로 `step(a_t)`가 `(o_{t+1}, r_t, terminated, truncated, info)`를 반환한다고 생각하자. `terminated`는 과제 자체의 종료이고 `truncated`는 시간 제한 같은 외부 중단일 수 있다. 가치 학습에서 진짜 종료 뒤의 미래 가치는 0으로 두지만, 시간만 잘린 경우에는 $V(o_{t+1})$를 이어서 추정해야 할 수 있다. 이 차이가 잘못 처리되면 모든 제한 시점의 가치를 과소평가할 수 있다.

3.35 슬라이드 35 · 완전 관측의 한계

MDP의 상태는 미래 예측에 필요한 정보를 충분히 담아야 한다. 실제 세계에서는 카메라가 가린 물체, 센서가 측정하지 않은 힘, 사람의 숨은 의도 때문에 관측만으로 전체 상태를 알기 어렵다. 이런 상황은 부분 관측 문제로 보는 편이 정확하다.

정책은 관측 이력 $o_{0:t}$을 사용하거나, 숨은 상태에 대한 믿음 분포 $b_t(s) = P(s_t = s \mid o_{0:t}, a_{0:t-1})$를 유지할 수 있다. 나중에 POMDP를 배우면 이 개념이 정식화된다. 현재는 관측 o_t와 충분한 상태 s_t를 무심코 같게 두지 않는 것이 중요하다.

컵이 불투명한 상자 뒤에 있을 때 카메라 영상 하나만으로 정확한 위치를 알 수 없더라도, 직전에 컵이 움직인 방향을 기억하면 추정할 수 있다. 이처럼 과거 이력을 요약한 내부 상태를 두는 정책은 한 장면만 쓰는 정책보다 정보가 많다. 단, 기억을 추가해도 센서가 전혀 제공하지 않는 정보는 추정의 불확실성이 남는다.

믿음 상태의 갱신은 두 단계다. 먼저 이전 믿음 $b_t(s)$와 행동 a_t로 다음 숨은 상태를 예측해 $\widetilde{b}_{t+1}(s') = \sum_s P(s' \mid s, a_t) b_t(s)$를 만든다. 다음 관측 o_{t+1}이 나오면 그 관측이 각 상태에서 얼마나 그럴듯한지 $O(o_{t+1} \mid s')$를 곱하고 전체 합이 1이 되도록 정규화한다. 예를 들어 컵이 왼쪽/오른쪽에 있을 사전 확률이 각각 0.5이고, 관측된 그림이 왼쪽일 가능도가 두 상태에서 0.8/0.2라면 관측 뒤 왼쪽 믿음은 $0.8 \cdot 0.5 / (0.8 \cdot 0.5 + 0.2 \cdot 0.5) = 0.8$이다. **관측을 곧 상태로 확정하지 않고 불확실성을 갱신**하는 방식이다.

3.36 슬라이드 36 · Ant 환경을 MDP로 쓰기

Ant 예시의 상태는 몸통 속도·높이·관절 상태, 행동은 관절 토크다. 전진 성과와 넘어짐 여부 등을 보상으로 측정한다.

3.37 슬라이드 37 · 게임에서의 상태와 보상

게임의 화면과 내부 정보가 상태, 플레이어의 명령이 행동이 된다. 승패 보상 외에 중간 목표를 보상에 넣으면 정책이 무엇을 우선하는지도 달라진다.

3.38 슬라이드 38 · 실제 로봇에서의 상태와 보상

실제 로봇에서는 영상과 관절 자세를 관측하고 모터 명령을 행동으로 낸다. 전이에는 접촉과 장치의 물리적 불확실성이 포함된다.

3.39 슬라이드 39 · 대화의 상태와 보상

대화에서는 이전 대화가 상태, 다음 응답이 행동이다. 사용자 반응이나 선호 점수는 보상의 한 예이며, 보상을 어떻게 정의하는지가 목표를 결정한다.

3.40 슬라이드 40 · 용어를 한 궤적에 모으기

한 시점의 관측 o_t와 상태 s_t를 받아 정책 $\pi(a_t \mid s_t)$가 행동 a_t를 선택한다. 환경은 $P(s_{t+1} \mid s_t, a_t)$에 따라 다음 상태로 움직이고 보상을 낸다. 이 반복의 기록이 trajectory 또는 rollout이다. episode는 보통 종료까지의 한 궤적을 가리킨다.

제어 분야에서는 행동을 u_t, 상태의 구성을 x_t라고 쓰기도 한다. 기호가 바뀌어도 입력, 제어 명령, 환경의 변화, 평가 신호라는 역할을 유지해 읽으면 된다.

용어를 시간 순서에 놓으면 s_0에서 a_0를 선택하고 r_0, s_1을 받은 뒤 a_1을 고른다. r_0가 행동 전에 주어진 점수인지 행동 뒤 결과인지 표기가 자료마다 다를 수 있으므로 **시간 인덱스의 약속**을 먼저 확인하자. 이 작은 차이를 놓치면 코드에서 현재 보상과 다음 보상을 한 칸 잘못 정렬할 수 있다.

3.41 슬라이드 41 · 정책경사로의 연결

다음 장에서는 정책 파라미터를 직접 바꾸는 policy gradient를 유도한다.

4 02-2. 정책경사와 REINFORCE

정책경사는 신경망 정책 π_θ가 만드는 궤적의 기대 반환을 직접 높인다. 어려운 점은 환경의 전이를 미분할 수 없고, 표본 궤적마다 보상이 크게 다르다는 것이다. 이 장은 확률밀도의 로그미분으로 첫 문제를 풀고, 다음 장에서 다룰 분산 감소 문제를 정확히 드러낸다.

4.1 슬라이드 01 · 정책경사의 질문

정책 파라미터를 직접 조정해 기대 누적 보상을 높이는 policy gradient를 유도한다.

4.2 슬라이드 02 · 반환과 기울기

MDP와 정책을 복습한 뒤, 궤적 확률에서 기울기를 유도하고 표본 추정의 분산을 살펴본다.

4.3 슬라이드 03 · 한 궤적의 기호

시점 t에서 상태 s_t가 주어지면 $a_t \sim \pi_\theta(\cdot \mid s_t)$를 뽑고, 환경은 다음 상태 s_{t+1}와 보상 r_t를 만든다. 이 반복에서 얻은 $\tau = (s_0, a_0, r_0, s_1, \ldots, a_{T-1}, r_{T-1}, s_T)$가 rollout이다. 보상 합 $R(\tau) = \sum_{t=0}^{T-1} r_t$는 그 궤적의 성과다.

정책경사 식에서 $\nabla_\theta \log \pi_\theta(a_t \mid s_t)$는 행동의 **로그확률이 파라미터 변화에 어떻게 반응하는지**를 나타낸다. 확률값 자체와 보상값을 혼동하지 말자.

표본에서 r_0가 크다고 a_0만 성과를 만든 것은 아니다. 나중에 실패한 a_3가 최종 보상을 없앴을 수도 있다. 궤적 전체의 반환을 모든 행동에 곱하는 기본형은 계산은 간단하지만 신용 할당이 거칠다. reward-to-go가 나중에 각 시점의 **이후 보상**만 남기는 이유를 이 한계를 통해 이해할 수 있다.

4.4 슬라이드 04 · MDP와 정책의 역할

MDP의 초기 상태와 전이 규칙은 정책을 고르기 전의 과업을 정의한다. 정책은 각 상태에서 행동 확률을 지정하므로 MDP와 합쳐져 특정 궤적의 확률을 만든다. 같은 MDP에서도 정책이 달라지면 어떤 상태를 얼마나 자주 보는지 달라진다.

목적함수의 기대값은 그 달라진 궤적 분포에 대해 계산한다. 그래서 한 번 모은 데이터에서 손실을 낮추는 것과 현재 정책의 진짜 기대 반환을 높이는 것이 정확히 같은 문제는 아니다.

예를 들어 동일한 교차로에서 정책 A가 오른쪽으로 갈 확률을 0.2, 정책 B가 0.8로 준다고 하자. 우회전 뒤의 보상이 좋다면 B는 그 결과를 더 자주 관측한다. 따라서 정책 변화는 단순히 현재 행동의 빈도뿐 아니라 **그 행동 뒤에 도달하는 상태의 데이터 분포**도 바꾼다.

4.5 슬라이드 05 · 확률적 정책의 의미

확률적 정책은 상태에서 하나의 행동만 고정해 내지 않고 가능한 행동의 분포를 만든다. 예컨대 두 경로 중 왼쪽을 0.7, 오른쪽을 0.3의 확률로 선택할 수 있다. 이런 무작위성은 탐색을 돕고 로그확률 기울기를 정의하게 한다.

드문 행동도 샘플링될 수 있고, 그 뒤의 반환이 높다면 해당 행동의 로그확률 기울기가 양의 학습 신호를 받는다. 표본의 변동에는 **정책이 행동을 무작위로 뽑는 효과**와 **환경이 같은 행동에 다른 결과를 내는 효과**가 함께 있다. 여러 궤적의 평균이 두 효과를 통과해 기대 기울기를 추정한다.

4.6 슬라이드 06 · 지도학습과 다른 손실

정답 행동이 주어지지 않을 때 기대 반환을 정책 파라미터로 어떻게 미분할 수 있는지 살펴본다.

4.7 슬라이드 07 · 정책 손실을 바로 쓸 수 없는 이유

$\mathcal{D} = \{(s_t, a_t, r_t)\}$가 있다고 해서 각 a_t를 정답처럼 더 자주 내도록 학습하면, 낮은 보상을 만든 행동까지 따라 하게 된다. 반대로 보상이 큰 궤적의 행동에는 더 높은 가중치를 주고 싶다. 문제는 그 가중치를 어떤 수학적 이유로 정하느냐다.

정책경사는 확률적 정책이 만든 궤적의 기대값을 미분해 그 가중치를 유도한다. 이 유도 없이 임의로 보상과 지도학습 손실을 곱하면 원하는 목적의 기울기인지 보장하기 어렵다.

두 궤적이 같은 상태를 거쳤어도 한쪽의 결과가 좋았다는 이유만으로 그 안의 모든 행동이 더 낫다고 할 수는 없다. 보상 가중치를 둔 업데이트는 **평균적으로 기대 반환의 기울기와 일치한다**는 수학적 성질에 의존한다. 이 성질은 개별 행동에 정답 라벨을 준다는 주장보다 약하지만, 미분할 수 없는 환경에서 목적함수를 개선할 근거가 된다.

4.8 슬라이드 08 · 궤적 확률의 곱

앞에서 정의한 길이 T의 궤적에는 보상도 들어 있다. $p_0(s_0)$는 초기 상태의 확률, $\pi_\theta(a_t \mid s_t)$는 정책의 행동 확률, $P(s_{t+1}, r_t \mid s_t, a_t)$는 환경이 다음 상태와 보상을 함께 낼 확률이다. 이들의 곱 $p_\theta(\tau)$가 정책 파라미터 θ에서 그 궤적이 나올 확률이다. 보상이 (s_t, a_t, s_{t+1})의 결정론적 함수인 경우에는 보상 확률 항을 생략하고 상태 전이 확률만 써도 된다.

$$p_\theta(\tau) = p_0(s_0) \prod_{t=0}^{T-1} \pi_\theta(a_t \mid s_t) P(s_{t+1}, r_t \mid s_t, a_t).$$

기대 반환은 $J(\theta) = \int p_\theta(\tau) R(\tau)\, d\tau$로 쓸 수 있다. 이산적인 상태와 행동이면 적분 대신 모든 궤적에 대한 합이다. θ는 정책에만 들어간다고 가정한다. 환경의 전이를 몰라도 정책의 확률은 계산할 수 있다.

세 단계 궤적이라면 확률은 $p_0(s_0)$와 세 정책 확률, 세 환경 전이 확률의 곱이다. 로그를 취하면 곱이 합으로 바뀌므로 정책 파라미터에 대한 미분이 각 시점의 로그확률 미분으로 분해된다. 이것이 신경망 출력의 미분을 전체 episode의 성과에 연결하는 계산의 핵심이다.

4.9 슬라이드 09 · 로그미분 항등식

양의 확률밀도 $p_\theta(\tau)$에 대해 $\nabla_\theta p_\theta(\tau) = p_\theta(\tau) \nabla_\theta \log p_\theta(\tau)$다. 여기서 ∇_θ는 파라미터 벡터 θ의 각 성분에 대한 편미분을 모은 기울기다. 연쇄법칙으로 $\nabla_\theta \log p = (\nabla_\theta p)/p$를 얻고 양변에 p를 곱하면 확인된다.

이 항등식을 기대 반환에 적용하면 다음과 같다.

$$\begin{aligned} \nabla_\theta J(\theta) &= \nabla_\theta \int p_\theta(\tau) R(\tau)\, d\tau \\ &= \int \nabla_\theta p_\theta(\tau) R(\tau)\, d\tau \\ &= E_{\tau \sim p_\theta}\left[\nabla_\theta \log p_\theta(\tau) R(\tau)\right]. \end{aligned}$$

두 번째 줄로 옮길 때는 $R(\tau)$가 궤적 τ를 고정하면 θ에 직접 의존하지 않고, 적분과 미분을 교환할 수 있는 매끄러움·적분 가능성 조건을 가정한다. 마지막 줄은 $\nabla_\theta p_\theta = p_\theta \nabla_\theta \log p_\theta$를 대입해 적분을

기대값으로 다시 쓴 것이다. 궤적을 더 자주 만들 방향을 그 궤적의 성과로 가중한다.

직관을 확인하려면 확률이 p인 결과에 보상 R_1, 나머지에 R_2를 주는 두 경우를 생각하면 된다. 기대 반환은 $J = pR_1 + (1-p)R_2$이고, p에 대한 미분은 $R_1 - R_2$다. 높은 보상의 결과가 더 자주 나오도록 p를 바꾸어야 한다는 방향이 로그미분 추정량에서도 그대로 나온다.

4.10 슬라이드 10 · 환경 전이 항이 사라지는 이유

곱으로 쓴 궤적 확률에 로그를 취하면 합이 된다. 초기 분포 p_0와 환경의 상태·보상 법칙 P가 정책 파라미터 θ에 직접 의존하지 않는다면 그 로그의 θ 미분은 0이다.

$$\nabla_\theta \log p_\theta(\tau) = \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t \mid s_t).$$

이는 행동이 상태 분포에 영향을 주지 않는다는 뜻이 아니다. 그 영향은 정책이 만든 궤적을 표본으로 뽑는 기대값 안에 이미 들어 있다. 따라서 환경의 미분 가능한 모델 없이도 정책경사를 추정할 수 있다.

4.11 슬라이드 11 · Monte Carlo 정책경사

앞의 두 단계를 합치면 기본 정책경사가 나온다.

$$\nabla_\theta J(\theta) = E_{\tau \sim p_\theta}\left[\left(\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t \mid s_t)\right) R(\tau)\right].$$

N개의 현재 정책 rollout으로 기대값을 표본 평균하면 $N^{-1}\sum_i \sum_t \nabla_\theta \log \pi_\theta(a_t^i \mid s_t^i) R(\tau_i)$가 된다. 위첨자 i는 i번째 궤적, $\widehat{\nabla J}$는 그 평균으로 추정한 기울기, $\alpha > 0$은 업데이트 크기를 정하는 학습률이다. 경사 상승은 $\theta \leftarrow \theta + \alpha \widehat{\nabla J}$로 쓰고, 손실을 최소화하는 optimizer에서는 목적에 음의 부호를 붙인다.

한 상태에서 두 행동만 가능한 예를 보자. $p = \pi_\theta(a_1) = \sigma(\theta)$이고 $\sigma(x) = 1/(1+e^{-x})$는 sigmoid 함수이며, $\pi_\theta(a_2) = 1 - p$다. 보상이 각각 1과 0이면 $J(\theta) = p$이므로 $\nabla_\theta J = p(1-p)$다. 로그확률 기울기를 보상으로 가중한 식도 $p\,\nabla_\theta \log p \cdot 1 + (1-p)\,\nabla_\theta \log(1-p) \cdot 0 = p(1-p)$를 준다.

4.12 슬라이드 12 · REINFORCE의 반복

REINFORCE는 현재 정책으로 여러 episode를 수집하고, 각 episode의 반환을 계산하며, 로그확률 합에 반환을 곱해 정책을 갱신한다. 갱신 뒤 정책이 바뀌므로 다음 반복은 새 정책으로 데이터를 다시 모은다.

높은 반환을 낸 궤적의 행동 확률을 늘리는 직관은 맞지만, 같은 궤적 안의 모든 행동을 동일한 전체 반환으로 가중하면 공로가 없는 행동도 함께 강화될 수 있다. 예컨대 마지막 점수와 무관한 초반 동작도 같은 점수를 받는다. 다음 장의 reward-to-go가 이 낭비를 줄인다.

한 번의 업데이트를 코드로 옮기면 `loss = -(log_probs.sum() * episode_return)` 형태가 된다. 여러 episode의 손실을 평균한 뒤 optimizer가 이를 최소화한다. `log_probs`는 행동을 뽑던 당시 정책에서 계산해 보관해야 하며, 학습 시점의 새 정책으로 다시 계산해 같은 값인 척하면 on-policy 유도와 맞지 않는다.

반환이 음수인 궤적은 어떻게 되나? 보상이 음수일 때는 그 궤적의 행동 로그확률을 낮추는 방향의 항이 생긴다. 하지만 보상의 기준점을 어디에 두었는지에 따라 한 표본의 부호가 달라질 수 있다. 절대적인 좋고 나쁨보다 같은 상태의 평균 성과와 비교하는 advantage가 유용한 이유다.

4.13 슬라이드 13 · 유도의 네 단계 복기

정책경사 유도는 네 고리로 정리된다. 첫째, J를 궤적 분포의 기대값으로 쓴다. 둘째, 로그미분으로 분포 미분을 기대값 안의 점수 함수로 옮긴다. 셋째, 궤적 로그확률을 정책 로그확률의 합으로 분해한다. 넷째, 기대값을 현재 정책의 표본 평균으로 근사한다.

각 고리에 가정이 있다. 보상 함수가 θ에 직접 의존하지 않고, 환경 전이도 정책 파라미터를 직접 공유하지 않으며, 표본은 해당 정책에서 나와야 한다. 응용에서 이 조건이 달라지면 추정량도 바뀐다.

이 연결 덕분에 최종 식에는 환경 전이의 미분이 없지만, 표본 궤적은 여전히 그 환경에서 현재 정책으로 수집해야 한다. 네 번째 단계의 표본 평균이 어느 분포를 근사하는지 확인하는 습관이 뒤의 on-policy 조건과 중요도 보정으로 이어진다.

4.14 슬라이드 14 · 장점과 두 한계

정책경사는 실제 목적 J의 상승 방향을 직접 겨냥할 수 있다. 연속 행동에서도 확률밀도를 정의하면 적용할 수 있다. 하지만 궤적마다 보상이 크게 달라 표본 기울기의 분산이 높고, 현재 정책 분포를 가정해 유도했으므로 오래된 정책 데이터는 그대로 재사용하기 어렵다.

"높은 반환이면 행동을 더 자주"라는 직관은 평균적으로 성립하는 미분 방향이지, 한 rollout의 각 행동이 모두 좋았다는 증명은 아니다. 표본 수와 분산 감소가 성능을 좌우한다.

연속 행동 로봇에서 정책이 Gaussian 평균과 분산을 출력한다고 하면 실제 샘플 행동의 로그밀도를 계산할 수 있다. 그러나 큰 행동 공간에서 한 궤적의 반환은 여러 관절 행동과 환경 잡음을 한꺼번에 반영한다. 기울기 식을 쓸 수 있다는 것과 적은 시도로 안정적으로 학습할 수 있다는 것은 다른 문제다.

4.15 슬라이드 15 · On-policy 데이터 조건

$E_{\tau\sim p_\theta}$라는 기호에는 **현재 θ에서 생성한 궤적**이라는 조건이 들어 있다. 예전 정책 π_{old}의 데이터를 그대로 평균하면 다른 분포에 대한 기대값을 계산하는 셈이다. 정책이 거의 변하지 않았다면 근사 오차가 작을 수 있지만 일반적으로 편향이 생긴다.

오래된 데이터를 쓰려면 중요도 가중치 $\pi_\theta(a \mid s)/\pi_{\text{old}}(a \mid s)$로 분포 차이를 보정하거나, off-policy 학습을 위한 다른 목적을 사용한다. 이 보정 자체도 분산을 키울 수 있어 뒤의 PPO가 정책 변화를 조심스럽게 제한한다.

한 시점의 중요도 비율은 $\pi_\theta(a_t \mid s_t)/\pi_{\text{old}}(a_t \mid s_t)$이지만, 완전한 궤적 분포를 정확히 보정하려면 일반적으로 시점별 비율을 곱해야 한다. 지평이 길수록 곱이 아주 크거나 작아져 추정 분산이 폭증할 수 있다. PPO의 한 단계 surrogate와 ratio clipping은 이 어려움을 다루는 실용적 근사이며, 임의로 오래된 자료를 정확하게 재사용할 수 있다는 보장은 아니다.

on-policy인데 minibatch를 여러 번 돌리면 바로 틀린가? 자료를 모은 뒤 파라미터가 바뀌므로 엄밀하게는 새 정책의 자료가 아니다. PPO는 수집 당시의 로그확률을 저장하고 현재 정책과의 비율을 계산하며, 변화가 지나치지 않도록 surrogate를 제한해 제한된 재사용을 허용한다.

4.16 슬라이드 16 · 표본 기울기의 분산

기울기 추정량을 X_i라고 하면 $\widehat{g} = N^{-1}\sum_i X_i$다. 독립적인 표본에 대해 $E[\widehat{g}] = g$이어도 $\mathrm{Var}(\widehat{g}) = \mathrm{Var}(X_i)/N$이므로 표본이 적으면 실제 기울기와 크게 어긋날 수 있다.

긴 episode의 전체 반환에는 여러 행동의 결과와 환경 잡음이 섞인다. 예를 들어 같은 행동에서도 환경이 성공과 실패를 반반씩 주면 두 궤적의 기울기 항이 반대 방향으로 흔들릴 수 있다. 개별 추정량의 분산을 줄이는 것이 표본 수를 늘리는 것과 함께 필요한 이유다.

4.17 슬라이드 17 · 분산을 줄이는 두 방향

첫 방법은 독립적인 rollout 수 N을 늘려 표본 평균의 불확실성을 줄이는 것이다. 표준오차는 대체로 $1/\sqrt{N}$ 비율로 줄어드므로 절반으로 줄이려면 약 네 배의 독립 표본이 필요하다. 실제 환경에서는 비용이 크다.

둘째 방법은 기대 기울기를 크게 바꾸지 않으면서 각 표본이 지닌 불필요한 변동을 줄이는 것이다. 과거 보상을 빼는 reward-to-go, 상태별 평균 성과를 빼는 baseline, 학습한 가치함수로 미래를 추정하는 actor–critic이 이 방향이다.

로봇에서 한 rollout에 2분이 걸리면 100개를 모으는 실행 시간만 200분이다. 초기화와 충돌 복구까지 더하면 비용은 커진다. Reward-to-go, baseline, critic은 같은 자료에서 기울기의 불필요한 변동을 줄이려는 장치다.

4.18 슬라이드 18 · 표본 수만 늘리기 어려운 이유

한 업데이트에 더 많은 rollout을 모으면 추정은 안정되지만 로봇 시간과 시뮬레이터 계산이 늘어난다. 하나의 100단계 episode에서 얻은 시점 100개는 같은 초기 상태와 실패 사건을 공유한다. 독립 episode 100개와 정보량이 다르므로 총 환경 단계 수와 독립 시작 횟수를 함께 세어야 한다.

4.19 슬라이드 19 · 더 나은 가중치의 계보

이 장의 비할인 설정에서 reward-to-go를 $G_t = \sum_{k=t}^{T-1} r_k$라 두자. 정책 $\pi = \pi_\theta$를 따를 때의 조건부 기대값 $Q^\pi(s,a) = E_\pi[G_t \mid s_t = s, a_t = a]$는 행동가치, $V^\pi(s) = E_{a\sim\pi(\cdot|s)}Q^\pi(s,a)$는 상태가치다. 상대적인 장점은 $A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$다. 과거 보상은 현재 행동의 선택에 의존하지 않아 기대 기울기에서 빠진다.

$$\nabla_\theta J = E\left[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, Q^\pi(s_t,a_t)\right] = E\left[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, A^\pi(s_t,a_t)\right].$$

두 번째 등식은 행동과 무관한 상태 baseline의 기대 기여가 0이라는 성질을 사용한다. Q와 V를 신경망으로 근사하면 분산은 줄 수 있지만 근사 오차가 새로 생긴다.

baseline 항의 기대 기여는 한 상태에서 직접 보일 수 있다. $b(s)$가 행동에 의존하지 않으면 $\sum_a \pi_\theta(a \mid s)\nabla_\theta \log \pi_\theta(a \mid s) b(s) = b(s)\nabla_\theta \sum_a \pi_\theta(a \mid s) = 0$이다. 그래서 평균적으로는 기울기를 바꾸지 않으면서, 행동 결과가 그 상태의 보통 성과보다 나았는지 판단하는 기준을 제공한다.

여기까지의 간결한 식은 이 장에서 정의한 **비할인** 반환 $R(\tau) = \sum_k r_k$에 대응한다. 할인 목적 $J_\gamma = E[\sum_k \gamma^k r_k]$를 그대로 미분한다면 시점 t의 과거 보상은 인과성으로 빠지고 남는 항은 $\sum_{k\ge t} \gamma^k r_k = \gamma^t G_t^\gamma$다. 따라서 $G_t^\gamma = \sum_{k\ge t} \gamma^{k-t} r_k$를 쓸 때는 바깥의 γ^t가 필요한 관례가 있다. 할인 목적과 코드의 reward-to-go 정의를 먼저 고정하지 않으면 할인 지수를 한 번 빠뜨리거나 두 번 곱할 수 있다.

4.20 슬라이드 20 · Actor–Critic으로의 연결

다음 장에서는 같은 정책경사의 기대값을 유지하면서 추정 분산을 낮추는 actor–critic 방법을 다룬다.

5 03-1. 가치함수와 Actor–Critic

정책경사는 기대 반환의 기울기를 정확한 형태로 표현하지만 표본으로 계산하면 크게 흔들린다. 이 장은 행동 이전에 이미 정해진 보상을 제거하고, 미래 보상을 가치함수로 추정하며, 상태의 평균 성과를 기준으로 행동의 상대적 장점을 계산한다. 이 과정을 거치면 **정책을 바꾸는 actor와 미래를 평가하는 critic**이 자연스럽게 등장한다.

5.1 슬라이드 01 · Actor–Critic의 출발점

정책경사의 분산을 줄이기 위해 reward-to-go, 가치함수, baseline, advantage를 도입한다.

5.2 슬라이드 02 · 분산 감소의 세 단계

정책경사를 복습하고, 미래 보상만 남긴 뒤 가치 추정과 baseline을 이용해 가중치를 개선한다.

5.3 슬라이드 03 · 로그미분 유도 다시 보기

정책 파라미터를 θ, 한 궤적을 τ, 그 궤적의 전체 반환을 $R(\tau)$, 정책이 만든 궤적 확률밀도를 $p_\theta(\tau)$라고 하자. 기대 반환 $J(\theta) = \int p_\theta(\tau) R(\tau) d\tau$를 미분할 때 $\nabla_\theta p_\theta = p_\theta \nabla_\theta \log p_\theta$를 사용하면

$$\nabla_\theta J(\theta) = E_{\tau \sim p_\theta}[\nabla_\theta \log p_\theta(\tau) R(\tau)]$$

가 된다. 이 표현은 환경의 내부 물리를 미분하지 않고 정책이 생성한 궤적을 표본으로 평균할 수 있게 한다. 단, 기대값을 표본으로 바꾸는 순간 추정 오차가 생긴다.

5.4 슬라이드 04 · 정책 로그확률만 남기기

궤적 확률은 초기 상태, 정책의 행동 확률, 환경 전이 확률의 곱이다. 로그를 취하면 합이 되고, 환경의 확률 법칙이 정책 파라미터 θ에 직접 의존하지 않는다면 다음이 된다.

$$\nabla_\theta \log p_\theta(\tau) = \sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t).$$

환경을 미분하지 않지만 뒤 보상의 영향은 사라지지 않는다. 행동이 이후 상태를 바꾸는 효과가 실제 rollout의 반환 $R(\tau)$에 포함되어 이 로그확률 합을 가중한다.

5.5 슬라이드 05 · 표본 정책경사의 구조

N개의 현재 정책 궤적을 모으자. 위첨자 i는 i번째 궤적, T_i는 그 길이, $\widehat{g}$는 표본으로 추정한 정책경사 벡터다. 각 행동의 로그확률 기울기에 해당 궤적의 반환을 곱한 뒤 평균한다.

$$\widehat{g} = \frac{1}{N} \sum_{i=1}^{N} \sum_{t=0}^{T_i - 1} \nabla_\theta \log \pi_\theta(a_t^i \mid s_t^i) R(\tau_i).$$

이 추정량은 기대값의 Monte Carlo 근사다. $R(\tau_i)$가 우연한 전이와 다른 시점의 행동 결과를 모두 포함하므로, 목적과 무관한 잡음이 한 행동의 업데이트에도 들어간다. 이번 장은 그 곱에 붙는 가중치를 바꿔 분산을 줄인다.

한 episode가 성공하면 초반 탐색 행동과 마지막 집기 행동이 모두 같은 $R(\tau)$를 받는다. 성공과 상관없는 초반의 우연한 흔들림도 강화될 수 있다. 이 때문에 기울기 식이 평균적으로 맞아도 한 batch의 업데이트는 실제 성능을 악화시킬 수 있다. **unbiased estimator(불편추정량)**라는 성질은 개별 업데이트의 품질을 보장하지 않는다.

5.6 슬라이드 06 · On-policy와 분산의 문제

현재 정책에서 모은 데이터라면 기대 정책경사와 연결되지만 표본 수가 적으면 방향이 심하게 흔들린다. 많이 모으면 비용이 크다. 오래된 정책의 데이터를 그대로 재사용하면 분포가 달라 또 다른 오차가 생긴다.

Actor–Critic은 critic을 학습해 한 전이에서도 미래 가치를 가늠하게 한다. 덕분에 짧은 rollout으로도 갱신할 수 있지만, critic의 근사 오차가 actor에게 전달된다. 같은 batch를 여러 번 재사용하면 수집 정책과 현재 정책도 멀어지므로, 가치 추정의 정확도와 데이터 분포 변화를 함께 살펴야 한다.

5.7 슬라이드 07 · 가중치를 단계적으로 바꾸기

먼저 현재 행동보다 앞선 보상이 정책경사 추정에 왜 필요 없는지 살펴본다.

5.8 슬라이드 08 · 먼저 필요한 점수 함수의 성질

다음 슬라이드의 핵심 도구는 확률분포의 **score function 기대값이 0**이라는 항등식이다.

5.9 슬라이드 09 · 점수 함수의 기대값이 0인 이유

$p_\theta(x)$를 파라미터 θ로 정해지는 확률밀도, x를 그 분포에서 뽑은 값이라 하자. $\nabla_\theta \log p_\theta(x)$는 관측 x의 로그확률을 바꾸는 **score function**이다. p_θ가 미분 가능하고 미분과 적분의 순서를 바꿀 수 있으며 적분 영역이 θ에 따라 변하지 않는다고 가정한다. $\nabla \log p = (\nabla p)/p$와 확률의 정규화 $\int p_\theta(x)\,dx = 1$을 차례로 쓰면

$$\begin{aligned}
\mathbb{E}_{x\sim p_\theta}[\nabla_\theta \log p_\theta(x)] &= \int p_\theta(x) \frac{\nabla_\theta p_\theta(x)}{p_\theta(x)}\,dx \\
&= \int \nabla_\theta p_\theta(x)\,dx \\
&= \nabla_\theta \int p_\theta(x)\,dx \\
&= \nabla_\theta 1 = 0.
\end{aligned}$$

이산 분포이면 적분을 합으로 바꾼다. 각 표본의 score가 0이라는 뜻이 아니라 **분포 전체에서 평균하면 0**이라는 뜻이다. 뒤에서 과거 보상이나 행동과 무관한 상태 baseline을 기울기 표본에서 빼는 근거가 된다.

5.10 슬라이드 10 · 과거 보상은 현재 행동의 결과가 아니다

시점 t의 행동을 정하기 전에 이미 받은 $r_0, \ldots, r_{t-1}$은 a_t로 바꿀 수 없다. 그런데 전체 궤적 반환을 곱하면 이 과거 보상도 $\nabla_\theta \log \pi_\theta(a_t \mid s_t)$를 가중한다. 기대값에는 도움이 없고 표본을 흔드는 성분이다.

예를 들어 로봇이 이미 첫 구간에서 보상을 받은 뒤 마지막에 컵을 놓친다면, 마지막 행동이 앞선 성공 보상을 받는 것은 공로를 잘못 배분한 것이다. 미래 보상만 쓰는 reward-to-go가 더 자연스러운 가중치다.

시점 0에서 동전을 던져 우연히 +10 또는 -10을 받았고, 시점 1에서 현재 행동의 결과로 +1 또는 0을 받는다고 하자. 시점 1의 행동 업데이트에 전체 반환을 쓰면 동전의 ±10이 큰 잡음이 된다. 동전 결과는 이미 정해졌으므로 그 항을 빼면 현재 행동의 효과를 더 선명하게 볼 수 있다.

5.11 슬라이드 11 · 전체 반환을 시점별로 나누기

정책경사의 시점 t 항은 $\nabla_\theta \log \pi_\theta(a_t \mid s_t) \sum_{k=0}^{T-1} r_k$ 다. 보상 합을 $k < t$의 과거와 $k \geq t$의 현재·미래로 나누면, 현재 행동과 무관한 과거 성분을 따로 볼 수 있다.

$$\sum_{k=0}^{T-1} r_k = \underbrace{\sum_{k=0}^{t-1} r_k}_{\text{이미 일어난 결과}} + \underbrace{\sum_{k=t}^{T-1} r_k}_{\text{현재 이후의 결과}} .$$

이 분해가 곧 reward-to-go 유도의 출발점이다. 어떤 보상이 언제 관측되는지 코드 배열의 인덱스와 맞추어 보자.

예를 들어 보상이 $r_0 = 2, r_1 = -1, r_2 = 5$이고 $t = 2$라면 전체 반환은 6이지만 현재 행동 이후의 반환은 5다. 앞의 $2 - 1 = 1$은 a_2보다 먼저 관측되었다. 보상 배열과 행동 배열의 시간축을 맞추어 이 분해를 코드로 구현해야 reward-to-go가 한 칸 밀리지 않는다.

5.12 슬라이드 12 · 과거 보상 항의 기대값

과거 기록 $h_t = (s_0, a_0, r_0, \ldots, s_t)$를 고정하면 그 시점의 과거 보상 합도 고정된다. 이때 현재 행동만 평균내면 점수 함수의 기대값이 0이므로

$$E\left[\nabla_\theta \log \pi_\theta(a_t \mid s_t) \sum_{k<t} r_k \mid h_t\right] = \left(\sum_{k<t} r_k\right) E_{a_t \sim \pi_\theta(\cdot \mid s_t)}[\nabla_\theta \log \pi_\theta(a_t \mid s_t)] = 0.$$

전체 기대값도 0이다. 과거 보상과 현재 상태는 상관될 수 있으므로, 과거 기록 h_t를 조건으로 고정한 뒤 현재 행동에 대해서만 평균하는 순서가 중요하다.

5.13 슬라이드 13 · Reward-to-go가 남긴 것

기대값이 0인 과거 항을 빼면 시점별 가중치는 $G_t = \sum_{k=t}^{T-1} r_k$가 된다. 정책경사는 $E[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t) G_t]$로 쓸 수 있다. 할인 보상에서는 $G_t = \sum_{k=t}^{T-1} \gamma^{k-t} r_k$를 쓰되 목적함수의 할인 관례와 일치시켜야 한다.

과거 항은 평균이 0이어도 각 rollout에서는 0이 아니므로 추정량에 잡음을 더한다. 제거하면 일반적으로 분산을 줄이는 방향이지만, 구체적 분산 크기는 항 사이의 공분산에도 영향을 받는다.

할인 목적을 엄밀히 다룰 때는 전체 목표 $E[\sum_t \gamma^t r_t]$와 각 시점에서 다시 시작한 reward-to-go G_t를 구분해야 한다. 같은 기호라도 외부의 γ^t 가중치가 남는 정책경사 관례가 있다. 강의나 구현의 식을 옮길 때 할인 지수를 한 번 더 곱하거나 빼지 않도록 목적함수부터 적자.

5.14 슬라이드 14 · 인과성의 직관

reward-to-go는 현재 행동이 **앞으로** 받는 결과만 가중한다. 전체 궤적 반환과 기대 기울기가 같은 이유는 이미 일어난 결과의 점수 함수 기여가 평균적으로 0이기 때문이다. 이는 보상을 미래에만 주는 새로운 과업을 정의한 것이 아니라 같은 목적의 더 나은 표본 추정량을 만든 것이다.

실제 코드는 뒤에서 앞으로 반환을 누적하면 효율적이다. 마지막 보상에서 시작해 $G_t = r_t + \gamma G_{t+1}$로 계산한다. 종료 이후의 가치는 더하지 않아야 한다.

뒤에서부터 계산하는 작은 예에서 $r_0 = 1, r_1 = 2, r_2 = 3, \gamma = 0.5$라면 $G_2 = 3$, $G_1 = 2 + 0.5 \cdot 3 = 3.5$, $G_0 = 1 + 0.5 \cdot 3.5 = 2.75$다. 같은 재귀식이 critic의 TD 목표 $r_t + \gamma V(s_{t+1})$에도 등장하지만, 실제 남은 반환 대신 **추정한 다음 상태 가치**를 쓴다는 점이 다르다.

5.15 슬라이드 15 · 가치함수가 필요한 순간

한 궤적의 실제 미래 보상은 변동이 크다. 앞으로는 여러 가능한 미래의 평균을 예측하는 가치함수를 도입한다.

5.16 슬라이드 16 · V^π와 Q^π의 차이

상태가치 $V^\pi(s)$는 상태 s에서 정책 π를 따를 때의 기대 반환이다. 행동가치 $Q^\pi(s,a)$는 첫 행동 a를 **고정한 뒤** 이후에 π를 따를 때의 기대 반환이다.

$$V^\pi(s) = E_\pi[G_t \mid s_t = s], \qquad Q^\pi(s,a) = E_\pi[G_t \mid s_t = s, a_t = a].$$

따라서 $V^\pi(s) = E_{a\sim\pi(\cdot|s)} Q^\pi(s,a)$다. $Q^\pi(s,a) = E[r_t + \gamma V^\pi(s_{t+1}) \mid s_t = s, a_t = a]$도 성립한다. 첫 식은 행동 평균, 둘째는 한 단계 보상과 남은 미래를 분리한 Bellman 관계다.

상태 s에서 정책이 행동 A와 B를 각각 0.25, 0.75 확률로 고르고 $Q^\pi(s,A) = 8$, $Q^\pi(s,B) = 4$라면 $V^\pi(s) = 0.25 \cdot 8 + 0.75 \cdot 4 = 5$다. 정책이 이 상태에서 행동을 뽑기 전의 평균 전망이 5라는 뜻이다.

5.17 슬라이드 17 · 가치와 advantage를 한 표기로

두 가치의 차이를 정의하면

$$A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$$

가 advantage다. 양수이면 현재 정책의 평균 행동보다 낫고, 음수이면 평균보다 나쁘다. "좋다"는 절대적인 보상값이 아니라 **그 상태와 그 정책에 대한 상대적 비교**다.

앞 슬라이드의 예에서는 $A^\pi(s,A) = 8 - 5 = 3$, $A^\pi(s,B) = 4 - 5 = -1$이다. 정책 확률로 평균하면 $0.25 \cdot 3 + 0.75 \cdot (-1) = 0$이다. 상태별 평균을 기준으로 정의했기 때문에 생기는 성질이며, 정책이 바뀌면 V^π와 두 advantage도 달라진다.

5.18 슬라이드 18 · 상태가치의 학습 목표

진짜 상태가치 V^π 대신 파라미터 ϕ의 신경망 $V_\phi(s)$를 학습한다. 목표 y_t는 실제 반환 G_t로 둘 수도 있고, 한 단계 TD 값 $r_t + \gamma V_{\bar\phi}(s_{t+1})$로 둘 수도 있다. 여기서 $\bar\phi$는 목표를 계산할 때 고정한 가치망 파라미터다. 종료 상태에서는 미래 가치 항을 0으로 둔다. 다음 기대값은 수집한 상태들의 분포에 대한 평균이다.

$$\mathcal{L}_V(\phi) = E\big[(V_\phi(s_t) - y_t)^2\big].$$

TD 목표는 실제 전체 궤적을 기다리지 않아도 되지만 목표에 또 다른 가치 추정치가 들어간다. 이를 **부트스트랩**이라고 한다. 목표 네트워크 $\bar\phi$를 분리할 수도 있고, 현재 네트워크의 값을 gradient가 흐르지 않는 목표로 사용할 수도 있다.

TD 손실이 작으면 가치 추정도 정확한가? 반드시 그렇지는 않다. 목표 $r + \gamma V_\phi(s')$ 자체가 같은 근사기의 값이어서 일관되게 틀릴 수 있다. 보상 척도, 데이터 범위와 Monte Carlo 반환에 대한 별도 비교가 도움이 된다.

TD 목표를 계산할 때는 y_t를 상수처럼 취급해 현재 $V_\phi(s_t)$와의 차이를 줄인다. 같은 네트워크로 다음 상태를 평가하더라도 목표 쪽 gradient를 끊지 않으면 두 값이 동시에 움직여 의도한 회귀 문제가 달라질 수 있다. 코드에서 `detach`나 target network가 어떤 역할을 하는지 이 식과 연결해 보자.

5.19 슬라이드 19 · 할인율의 두 역할

할인 반환 $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$에서 γ가 1에 가까우면 먼 미래를 많이 반영하고, 작으면 가까운 보상에 집중한다. $\gamma = 0.9$라면 네 단계 뒤의 보상에는 $0.9^4 \approx 0.656$의 가중치가 붙는다.

또한 계속되는 과업에서 유계 보상의 무한합을 다루기 쉽게 한다. 예를 들어 매번 1의 보상을 받으면 $\gamma = 1$에서는 합이 발산하지만 $0 \le \gamma < 1$에서는 $1/(1-\gamma)$다. 할인율은 시간 선호와 수학적 안정성 두 관점에서 읽는다.

$\gamma = 0.99$와 0.9는 가까워 보여도 100단계 뒤 보상에 붙는 가중치는 각각 약 0.366과 0.000027이다. 긴 과업에서 할인율의 작은 차이가 학습 목표의 시간 범위를 크게 바꾼다. 에피소드가 짧은지, 중요한 보상이 몇 단계 뒤에 오는지와 함께 값을 정해야 한다.

5.20 슬라이드 20 · Bellman 식에 할인율 넣기

미래 반환을 $G_t = r_t + \gamma G_{t+1}$로 나누고 조건부 기대값을 취하면

$$Q^\pi(s,a) = E[r(s,a) + \gamma V^\pi(s') \mid s, a],$$
$$V^\pi(s) = E_{a\sim\pi(\cdot|s),\, s'\sim P(\cdot|s,a)}[r(s,a) + \gamma V^\pi(s')].$$

이때 즉시 보상에 붙는 가중치는 1이고 다음 상태 가치에 γ가 붙는다. 코드에서 γ를 현재 보상에도 곱하거나 종료 상태의 다음 가치를 남겨 두면 다른 목표를 학습하게 된다.

상태 s에서 행동 a를 고른 뒤 즉시 보상 2를 받고 다음 상태 가치가 5이며 $\gamma = 0.9$라면 Bellman 목표는 $2 + 0.9 \cdot 5 = 6.5$다. 현재 보상과 미래 보상을 둘 다 0.9배 하면 6.3이 되어 다른 값을 학습한다. 종료라면 뒤의 5를 넣지 않아 목표는 2다. 작은 수치로 구현의 마스크를 점검할 수 있다.

5.21 슬라이드 21 · Reward-to-go를 더 안정적으로 추정하기

한 번 관측한 G_t는 $Q^\pi(s_t, a_t)$의 표본이다. 같은 상태·행동의 미래가 매번 다르면 표본 하나는 크게 흔들린다. 여러 궤적에서 조건부 평균을 배운 $Q_\phi(s,a)$는 그 우연성을 평활화한다. 다만 신경망의 예측 오차가 들어오므로 표본 분산 감소와 근사 편향 사이에 절충이 생긴다.

특히 자료가 한 행동 주변에만 있으면 다른 행동의 Q를 배울 근거가 약하다. Actor가 그런 행동의 우연히 높은 예측값을 찾아갈 위험은 뒤의 off-policy 및 offline RL에서 중요해진다.

5.22 슬라이드 22 · Q가 행동의 공로를 평가한다

행동의 공로를 평가할 때는 그 행동을 고정하고 이후에는 정책 π를 따르는 조건을 유지한다. $Q^\pi(s,a)$가 높을 때 그 선택이 같은 상태의 평균 행동보다도 유리한지는 상태 기준값 $V^\pi(s)$와 비교해 판단한다.

실제로는 Q_ϕ를 데이터에서 학습한다. 정책이 바뀌면 같은 (s,a) 뒤에 따를 미래 행동도 달라지므로, critic이 추정해야 하는 Q^π 자체가 움직인다.

한 번의 성공 궤적만 보면 마지막 손가락 동작의 G_t도 높다. 하지만 같은 상태·행동을 100번 반복했을 때 10번만 성공한다면 평균 Q^π는 그 우연한 한 성공보다 훨씬 낮다. 행동의 공로를 평가하려면 **그 행동을 고정한 조건에서 가능한 미래를 평균**해야 한다는 뜻이다.

5.23 슬라이드 23 · 진짜 Q를 얻는 두 방법

한 상태–행동에서 아주 많은 미래 궤적을 수집해 평균내면 Q^π에 접근할 수 있다. 하지만 실제 로봇에서는 같은 상태로 되돌리는 것부터 어렵다. 그래서 관측한 전이에서 Bellman 관계를 이용해 Q_ϕ를 학습하는 방법을 쓴다.

가치 추정으로 정책경사를 쓰면 $\nabla_\theta J = E[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t) Q^\pi(s_t, a_t)]$다. 여기서 Q_ϕ로 대체하면 계산할 수 있는 알고리즘이 되지만, 진짜 정책경사와의 차이를 critic의 오차가 만든다.

Monte Carlo 방법은 실제 끝까지 실행한 결과를 쓰므로 목표가 직접적이지만 완료까지 기다려야 하고 분산이 크다. TD 방법은 $r + \gamma V(s')$로 이른 갱신을 하지만 미래의 추정값을 다시 목표로 쓰므로 편향이 생길 수 있다. 어느 쪽이 좋은지는 episode 길이, 보상 희소성, critic 정확도에 달려 있다.

5.24 슬라이드 24 · 상태 baseline을 생각하기

같은 상태에서 평균적으로 이미 높은 점수를 얻는다면 한 행동의 큰 reward-to-go가 정말 특별한 성과인지 알기 어렵다. 상태별 기준 $b(s_t)$를 빼면 행동의 상대 성과를 보게 된다.

$b(s_t)$는 현재 행동을 고르기 전에 상태만으로 결정되어야 한다. 현재 행동 a_t에 의존하는 값을 마음대로 빼면 기대 정책경사가 달라질 수 있다. 다음 페이지에서 이 조건을 수식으로 확인한다.

쉬운 상태에서는 어떤 행동을 해도 10점 정도를 얻고 어려운 상태에서는 1점만 얻는다고 하자. 반환 5점을 받은 행동은 쉬운 상태에서는 나쁘고 어려운 상태에서는 좋다. 전체 점수를 그대로 비교하면 이 차이를 놓친다. $b(s)$는 상태별 난도의 기준점 역할을 한다.

5.25 슬라이드 25 · Baseline을 빼도 기대값이 같은 증명

현재 상태 s를 고정하면 $b(s)$는 행동과 무관하다. 따라서

$$
\begin{aligned}
E_{a\sim\pi_\theta(\cdot|s)}[\nabla_\theta \log \pi_\theta(a \mid s) b(s)] &= b(s) \sum_a \pi_\theta(a \mid s) \frac{\nabla_\theta \pi_\theta(a \mid s)}{\pi_\theta(a \mid s)} \\
&= b(s) \nabla_\theta \sum_a \pi_\theta(a \mid s) = 0.
\end{aligned}
$$

연속 행동이면 합 대신 적분을 쓴다. 이 성질로 G_t에서 $b(s_t)$를 빼도 정책경사의 기대값은 같다. 중요한 조건은 상태가 주어졌을 때 baseline이 **그때 샘플링한 행동에 의존하지 않는 것**이다.

actor와 critic이 파라미터를 공유해도 baseline을 뺄 수 있나? 한 정책 파라미터 값에서 baseline이 현재 샘플 행동에 의존하지 않으면 점수 함수와 baseline의 기대 곱은 0이다. 그러나 구현에서 actor 손실 $-\log \pi_\theta(a \mid s)(G - b_\theta(s))$ 전체를 그대로 자동미분하면 b_θ를 미분하는 추가 경로가 생긴다. 표준 정책경사 추정에서는 advantage를 actor 손실에서 stop-gradient 처리하고 critic은 별도 가치 손실로 학습한다.

5.26 슬라이드 26 · 기대값 보존과 분산 감소는 별개

상태에만 의존하는 baseline $b(s)$는 기대 정책경사를 유지한다. 분산 감소 효과는 $b(s)$가 미래 반환 G_t의 규모를 얼마나 잘 설명하는지에 달려 있다.

좋은 baseline은 그 상태에서 예상되는 반환을 잘 설명해, 행동 결과의 **예상 밖 부분**만 남긴다. 상태가치 $V^{\pi}(s)$가 자연스러운 후보인 이유다. 학습 초기에 가치 추정이 불안정하면 정책 학습에도 영향을 주므로 baseline 손실과 정책 반환을 함께 본다.

예를 들어 G_t가 대략 10이고 $b(s) = -100$이면 가중치는 약 110이다. 큰 표본 기울기가 생길 수 있으므로 critic의 가치 손실과 정책 반환을 함께 확인한다. 다음 슬라이드에서 분산을 최소화하는 baseline의 조건을 계산한다.

5.27 슬라이드 27 · 분산을 실제로 비교하는 식

추정량의 평균은 baseline 전후에 같아도 분산은 $E[X^2] - E[X]^2$의 첫 항이 달라진다. 한 시점에서 점수 함수를 $g_t = \nabla_\theta \log \pi_\theta(a_t \mid s_t)$라고 두면 표본은 $g_t(G_t - b(s_t))$가 된다.

직관적으로 $G_t - b(s_t)$가 작을수록 표본의 크기와 흔들림이 줄 가능성이 있다. 하지만 최적 baseline은 일반적으로 단순한 $E[G_t \mid s_t]$와 정확히 같지는 않다. g_t의 크기까지 가중한 조건부 분산 최소화 해가 존재하며, 여러 시점의 공분산도 전체 분산에 영향을 준다.

한 시점의 분산을 최소화하는 baseline을 직관적으로 구하면 $E[\|g_t\|^2(G_t - b)^2 \mid s]$를 b에 대해 미분한다. $-2E[\|g_t\|^2(G_t - b) \mid s] = 0$이므로 $b^*(s) = E[\|g_t\|^2 G_t \mid s] / E[\|g_t\|^2 \mid s]$다. 점수 함수의 크기가 결과와 무관할 때에만 단순 평균 $V^{\pi}(s)$와 같다.

5.28 슬라이드 28 · 왜 $V^{\pi}(s)$를 선택하는가

$V^{\pi}(s)$는 행동을 고르기 전의 평균 미래 반환이므로 baseline으로 해석하기 쉽다. $G_t - V^{\pi}(s_t)$는 한 번 실현된 미래가 그 상태의 평균에서 얼마나 벗어났는지 측정한다. 반면 $Q^{\pi}(s,a) - V^{\pi}(s)$는 **특정 행동의 조건부 평균 성과**가 상태 평균과 얼마나 다른지 측정한다. 앞의 값에는 단일 궤적의 우연성이, 뒤의 값에는 critic의 근사 오차가 남는다.

슬라이드 27의 분산 최소해와 정확히 일치하지 않아도 V^{π}는 Bellman 관계로 학습할 수 있고, 상태별 난도를 빼는 기준값으로 쓰기 편하다.

5.29 슬라이드 29 · 평균보다 좋은 행동 강화하기

정책경사의 표본 가중치를 $G_t - V_\phi(s_t)$로 바꾼다. 양수면 그 행동의 로그확률을 높이고, 음수면 낮춘다. 보상값이 모두 양수라고 무조건 모든 행동의 확률을 높이는 직관보다 정확하다.

$$\widehat{g} = \frac{1}{N} \sum_{i,t} \nabla_\theta \log \pi_\theta(a_t^i \mid s_t^i) \big(G_t^i - V_\phi(s_t^i)\big).$$

가치함수는 현재 상태에서 기대되는 미래를 학습해야 한다. 다른 정책의 오래된 데이터로 학습한 값이라면 현재 정책의 기준선과 달라질 수 있다.

정책 손실을 구현할 때 $V_\phi(s_t)$를 actor의 가중치로 쓰는 경로와 critic의 회귀 손실을 구분해야 한다. Baseline을 추정하는 ϕ의 학습은 가치 목표를 맞히고, θ의 학습은 advantage에 따라 행동 확률을 바꾼다. 두 손실을 한 숫자로 합칠 수 있어도 각 항이 어떤 파라미터를 갱신하는지 확인해야 한다.

5.30 슬라이드 30 · Advantage와 Actor–Critic

한 궤적의 G_t 대신 critic의 $Q_\phi(s_t, a_t)$를 쓰면, baseline을 뺀 값 $A_\phi(s_t, a_t) = Q_\phi(s_t, a_t) - V_\phi(s_t)$를 actor의 업데이트 가중치로 쓸 수 있다. 이것이 actor–critic의 대표적인 형태다.

$$\widehat{g}_{\mathrm{AC}} = \frac{1}{N}\sum_{i,t} \nabla_\theta \log \pi_\theta(a_t^i \mid s_t^i) A_\phi(s_t^i, a_t^i).$$

정책은 행동을 생성하고 critic은 그 행동이 상태 평균보다 나았는지 평가한다. critic을 배우는 과정에는 TD 부트스트랩이 들어갈 수 있으므로 낮은 분산의 대가로 가치 근사 오차를 받아들인다.

Actor는 평가자가 준 advantage가 양수인 행동을 늘리고 음수인 행동을 줄인다. 하지만 critic이 실제로 나쁜 행동에 높은 Q_ϕ를 주면 actor는 바로 그 행동을 강화할 수 있다. 따라서 critic의 TD 손실이 낮은지만 보지 말고 실제 rollout 반환과 가치 예측이 함께 움직이는지도 살펴야 한다.

5.31 슬라이드 31 · 네 가지 가중치의 비교

전체 반환은 한 episode의 성과를 모든 행동에 준다. Reward-to-go는 현재 이후 보상만 남긴다. $Q_\phi(s,a)$는 같은 조건의 여러 미래를 평균하려 하고, advantage는 상태의 평균을 빼서 상대 성과를 본다.

이 네 값의 차이는 **어떤 잡음을 제거하고 어떤 추정을 도입했는가**에 있다. 실제 반환에는 표본의 우연성이 있고, 학습된 Q_ϕ에는 critic의 근사 오차가 있다.

같은 한 궤적의 숫자로 비교해 보자. 전체 반환은 20, 현재 시점 이전 보상은 12라면 reward-to-go는 8이다. Critic의 $Q_\phi(s,a) = 7$, $V_\phi(s) = 6$이면 advantage는 1이다. 20→8은 인과성으로 과거를 지운 것이고, 8→7은 모델의 평균 추정, 7→1은 상태 기준선을 뺀 것이다. 마지막 숫자가 작다고 반드시 더 좋은 추정량이라는 뜻은 아니다.

5.32 슬라이드 32 · REINFORCE의 데이터 요구

REINFORCE는 현재 정책의 완성된 궤적을 모아 전체 반환을 계산하고 한 번의 정책 업데이트에 사용한다. 종료 보상만 있는 과업에서는 episode가 끝날 때까지 기다려야 한다. 길이가 1000인 과업이라면 한 정책 업데이트에 최소 1000개의 환경 단계가 필요할 수 있다.

이 단순함은 장점도 있다. 가치함수를 따로 학습하지 않으므로 critic의 부트스트랩 편향은 없다. 짧은 과업이나 유도·디버깅에는 좋은 기준선이며, 다른 알고리즘의 편향이 성능에 어떤 영향을 주는지 비교할 때 유용하다.

Actor–Critic은 TD 목표로 episode 중간에도 갱신할 수 있다. 그때 확보하는 빠른 신호는 아직 관측하지 않은 미래를 critic으로 근사한 값에 의존한다. 두 방법을 비교할 때는 환경 단계당 갱신 횟수와 실제 반환을 함께 본다.

5.33 슬라이드 33 · Actor와 critic의 반복

Actor–Critic은 현재 정책으로 모은 전이에서 critic의 TD 목표를 만들고, 그 예측으로 계산한 advantage를 actor 갱신에 사용한다. 이 과정은 episode 중간의 짧은 rollout에서도 수행할 수 있다.

갱신 뒤에는 달라진 actor가 다른 상태를 방문한다. Critic이 그 새 분포를 평가하기 전에 actor가 크게 움직이면 두 망의 오차가 서로 증폭될 수 있다. 이때 critic 손실, advantage 분포, 정책 변화량을 함께 보면 어느 단계에서 불안정해졌는지 추적할 수 있다.

5.34 슬라이드 34 · 표본 효율의 정확한 비교

그림은 길이 1000인 episode마다 갱신하는 REINFORCE와 10단계씩 모아 갱신하는 Actor–Critic을 비교한다. 짧은 rollout은 **같은 환경 단계 수에서 더 많은 갱신 기회**를 주지만 아직 보지 않은 미래를 critic에 맡긴다. Critic 편향이 크면 갱신을 늘려도 평가 반환은 오르지 않을 수 있다.

Sample efficiency는 예컨대 평가 반환 200에 도달할 때까지 사용한 **환경 전이 수**로 잰다. 같은 전이에서 여러 번 gradient 계산을 했다면 학습 계산량도 따로 기록한다. 두 비용을 나란히 보아야 잦은 갱신이 실제로 효율적인지 판단할 수 있다.

5.35 슬라이드 35 · 다음 단계: 길이와 재사용

다음 장에서는 n단계 반환으로 가치 추정의 편향·분산을 조절하고, 중요도 샘플링과 PPO로 데이터 재사용을 다룬다.

6 03-2. GAE와 PPO

Actor–Critic은 가치 추정으로 정책을 자주 갱신하지만 두 문제가 남는다. 짧은 TD 목표는 critic 오류에 민감하고, 현재 정책의 rollout을 한 번 쓰고 버리면 데이터가 비싸다. **GAE**는 advantage 추정의 길이를 조절하고, **PPO**는 수집한 batch에서 여러 번 갱신할 때 정책 변화가 지나치게 커지는 일을 완화한다.

6.1 슬라이드 01 · PPO가 해결하려는 문제

PPO는 정책을 여러 번 갱신하면서도 한 번에 지나치게 크게 바뀌지 않도록 하는 방법이다.

6.2 슬라이드 02 · 두 개선 방향

Actor–Critic을 복습한 뒤, advantage 추정과 데이터 재사용이라는 두 문제를 순서대로 다룬다.

6.3 슬라이드 03 · 정책경사 가중치의 계보

전체 반환, reward-to-go, $Q^\pi(s,a)$, advantage는 모두 점수 함수 $\nabla_\theta \log \pi_\theta(a \mid s)$에 곱할 성과 신호다. Advantage는 같은 상태에서 평균적인 가치 $V^\pi(s)$를 $Q^\pi(s,a)$에서 뺀 값이다. 각각 과거 보상의 잡음과 상태 난도의 영향을 다르게 제거한다.

실제 PPO에서는 A^π를 정확히 알 수 없으므로 가치망과 관측 보상으로 $\widehat{A}_t$를 만든다. 정책 목적함수의 공식이 정확해도 advantage가 심하게 잘못되면 갱신 방향이 틀릴 수 있다.

예를 들어 같은 상태의 보통 반환이 20인데 한 행동 뒤의 추정 반환이 15라면, 보상이 양수라도 advantage는 -5다. PPO는 그 행동의 확률을 낮추려 한다. 보상의 부호와 advantage의 부호를 동일하게 생각하면 clipping을 해석할 때 특히 혼동하기 쉽다.

6.4 슬라이드 04 · REINFORCE가 기다리는 것

REINFORCE의 기본형은 끝난 궤적의 전체 보상으로 정책을 업데이트한다. 장기 과업에서 한 episode가 길고 보상이 마지막에만 있으면 갱신까지 많은 환경 단계가 필요하다. 가치함수가 없으므로 단순하고 설명하기 좋지만 표본 분산이 클 수 있다.

PPO의 개선을 이해할 때 REINFORCE를 단순히 "옛 알고리즘"으로 보면 안 된다. 어떤 편향을 추가해서 데이터 사용을 줄였는지 비교하는 기준선이다.

가치망의 근사 오류 없이 실제 반환을 쓰는 REINFORCE는 뒤에서 도입할 부트스트랩 방법의 기준점이다. 두 방법을 비교할 때에는 **같은 환경 단계에서의 평가 반환**과 추정량의 분산을 함께 본다.

6.5 슬라이드 05 · Actor–Critic의 한 단계

Actor–Critic은 상태가치나 행동가치를 학습해 완전한 episode를 기다리지 않고 미래를 추정한다. actor는 $\widehat{A}_t$의 부호와 크기를 이용해 정책 로그확률을 갱신한다. $\widehat{A}_t > 0$이면 관측 행동의 확률을 높이고, 음수면 낮춘다.

critic의 학습 목표가 $r_t + \gamma V_\phi(s_{t+1})$처럼 자기 추정에 의존하면 갱신이 빨라지는 대신 편향이 생긴다. 다음 단계는 여러 단계의 실제 보상과 끝의 가치 추정을 섞어 이 절충을 조절한다.

현재 가치 추정이 $V(s_t) = 4$, 다음 상태 추정이 $V(s_{t+1}) = 6$, 보상이 1, $\gamma = 0.9$라면 한 단계 TD 잔차는 $\delta_t = 1 + 0.9 \cdot 6 - 4 = 2.4$다. 이 값은 "현재 행동 뒤의 결과가 가치망의 이전 전망보다 얼마나 좋았는가"를 나타낸다. 뒤의 GAE는 이런 잔차를 여러 시점에서 합친다.

6.6 슬라이드 06 · 자주 갱신할 수 있다는 뜻

한 episode가 1000단계인데 actor–critic이 10단계마다 가치 추정으로 갱신할 수 있다면 같은 경험에서 훨씬 많은 갱신 기회를 얻는다. 하지만 10단계 뒤의 나머지 결과는 critic에 의존한다. 업데이트 수 증가와 실제 성능 개선은 별개다.

표본 효율은 목표 점수에 도달하는 데 필요한 환경 단계 수로 평가한다. critic의 추정 오차가 크면 잦은 갱신이 오히려 학습을 불안정하게 할 수 있다.

업데이트를 자주 한다는 말은 같은 데이터를 무한히 써도 된다는 뜻이 아니다. 10단계마다 새 표본을 모은다면 데이터는 계속 현재 정책에서 나오지만, 한 10단계 batch만 계속 재사용하면 정책과 수집 분포의 차이가 커진다. **갱신 빈도**와 **한 batch의 재사용 횟수**를 구분해야 한다.

6.7 슬라이드 07 · 남은 두 약점

짧은 반환 추정은 분산을 낮추지만 가치망이 틀린 만큼 편향될 수 있다. 또 이미 모은 batch에서 actor를 여러 번 갱신하면 새 정책과 수집 정책의 간격이 벌어진다.

GAE는 **미래를 몇 단계 실제 보상으로 추적할지**, PPO clipping은 **그 추정을 이용해 정책 확률을 얼마나 바꿀지** 조절한다. 다음 식들에서는 λ가 반환의 길이, ϵ이 확률비의 허용 폭을 정한다.

6.8 슬라이드 08 · n단계 반환

r_{t+l}은 시점 $t+l$의 보상, γ는 한 단계 할인율, n은 실제로 관측해 합칠 보상의 개수다. $V_\phi(s)$는 파라미터 ϕ로 학습한 상태가치 추정치다. 모자 쓴 $\widehat{Q}_t^{(n)}$과 $\widehat{A}_t^{(n)}$은 각각 시점 t의 n단계 행동가치와 advantage **추정치**를 뜻한다.

$$\widehat{Q}_t^{(n)} = \sum_{l=0}^{n-1} \gamma^l r_{t+l} + \gamma^n V_\phi(s_{t+n}), \qquad \widehat{A}_t^{(n)} = \widehat{Q}_t^{(n)} - V_\phi(s_t).$$

$n=1$은 빠르지만 가치망을 많이 믿고, n이 episode 끝까지 가면 실제 반환을 많이 반영한다. 끝 상태가 진짜 종료라면 마지막 부트스트랩 항은 0이다. 어떤 n이 좋은지는 과업의 보상 지연과 가치망의 정확도에 따라 달라진다.

$n=2$라면 $\widehat{Q}_t^{(2)} = r_t + \gamma r_{t+1} + \gamma^2 V(s_{t+2})$다. 예를 들어 보상 1, 2와 $V(s_{t+2})=5$, $\gamma=0.9$를 쓰면 $1 + 1.8 + 4.05 = 6.85$다. 한 단계 목표보다 실제로 관측한 보상 하나를 더 쓰지만, 끝의 가치 추정 오차는 여전히 남는다.

rollout이 시간 제한으로 잘리면 마지막 가치가 항상 0일까? 진짜 종료 상태라면 뒤에 받을 보상이 없으므로 0이다. 단순히 수집 시간을 채워 잘렸고 과업은 계속된다면 다음 상태의 가치를 bootstrap해야 한다. 두 종료 원인을 같은 마스크로 처리하면 가치와 GAE가 왜곡된다.

6.9 슬라이드 09 · GAE의 가중 평균과 재귀

GAE의 $\lambda \in [0,1]$는 긴 반환에 줄 가중치를 정한다. T를 현재 rollout이 끝나는 시점, $\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$를 한 단계 TD 잔차라 두자. 이 잔차를 뒤에서부터 합치면 여러 길이의 advantage를 평균한 값과 같다. 아래 재귀식의 $\widehat{A}_{t+1}$도 같은 GAE 추정치이며 끝에서는 $\widehat{A}_T = 0$으로 놓는다.

$$\widehat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \delta_{t+l} = \delta_t + \gamma\lambda \widehat{A}_{t+1}.$$

$\lambda = 0$이면 한 단계 TD advantage이고, $\lambda = 1$에 가까우면 긴 실제 반환을 많이 반영한다. λ를 키울수록 가치 근사에 따른 짧은 지평 편향은 줄 수 있지만 표본 분산은 커질 수 있다. 종료와 시간 제한 처리에 따라 경계항이 달라진다. GAE 논문

GAE를 단순한 평균으로 보면 왜 부족한가? 각 길이의 advantage를 동일 가중으로 평균하지 않는다. λ가 긴 길이의 비중을 지수적으로 조절하고 γ가 보상의 시간 가중을 함께 정한다. 구현은 TD 잔차를 뒤에서 앞으로 재귀적으로 합치는 편이 정확하다.

두 단계만 있는 예에서 $\delta_t = 2$, $\delta_{t+1} = -1$, $\gamma = 0.9$, $\lambda = 0.5$라면 $\widehat{A}_t = 2 + 0.9 \cdot 0.5 \cdot (-1) = 1.55$다. 한 단계 TD만 쓰면 2이고, 더 긴 결과를 반영하면 뒤의 나쁜 결과가 현재 행동의 평가를 낮춘다. λ는 먼 TD 잔차의 반영 정도를 조절한다.

왜 여러 길이의 반환이 TD 잔차 합으로 바뀌는지 직접 전개해 보자. $\delta_{t+j} = r_{t+j} + \gamma V(s_{t+j+1}) - V(s_{t+j})$를 $\sum_{j=0}^{n-1} \gamma^j \delta_{t+j}$에 대입하면 가운데의 $+\gamma^{j+1} V(s_{t+j+1})$와 다음 항의 $-\gamma^{j+1} V(s_{t+j+1})$가 차례로 상쇄된다. 처음의 $-V(s_t)$와 마지막의 $+\gamma^n V(s_{t+n})$만 남으므로 이 합은 정확히 $\widehat{Q}_t^{(n)} - V(s_t) = \widehat{A}_t^{(n)}$다. GAE는 길이가 다른 이 합들에 λ 가중치를 주므로, 오래된 잔차일수록 $(\gamma\lambda)^j$가 붙는다.

남은 길이를 $H = T - t$라고 놓고, 길이 $n < H$의 가중치를 $w_n = (1-\lambda)\lambda^{n-1}$, 마지막 길이의 가중치를 $w_H = \lambda^{H-1}$로 정하자. 실제로 이 가중치의 합은 $(1-\lambda)\sum_{n=0}^{H-2} \lambda^n + \lambda^{H-1} = 1$이다. 따라서

$$\begin{aligned} \widehat{A}_t^{\mathrm{GAE}} &= (1-\lambda) \sum_{n=1}^{H-1} \lambda^{n-1} \widehat{A}_t^{(n)} + \lambda^{H-1} \widehat{A}_t^{(H)} \\ &= \sum_{j=0}^{H-1} \gamma^j \delta_{t+j} \underbrace{\sum_{n=j+1}^{H} w_n}_{=\lambda^j} = \sum_{j=0}^{H-1} (\gamma\lambda)^j \delta_{t+j}. \end{aligned}$$

내부 합 $\sum_{n=j+1}^{H} w_n = \lambda^j$는 j번째 잔차를 포함하는 모든 길이의 총가중치다. 시간 제한으로 rollout을 잘랐다면 마지막 $\widehat{A}_t^{(H)}$에는 $V(s_T)$ 부트스트랩이 남는다.

$\lambda = 1$이면 언제 Monte Carlo advantage와 같아지나? 수집 구간이 진짜 종료까지 이어지고 종료 상태의 V를 0으로 두면 잔차 합의 중간 가치가 모두 지워져 $G_t - V(s_t)$가 된다. 단순 시간 제한에서 수집을 멈췄다면 마지막 $V(s_T)$를 남겨 부트스트랩하므로 순수 Monte Carlo 반환과 같지 않다.

6.10 슬라이드 10 · Advantage 추정과 데이터 재사용은 별개

앞에서는 n단계 반환과 GAE로 advantage 추정을 개선했다. 남은 문제는 같은 데이터로 정책을 여러 번 갱신하는 것이다.

6.11 슬라이드 11 · 왜 현재 정책 데이터가 필요한가

기본 정책경사의 기대값은 $\tau \sim p_\theta$에 대한 것이다. 파라미터를 θ에서 θ'로 바꾸면 원하는 기울기는 $E_{\tau \sim p_{\theta'}}$가 되지만 손에 든 batch는 여전히 p_θ에서 왔다.

정책 변화가 작으면 근사적으로 쓸 수 있어도 여러 업데이트를 반복하면 차이가 커진다. 단순히 같은 로그확률 손실을 여러 번 줄이는 것은 원래 유도의 표본 가정을 벗어난다.

예를 들어 이전 정책이 어떤 행동을 0.2 확률로 골랐는데 새 정책은 0.8 확률로 고른다면, 이전 batch에는 그 행동의 결과가 새 정책에서 필요한 빈도보다 훨씬 적다. 기존 한 표본을 네 배로 가중해 평균을 맞추려 할 수 있지만, 실제 표본 하나가 우연한 성공이었다면 그 우연도 네 배가 된다. 분포 보정과 분산 문제는 함께 생긴다.

6.12 슬라이드 12 · 중요도 샘플링의 기본식

원하는 표본분포의 밀도를 $p(x)$, 실제 자료를 뽑은 분포의 밀도를 $q(x)$, 평균낼 함수를 $f(x)$라 두자. $E_{x\sim p}[f(x)]$는 p에서 뽑았을 때의 함수 평균이다. 적분 안에서 $q(x)$를 곱하고 나누면 q 자료만으로 같은 기대값을 표현할 수 있다.

$$E_{x\sim p}[f(x)] = \int q(x)\frac{p(x)}{q(x)} f(x)\,dx = E_{x\sim q}\left[\frac{p(x)}{q(x)} f(x)\right].$$

이 식이 성립하려면 $p(x) > 0$인 곳에서 $q(x) > 0$이어야 하고, 해당 기대값이 존재해야 한다. 비율 $w(x) = p(x)/q(x)$를 **중요도 가중치**라 한다. 이전 정책이 전혀 선택하지 않은 행동의 결과는 데이터 재가중만으로 알 수 없으며, 작은 $q(x)$ 때문에 $w(x)$가 크면 추정량의 분산도 커진다.

두 값의 장난감 예로 $q(a_1) = 0.25$, $p(a_1) = 0.5$라면 a_1 표본의 비율은 2다. 반대로 $q(a_2) = 0.75$, $p(a_2) = 0.5$이면 비율은 $2/3$이다. q에서 얻은 표본을 이 비율로 가중하면 p가 더 자주 낼 행동은 키우고 덜 낼 행동은 낮춘다. $q(a_1) = 0$이면 비율 자체가 정의되지 않으므로 **없는 자료를 가중치로 만들어 낼 수는 없다.**

6.13 슬라이드 13 · 궤적 비율에서 행동 비율로

길이 T의 궤적 $\tau = (s_0, a_0, \ldots, s_T)$를 생각하자. π_{old}는 이 궤적을 수집한 정책, π_θ는 파라미터 θ로 갱신하는 정책이다. $p_{\text{old}}(\tau)$와 $p_\theta(\tau)$는 각각 두 정책 아래 같은 궤적의 확률이다. 초기 상태분포와 환경 전이확률이 같고 분모가 양수인 궤적에서는 그 항들이 비율에서 상쇄된다.

$$\frac{p_\theta(\tau)}{p_{\text{old}}(\tau)} = \prod_{t=0}^{T-1} \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\text{old}}(a_t \mid s_t)}.$$

한 시점의 행동 확률비를 $\rho_t(\theta) = \pi_\theta(a_t \mid s_t)/\pi_{\text{old}}(a_t \mid s_t)$라고 정의하자. 전체 궤적 비율은 $\prod_t \rho_t(\theta)$다. 긴 궤적에서는 이 곱의 분산이 커지므로 PPO는 시점별 ρ_t를 surrogate 목적에 쓴다. 보상 r_t와 확률비 ρ_t를 구별하자.

각 단계의 비율이 1.1이어도 길이 100의 궤적 비율은 $1.1^{100} \approx 13780$이다. PPO의 한 단계 비율은 전체 궤적 기대값을 정확히 보정하는 가중치가 아니라, 정책을 점진적으로 갱신하기 위한 surrogate의 요소다.

6.14 슬라이드 14 · 여러 갱신에서 이전 정책 고정하기

rollout 직후 행동마다 수집 정책의 로그확률 $\log \pi_{\text{old}}(a_t \mid s_t)$를 저장하고 갱신 중 고정한다. 시점 t에 수집한 행동 a_t의 **새 정책 확률질량 또는 밀도를 옛 정책의 값으로 나눈 비**를 $\rho_t(\theta) = \pi_\theta(a_t \mid s_t)/\pi_{\text{old}}(a_t \mid s_t)$라 정의한다. 분모가 양수인 표본에서 현재 정책의 로그확률을 구해 $\rho_t(\theta) = \exp\{\log \pi_\theta(a_t \mid s_t) - \log \pi_{\text{old}}(a_t \mid s_t)\}$로 계산한다.

같은 batch를 여러 epoch 학습하려면 분모를 업데이트마다 새로 계산하면 안 된다. 그러면 "수집 당시 정책과 지금 정책"의 비가 아니게 된다. 중요도 비율은 계산만으로 안정성을 보장하지 않아 추가적인 갱신 제어가 필요하다.

저장된 값이 $\log \pi_{\text{old}}(a_t \mid s_t) = -1.2$이고 현재 값이 -1.0이라면 비율은 $\exp(0.2) \approx 1.22$다. 로그확률을 빼고 지수화하면 매우 작은 확률도 직접 나누는 것보다 계산이 안정적이다. 분모를 현재 정책으로 매번 다시 계산하면 비율이 늘 1 근처가 되어 clipping의 의미가 사라진다.

6.15 슬라이드 15 · 큰 중요도 비율과 정책 변화

좋은 결과를 낸 행동의 확률을 계속 높이면 ρ_t가 커진다. 한 batch를 반복 학습할수록 몇 표본의 큰 비율이 손실과 기울기를 지배하고, 그 표본을 낳은 상태 분포와 새 정책의 상태 분포도 멀어진다.

파라미터 거리 $\|\theta - \theta_{\text{old}}\|$는 정책 거리의 좋은 대리변수가 아닐 수 있다. 신경망에서는 작은 파라미터 이동도 출력 확률을 크게 바꿀 수 있기 때문이다. TRPO는 정책 간 KL 발산을 제한하는 신뢰영역 관점을 쓰고, PPO는 훨씬 간단한 목적함수의 clipping으로 큰 변화를 억제한다. **PPO clipping은 KL 제약을 엄밀하게 보장하지 않는다.** TRPO, PPO

실험에서는 평균 KL과 clip fraction(비율이 clip 구간 밖에 놓인 표본의 비율)을 함께 관찰한다. PPO의 clipping이 목적함수에 주는 제어와 실제 정책 변화의 크기를 나란히 확인할 수 있다.

6.16 슬라이드 16 · PPO를 구성하는 네 조각

PPO-Clip은 actor–critic의 가치 추정, GAE advantage, 행동 확률비, 비율 clipping을 결합한다. 수집한 batch의 표본 수를 N, 각 표본에서 이미 계산해 고정한 advantage를 $\widehat{A}_t$라 하자. 출발점인 surrogate $\widehat{L}(\theta) = N^{-1}\sum_t \rho_t(\theta)\widehat{A}_t$는 좋은 결과의 행동 확률을 올리고 나쁜 결과의 확률을 내린다. 같은 batch를 오래 최적화하면 비율이 과하게 변한다.

수집 당시의 로그확률과 $\widehat{A}_t$는 여러 actor 갱신 동안 고정한다. 구현에서는 advantage를 `detach`해 actor 손실의 경사가 critic으로 흐르지 않게 하고, critic은 별도의 가치 손실로 학습한다. PPO 원논문

6.17 슬라이드 17 · Advantage 부호에 따른 clipping 직관

수집 정책 대비 새 정책이 같은 행동에 주는 확률의 비율을 ρ_t라 하고, 그 행동의 advantage 추정치를 $\widehat{A}_t$라 하자. $\widehat{A}_t > 0$이면 행동 확률을 높이는 이득을 $1+\epsilon$에서, $\widehat{A}_t < 0$이면 낮추는 이득을 $1-\epsilon$에서 제한한다. 여기서 ϵ은 허용 비율 변화의 폭이다. 다음 슬라이드가 이 두 경우를 하나의 목적함수로 합친다.

6.18 슬라이드 18 · 하나의 PPO-Clip 목적함수로 합치기

수집 당시 정책을 π_{old}, 현재 정책을 π_θ라고 하자. $\rho_t(\theta) = \pi_\theta(a_t \mid s_t)/\pi_{\text{old}}(a_t \mid s_t)$는 수집한 행동의 확률비, $\widehat{A}_t$는 그 행동의 고정된 advantage 추정치, $\epsilon \in (0,1)$은 허용 비율의 폭이다. $\mathcal{B}$는 수집 rollout의 시점 인덱스들이며, $E_{t\in\mathcal{B}}$는 그 표본 평균이다. $\text{clip}(x, l, u)$는 x를 구간 $[l,u]$의 가까운 끝점으로 자르는 함수다. PPO가 **최대화**하는 actor 목적은

$$L^{\text{CLIP}}(\theta) = E_{t\in\mathcal{B}}\left[\min\left(\rho_t(\theta)\widehat{A}_t,\ \text{clip}(\rho_t(\theta), 1-\epsilon, 1+\epsilon)\widehat{A}_t\right)\right].$$

양의 advantage에서는 비율 상단, 음의 advantage에서는 비율 하단의 추가 이득이 잘린다. clipping 범위를 벗어난 모든 표본의 기울기가 0이 되는 것은 아니다. 예컨대 양의 advantage인데 비율이 지나치게 **작으면**, 목적은 확률을 회복시키도록 여전히 기울기를 준다. min은 과도하게 유리한 변화만 제한하는 보수적 surrogate다. PPO 원논문

비율을 자르면 새 정책이 반드시 가까운가? 아니다. 표본에서 목적의 유리한 방향을 자르는 장치다. 관측하지 않은 상태에서 정책이 크게 변할 수도 있고, 실제 KL이 커질 수도 있다. 실무에서는 평균 KL을 모니터링하거나 조기 중단을 더한다.

한 표본의 advantage를 A, 확률비를 ρ라고 줄여 쓰자. $f(\rho, A) = \min\{\rho A, \text{clip}(\rho, 1-\epsilon, 1+\epsilon)A\}$를 부호별로 풀면

$$f(\rho, A) = \begin{cases} A \min(\rho, 1+\epsilon), & A \geq 0, \\ A \max(\rho, 1-\epsilon), & A < 0. \end{cases}$$

$A < 0$에서 음수를 곱하면 부등호 방향이 뒤집혀 $\max(\rho, 1-\epsilon)$이 나온다. $A = -2$, $\rho = 1.3$처럼 나쁜 행동의 확률이 증가한 경우에는 목적이 -2.6으로 내려가므로 이를 되돌릴 기울기가 남는다.

6.19 슬라이드 19 · PPO 학습 반복의 순서

현재 정책으로 rollout을 수집하고 행동 로그확률, 보상, 상태, 종료 여부를 저장한다. critic으로 가치와 GAE를 계산한 다음 수집 정책을 π_{old}로 고정한다. 동일 batch를 여러 epoch와 minibatch에 나누어 L^{CLIP}를 최대화하고, 가치 손실과 정책 엔트로피 보너스를 함께 학습한다. 이후 **새 정책으로 다시 rollout**을 수집한다.

엔트로피 $H(\pi(\cdot \mid s)) = -\sum_a \pi(a \mid s) \log \pi(a \mid s)$는 정책이 너무 일찍 한 행동으로 수렴하지 않도록 한다. 연속 행동에서는 해당 밀도의 미분 엔트로피를 쓴다. 가치 손실·엔트로피 계수의 스케일은 보상 스케일과 함께 점검해야 한다. 코드에서 분모의 old log probability를 매 epoch 재계산하면 알고리즘이 달라진다.

Rollout을 시간 제한으로 끊은 경우에는 진짜 종료인지 수집만 중단했는지 표시해야 한다. 전자는 이후 가치가 0이고, 후자는 마지막 상태의 가치로 GAE를 부트스트랩한다. 이 마스크를 잘못 쓰면 actor의 advantage와 critic의 목표가 동시에 바뀐다.

6.20 슬라이드 20 · PPO가 널리 쓰이는 이유와 한계

같은 clipping 식이 이산 행동과 연속 행동의 정책 로그확률에 적용되고, 여러 환경을 병렬 실행해 rollout을 모으기 쉽다. 구현의 핵심도 복잡한 제약 최적화 대신 보통의 경사 최적화로 표현된다. 그래서 로봇 제어와 언어모델 학습을 포함한 여러 맥락에서 출발점으로 쓰인다.

rollout 길이, minibatch 크기, 업데이트 epoch 수, 학습률, ϵ, GAE λ, entropy 계수, 보상·advantage 정규화가 성능을 좌우한다. 재현할 때는 평가 환경을 학습 환경과 분리하고 여러 seed의 평균과 분산을 함께 보고한다.

실험에서 episode 반환만 보면 정책이 왜 불안정한지 찾기 어렵다. 평균 KL이 계속 커지면 갱신 폭을, clip fraction이 지나치게 높으면 많은 표본이 더 이상 유리한 기울기를 주지 못하는지를, value loss가 높으면 advantage의 질을 확인한다. 정책 엔트로피가 갑자기 0에 가까워지면 탐색이 너무 일찍 줄었는지도 살핀다.

6.21 슬라이드 21 · 접촉이 많은 로봇 조립

Factory는 접촉이 많은 로봇 조립을 시뮬레이션에서 학습·평가한 사례다. 접촉의 정확한 물리 계산과 병렬 수집이 중요하다.

6.22 슬라이드 22 · SAPG와 병렬 학습

SAPG는 병렬 정책경사 학습의 확장 사례다. 여기서는 PPO 계열 방법이 많은 시뮬레이션 경험을 모아 로봇 정책을 학습할 수 있다는 점을 보여 준다.

6.23 슬라이드 23 · 사족 보행의 실제 로봇 시연

네 발 로봇의 실제 보행 시연이다. 시뮬레이션에서 학습한 정책을 실제 로봇에서 실행할 때의 안정성을 보여 준다.

6.24 슬라이드 24 · 보행을 넘어서는 전신 제어

보행 외 전신 제어 동작의 예시다. 복잡한 몸체의 여러 관절을 동시에 조절해야 한다.

6.25 슬라이드 25 · 조작 정책 시연을 읽는 법

로봇 조작 정책의 실행 영상이다. 화면은 결과를 보여 주며 구체적인 학습 알고리즘은 별도로 확인해야 한다.

6.26 슬라이드 26 · 사람 피드백과 언어모델

RLHF에서는 사람의 선호로 만든 보상모델을 사용해 언어모델 정책을 개선한다. 응답 전체에 대한 선호가 학습 신호가 된다는 사례다.

6.27 슬라이드 27 · 왜 다음은 off-policy인가

PPO는 한 rollout batch를 몇 번 재사용하지만, 새 정책이 수집 정책에서 너무 멀어지면 자료를 버리고 다시 모아야 한다. 서로 다른 과거 정책의 방대한 replay buffer를 자유롭게 활용하는 방식과는 다르므로 보통 on-policy 계열로 분류한다.

Q-learning과 DQN은 행동 정책과 다른 greedy 정책의 가치를 학습할 수 있다. DDPG·TD3·SAC는 연속 행동에서 replay를 활용한다. 다음 장의 핵심 질문은 **새 정책의 행동을 실험하지 않은 전이 자료만으로 어떻게 평가하고 개선할 것인가**다.

한 전이 (s, a, r, s')는 과거 행동 정책이 수집했어도, 그 결과 r, s'는 환경의 같은 전이 법칙에서 나왔다. Off-policy 방법은 이 자료를 이용해 현재와 다른 목표 정책의 가치를 추정한다. 다만 데이터가 없는 행동의 결과를 자동으로 아는 것은 아니며, Q함수의 일반화와 탐색이 중요하다.

7 04-1 Q-learning과 DQN

7.1 슬라이드 01 · Q-learning의 출발

Q-learning은 정책의 기울기를 직접 구하지 않고 행동가치 $Q(s,a)$를 학습해 행동을 고른다.

7.2 슬라이드 02 · 지금까지의 정책경사

Actor–critic에서는 현재 정책으로 자료를 모으고 $\widehat{A}(s,a)$를 추정하여 $\nabla_\theta \log \pi_\theta(a \mid s)\widehat{A}$로 actor를 개선했다. 여기서 advantage는 **현재 정책 평균에 비해** 그 행동이 얼마나 좋은지를 나타낸다. 정책이 좋아진 뒤에는 같은 Q^π도 달라진다. 결국 정책 평가와 정책 개선을 번갈아 수행한다.

Actor–Critic의 critic은 현재 정책이 앞으로 따를 행동까지 포함한 Q^π를 평가한다. 반면 Q-learning의 target에는 다음 상태에서의 $\max_{a'} Q$가 들어간다. 수집 정책이 어떤 행동을 했든 다음 상태에서는 가장 좋다고 추정한 행동을 택한다고 가정하는 차이가 off-policy 성질의 핵심이다.

7.3 슬라이드 03 · 정책 반복에서 가치 반복으로

어떤 정책의 $Q^\pi(s,a)$를 알고 있다면 $a^*(s) = \arg\max_a Q^\pi(s,a)$를 택하는 greedy 정책으로 개선할 수 있다. 이 절차를 평가와 개선으로 나누면 정책 반복이다. 가치 반복은 매 단계에서 "다음 상태에서는 이미 최선의 행동을 고른다"고 가정해 최적 가치의 고정점을 찾는다.

이산 행동이 몇 개뿐이면 모든 a의 가치를 비교하기 쉽다. 관절 토크처럼 연속 벡터인 행동에서는 $\arg\max_a Q(s,a)$ 자체가 고차원 최적화 문제가 된다. 이 한계가 뒤의 DDPG·TD3·SAC로 이어진다.

작은 표에서 $Q^\pi(s,A)=4, Q^\pi(s,B)=7$이고 현재 정책이 A를 80% 택한다면 $V^\pi(s) = 0.8\cdot 4 + 0.2\cdot 7 = 4.6$이다. B를 greedy하게 선택하는 새 정책은 그 상태의 기대 가치를 높일 가능성이 있다. 단, 이후 상태에서도 정책이 바뀌므로 실제 개선을 분석하려면 평가와 개선의 전체 연결이 필요하다.

7.4 슬라이드 04 · V와 Q의 관계

$V^\pi(s) = E_\pi[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \mid s_t = s]$, $Q^\pi(s,a) = E_\pi[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \mid s_t = s, a_t = a]$라고 두자. 첫 행동을 정책 π로 뽑아 평균하면 $V^\pi(s) = E_{a\sim\pi(\cdot|s)} Q^\pi(s,a)$다. 최적 정책에서는 $V^*(s) = \max_a Q^*(s,a)$가 된다.

한 step을 떼어 내면 $Q^\pi(s,a) = E[r + \gamma V^\pi(s') \mid s, a]$다. 이 식은 다음 상태의 미지의 긴 미래를 가치함수 하나로 대체하는 **부트스트랩**의 출발점이다. γ가 없는 강의식은 $\gamma = 1$인 유한 지평의 특수 경우로 읽을 수 있다.

앞 예의 $V^\pi = 4.6$은 현재 정책의 행동 평균이고 $\max_a Q^\pi = 7$은 **첫 행동만** greedy하게 바꾼 값이다. 이후 모든 상태에서 최선의 행동을 반복하는 Q^*와도 다르다. 이 세 값을 나누면 Bellman 식에서 최댓값을 어디에 두는지 분명해진다.

7.5 슬라이드 05 · 모델을 요구하는 fitted value iteration

$V^*(s) = \max_a E_{s'\sim p(\cdot|s,a)}[r(s,a,s') + \gamma V^*(s')]$를 표본으로 맞추려면 상태 s에서 **각 후보 행동**의 다음 상태를 샘플링하거나 전이모델 p를 알아야 한다. 기록에 A를 실행한 전이만 있다면 B의 결과 s'는 비어 있다.

Fitted Q-iteration은 관측된 (s,a,r,s')로 **실제로 한 행동의** $Q(s,a)$를 회귀한다. 다음 상태에서의 행동 비교는 학습한 Q값의 최댓값으로 옮긴다. 이 전환이 모델을 모르는 자료에서 Bellman 갱신을 가능하게 한다.

7.6 슬라이드 06 · 최적 Bellman 방정식

$Q^*(s,a)$는 상태 s에서 첫 행동 a를 한 뒤 최적으로 행동할 때의 기대 할인 반환이다. r은 그 첫 행동의 보상, s'은 다음 상태, a'는 다음 상태에서 고를 수 있는 행동, $\gamma \in [0,1)$는 할인율이다. 첫 보상을 떼고 이후의 최적 선택을 남기면

$$Q^*(s,a) = E\Big[\, r + \gamma \max_{a'} Q^*(s',a') \mid s,a \,\Big].$$

왼쪽은 현재 상태–행동 쌍의 전체 미래 가치, 오른쪽은 즉시 보상과 다음 상태에서 가능한 최선의 가치다. 이것은 정의에서 첫 보상을 분리하고 최적 부분문제를 남긴 식이다. 실제 transition (s,a,r,s') 하나로는 기대값을 계산하지 못하므로 $r + \gamma \max_{a'} Q(s',a')$를 **표본 목표**로 사용한다.

식의 재귀를 수치로 읽어 보자. 현재 보상이 2, 다음 상태의 최대 Q가 5, $\gamma = 0.9$라면 목표는 $2 + 0.9 \cdot 5 = 6.5$다. 종료라면 미래가 없으므로 2만 남는다. 현재 Q가 4라면 6.5 쪽으로 올리는 회귀를 하지만, 5 자체가 틀린 추정이면 그 오류도 함께 전파된다.

정확한 표 형태 계산에서 이 반복이 수렴하는 이유도 볼 수 있다. $(\mathcal{T}^*Q)(s,a) = E[r + \gamma \max_b Q(s',b) \mid s,a]$를 **최적 Bellman 연산자**라 정의하자. Q_1, Q_2는 비교할 두 행동가치 함수다. $\|Q_1 - Q_2\|_\infty$는 모든 상태·행동에서의 최대 차이이며, 정확히 $\max_{s,a} |Q_1(s,a) - Q_2(s,a)|$다. 최댓값의 차이는 항별 최대 차이를 넘지 못하므로

$$\begin{aligned} |(\mathcal{T}^*Q_1)(s,a) - (\mathcal{T}^*Q_2)(s,a)| &\le \gamma E[\,|\max_b Q_1(s',b) - \max_b Q_2(s',b)| \mid s,a] \\ &\le \gamma \|Q_1 - Q_2\|_\infty. \end{aligned}$$

$0 \le \gamma < 1$이면 모든 (s,a)에 대한 최댓값 거리도 매 반복 γ배 이하로 줄어드는 **수축**이다. 그래서 정확한 Bellman 연산의 고정점은 유일하다. 신경망의 회귀·표본 오차·부분적인 상태 방문이 끼면 실제 DQN 업데이트가 이 정확한 연산과 달라지므로, 이 증명을 DQN의 무조건적 수렴 보장으로 옮기면 안 된다.

7.7 슬라이드 07 · Fitted Q-iteration

자료 $\mathcal{D} = \{(s_i, a_i, r_i, s_i', d_i)\}$의 i번째 항은 상태·행동·보상·다음 상태·종료 표시다. Q_ϕ는 파라미터 ϕ로 학습하는 Q망, $Q_{\bar\phi}$는 목표 계산 동안 고정하는 target 망, y_i는 회귀 목표다. 진짜 종료에는 $d_i = 1$을 두어 미래 항을 없앤다. 전이마다 $y_i = r_i + \gamma(1-d_i)\max_{a'} Q_{\bar\phi}(s_i', a')$를 만들고 $\sum_i (Q_\phi(s_i,a_i) - y_i)^2$를 줄인다.

목표에는 자료 수집 정책 β의 행동 확률이 직접 들어가지 않는다. β가 택한 현재 행동의 전이를 사용하면서 **다음 상태에서는 greedy 행동**을 평가하므로 off-policy다. 필요한 상태–행동 coverage는 자료에서 확보해야 한다.

한 전이가 $r = 1$, s'의 target Q가 A에 4, B에 6이고 $\gamma = 0.5$라면 목표는 $1 + 0.5 \cdot 6 = 4$다. 자료에서 현재 행동이 A였는지 B였는지는 **현재 회귀할 $Q(s,a)$의 인덱스**를 정하고, 다음 상태의 최댓값은 목표 정책을 정한다. 현재 행동과 다음 행동의 역할을 섞지 말자.

7.8 슬라이드 08 · 세 가지 반복을 구분하기

정책 반복은 Q^π를 평가한 뒤 π를 개선한다. 모델 기반 fitted value iteration은 V를 맞추면서 각 후보 행동의 전이 기대값을 요구한다. fitted Q-iteration은 관측된 transition에서 $Q(s,a)$를 맞추고 정책을 $\arg\max_a Q(s,a)$로 **암묵적으로** 정의한다.

세 방법의 차이는 **현재 상태에서 후보 행동의 전이가 필요한가**다. Fitted Q-iteration은 현재 행동에 기록된 전이를 쓰지만, 다음 상태의 $\max_a Q(s',a)$에서는 기록에 드문 행동까지 평가한다. 자료 coverage

가 좁으면 그 최댓값이 근사 오차에 민감해진다.

7.9 슬라이드 09 · Q-learning의 장점과 위험

Q-learning은 과거 transition을 다시 쓸 수 있고, 다음 상태의 greedy 가치를 목표로 삼는다. 목표 $y = r + \gamma \max Q_\phi(s', a')$가 학습 중 함께 움직이므로 추정 오차가 이전 상태로 전파될 수 있다.

Deadly triad는 함수 근사, 부트스트랩, off-policy 자료가 결합할 때 생기는 불안정을 가리킨다. Replay는 오래된 정책의 자료를 남기고, 부트스트랩은 추정 Q로 새 목표를 만들며, 신경망은 드문 행동에도 값을 낸다. 이후 장치들은 자료 상관, 목표 이동, 탐색 부족, 최대값 편향을 각각 다룬다.

7.10 슬라이드 10 · Deep Q-Network

DQN은 신경망으로 Q값을 근사하는 방법이다. 다음 슬라이드에서 안정적 학습에 필요한 장치를 살펴본다.

7.11 슬라이드 11 · DQN을 지탱하는 세 장치

연속된 게임 화면은 서로 비슷해 minibatch가 편향되기 쉽다. 가치망으로 만든 목표가 매 step 바뀌면 최적화가 불안정하다. greedy 행동만 고르면 아직 보지 않은 좋은 행동을 발견하기 어렵다. DQN은 각각 **experience replay**, **target network**, **ϵ-greedy 탐색**으로 대응한다. 이 셋은 서로 다른 문제를 해결한다.

따라서 세 장치의 역할은 데이터 **순서**, 학습 **목표**, 행동 **coverage**로 나뉜다. 어느 장치를 바꾸는지에 따라 해결되는 병목도 달라진다.

7.12 슬라이드 12 · 연속 transition의 상관

공이 오른쪽으로 움직이는 100프레임을 차례로 학습하면 거의 같은 화면의 기울기가 연속해서 들어온다. 이어 왼쪽으로 움직이는 구간이 시작되면 입력 분포도 갑자기 바뀐다. Buffer에서 두 구간을 섞어 뽑으면 한 minibatch에 서로 다른 움직임이 들어와 인접 프레임의 상관을 줄인다. 아직 방문하지 않은 상태의 전이는 replay에도 없다.

7.13 슬라이드 13 · Replay buffer의 역할

전이 (s, a, r, s', d)를 FIFO buffer $\mathcal{B}$에 저장하고, 그 안에서 무작위 minibatch를 뽑아 여러 차례 학습한다. 경험 한 번으로 여러 업데이트를 할 수 있어 환경 상호작용당 학습량이 증가한다. 동시에 오래된 정책의 자료도 남으므로 off-policy Q-learning과 잘 맞는다.

buffer가 작으면 비슷한 최근 전이가 많이 뽑히고, 크면 오래된 정책의 자료가 오래 남는다. 용량은 자료의 다양성과 최근성 사이의 절충이다.

전이 하나를 저장할 때 s와 s'의 영상, 행동, 보상, 종료 여부가 같은 시각 기준으로 맞아야 한다. 특히 frame stack이 있는 환경은 다음 상태의 stack이 한 프레임 밀리므로 버퍼에서 복원할 때 주의해야 한다. Replay는 경험의 재사용 장치인 동시에 잘못 정렬한 경험을 계속 재사용하게 만드는 장치이기도 하다.

7.14 슬라이드 14 · 움직이는 목표 문제

replay는 입력 표본을 섞지만 Q_ϕ의 목표에 같은 ϕ를 쓰면 파라미터가 변할 때 목표 y_i도 즉시 변한다. 고정된 정답에 회귀하는 감독학습과 다르다. 목표의 급격한 변화를 늦추는 장치가 target network다.

예를 들어 다음 상태의 최대 Q가 5에서 8로 바뀌면 같은 저장 전이의 목표도 3γ 만큼 올라간다. 이 변화를 늦추기 위해 target network를 별도로 유지한다.

7.15 슬라이드 15 · 자기 자신을 쫓는 회귀

예를 들어 진짜 값이 5인데 다음 상태 $Q_\phi(s', a')$를 우연히 8로 추정하면 현 상태의 목표도 크게 잡힌다. 현 상태의 Q를 올리면 다시 다른 이전 상태의 목표가 올라가는 식으로 오차가 전파될 수 있다. replay로 자료 순서를 섞어도 이 부트스트랩 고리는 그대로다. 최적화 대상과 목표 계산망을 잠시 분리해야 한다.

s_2의 Q가 실제보다 3 높으면 s_1의 목표는 약 3γ 커지고, 그 오차가 s_0의 목표로 다시 전달된다. Target network는 목표를 잠시 고정해 이 연쇄의 속도를 늦춘다.

7.16 슬라이드 16 · Target network

온라인 망 Q_ϕ는 매 minibatch 최적화하지만 target 망 $Q_{\bar{\phi}}$는 일정 주기마다 $\bar{\phi} \leftarrow \phi$로 복사한다. 그 사이 목표 $y = r + \gamma(1-d)\max_{a'} Q_{\bar{\phi}}(s', a')$는 target 파라미터에 관해 고정된다. 소프트 업데이트 $\bar{\phi} \leftarrow (1-\tau)\bar{\phi} + \tau\phi$도 쓰지만 원래 DQN의 주기적 복사와 구분해야 한다.

현재 망의 예측값을 목표와 맞추되 target 식으로 기울기가 흐르지 않게 하는 stop-gradient가 구현상 중요하다.

복사 주기가 너무 짧으면 target이 온라인 망을 거의 그대로 따라가고, 너무 길면 오래된 가치로 회귀한다. 소프트 업데이트의 τ도 같은 속도를 연속적으로 조절한다. 구현에서 target 망을 optimizer에 넣지 않고 목표값 계산의 gradient를 끊어야 한다는 두 조건을 함께 확인하자.

7.17 슬라이드 17 · 탐색이 남는다

replay와 target network를 넣어도 한 번도 시도하지 않은 행동의 보상은 알 수 없다. 현재 Q 최대 행동만 고르면 초기에 잘못 높게 추정한 행동을 계속 택할 수 있다. 수집 정책에 의도적인 탐색을 넣어 자료의 행동 다양성을 확보한다.

예를 들어 왼쪽 행동을 한 번도 시도하지 않아 Q를 낮게 초기화했다면 greedy 정책은 계속 오른쪽만 고를 수 있다. 그 결과 replay에는 오른쪽 전이만 쌓여 왼쪽 Q를 고칠 근거가 없다. 탐색은 학습 초기에 **자료를 얻을 행동**을 정하는 문제이기도 하다.

7.18 슬라이드 18 · ϵ-greedy의 확률

$|\mathcal{A}| = m$인 이산 행동에서 확률 ϵ으로 균등 무작위 행동을 택하고 나머지는 greedy 행동을 고른다. greedy 행동의 실제 선택 확률은 $(1-\epsilon) + \epsilon/m$, 다른 각 행동은 ϵ/m이다. 따라서 "ϵ이면 greedy를 절대 선택하지 않는다"는 해석은 틀리다.

초기에는 큰 ϵ으로 탐색하고 학습이 진행되며 줄인다. 평가 때도 탐색 확률을 그대로 둘지 별도로 정의해야 비교가 공정하다. 탐색의 성공은 상태 방문 범위와 보상 구조에 좌우된다.

행동이 4개이고 $\epsilon = 0.2$라면 greedy 행동 확률은 $0.8 + 0.2/4 = 0.85$, 나머지 각 행동 확률은 0.05다. 학습과 평가에서 ϵ을 다르게 두는 이유는 탐색의 효과와 greedy 정책의 성능을 분리하기 위해서다.

7.19 슬라이드 19 · Frame skip과 frame stack

Atari에서 매 frame마다 행동을 바꾸는 것은 필요하지 않을 수 있다. 같은 행동을 몇 frame 반복하면 결정 횟수를 줄이고 보상 지연의 시간 단위를 바꾼다. 반대로 한 장의 화면은 공의 속도 방향을 알려

주지 못한다. 최근 4장을 쌓으면 같은 위치에 있는 공이 왼쪽으로 오는지 오른쪽으로 가는지 구분할 수 있다.

frame skip이 4라면 정책의 한 행동이 게임 내부의 네 프레임에 적용된다. Frame stack 4는 최근 네 관측을 한 입력으로 묶는다. 전자는 **결정 주기**, 후자는 **속도 방향을 추론할 정보량**을 바꾼다. 환경 단계 수를 비교할 때 내부 프레임과 에이전트의 결정 횟수를 구분한다.

7.20 슬라이드 20 · 시각 자료: Atari 화면

여러 Atari 게임에서 DQN의 성능을 비교한 막대그래프다. 과제마다 향상 폭이 다르므로 평균뿐 아니라 개별 게임의 결과도 읽어야 한다.

7.21 슬라이드 21 · DQN 구조를 다시 묶기

DQN은 replay buffer로 연속 표본의 상관을 줄이고, target network로 목표값의 급격한 변화를 늦추며, 탐색 정책으로 새로운 행동을 시도한다.

7.22 슬라이드 22 · DQN을 개선하는 질문

이제 기본 DQN의 가치 과대평가, 학습 목표, 표본 선택과 반환 분포를 개선하는 방법으로 넘어간다.

7.23 슬라이드 23 · 네 가지 개선 축

Double DQN은 **다음 행동의 선택과 평가**를 분리한다. n단계 목표는 **직접 관측한 보상의 개수**를 늘린다. Prioritized replay는 **전이의 샘플 확률**을, C51은 **가치의 표현과 손실**을 바꾼다. 같은 DQN 계열이어도 수정하는 연산의 위치가 다르다.

7.24 슬라이드 24 · 큰 Q와 큰 실제 반환은 같은가

Q가 예측한 값과 실제 실행 반환의 평균을 비교하는 것을 **가치의 calibration**이라고 한다. 같은 초기 상태에서 같은 할인율과 보상 처리로 반환을 계산해야 두 숫자를 직접 비교할 수 있다. 예측 Q가 20이고 평가 점수가 12여도 한쪽은 할인 반환, 다른 쪽은 비할인 episode 합계라면 먼저 정의를 맞춘다.

7.25 슬라이드 25 · 과대평가가 드러나는 경우

Double DQN 연구의 평가 그림이다. Q값이 높게 추정되어도 실제 게임 점수가 함께 높아지는 것은 아닐 수 있다.

7.26 슬라이드 26 · 최대값 편향의 간단한 유도

행동 i의 참 가치를 Q_i, 추정치를 $\widehat{Q}_i = Q_i + \varepsilon_i$라 하자. 두 행동의 참 가치가 모두 0이고 추정 오차 $(\varepsilon_1, \varepsilon_2)$가 $(+1,-1)$ 또는 $(-1,+1)$로 같은 확률로 생기면, 각 $\widehat{Q}_i$의 평균은 0이지만 $\max_i \widehat{Q}_i = 1$은 매번 성립한다. 따라서 $E[\max_i \widehat{Q}_i] = 1 > \max_i E[\widehat{Q}_i] = 0$이다. 일반적으로 max는 볼록하므로 Jensen 부등식으로 $E[\max_i \widehat{Q}_i] \geq \max_i E[\widehat{Q}_i]$다. 같은 Q망으로 다음 행동을 고르고 그 행동의 가치를 평가하는 target에 이 선택 편향이 들어간다.

7.27 슬라이드 27 · Double DQN의 분리

표준 DQN의 다음 상태 항은 $Q_{\bar{\phi}}(s', \arg\max_{a'} Q_{\bar{\phi}}(s', a'))$ 다. Double DQN에서는 학습 중인 온라인 망 Q_ϕ로 다음 행동을 고르고, 고정된 target 망 $Q_{\bar{\phi}}$로 그 행동의 값을 평가한다. $d = 1$이면 현재 전이가 종료되어 미래 가치를 0으로 만든다.

$$y^{\text{DDQN}} = r + \gamma(1-d) Q_{\bar{\phi}}\Big(s', \arg\max_{a'} Q_\phi(s', a')\Big).$$

두 망의 오차가 완전히 독립인 것은 아니지만 선택과 평가가 일치해 생기는 낙관 편향을 줄인다. 이름의 "Double"이 두 개의 완전 독립 Q망을 뜻한다고 외우면 구현을 잘못 이해하기 쉽다. Double DQN 논문

온라인 Q가 A=7, B=6이고 target Q가 A=4, B=5라면 온라인 망은 A를 고르고 target 망은 **A의 값 4**를 목표에 넣는다. 표준 DQN은 target 망에서 직접 최고인 B의 값 5를 쓴다. 두 망의 오차가 다를 때 목표가 어떻게 달라지는지 보인다.

7.28 슬라이드 28 · 다음 개선: 목표의 지평

한 단계 target은 다음 상태 가치에 크게 의존한다. 먼 보상은 Bellman 업데이트를 여러 번 거쳐야 이전 상태까지 전파된다. 실제 보상 몇 개를 더 관측하고 마지막에만 부트스트랩하면 초기의 부정확한 Q에 대한 의존이 줄어든다. 이 생각이 n단계 반환이다.

마지막에만 +10을 받는 5단계 경로에서 한 단계 target은 +10을 한 번에 첫 상태로 가져오지 못한다. 반대로 5단계 목표는 실제 다섯 보상을 한 번에 포함할 수 있다. 보상이 늦게 오는 과업에서 전파가 빨라지는 이유지만, 중간에 수집 정책의 행동이 들어간다는 새 가정도 생긴다.

7.29 슬라이드 29 · n단계 Q 목표

시점 t에서 목표 지평을 n단계로 정하자. $m \le n$은 실제 종료 또는 자료의 끝까지 관측할 수 있는 보상 개수다. 따라서 $r_t, \dots, r_{t+m-1}$을 사용하고, $d_{t+m} = 1$은 마지막 보상 뒤 **진짜 종료**가 일어났음을 뜻한다. $d_{t+m} = 0$이면 수집 구간이 끝났어도 과업은 계속되어 target Q로 미래를 잇는다. $y_t^{(n)}$은 이 자료에서 만든 n단계 회귀 목표다.

$$y_t^{(n)} = \sum_{k=0}^{m-1} \gamma^k r_{t+k} + \gamma^m (1 - d_{t+m}) \max_a Q_{\bar{\phi}}(s_{t+m}, a).$$

진짜 종료가 n보다 이르면 $m < n$이고 부트스트랩은 사라진다. 종료 없이 n개 보상을 얻으면 $m = n$이다. $n = 1$이면 원래 한 단계 Q target이며, n이 커질수록 먼 보상이 목표에 직접 들어온다.

세 단계 보상이 $1, 0, 5$, $\gamma = 0.9$이고 세 단계 뒤 target Q가 4라면 목표는 $1 + 0.9 \cdot 0 + 0.9^2 \cdot 5 + 0.9^3 \cdot 4 = 7.966$이다. 두 번째 단계에서 진짜 종료되었다면 이후의 5와 4는 들어갈 수 없다. 구현에서는 길이 n만 고정해 더하기보다 종료 위치를 먼저 확인해야 한다.

7.30 슬라이드 30 · n단계의 off-policy 주의점

자료 안의 중간 행동 $a_{t+1}, \dots, a_{t+n-1}$은 수집 정책이 실행했다. 마지막에만 greedy 최댓값을 넣는 n단계 목표는 중간 행동도 목표 정책에서 왔다고 가정하는 완전한 off-policy Bellman target과 같지 않다. 정책 차이가 크면 편향이 생긴다. 그럼에도 실무에서는 단순한 n단계 target이 종종 빠른 학습을 준다. 정확한 보정을 원하면 중요도 샘플링 같은 장치가 필요하고 분산 비용이 따른다.

예를 들어 buffer의 기록 정책이 중간 단계에서 안전한 우회 행동을 골랐지만 목표 정책은 곧장 전진했을 상황이라면, 기록된 n 단계 보상은 목표 정책의 정확한 미래가 아니다. 한 단계 Q-learning은 실제 현재 행동 뒤 다음 상태에서 바로 greedy 값을 넣지만, n 단계는 그 사이 기록 행동을 더 따라간다. n을 늘릴 때 생기는 off-policy 편향을 이 경로 차이로 이해할 수 있다.

7.31 슬라이드 31 · 자료를 뽑는 방법도 중요하다

Uniform replay는 모든 저장 전이를 같은 확률로 뽑는다. 벽에 맞아 공의 방향이 바뀌는 드문 전이는 학습 오차가 커도 평범한 프레임과 같은 빈도로 들어온다. Prioritized replay는 TD 오차를 샘플 확률의 근거로 사용한다.

센서 잡음으로 보상이 튄 전이도 TD 오차가 크다. 따라서 우선순위는 학습에 **유용할 가능성**을 나타내는 휴리스틱이고, 다음 슬라이드의 확률적 샘플링과 가중치가 편향을 조절한다.

7.32 슬라이드 32 · Prioritized experience replay

Replay buffer에 N개 전이가 있다고 하자. $\delta_i = y_i - Q_\phi(s_i, a_i)$는 전이 i의 TD 오차, $\varepsilon_0 > 0$은 우선순위가 0이 되지 않게 하는 작은 상수다. 우선순위 $p_i = |\delta_i| + \varepsilon_0$에서 샘플 확률 $P(i) = p_i^\alpha / \sum_j p_j^\alpha$를 만들며, 지수 $\alpha = 0$이면 uniform replay다. 분포 변화의 보정 가중치는 $w_i = (1/(NP(i)))^\beta$다. $\beta = 1$은 평균 손실의 완전 보정이고, 작은 β는 큰 가중치의 분산을 줄인다.

우선순위는 망이 갱신되면 다시 계산해야 한다. 오래된 $|\delta_i|$는 현재 망의 오차를 나타내지 않을 수 있다. PER 논문

두 전이의 우선순위가 9와 1이고 $\alpha = 1$이면 샘플 확률은 각각 0.9와 0.1이다. Uniform이었다면 둘 다 0.5다. 드문 두 번째 전이의 손실을 그대로 평균하면 buffer 전체의 평균과 다른 학습 목표가 되므로, 샘플 확률의 역수에 기반한 중요도 가중치를 사용한다. 그러나 그 가중치가 너무 커지면 다시 분산이 늘 수 있다.

가중치가 필요한 이유를 평균 손실에서 유도하자. ℓ_i를 전이 i의 회귀 손실, $L = N^{-1}\sum_i \ell_i$를 buffer 전체의 균등 평균 손실이라 하자. $i \sim P$는 전이 인덱스를 앞서 정의한 확률 $P(i)$로 뽑는다는 뜻이다. $\beta = 1$의 정확한 보정 가중치 $w_i = 1/(NP(i))$를 곱하면

$$E_{i\sim P}[w_i \ell_i] = \sum_i P(i) \frac{\ell_i}{NP(i)} = \frac{1}{N}\sum_i \ell_i = L.$$

실제 PER의 $\beta < 1$은 이 보정을 일부만 적용해 큰 역확률 가중치의 분산을 줄인다. 이는 **buffer에서 뽑는 확률**의 보정이며, 예전 행동 정책의 자료를 현재 정책의 자료로 바꾸는 보정과는 다른 문제다.

TD 오차가 큰 전이만 계속 뽑으면 더 빨리 배울까? 처음 보는 전이와 작은 오차의 전이도 일정 확률로 뽑혀야 한다. 큰 오차가 관측 잡음 때문일 수도 있고, 우선순위는 마지막 갱신 뒤 오래되어 현재 오차를 반영하지 못할 수도 있다. 확률적 샘플링과 우선순위 갱신이 필요한 이유다.

7.33 슬라이드 33 · 평균 가치만으로 충분한가

같은 기대 반환 5라도 확정적으로 5를 받는 행동과 절반 확률로 0 또는 10을 받는 행동은 위험 수준이 다르다. 평균 Q는 두 행동을 같게 보지만, 반환 분포는 차이를 보존한다. 위험을 싫어하는 로봇이라면 낮은 꼬리를, 높은 보상을 노리는 게임 정책이라면 큰 결과의 가능성을 행동 선택 기준에 넣을 수 있다. 기본 C51은 평균으로 행동을 고른다.

7.34 슬라이드 34 · 반환 분포와 평균의 차이

$Z^\pi(s,a) = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$를 상태–행동에 조건부인 **확률변수**로 보자. 보통의 $Q^\pi(s,a) = E[Z^\pi(s,a)]$는 그 평균 하나다. 확정 5와 0/10 반반은 모두 $Q = 5$지만 분산은 각각 0과 25다. 분포를 알면 여러 결과 가능성을 표현할 수 있다. 다만 표준 greedy 정책은 여전히 평균이 가장 큰 행동을 선택한다. Distributional RL 논문

C51은 이 분포를 고정된 51개 반환값에 대한 확률로 근사한다. 다음 슬라이드에서 분포의 두 종류 불확실성을 구분한다.

7.35 슬라이드 35 · 불확실성의 출처

고정한 (s,a)에서도 이후 환경 전이가 확률적이고, 미래 정책 행동이 확률적이며, 보상 자체도 달라질 수 있다. 이것이 **반환의 환경적 분포**다. 모델 파라미터를 잘 몰라 생기는 추정 불확실성과는 구분해야 한다. C51이 반환 분포를 예측한다고 해서 신경망의 인식론적 불확실성을 자동으로 해결하지는 않는다.

예를 들어 공이 같은 행동 뒤 왼쪽 또는 오른쪽으로 튕기는 무작위성은 더 많은 자료를 모아도 남는다. 반면 특정 화면을 거의 보지 않아 Q망의 값이 불안정한 문제는 관련 자료가 늘면 줄어들 수 있다. C51이 넓게 예측한 반환 분포는 주로 첫 종류를 나타낸다.

7.36 슬라이드 36 · 분포적 Bellman 관계

정책 π를 따를 때 상태 s와 행동 a 뒤에 받을 **할인 반환의 확률분포**를 $Z^\pi(s,a)$라 하자. 그 평균이 앞서 학습한 행동가치 $Q^\pi(s,a) = E[Z^\pi(s,a)]$다. r은 첫 보상, s'은 다음 상태, $a' \sim \pi(\cdot \mid s')$는 다음 행동이다. 반환을 첫 보상과 이후 반환으로 나누면 분포의 Bellman 관계는

$$Z^\pi(s,a) \overset{D}{=} r + \gamma Z^\pi(s',a'), \qquad a' \sim \pi(\cdot \mid s').$$

$\overset{D}{=}$는 양쪽에서 뽑은 값이 늘 같다는 뜻이 아니라 **확률분포가 같다**는 뜻이다. 예를 들어 다음 상태의 반환이 0 또는 10이고 현재 보상이 2, $\gamma = 0.5$라면 현 반환 분포는 2 또는 7로 이동·축소된다. 양쪽의 평균을 취하고 기댓값의 선형성을 쓰면 $Q^\pi(s,a) = E[r + \gamma Q^\pi(s',a') \mid s,a]$가 된다.

이 예에서 다음 반환의 평균은 5이므로 현재 Q값은 $2 + 0.5 \cdot 5 = 4.5$다. 분포식은 그 평균을 만드는 두 결과 2와 7도 보존한다.

7.37 슬라이드 37 · C51의 고정 지지점

C51은 가능한 반환값의 범위 $[V_{\min}, V_{\max}]$에 51개 고정 원자 $z_i = V_{\min} + i\Delta z$를 놓는다. 인덱스 $i = 0, \dots, 50$이고 원자 간격은 $\Delta z = (V_{\max} - V_{\min})/50$이다. 신경망 파라미터 ϕ가 예측하는 $p_{\phi,i}(s,a)$는 상태·행동 (s,a)에서 원자 z_i에 줄 확률이며 $\sum_i p_{\phi,i} = 1$이다. δ_{z_i}를 값 z_i에 모든 질량이 모인 점분포라 쓰면 $Z_\phi(s,a) = \sum_i p_{\phi,i}(s,a)\delta_{z_i}$이고 평균 Q는 $\sum_i z_i p_{\phi,i}(s,a)$다.

학습 대상은 원자의 위치가 아니라 각 위치의 확률이다. 행동 선택은 이 분포들의 평균을 비교한다. C51 원논문

예를 들어 $[V_{\min}, V_{\max}] = [-10, 10]$이면 $\Delta z = 0.4$다. 원자 51개는 행동의 개수가 아니라 **반환값의 표현 해상도**다.

7.38 슬라이드 38 · C51 목표 분포의 투영

다음 상태 s'에서 target 분포의 평균이 가장 큰 행동을 a^*로 고른다. 그 행동의 원자 z_i와 확률 $p_i = p_{\bar{\phi},i}(s', a^*)$를 가져와, 즉시 보상 r과 할인율 γ로 원자 위치를 $r + \gamma z_i$로 옮긴다. $\bar{\phi}$는 목표 계산 동안 고정된 target 망이다. 새 위치가 $[V_{\min}, V_{\max}]$ 밖이면 끝점으로 자르고, 두 원자 사이면 질량을 **거리 비례**로 나눈다. 이 투영 목표와 현재 예측 분포의 교차엔트로피를 줄인다.

예를 들어 지지점이 0과 10이고 옮긴 값이 3이면 질량의 70%를 0, 30%를 10에 둔다. 범위 밖을 clip하면 꼬리 정보가 사라지므로 지지 범위 설정이 중요하다.

왜 그 비율인지 확인하자. 옮겨진 값 $y = r + \gamma z_i$가 인접 지지점 $z_j \le y \le z_{j+1}$ 사이에 있다고 하자. j는 그 아래 원자의 인덱스이고 p_i는 옮겨진 원자가 지닌 원래 확률질량이다. m_j, m_{j+1}을 목표 분포의 두 지지점에 **이 원자 하나가 보태는 질량**이라 두자. 전체 질량과 평균을 보존하려면 $m_j + m_{j+1} = p_i$, $m_j z_j + m_{j+1} z_{j+1} = p_i y$여야 한다. 첫 식에서 $m_{j+1} = p_i - m_j$를 둘째 식에 대입하고 $z_{j+1} - z_j = \Delta z$로 나누면

$$m_j = p_i \frac{z_{j+1} - y}{\Delta z}, \qquad m_{j+1} = p_i \frac{y - z_j}{\Delta z}$$

로 더한다. 두 가중치는 y가 가까운 쪽에 더 많은 질량을 주며 합은 p_i다. 지지 구간 **안에서는** 원자의 기대값도 보존된다. y가 지지 구간 밖이면 끝점으로 잘라 투영하므로 원래 기대값까지 보존되지는 않는다. 여러 다음 원자의 기여를 같은 지지점에 더해 최종 목표 분포를 만든다.

격자 좌표로 C51 투영을 구현하려면? 옮긴 원자 $y_j = \mathrm{clip}(r + \gamma z_j, V_{\min}, V_{\max})$를 격자 좌표 $b_j = (y_j - V_{\min})/\Delta z$로 바꾼다. $l = \lfloor b_j \rfloor, u = \lceil b_j \rceil$이면 확률 p_j를 아래 격자에 $p_j(u - b_j)$, 위 격자에 $p_j(b_j - l)$만큼 더한다. $l = u$이면 전량을 그 원자에 둔다. 범위 밖 원자를 끝점으로 자르면 질량은 보존되지만 원래 기대값은 바뀔 수 있다.

C51이 분포를 예측하면 행동 선택도 위험 회피적으로 바뀌나? 기본 C51의 greedy 행동은 각 행동 분포의 평균 $\sum_i z_i p_i(s, a)$가 가장 큰 쪽이다. 분포 전체를 학습하지만 별도의 위험 기준을 넣지 않는 한 낮은 꼬리를 특별히 피하도록 행동을 고르지는 않는다.

7.39 슬라이드 39 · 분포적 학습이 바꾸는 것

DQN은 $y = r + \gamma \max Q$라는 스칼라 목표를, C51은 보상에 의해 이동·축소한 **확률 분포 목표**를 맞춘다. replay, target network, 탐색 등 DQN의 주요 구성은 그대로 결합할 수 있다. 분포가 다봉형이어도 평균이 같으면 greedy 행동은 같을 수 있으므로 "분포를 예측한다 = 위험 민감 정책을 쓴다"는 등식은 틀리다.

표준 DQN의 손실은 숫자 Q와 숫자 목표의 차이를 회귀한다. C51은 현재 행동의 51개 확률과 투영한 목표 확률의 교차엔트로피를 비교한다. 그래도 탐색, replay, target 망은 여전히 필요하다. 반환 분포라는 새 표현이 데이터 coverage나 목표 이동 문제를 자동으로 없애지는 않는다.

7.40 슬라이드 40 · 개선들의 상호작용

Double DQN, n단계 목표, PER, C51은 각각 독립적으로 정의할 수 있으나 함께 쓰면 효과가 단순 합산되지는 않는다. 예를 들어 n단계 목표는 TD 오차의 분포를 바꾸어 PER의 샘플링 빈도도 바꾼다. 결합 알고리즘을 평가할 때는 구성요소 하나를 제거한 ablation이 필요하다.

PER과 n단계를 함께 쓰면 n단계 목표의 큰 오차를 가진 전이를 우선해서 뽑는다. C51을 쓰면 우선순위로 무엇을 오차로 측정할지도 달라질 수 있다. 한 구성요소가 다른 구성요소의 입력 분포를 바꾸므로, 각 기술의 단독 효과를 합산해 Rainbow의 성능을 예측할 수 없다.

7.41 슬라이드 41 · Rainbow

Rainbow는 Double DQN, multi-step, prioritized replay, distributional RL에 dueling architecture와 noisy networks까지 결합해 Atari에서 평가했다. “무지개”라는 이름은 한 핵심 공식이 아니라 여러 개선의 조합을 가리킨다. 성능을 읽을 때 어떤 전처리·환경 step 기준·평가 seed가 쓰였는지 확인해야 한다. Rainbow 논문

Rainbow의 구성요소 제거 실험을 읽으면 어느 기법이 해당 Atari 설정에서 큰 기여를 했는지 볼 수 있다. 그러나 그 순위를 다른 환경으로 그대로 옮길 수는 없다. 보상이 희소한 게임, 행동이 많은 게임, 화면 변화가 빠른 게임은 각각 병목이 다르기 때문이다. 조합의 성능과 각 요소의 일반적인 우수성을 구분하자.

7.42 슬라이드 42 · 연속 행동으로 넘어가기

다음 장에서는 행동이 연속값일 때 Q값의 최댓값을 어떻게 찾을지 다룬다.

8 05-1 Off-policy Actor–Critic

8.1 슬라이드 01 · 연속 행동을 향하여

연속 행동 공간에서는 모든 행동의 Q값을 비교할 수 없다. DDPG, TD3, SAC는 이 최대화 문제를 actor와 critic으로 다룬다.

8.2 슬라이드 02 · 다룰 알고리즘의 관계

DQN을 복습한 뒤, 연속 행동에서 결정적 actor를 쓰는 DDPG·TD3와 확률적 actor를 쓰는 SAC로 이어진다.

8.3 슬라이드 03 · 정책 개선의 두 경로

정책경사는 Q^π가 주어졌을 때 $\nabla_\theta \log \pi_\theta(a \mid s) A^\pi(s,a)$로 확률을 조정한다. 가치 기반 접근은 Q^π가 큰 행동을 직접 선택해 개선한다. 둘 다 가치의 품질에 의존한다. Q가 잘못되면 actor는 잘못된 행동의 확률을 높이거나 그 행동을 직접 출력할 수 있다.

두 경로 모두 critic의 오차에 민감하지만 오류를 이용하는 방식은 다르다. 확률 정책은 높은 advantage를 가진 표본 행동의 확률을 높이고, 결정적 actor는 Q의 행동 방향 기울기를 따라 직접 움직인다. Q망이 데이터 밖에서 비현실적인 봉우리를 만들었다면 두 번째 방식은 그 봉우리를 더 빠르게 찾아갈 수 있다.

8.4 슬라이드 04 · Bellman 식의 재확인

$Q^\pi(s,a) = E[r + \gamma V^\pi(s') \mid s,a]$이고 $V^\pi(s') = E_{a' \sim \pi} Q^\pi(s',a')$다. 최적 정책의 경우 $V^*(s') = \max_{a'} Q^*(s',a')$로 바뀐다. 따라서 학습할 때 현재 전이의 보상과 다음 상태 가치를 합한 목표를 만든다. 연속 행동에서 어려운 지점은 방정식 자체가 아니라 $\max_{a'}$를 빠르고 안정적으로 근사하는 부분이다.

한 단계 보상이 1이고 다음 상태의 최대 Q가 4, $\gamma = 0.9$라면 최적 Bellman 목표는 4.6이다. DDPG에서 target actor가 낸 행동의 Q가 3이라면 목표는 3.7이다. 이는 actor가 모든 가능한 행동의 진짜 최댓값을 찾았다는 보장이 없기 때문이다. 계산 가능성을 얻는 대신 최대화의 근사 오차를 받아들인다.

8.5 슬라이드 05 · DQN의 안전장치

Replay buffer는 시간적으로 이웃한 표본의 상관을 줄이고 경험을 재사용한다. Target network는 부트스트랩 목표를 잠시 고정한다. ϵ-greedy는 이산 행동의 탐색을 만든다. DDPG는 앞의 두 장치를 가져오지만 연속 행동에서 균등 무작위 행동 하나를 섞는 식의 탐색은 비효율적일 수 있어 actor 출력에 연속 잡음을 더한다.

연속 행동에서 행동 범위가 $[-1,1]^8$라면 균등 무작위로 뽑은 벡터는 actor가 현재 유망하다고 보는 행동에서 멀 수 있다. Actor 출력 주변에 작은 Gaussian 잡음을 더하면 현재 행동을 중심으로 국소 탐색을 한다. 반면 국소 잡음만 쓰면 먼 곳의 좋은 행동을 놓칠 수 있어 잡음 규모와 초기 경험의 다양성을 같이 설계한다.

8.6 슬라이드 06 · DQN 한 사이클

수집 정책으로 상태 s, 행동 a, 보상 r, 다음 상태 s', 진짜 종료 표시 $d \in \{0,1\}$를 replay에 저장한다. Q_ϕ는 학습 중인 Q망, $Q_{\bar{\phi}}$는 목표 계산 동안 고정한 target 망, γ는 할인율이다. Replay 표본마다 Bellman

목표 y를 만들고 $Q_\phi(s,a)$와의 차이를 줄인다.

$$y = r + \gamma(1-d)\max_{a'} Q_{\bar{\phi}}(s', a')$$

이 과정은 수집 당시 정책의 행동 확률이 목표 식에 직접 들어가지 않는다는 점에서 off-policy다. 단, 충분한 행동 coverage가 없으면 좋은 행동의 가치를 배울 표본도 없다.

Replay의 현재 행동 a는 수집 정책의 기록이고, 목표의 $\max_{a'}$는 다음 상태에서 Q가 큰 행동을 택한다. 이 두 선택이 다르기 때문에 과거 자료를 재사용할 수 있다.

8.7 슬라이드 07 · DQN 개선의 역할

Double DQN은 행동 선택과 가치 평가를 분리하고, n단계 목표는 실제 보상을 더 길게 반영한다. Prioritized replay는 TD 오차가 큰 자료를 더 자주 사용하고, 분포적 RL은 반환의 확률분포를 학습한다. 연속 행동 알고리즘도 비슷한 문제를 만나지만 DQN의 해법을 그대로 복사하지는 않는다.

Double DQN과 TD3는 모두 높은 Q 오류를 걱정하지만 방법은 다르다. Double DQN은 하나의 행동을 **고르는 망**과 그 행동을 **평가하는 망**을 분리한다. TD3는 연속 행동에서 actor가 고른 행동을 **두 critic의 작은 값**으로 평가한다. 이름의 유사성보다 목표식에 어느 망이 들어가는지를 봐야 한다.

8.8 슬라이드 08 · 최댓값은 오차를 고른다

두 행동의 참값이 모두 0이고 각각의 추정 오차가 ± 1이라면 max는 양의 오차를 더 자주 선택한다. 일반적으로 $E[\max_a \widehat{Q}_a] \geq \max_a E[\widehat{Q}_a]$다. 정책이 Q가 큰 행동을 적극적으로 찾을수록 critic의 우연한 과대평가를 악용할 위험도 커진다. 이 현상은 연속 행동 actor가 미분을 통해 arg max를 좇을 때 더욱 중요하다.

두 행동의 실제 Q가 5인데 critic의 추정이 4와 9라면 actor는 9라고 믿는 행동 쪽으로 이동한다. 그 행동의 실제 반환을 확인하지 않고 Q만 반복 최적화하면 더 높은 오차를 찾는 셈이 될 수 있다. 연속 공간에서는 아주 좁은 잘못된 봉우리도 기울기를 통해 따라갈 수 있어 target smoothing이 중요해진다.

8.9 슬라이드 09 · Double DQN의 기준

Double DQN은 $a' = \arg\max_a Q_\phi(s', a)$로 행동을 고르고 $Q_{\bar{\phi}}(s', a')$로 평가한다. 두 단계가 같은 추정 오차를 완전히 공유하지 않게 해 최대값 편향을 줄인다. 연속 행동에서는 arg max 자체를 열거하지 못하므로 actor와 **두 critic의 최솟값**을 쓰는 TD3의 다른 형태로 이어진다. 두 방법의 이름에 "double"이 있어도 수식은 같지 않다. Double DQN 논문

온라인 망이 행동 A를 최고로 골랐다고 target 망까지 A를 최고로 평가할 필요는 없다. 두 망이 모두 같은 자료에서 학습해 오차가 상관되어 있어도, 선택한 값과 평가한 값을 완전히 동일한 계산에서 얻는 것보다 낙관 편향을 낮출 수 있다. TD3의 twin critic은 별도의 두 평가기를 두는 다른 구조다.

8.10 슬라이드 10 · 개선책의 한계도 이어진다

Replay와 target을 도입하면 표본 재사용과 목표 안정성이 좋아지지만, 정책이 아예 방문하지 않는 영역의 가치는 알 수 없다. 연속 행동에서도 actor가 높은 Q를 찾아 움직일 때 데이터 지지 밖의 critic 예측을 과신하면 실패한다. 이는 뒤의 offline RL에서 핵심 문제가 된다.

온라인 off-policy 방법은 현재 actor가 불확실한 행동을 시도해 새 데이터를 buffer에 넣을 수 있다. Offline RL에서는 이 교정 경로가 없다. 같은 Bellman 목표와 같은 신경망을 써도 자료를 더 얻을 수 있는지에 따라 critic 오류의 위험이 달라진다. 이 차이를 뒤 장에서 다시 살핀다.

8.11 슬라이드 11 · Q-learning의 정책은 암묵적이다

이산 Q-learning에서는 Q_ϕ만 학습해도 $\pi(s) = \arg\max_a Q_\phi(s,a)$로 정책이 결정된다. 수집 정책은 그와 달라도 된다. 반대로 연속 행동에서는 매 순간 $Q_\phi(s,a)$를 최적화해 행동을 찾는 비용이 커서, 이 최적화 결과를 빠르게 흉내 내는 함수 $\mu_\theta(s)$를 별도로 학습한다.

이산 행동 4개라면 Q망의 네 출력 중 가장 큰 인덱스를 고르면 된다. 연속 행동의 Q망에는 평가할 행동 벡터를 입력해야 하므로 후보 생성이 별도 계산이다. 결정적 actor는 그 후보를 상태에서 바로 만드는 **학습된 최적화 근사기**다.

8.12 슬라이드 12 · DDPG로 넘어가는 단계

연속 행동에서 Q값을 최대화할 행동을 찾는 문제로 넘어간다.

8.13 슬라이드 13 · 연속 행동에서 전수 비교가 막힌다

10개 관절 각각에 100개 후보 토크를 두면 가능한 조합은 100^{10}개다. 각 시점에 전부 평가하는 것은 어렵다. 상태마다 gradient ascent로 $\max_a Q(s,a)$를 풀 수도 있지만 비볼록 Q망에서는 여러 시작점과 반복이 필요하고 국소 최댓값에 멈출 수 있다. Actor $\mu_\theta(s)$는 이 탐색 비용을 학습 단계에 옮겨 실행 시 하나의 행동을 제안한다.

8.14 슬라이드 14 · 결정적 actor로 최대화를 근사

Replay buffer에서 뽑는 상태분포를 $\mathcal{B}$, actor 파라미터를 θ, critic 파라미터를 ϕ라 하자. Critic을 고정하고 $J_{\text{actor}}(\theta) = E_{s\sim\mathcal{B}}[Q_\phi(s,\mu_\theta(s))]$를 최대화한다. 출력 $\mu_\theta(s)$가 critic의 행동 입력으로 이어지므로 자동미분으로 actor를 움직일 수 있다. DDPG 논문

Actor 목표의 평균은 replay에서 뽑은 상태에 대해 계산한다. 각 상태에서 actor가 제안한 $\mu_\theta(s)$의 Q를 묻기 때문에, critic은 저장된 행동 a뿐 아니라 actor가 현재 내는 행동 근처에서도 정확해야 한다. 다음 슬라이드는 그 Q를 actor 파라미터까지 미분하는 경로를 보인다.

8.15 슬라이드 15 · 결정적 정책경사의 연쇄법칙

$\mathcal{B}$를 replay buffer에서 뽑은 상태의 분포라 하고, 학습된 critic Q_ϕ를 고정한 actor의 대리 목적을 $J_{\text{actor}}(\theta) = E_{s\sim\mathcal{B}}[Q_\phi(s,\mu_\theta(s))]$라 두자. $\nabla_a Q_\phi(s,a)$는 행동을 바꿀 때 critic 값이 변하는 방향이고, $D_\theta\mu_\theta(s)$는 actor 파라미터가 행동 출력을 바꾸는 Jacobian이다. 행동이 m차원이고 파라미터가 p개라면 전자는 m차원 열벡터, 후자는 $m \times p$ 행렬이다. $\mathcal{B}$와 ϕ를 고정하고 연쇄법칙을 적용하면

$$\nabla_\theta J_{\text{actor}} = E_{s\sim\mathcal{B}}\Big[(D_\theta\mu_\theta(s))^{\mathsf{T}}\,\nabla_a Q_\phi(s,a)\big|_{a=\mu_\theta(s)}\Big].$$

Critic을 통해 행동의 가치를 직접 미분하므로 actor 기울기의 Monte Carlo 분산은 줄 수 있지만 **critic의 추정 편향**도 직접 전달된다. 이 식의 replay 상태분포 $\mathcal{B}$는 현재 정책이 실제로 방문할 상태분포와 다를 수 있다.

전치 기호 ^{T}는 $m \times p$ Jacobian을 $p \times m$으로 바꾼다. 따라서 오른쪽의 행렬–벡터 곱은 왼쪽 $\nabla_\theta J_{\text{actor}}$와 같은 p차원이다.

한 차원 예에서 actor가 $\mu_\theta(s) = \theta$, critic이 $Q_\phi(s,a) = -(a-2)^2$라고 예측하자. $\partial Q/\partial a = -2(a-2)$이고 $\partial\mu/\partial\theta = 1$이므로 $\theta = 0$에서 actor 기울기는 4다. 경사 상승은 행동을 2에 가깝게 움직인다. 하지만 실제 환경의 최적 행동이 2라는 증명은 아니다. **학습된 critic이 그렇게 예측했다**는 뜻이며, 데이터가 거의 없는 $a = 2$ 근처라면 이 기울기는 특히 의심해야 한다.

8.16 슬라이드 16 · 두 target 망의 소프트 갱신

DDPG는 critic target $\bar{\phi}$와 actor target $\bar{\theta}$를 모두 둔다. 매 step $\bar{\phi} \leftarrow \rho\bar{\phi} + (1-\rho)\phi$, $\bar{\theta} \leftarrow \rho\bar{\theta} + (1-\rho)\theta$로 천천히 따라오게 한다. ρ가 1에 가까울수록 목표 변화가 느리다. 다른 표기에서 $\tau = 1-\rho$를 쓰므로 코드의 숫자가 어느 방식인지 확인해야 한다.

$\rho = 0.995$라면 새 온라인 파라미터의 0.5%만 target에 반영한다. 한 번의 갱신이 target을 크게 바꾸지 않으므로 critic의 회귀 목표가 덜 흔들린다. 반면 target이 너무 늦으면 현재 actor의 행동을 오래된 critic이 평가할 수 있다.

8.17 슬라이드 17 · DDPG의 전체 업데이트

η는 자료 수집 때 actor 행동에 더하는 탐색 잡음이다. 전이 (s, a, r, s', d)를 저장한 뒤, 진짜 종료를 뜻하는 $d = 1$이면 미래 항을 제거한다. $\bar{\theta}, \bar{\phi}$는 갱신을 늦춘 target actor·critic 파라미터다. Critic의 회귀 목표는

$$y = r + \gamma(1-d)Q_{\bar{\phi}}(s', \mu_{\bar{\theta}}(s')).$$

탐색 잡음 η가 붙은 실제 행동 $a = \mu_\theta(s) + \eta$는 replay에 저장되어 critic의 현재 예측 $Q_\phi(s, a)$에 들어간다. 반면 회귀 목표의 다음 행동은 target actor의 $\mu_{\bar{\theta}}(s')$다. $Q_\phi(s, a)$를 y에 회귀한 뒤 actor가 $Q_\phi(s, \mu_\theta(s))$를 높이고, 두 target 망을 느리게 갱신한다. DDPG 논문

8.18 슬라이드 18 · DQN에서 DDPG로 바뀐 자리

이산 DQN의 ϵ-greedy는 연속 행동 탐색 잡음으로, $\max_a Q(s', a)$는 target actor 출력 $\mu_{\bar{\theta}}(s')$로 바뀐다. Replay와 critic target은 유지되고 actor의 정책 개선 단계가 추가된다. "DQN의 연속 버전"이라는 요약은 이 대응을 뜻하지만 결정적 actor에 대한 새 가정과 오차를 숨기지는 못한다.

8.19 슬라이드 19 · DDPG의 실용적 한계

결정적 정책이므로 별도 탐색 잡음 없이는 거의 같은 행동만 반복한다. Critic이 우연히 높은 값을 예측하는 행동을 actor가 학습하면 오차가 증폭될 수 있다. 행동 스케일, 잡음 크기, target 갱신 속도와 학습률에 민감하다. 재현 실험에서는 같은 seed 하나의 최고점보다 여러 seed의 곡선을 살펴야 한다.

실험에서 critic 예측 Q만 계속 높고 rollout 반환이 떨어진다면 actor가 critic 오류를 이용하는지 살펴야 한다. 탐색 잡음을 너무 작게 잡으면 필요한 행동을 못 만나고, 너무 크게 잡으면 대부분 낮은 품질의 전이를 모을 수 있다. DDPG의 민감도는 하나의 학습률보다 **자료 범위와 가치 추정의 상호작용**에서 생긴다.

8.20 슬라이드 20 · TD3가 해결할 세 문제

DDPG의 Q 과대평가와 정책 불안정을 줄이기 위한 TD3의 세 장치를 살펴본다.

8.21 슬라이드 21 · TD3의 세 안정화 장치

두 critic Q_{ϕ_1}, Q_{ϕ_2}를 따로 학습하고 느리게 갱신하는 target critic을 $Q_{\bar{\phi}_1}, Q_{\bar{\phi}_2}$라 하자. $\mu_{\bar{\theta}}$는 target actor다. $\xi \sim \mathcal{N}(0, \sigma^2_{\text{targ}} I)$는 target 행동에만 더하는 Gaussian 잡음이다. σ_{targ}는 좌표별 표준편차이고 I는 행동 차원의 항등행렬이므로 좌표 간 잡음의 공분산은 0이다. $c > 0$은 잡음의 좌표별 절댓값 상한, $[a_{\min}, a_{\max}]$는 가능한 행동 범위다. $\text{clip}(x, l, u)$는 x를 좌표별로 $[l, u]$ 안으로 자르는 함수다. Replay 전이 (s, a, r, s', d)의 종료 표시 d까지 넣으면

$$\tilde{a}' = \mathrm{clip}\big(\mu_{\bar{\theta}}(s') + \mathrm{clip}(\xi, -c, c), a_{\min}, a_{\max}\big), \qquad y = r + \gamma(1-d)\min_{i\in\{1,2\}} Q_{\bar{\phi}_i}(s', \tilde{a}').$$

Actor와 target 망은 critic보다 드물게 갱신한다.

최솟값은 과대평가를 억제하지만 지나치면 과소평가를 만들 수 있다. Target 잡음은 자료 수집용 탐색 잡음과 목적이 다르다. 전자는 **목표 Q의 날카로운 봉우리**를 완화하고 후자는 실제 새로운 상태–행동을 방문하게 한다. TD3 논문

자료 수집 잡음은 실제 환경에서 다양한 (s, a) 전이를 얻기 위해 actor 행동에 붙인다. Target policy smoothing 잡음은 이미 저장된 전이의 **다음 상태 target 행동**에만 붙여 Q 목표를 계산한다. 두 잡음의 크기를 코드에서 섞으면 탐색 범위와 critic의 보수성이 동시에 바뀌어 실험 원인을 해석하기 어렵다.

8.22 슬라이드 22 · 두 critic이 주는 보수성

한 critic이 특정 행동을 12, 다른 critic이 5로 예측한다면 TD3는 다음 상태 목표에 5를 사용한다. 실제 가치가 6이면 큰 과대평가를 피하지만, 실제 가치가 10이면 5만큼 과소평가한다. **두 추정의 최솟값이 참값의 하한이라는 보장은 없다.** 서로 다른 초기화와 minibatch로 두 오류가 같아지는 일을 줄일 수 있으나, 두 망은 같은 환경 자료에 의존한다.

Double DQN과 비교하면 둘 다 큰 Q를 무비판적으로 쓰지 않지만 계산 위치가 다르다. Double DQN은 **한 온라인 망으로 행동을 고르고 target 망으로 그 행동을 평가**한다. TD3는 target actor가 낸 연속 행동을 **두 target critic에 모두 넣어 작은 평가값**을 쓴다. 이 구분은 행동 선택 오류와 가치 평가 오류가 알고리즘에서 어디에 남는지 보여 준다.

8.23 슬라이드 23 · TD3 학습 흐름

TD3는 두 critic 중 작은 Q값을 쓰고, target action에 잡음을 더하며, actor 갱신을 늦춘다. 이 그림은 세 장치가 DDPG 업데이트에 들어가는 위치를 정리한다.

8.24 슬라이드 24 · 성능 그래프를 해석할 때

TD3 논문의 성능 비교 그래프다. 여러 환경과 반복 실험을 함께 봐야 단일 곡선의 차이를 학습 안정성으로 해석할 수 있다.

8.25 슬라이드 25 · TD3 요약

TD3는 deterministic actor와 replay를 유지하는 DDPG의 개선형이다. Twin critic, delayed policy update, target policy smoothing이 이름의 세 핵심이다. 적용은 연속 행동에서 자연스럽지만 관측, 보상, 탐색 잡음, 행동 경계가 달라지면 하이퍼파라미터를 다시 확인해야 한다.

TD3의 장치를 표로 요약하면 현재 행동은 deterministic actor가 내고, 자료 수집 때는 탐색 잡음을 더하며, target 행동에는 별도의 smoothing noise를 더하고, 두 Q 중 작은 값으로 목표를 만든다. 이 네 위치의 행동을 각각 기록해 보면 구현에서 noise를 잘못된 위치에 넣는 실수를 예방할 수 있다.

8.26 슬라이드 26 · 확률적 actor라는 다른 선택

다음은 확률적 actor와 엔트로피 보너스를 사용하는 SAC다.

8.27 슬라이드 27 · 평균과 분산을 모두 배우기

상태 s에서 actor 파라미터 θ가 출력하는 $\mu_\theta(s)$와 $\sigma_\theta(s) > 0$는 각각 Gaussian의 평균과 표준편차다. 먼저 **압축 전** 변수 $U \mid s \sim \mathcal{N}(\mu_\theta(s), \sigma_\theta(s)^2)$를 뽑는다. σ가 작으면 평균 근처에 집중하고, 크면 여러 행동을 시험한다. 행동 범위가 $[-1, 1]$이면 $a = \tanh U$로 압축한다. 따라서 실제 행동 a의 분포 $\pi_\theta(a \mid s)$는 Gaussian 자체가 아니다. 로그밀도를 구할 때 이 변환의 Jacobian을 반영해야 한다. SAC 논문

한 행동 좌표에서 표준정규 잡음 $\varepsilon \sim \mathcal{N}(0,1)$를 이용해 $u = \mu_\theta(s) + \sigma_\theta(s)\varepsilon$로 표본을 만들자. $p_U(u \mid s)$를 압축 전 변수 U의 밀도라 하면, 확률질량 보존 $\pi_\theta(a \mid s)\,da = p_U(u \mid s)\,du$에서 $\pi_\theta(a \mid s) = p_U(u \mid s)/|da/du|$다. $a = \tanh u$이고 $da/du = 1 - \tanh^2 u$이므로

$$\log \pi_\theta(a \mid s) = \log p_U(u \mid s) - \log(1 - \tanh^2 u).$$

여러 행동 차원에서는 좌표별 보정항을 합한다. u가 매우 크면 a가 ± 1 근처에 몰려 수치 오차가 커지므로, 구현은 이 항을 안정적인 형태로 계산한다. SAC 원논문

SAC의 확률 정책은 항상 두 개의 행동 모드를 표현할까? 일반적인 단일 Gaussian 정책은 한 중심 주변의 퍼짐을 나타낸다. 서로 떨어진 두 행동 경로를 정확히 표현하려면 mixture나 더 유연한 정책 분포가 필요할 수 있다. Entropy가 높다는 사실만으로 다봉 표현이 된다고 볼 수 없다.

8.28 슬라이드 28 · 최대 엔트로피 목적

여기서 $r(s_t, a_t)$는 시점 t의 보상, $\gamma \in [0, 1)$는 할인율, E_π는 정책 π로 만든 경로에 대한 기댓값이다. 상태 s의 정책 엔트로피 $H(\pi(\cdot \mid s)) = E_{a \sim \pi(\cdot|s)}[-\log \pi(a \mid s)]$는 행동의 퍼짐을 측정한다. 온도 $\alpha \geq 0$로 보상과 엔트로피의 비중을 정하면

$$J_{\text{soft}}(\pi) = E_\pi\left[\sum_t \gamma^t \{r(s_t, a_t) + \alpha H(\pi(\cdot \mid s_t))\}\right]$$

를 최대화한다. α가 크면 여러 행동을 유지하려는 압력이 강하다. 연속분포의 미분 엔트로피는 음수가 될 수 있으므로 그 숫자를 이산 엔트로피와 같은 범위로 해석하지 않는다.

두 행동이 같은 보상인데 정책이 항상 A만 고른다면 보상 항은 높아도 엔트로피는 낮다. $\alpha > 0$인 soft 목표는 B에도 일부 확률을 주는 정책을 선호할 수 있다. 이 선택은 탐색 가능성을 남기지만, 실제 배포에서 B가 위험하다면 별도의 안전 제약이 필요하다. 엔트로피는 위험의 무조건적 해답이 아니다.

한 상태에서 행동 A와 B의 soft Q가 각각 2와 0이라고 하자. 고정된 Q에 대해 $\sum_a \pi(a)Q(a) + \alpha H(\pi)$를 최대화하면 $\pi(a) \propto \exp(Q(a)/\alpha)$가 된다. $\alpha = 1$이면 A의 확률은 $e^2/(e^2+1) \approx 0.881$이고 B도 약 0.119를 받는다. $\alpha \to 0$에서는 거의 최댓값 행동만 선택하고, α가 매우 크면 두 행동이 비슷한 확률이 된다. 이 예는 온도가 **가치 차이를 얼마나 무작위 행동으로 희석할지** 정함을 보여 준다.

Boltzmann 형태는 엔트로피 항을 미분하면 나온다. 한 상태에서 $p_a = \pi(a \mid s)$, $Q_a = Q(s, a)$라 쓰고 $\sum_a p_a = 1$을 승수 c로 강제한다. $\alpha > 0$일 때

$$\begin{aligned} F(p, c) &= \sum_a p_a Q_a - \alpha \sum_a p_a \log p_a + c\left(\sum_a p_a - 1\right), \\ 0 = \frac{\partial F}{\partial p_a} &= Q_a - \alpha(\log p_a + 1) + c, \\ p_a &= \frac{\exp(Q_a/\alpha)}{\sum_b \exp(Q_b/\alpha)}. \end{aligned}$$

α로 나눈 Q 차이가 행동 확률의 비율을 정한다: $p_a/p_b = \exp\{(Q_a - Q_b)/\alpha\}$. 모든 Q에 같은 상수를 더해도 이 비율은 바뀌지 않는다. 연속 행동에서는 합 대신 적분과 기준 측도가 필요하며, 구현의 파라미터화된 actor가 항상 이 이상적 분포를 정확히 표현하는 것은 아니다.

8.29 슬라이드 29 · 무작위성의 효용과 비용

두 개의 서로 다른 경로가 모두 성공한다면 확률적 정책은 한 경로에 조기 고착하지 않고 둘 다 탐색할 수 있다. 환경의 마찰이 바뀌었을 때 선택지를 남겨 둔 정책이 더 빨리 적응할 가능성도 있다. 그러나 실제 제어에서 무작위 행동 자체가 안전하지 않을 수 있고, 높은 엔트로피가 항상 높은 최종 보상을 의미하지 않는다. α와 안전 제약을 맥락에 맞게 정해야 한다.

평가에서는 같은 상태에서 행동을 샘플할지, 평균 또는 최빈 행동을 사용할지도 결과에 영향을 준다. 연속 정책의 평균은 tanh 압축 전후에서 달라질 수 있으므로 실제 환경에 전달된 행동의 분포를 기준으로 비교한다.

8.30 슬라이드 30 · Soft Bellman 식

현재 정책 π의 soft 행동가치 $Q^{\pi}_{\text{soft}}(s,a)$는 상태 s에서 행동 a를 한 뒤 받을 보상과 미래 엔트로피 보너스의 기댓값이다. 다음 상태 s'에서 $a' \sim \pi(\cdot \mid s')$를 뽑고 그 다양성까지 평가한 상태가치를 $V^{\pi}_{\text{soft}}(s') = E_{a'\sim\pi(\cdot|s')}[Q^{\pi}_{\text{soft}}(s',a') - \alpha \log \pi(a' \mid s')]$로 정의한다. 전이와 보상의 확률성에 대한 조건부 기댓값을 취하면

$$Q^{\pi}_{\text{soft}}(s,a) = E[r + \gamma V^{\pi}_{\text{soft}}(s') \mid s,a].$$

$-\log\pi$의 정책 평균이 엔트로피이므로 V^{π}_{soft}에는 다음 상태의 보상 전망과 행동 다양성이 함께 들어간다. $\alpha = 0$이면 엔트로피 항이 사라져 일반 정책 평가식으로 돌아간다.

8.31 슬라이드 31 · SAC의 critic과 actor 목표

Replay buffer $\mathcal{B}$의 한 전이를 (s,a,r,s',d)라 하자. $d = 1$은 종료를 뜻한다. Q_{ϕ_1}, Q_{ϕ_2}는 두 critic, $Q_{\bar{\phi}_i}$는 각 critic의 느리게 갱신하는 target 망이다. 다음 행동 $a' \sim \pi_\theta(\cdot \mid s')$를 뽑아 $y = r + \gamma(1-d)[\min_{i\in\{1,2\}} Q_{\bar{\phi}_i}(s',a') - \alpha \log \pi_\theta(a' \mid s')]$를 만든다. y의 경사를 끊고 두 critic은 각각 $(Q_{\phi_i}(s,a) - y)^2$를 줄인다. Actor 파라미터 θ는

$$J_\pi(\theta) = E_{s\sim\mathcal{B},\, a\sim\pi_\theta}[\alpha \log \pi_\theta(a \mid s) - \min_i Q_{\phi_i}(s,a)]$$

를 최소화한다. $-Q$는 높은 가치를 선호하고 $\alpha \log \pi$는 행동 분포의 엔트로피를 유지한다. 실제 SAC 구현에는 tanh 밀도 보정과 target stop-gradient가 중요하다. SAC 논문

행동 차원의 표준정규 벡터 $\varepsilon \sim \mathcal{N}(0, I)$를 뽑고, $f_\theta(s,\varepsilon) = \tanh\{\mu_\theta(s) + \sigma_\theta(s) \odot \varepsilon\}$로 행동을 만들자. $\odot$는 좌표별 곱이다. $Q_{\min}(s,a) = \min_i Q_{\phi_i}(s,a)$라 쓰고 critic을 고정하면, 같은 actor 손실을 s, ε에 대한 기댓값으로 바꿀 수 있다.

$$J_\pi(\theta) = E_{s\sim\mathcal{B},\, \varepsilon\sim\mathcal{N}(0,I)}\left[\alpha \log \pi_\theta(f_\theta(s,\varepsilon) \mid s) - Q_{\min}(s, f_\theta(s,\varepsilon))\right].$$

θ로 미분할 때 로그밀도 항은 파라미터와 행동 f_θ를 통하는 두 경로를 모두 미분한다. $D_\theta f_\theta$를 actor 파라미터에 대한 행동의 Jacobian이라 하면, 가치 항은 연쇄법칙에 따라 $-(D_\theta f_\theta)^{\mathsf{T}} \nabla_a Q_{\min}(s,a)|_{a=f_\theta}$를 준다. 이처럼 표본 행동을 미분 가능하게 표현해야 critic의 행동 기울기가 actor에 전달된다.

8.32 슬라이드 32 · 온도 α를 자동 조절

목표 엔트로피 $\bar{H}$ 이상을 유지한다는 제약을 두면 $\alpha \geq 0$를 그 제약의 라그랑주 승수로 볼 수 있다. 정책 엔트로피가 목표보다 낮으면 α를 높여 퍼짐을 장려하고, 너무 높으면 낮춰 보상을 더 중시하게 한다. 실무 손실은 구현의 부호 관례에 따라 달라지므로 $-\log\pi$의 평균이 목표와 어느 방향으로 다른지 확인해야 한다.

자동 조절 뒤에도 목표 엔트로피의 설정, reward scale, 행동 차원은 남는다. 예를 들어 보상의 숫자 크기를 100배 키우면 같은 α의 상대적 영향이 줄어든다. 자동 온도 조절 연구

부호를 제약식에서 확인할 수 있다. $H(\pi) \geq \bar{H}$를 요구하면 정책 쪽에는 $J_{\text{reward}} + \alpha(H(\pi) - \bar{H})$를 최대화하게 하고, $\alpha \geq 0$는 반대 방향으로 최소화한다. $\partial/\partial\alpha = H(\pi) - \bar{H}$이므로 $H(\pi) < \bar{H}$일 때 α의 경사 하강은 α를 높인다. 실제 구현은 양수를 유지하려고 $\log\alpha$를 학습하기도 한다. 목적을 **최대화하는 actor**와 승수를 **최소화하는 조절기**의 방향을 혼동하지 않으면 손실 부호를 점검할 수 있다.

8.33 슬라이드 33 · SAC와 TD3의 공통 기반

TD3와 SAC는 replay, target critic, 두 Q망의 작은 값이라는 기반을 공유한다. TD3의 actor는 결정적이고, 자료 수집용 잡음과 target smoothing 잡음을 따로 더하며 actor를 지연 갱신한다. SAC의 actor는 분포를 출력하고, 표본 행동의 $\alpha\log\pi$를 critic 목표와 actor 목적에 넣는다. 코드에서는 **자료 수집 행동**, **target 행동**, **actor 업데이트 행동**을 나누어 보면 각 잡음의 역할이 보인다.

8.34 슬라이드 34 · SAC가 유용한 조건

연속 행동에서 고차원 탐색이 필요하고 환경 step이 비쌀 때 경험을 재사용하는 SAC가 강력한 기준선이다. 그러나 보상 설계가 잘못되면 엔트로피를 넣어도 원치 않는 행동을 학습할 수 있다. 분포의 표준편차, 온도, Q 과대평가, 평가 시 평균 행동 사용 여부를 함께 점검한다. "state of the art"는 당시 비교 환경의 결과로 읽어야 하며 모든 과제에서의 보편적 순위가 아니다.

배포 시 행동 선택은 학습 중의 샘플링 정책과 별도로 정한다. 매 제어 주기 표본을 뽑는 방식과 일정한 대표 행동을 택하는 방식은 같은 critic과 actor를 사용해도 서로 다른 제어 궤적을 만든다. 논문 점수를 비교할 때 평가 행동의 선택 방식과 episode 초기화를 확인하자.

8.35 슬라이드 35 · TD3와 SAC 실험 비교

그래프에서 두 방법의 수익 곡선은 환경마다 다르고 seed 간 편차도 있다. 동일한 환경 step, 네트워크 규모, 평가 주기, 보상 처리 아래에서 비교해야 알고리즘 효과를 분리할 수 있다. SAC 논문 그림과 TD3 논문 그림을 나란히 놓은 경우 원래 실험 설정이 동일하다고 가정해서는 안 된다. TD3, SAC

동일한 학습 곡선이라도 첫 10만 환경 단계에서 빨리 좋아지는 방법과 마지막에 높은 점수를 얻는 방법은 장점이 다르다. 평균 곡선 아래의 변동 폭이 크면 실험 재현성이 낮을 수 있다. 목표 점수 도달 단계 수, 마지막 성능, 여러 seed의 변동을 따로 보고 판단해야 한다.

8.36 슬라이드 36 · 지금까지의 학습 지형

방법을 데이터의 흐름으로 정리해 보자. BC는 전문가 시연을 복제하고, PPO는 현재 정책에서 새 rollout을 모으며, DQN·TD3·SAC는 과거 replay를 재사용하면서 새 자료도 얻는다. 다음 장의 offline RL은 **고정된 자료만** 사용한다. 이 조건에서는 critic이 높게 평가한 낯선 행동을 실제로 실행해 확인할 수 없으므로 분포 밖 가치 추정이 중심 문제가 된다.

8.37 슬라이드 37 · 벤치마크로 이어지는 질문

다음 장에서는 알고리즘의 최종 성능과 표본 효율을 비교하는 벤치마크를 살펴본다.

9 05-2 강화학습 벤치마크

9.1 슬라이드 01 · 벤치마크를 읽는 관점

강화학습 알고리즘을 비교할 때 과제, 관측·행동 공간, 학습 경험량과 평가 방법을 함께 본다.

9.2 슬라이드 02 · 최종 성능과 표본 효율

그래프의 **최종 성능**은 충분히 학습한 뒤의 점수이고, **표본 효율**은 같은 점수에 도달하는 데 필요한 환경 상호작용 수다. 알고리즘의 우열은 두 기준에서 다를 수 있다. 곡선은 여러 seed의 변동도 함께 읽어야 한다.

9.3 슬라이드 03 · 벤치마크의 지도

벤치마크는 연속 제어, 이산 게임, 로봇 조작과 언어 조건부 로봇 과제로 나뉜다. 서로 다른 범주의 점수는 직접 비교할 수 없다.

9.4 슬라이드 04 · Gymnasium MuJoCo와 DMC

Gymnasium의 MuJoCo 과제와 DeepMind Control Suite는 물리 시뮬레이션에서 연속 행동을 평가한다. 같은 이름의 동작이라도 관측·보상·시간 제한 설정을 맞춰야 한다.

9.5 슬라이드 05 · Atari의 관측과 예산

Atari는 화면 픽셀 또는 RAM을 보고 이산 행동을 고르는 게임 벤치마크다. frame skip, 학습 프레임 수, 시작 상태가 비교 결과에 영향을 준다.

9.6 슬라이드 06 · 로봇 벤치마크의 폭

robosuite, RLBench, Meta-World 등은 로봇 조작의 서로 다른 작업과 난도를 제공한다. 벤치마크 이름만으로 새 물체·새 배치에 대한 일반화 성능을 알 수는 없다.

9.7 슬라이드 07 · LIBERO와 평생학습

LIBERO는 순차적으로 새 로봇 과업을 학습하며 기존 지식을 유지·전이하는지를 평가한다. 새 과업 성공률과 이전 과업 성능 저하를 함께 본다.

9.8 슬라이드 08 · RoboTwin 2.0의 양팔 과제

RoboTwin 2.0은 양팔 조작 과제를 다양한 시뮬레이션 조건에서 평가한다. 양팔의 협응과 조건 변화에 대한 강건성이 핵심이다.

9.9 슬라이드 09 · SIMPLER: 현실 정책을 시뮬레이션에서 평가

SIMPLER는 실제 로봇 정책을 시뮬레이션에서 평가할 때 현실 성능과의 관련성을 묻는 사례다. 시뮬레이션 점수만 높다고 실제 성공률이 같아지는 것은 아니다.

9.10 슬라이드 10 · RoboArena의 분산형 실기 평가

RoboArena는 여러 현장에서 로봇 정책을 실물로 평가하고 선호·성공 기록을 모은 사례다. 시험 장소와 작업의 다양성이 중요하다.

9.11 슬라이드 11 · FurnitureBench의 재현 가능한 실물 조립

FurnitureBench는 실제 로봇의 가구 조립을 재현 가능한 조건에서 평가한다. 긴 과업과 접촉 조작을 함께 요구한다.

9.12 슬라이드 12 · HumanoidBench의 전신 제어

HumanoidBench는 휴머노이드의 이동과 조작을 평가하는 시뮬레이션 벤치마크다.

9.13 슬라이드 13 · 과제 그림에서 평가 설계로

허들 넘기, 균형 잡기, 물체 밀기 등 다양한 과제의 그림이다. 각 과제의 상태·행동·보상 정의를 확인한 뒤 성능을 비교해야 한다.

10 06 오프라인 강화학습

10.1 슬라이드 01 · 데이터가 고정되면 달라지는 문제

Offline RL은 고정된 데이터셋만으로 보상을 이용해 정책을 개선한다. 데이터 밖 행동의 가치를 잘못 추정하는 문제가 중심이다.

10.2 슬라이드 02 · 이 장의 방법 지도

방법들을 **어느 연산에서 데이터 밖 행동을 통제하는가**로 나눌 수 있다. TD3+BC는 actor를 기록 행동 근처에 묶고, CQL은 critic의 데이터 밖 Q를 낮춘다. Filtered BC는 높은 성과의 궤적을 남기고, AWR·IQL은 데이터 안 행동의 상대적 가치를 가중치로 사용한다. 뒤의 수식은 이 네 위치를 차례로 구체화한다.

10.3 슬라이드 03 · Offline과 online의 자료 흐름

Online RL은 자료를 수집하고 갱신한 뒤 바뀐 정책으로 다시 자료를 모은다. Offline RL은 $\mathcal{D}$가 고정되어 학습 중 행동해도 그 결과가 추가되지 않는다. 학습된 정책을 최종 평가할 때 환경에서 실행할 수는 있지만, 그 평가 자료로 다시 학습했다면 순수 offline 설정이 아니다. 실험 보고서에서 이 경계를 명확히 해야 한다.

온라인 SAC의 replay buffer가 현재 10만 전이를 담고 있어도, 다음 rollout으로 새 actor의 결과를 확인해 넣을 수 있다. Offline SAC에는 그 통로가 없다. 같은 저장 형식과 같은 손실 함수를 써도 **데이터 수집이 멈췄는가**라는 조건 하나가 학습의 위험을 바꾼다.

10.4 슬라이드 04 · 행동 정책과 목표 정책

$\mathcal{D} = \{(s, a, r, s', d)\}$의 행동은 알 수 없거나 혼합된 행동 정책 β에서 나왔다. 학습 목표는 새 정책 π_θ의 $J(\pi) = E_{\tau\sim\pi}[\sum_t \gamma^t r_t]$를 높이는 것이다. 자료의 상태–행동 분포 $d^\beta(s,a)$와 새 정책의 $d^\pi(s,a)$가 다르면, $\mathcal{D}$에서 작은 예측 오차도 실제 정책 아래에서는 크게 작용할 수 있다.

예를 들어 기록 정책 β가 어떤 상태에서 A를 99%, B를 1% 실행했는데 새 정책 π가 B를 90% 실행한다면, B의 소수 표본이 새 정책 성과를 좌우한다. 전체 데이터에서의 작은 Q 회귀 손실은 이 드문 행동의 오차를 가릴 수 있다. **정책이 바뀌는 방향과 기록의 밀도**를 함께 보아야 한다.

10.5 슬라이드 05 · BC와 궤적 연결

BC는 데이터의 각 상태에서 관측된 행동을 흉내 낸다. 예를 들어 데이터에 A→B와 B→C를 성공한 부분 경로가 따로 있으면 가치 기반 offline RL은 A에서 B로 간 뒤 B에서 C로 가는 새 조합을 찾아 A→C 과제를 해결할 수 있다. 이 **stitching**은 각 부분의 전이가 데이터에 충분히 존재하고, 상태 표현이 B를 같은 상태로 인식할 때 가능하다.

나쁜 시도도 그 행동의 결과와 보상을 알려 주므로 가치 학습에 정보가 된다. 반면 A→B와 B→C가 서로 다른 숨은 조건에서 기록됐다면 겉보기의 B가 같아도 연결이 실패할 수 있다. Stitching의 근거는 각 부분 경로의 **공통 상태 표현과 일관된 전이 조건**이다.

10.6 슬라이드 06 · 보지 못한 행동의 함정

데이터에서 상태 s'의 행동이 a_1 근처뿐인데 critic이 먼 a_2에 높은 $Q(s', a_2)$를 예측했다고 하자. 학습 손실은 데이터의 (s', a_2)를 검사하지 못하므로 그 큰 예측을 교정할 근거가 없다. 그런데 actor나

$\max_a Q$가 바로 a_2를 선택해 높은 값을 이전 상태로 전파한다.

예를 들어 실제 Q가 A=5, B=0인데 데이터에는 A만 있고 망은 B를 12로 예측했다고 하자. Actor가 B를 선택하면 다음 상태의 Bellman 목표에도 12가 들어가 이전 상태의 Q까지 높아질 수 있다. **OOD 추정 오류의 선택과 부트스트랩 전파**가 함께 일어난다. Offline RL 개관

10.7 슬라이드 07 · 언제 필요한가

로봇 충돌이나 임상 처치처럼 새 행동을 시험하기 비싸거나 위험한 곳에서 과거 기록이 출발점이 된다. 다만 기록된 행동의 선택 과정까지 이해해야 결과를 해석할 수 있다. 의료 기록에서는 환자의 중증도나 담당자의 판단이 처치 선택과 결과에 함께 영향을 줄 수 있다. 이런 숨은 요인은 기술적인 OOD Q 문제와 별도로 인과적 평가를 어렵게 한다.

10.8 슬라이드 08 · 보통 off-policy Q-learning을 그대로 쓰면

일반 목표 $y = r + \gamma \max_{a'} Q(s', a')$는 기록되지 않은 다음 행동도 최댓값 후보로 삼는다. 앞 예의 B에 Q=12가 매겨지면 현재 전이의 목표는 $r + 12\gamma$로 커진다. 이 값으로 학습한 현재 Q가 또 그 이전 상태의 목표로 들어간다. 고정 데이터에서 **예측 오류가 선택되어 역방향으로 전파**되는 경로다.

10.9 슬라이드 09 · 예측 Q와 실제 반환의 분리

정적 데이터로 SAC를 학습할 때 예측 Q는 오르지만 독립 평가 반환이 개선되지 않는 그래프가 있다. 예를 들어 평균 Q가 50에서 100으로 오르고 실제 반환은 20 근처라면, critic의 과대평가가 누적되었을 가능성을 살펴야 한다. **학습한 가치와 정책의 실제 성과**를 별도 축으로 읽는다. BEAR 연구

10.10 슬라이드 10 · Offline RL의 핵심 긴장

데이터 행동을 그대로 복제하면 BC 수준의 성과를 얻기 쉽지만, 기록에 섞인 나쁜 행동까지 따른다. 더 좋은 행동을 선택하려면 기록된 행동의 품질을 구별하거나 새로운 행동을 평가해야 한다. 후자에서는 critic의 외삽 오차가 커질 수 있으므로 **개선 폭과 데이터 coverage**를 함께 살핀다.

행동이 기록된 행동에 가까워도 그 행동이 이어 만든 상태는 데이터 범위 밖일 수 있다. 따라서 현재 상태에서의 행동 거리와 정책을 실행했을 때의 방문 상태 이동을 각각 점검해야 한다.

10.11 슬라이드 11 · 두 가지 주요 통제 위치

통제 위치는 세 곳이다. **Actor 제약**은 정책을 기록 행동 가까이에 두고, **critic 보수화**는 기록 밖 행동의 Q를 낮추며, **데이터 안 정책 추출**은 기록 행동 중 좋은 쪽을 더 자주 고른다. 각각 새로운 행동으로 이동할 수 있는 범위가 다르므로 단순 BC와 같은 자료에서 비교한다.

10.12 슬라이드 12 · TD3+BC의 목적

고정 데이터셋 $\mathcal{D}$에는 상태–행동 쌍 (s, a)가 있다. $\pi_\theta(s)$는 파라미터 θ인 actor의 행동, $Q_\phi(s, a)$는 파라미터 ϕ인 critic의 가치 예측이다. TD3+BC는 데이터에서 기대한 Q를 높이되 actor 행동과 기록 행동 사이의 유클리드 제곱거리 $\|\cdot\|_2^2$를 벌점으로 준다. 두 항의 상대 비중은 $\lambda > 0$가 정한다.

$$\max_\theta E_{(s,a)\sim\mathcal{D}} \left[\lambda Q_\phi(s, \pi_\theta(s)) - \|\pi_\theta(s) - a\|_2^2 \right].$$

λ가 크면 가치 개선을 더 중시하고 작으면 BC에 가까워진다. 논문의 실제 구현은 Q 크기를 정규화해 두 항의 척도를 맞춘다. 데이터 행동이 무작위적이고 매우 나쁘다면 지나친 제약이 개선을 막지만, 제약이 약하면 OOD Q 오류가 다시 나타난다. TD3+BC 논문

한 상태의 기록 행동이 $a = 0.2$이고 actor 출력이 0.8이라면 제곱거리 벌점은 $(0.8 - 0.2)^2 = 0.36$이다. Actor가 0.3으로 움직이면 벌점은 0.01로 줄지만 Q 개선이 작을 수 있다. λ는 Q의 숫자 단위에 민감하므로, 가치와 행동 거리 항의 크기를 맞추지 않으면 어느 한쪽만 손실을 지배한다.

두 항이 행동을 어느 방향으로 미는지도 직접 미분할 수 있다. 한 데이터 쌍 (s, a)를 고정하고, 그 기록 행동을 $a_{\mathcal{D}} = a$, actor 출력을 $u = \pi_\theta(s)$라 쓰자. $F(u)$는 이 표본의 최대화 목적이고 $\nabla_u F$는 행동 벡터 u에 대한 기울기다.

$$F(u) = \lambda Q_\phi(s, u) - \|u - a_{\mathcal{D}}\|_2^2, \qquad \nabla_u F = \lambda \nabla_u Q_\phi(s, u) - 2(u - a_{\mathcal{D}}).$$

첫 항은 critic이 예측한 가치 상승 방향이고, 둘째 항은 actor를 기록 행동으로 당긴다. Q가 국소적으로 $Q(s, u) = cu + b$인 1차원 예에서는 정지점이 $u^* = a_{\mathcal{D}} + \lambda c/2$다. 따라서 λ가 같아도 critic의 기울기 c가 크면 데이터에서 더 멀리 이동한다. 실제 행동 경계와 비선형 Q에서는 이 정지점이 그대로 최적해가 아니며, actor 파라미터에 대한 기울기는 위 식에 $\nabla_\theta \pi_\theta(s)$를 곱한 연쇄법칙으로 얻는다.

10.13 슬라이드 13 · 정책 제약과 가치 보수화

정책을 데이터에 묶는 방법은 이해하기 쉽지만 모든 상태에서 적절한 거리를 정하기 어렵다. 낮은 품질 데이터와 높은 품질 데이터가 섞였다면 단일 BC 벌점은 좋은 행동으로의 이동도 제한할 수 있다. CQL은 반대로 **Q함수의 학습 목표**를 바꿔 데이터 밖 행동의 매력을 낮춘다.

가치 보수화도 무조건 낮은 값을 만들면 좋은 개선 가능성을 지워 버린다. 목표는 데이터 안 행동과 데이터 밖 행동 사이의 상대적 값에 신뢰 가능한 간격을 주는 것이다.

행동 거리의 의미는 과업마다 다르다. 관절 토크 벡터에서 0.1 차이가 작은 변화일 수 있지만 접촉 직전 그리퍼 닫힘 명령의 0.1 차이는 성공과 실패를 가를 수 있다. 단일 L2 벌점은 이런 중요도를 자동으로 알지 못한다. 정책 제약을 쓸 때 행동 정규화와 차원별 단위를 먼저 확인해야 한다.

10.14 슬라이드 14 · 데이터 밖 Q를 낮추는 이유

Bellman 오차는 기록된 (s, a)에서 계산된다. A 행동만 기록됐다면 B의 Q가 100이어도 직접 교정되지 않는다. CQL은 후보 행동의 높은 Q에 비용을 주고 기록 행동의 Q를 상대적으로 유지해 actor가 추정의 허점으로 이동할 유인을 줄인다. 실제로 좋은 미관측 행동까지 낮출 수 있다는 것이 보수성의 비용이다.

10.15 슬라이드 15 · CQL의 첫 목적식

먼저 $\mathcal{L}_{\text{Bellman}}(Q)$를 기록 전이에 맞춘 critic의 Bellman 회귀 손실이라 하자. $\mathcal{D}$에서 상태 s를 뽑고 제안분포 $\mu(a \mid s)$에서 후보 행동 a를 뽑는다. 후보의 높은 Q에 계수 $\alpha > 0$의 벌점을 더하면

$$\min_Q \; \mathcal{L}_{\text{Bellman}}(Q) + \alpha E_{s \sim \mathcal{D},\, a \sim \mu(\cdot|s)} Q(s, a)$$

로 쓸 수 있다. 최소화하므로 두 번째 항은 μ가 자주 제안하는 행동의 Q를 내린다. 그러나 모든 행동을 함께 내리면 데이터 행동의 가치까지 과하게 낮아질 수 있다. 다음 단계에서 데이터 행동의 Q를 끌어올리는 상대적 항을 넣는다. CQL 논문

후보 분포 μ가 데이터 밖 행동 B를 많이 뽑는다면 $E_{a \sim \mu} Q(s, a)$ 벌점은 B의 값을 낮춘다. 하지만 μ가 데이터 행동만 뽑거나 중요한 OOD 행동을 놓치면 원하는 효과가 약해진다. 그래서 CQL의 구현을 볼 때는 정규화 계수뿐 아니라 **후보 행동을 어떻게 샘플했는가**를 함께 확인해야 한다.

10.16 슬라이드 16 · 데이터 행동과의 상대적 보수성

여기서 $E_{s,a\sim\mu}$는 실제로 $s\sim\mathcal{D}$, $a\sim\mu(\cdot\mid s)$를 차례로 뽑은 기댓값의 약기다. $E_{(s,a)\sim\mathcal{D}}$는 기록된 상태–행동 쌍의 기댓값이다. 두 분포의 Q를 비교하는 CQL 정규화 항은

$$\alpha\left(E_{s,a\sim\mu}Q(s,a)-E_{(s,a)\sim\mathcal{D}}Q(s,a)\right)$$

이다. 최소화하면 후보 행동의 Q는 내리고 **실제 데이터 행동**의 Q는 상대적으로 올린다. "모든 (s,a)에서 학습 Q가 참 Q보다 낮다"는 강한 보장으로 읽으면 안 된다. 논문이 분석하는 보수성은 특정 정책과 데이터 분포 아래의 기대 가치에 관한 조건부 주장이다. α가 너무 크면 실제로 좋은 행동까지 억누를 수 있다.

데이터 행동 Q가 5, 데이터 밖 후보 Q가 8이면 두 평균의 차이 3이 비용으로 들어간다. Critic은 이 격차를 줄이도록 갱신된다. 데이터 행동도 함께 내리는 해보다 후보만 낮추는 해가 상대 항을 더 빨리 줄일 수 있다.

10.17 슬라이드 17 · Log-sum-exp가 나오는 이유

고정 상태 s에서 이산 행동의 제안분포를 $\mu_a=\mu(a\mid s)$, 가치 예측을 $Q_a=Q(s,a)$라 쓰자. $H(\mu)=-\sum_a\mu_a\log\mu_a$는 엔트로피다. 가치가 높은 행동을 선호하면서 분포가 한 행동으로 몰리지 않도록 $E_{a\sim\mu}Q_a+H(\mu)$를 최대화한다. 확률의 합 $\sum_a\mu_a=1$을 제약하면 최적값은

$$\max_{\mu}\{E_\mu Q+H(\mu)\}=\log\sum_a e^{Q(s,a)}.$$

이를 유도하려면 $F(\mu)=\sum_a\mu_aQ_a-\sum_a\mu_a\log\mu_a$에 정규화 제약의 승수 $c(\sum_a\mu_a-1)$를 더한다. μ_a로 미분한 정지 조건은 $Q_a-\log\mu_a-1+c=0$이다. $Z=\sum_b e^{Q_b}$라 정의하면 최적 후보분포는 $\mu_a^*=e^{Q_a}/Z$다. 이때 $\log\mu_a^*=Q_a-\log Z$이므로 $F(\mu^*)=\sum_a\mu_a^*Q_a-\sum_a\mu_a^*(Q_a-\log Z)=\log Z$가 되어 위 식을 얻는다.

두 행동 Q가 2와 0이면 $\log(e^2+e^0)\approx2.13$이다. 데이터 행동의 Q가 2이면 CQL의 차이 항은 약 0.13, Q가 0이면 약 2.13이다. 또한 log-sum-exp를 Q_a로 미분하면 μ_a이므로 큰 Q를 가진 후보 행동일수록 더 큰 하향 압력을 받는다. 연속 행동에서 이 합을 근사할 때는 후보 행동을 뽑은 분포와 밀도를 계산에 반영한다. CQL 원논문

10.18 슬라이드 18 · CQL 학습과 조절 계수

이산 행동에서 $\mathcal{D}(a\mid s)$를 기록 행동의 조건부분포, $\alpha>0$를 보수화 강도로 두면 Bellman 회귀에 다음 정규화 항을 더한다.

$$\mathcal{L}_{\text{CQL}}=\mathcal{L}_{\text{Bellman}}+\alpha E_{s\sim\mathcal{D}}\left[\log\sum_a e^{Q(s,a)}-E_{a\sim\mathcal{D}(\cdot|s)}Q(s,a)\right].$$

연속 행동 SAC와 결합할 때는 적절한 행동 샘플링으로 log-sum-exp를 근사한다. α가 크면 분포 밖 낙관을 줄이지만 정책이 너무 보수적으로 남을 수 있다. 값의 척도와 샘플 수를 함께 조절해야 한다. CQL 논문

연속 행동 공간 $\mathcal{A}$에서는 이산 합에 대응하는 $I(s)=\log\int_{\mathcal{A}}\exp\{Q(s,a)\}\,da$를 계산해야 한다. 제안 밀도 $q(a\mid s)>0$에서 독립적인 후보 $a_1,\dots,a_N$을 뽑는다고 하자. q를 곱하고 나누면 적분 안쪽에 중요도 샘플링 항등식을 적용할 수 있다.

$$\int_{\mathcal{A}} e^{Q(s,a)}\,da = E_{a\sim q}\left[\frac{e^{Q(s,a)}}{q(a\mid s)}\right] \approx \frac{1}{N}\sum_{j=1}^{N}\frac{e^{Q(s,a_j)}}{q(a_j\mid s)}.$$

마지막 합에 로그를 취해 $I(s)$를 근사한다. 특정 행동을 자주 뽑는 제안분포 q를 사용하면서 $1/q$를 빼면 그 행동의 높은 Q를 과도하게 반영한다. 반대로 q가 중요한 영역에 거의 질량을 주지 않으면 몇 개 표본의 가중치가 커져 근사가 불안정하다. 유한 표본에서 **로그를 취한 추정량 자체는 편향**될 수 있으며, 구현의 행동 샘플링·밀도 보정·수치 안정화가 결과에 영향을 준다.

α가 작으면 Bellman 손실이, 크면 보수화 항이 갱신을 지배한다. 두 항의 크기와 정책이 선택한 행동의 기록 데이터 밀도를 함께 보면 보수성의 강도를 해석할 수 있다.

10.19 슬라이드 19 · Offline RL의 평가는 왜 까다로운가

고정 데이터에서 만든 정책을 학습 중 환경에 실행할 수 없다면, 성능 평가에도 별도의 자료나 실험이 필요하다.

10.20 슬라이드 20 · D4RL 데이터셋

D4RL은 고정된 MuJoCo 및 다른 환경 자료를 제공해 offline 알고리즘을 같은 데이터로 비교하게 한다. random, medium, medium-replay, medium-expert 등은 수집 행동의 품질과 혼합 방식이 다르다. Expert 자료가 많을수록 BC가 강한 기준선이 되지만, 다양한 품질의 replay에는 RL의 재가중·stitching 여지가 커질 수 있다.

점수는 보통 환경별 reference score로 정규화된다. 그래서 서로 다른 환경의 raw return이나 버전이 다른 정규화 점수를 단순 합산하면 안 된다. D4RL 논문

같은 Ant 과제에서도 expert 데이터는 높은 반환 행동이 많지만 상태 범위가 좁을 수 있고, medium-replay 데이터는 학습 중 여러 수준의 행동을 포함해 stitching 가능성이 있다. 어느 쪽이 무조건 쉽다고 볼 수 없다. 알고리즘 점수를 읽을 때 데이터셋 이름 뒤에 숨은 수집 정책의 품질과 coverage를 확인해야 한다.

10.21 슬라이드 21 · 데이터의 질과 다양성

D4RL의 여러 데이터 유형에서 BC, TD3+BC, CQL을 비교한 표다. 같은 알고리즘도 데이터의 품질과 범위에 따라 성능이 달라진다.

10.22 슬라이드 22 · OGBench와 목표 조건부 학습

OGBench는 주어진 목표 g를 달성하는 $\pi(a\mid s,g)$를 offline 데이터에서 학습하는 과제를 제공한다. 고정 데이터의 부분 경로를 연결해 보지 못한 시작–목표 조합을 해결하는 능력을 시험한다. 한 목표에서 높은 반환을 내는 정책과 다양한 목표를 처리하는 정책의 평가는 다르다. 데이터의 goal coverage, 성공 판정과 평가 목표 분포를 함께 확인한다. OGBench 논문

목표가 B인 궤적과 목표가 C인 궤적이 따로 있을 때, 같은 중간 상태를 지나면 부분 경로를 이어 새로운 시작–목표 조합을 해결할 수 있다. 그러나 상태가 비슷해 보인다는 이유만으로 연결하면 숨겨진 물체 위치나 속도의 차이 때문에 실패할 수 있다. 목표 조건부 정책의 성과를 볼 때 **보지 못한 목표 조합**과 **새로운 환경 상태**를 따로 평가해야 한다.

10.23 슬라이드 23 · TD3+BC와 CQL을 비교하기

TD3+BC는 actor 손실에서 기록 행동과의 거리를 벌점으로 주고, CQL은 critic의 데이터 밖 행동 값을 낮춘다. 연속 행동의 거리 벌점은 관절별 단위와 정규화에 민감하며, CQL은 정규화 계수 α와 후보 행동의 샘플링 분포에 민감하다. 제약이 너무 강하면 데이터 밖에 있는 좋은 행동도 선택하기 어렵다.

두 방법이 같은 벤치마크에서 비슷한 반환을 얻어도 억제한 오류는 다를 수 있다. 기록 행동의 범위를 바꾸어 실험하고, actor가 고른 행동의 거리와 critic의 Q 예측을 함께 보면 어느 제약이 성과를 좌우했는지 파악할 수 있다.

10.24 슬라이드 24 · 다른 개선 경로의 질문

Filtered BC, AWR, IQL은 **기록된 행동 중 좋은 쪽을 더 자주 선택**하는 방향으로 간다. 데이터 안 행동의 반환이나 상대적 가치를 이용하므로 임의의 새 행동을 Q로 평가하는 부담이 줄어든다. 좋은 행동이 기록 어딘가에 있어야 이 선택이 가능하다.

10.25 슬라이드 25 · OOD 평가가 남는 자리

TD3+BC의 actor 항 $Q(s, \pi(s))$는 $\pi(s)$가 기록 행동과 조금만 달라도 critic의 일반화에 의존한다. CQL 역시 후보 행동의 Q를 계산한다. AWR은 actor 업데이트에서 기록 행동의 로그확률만 학습하므로 이 평가를 피하지만, 정책 개선의 방향은 기록된 행동의 품질에 묶인다.

10.26 슬라이드 26 · 보상을 쓰는 행동복제

다음 방법들은 Q값으로 데이터 밖 행동을 직접 선택하는 대신, 데이터 속 행동을 보상에 따라 다르게 모방한다.

10.27 슬라이드 27 · Filtered BC

각 궤적의 반환 $R(\tau) = \sum_t \gamma^t r_t$를 계산하고 상위 $K\%$ 또는 임계값 η 이상인 궤적만 남긴다. 이어 $\max_\theta E_{(s,a)\sim\widehat{\mathcal{D}}} \log \pi_\theta(a \mid s)$로 모방한다. 반환이 큰 궤적이 반드시 모든 상태에서 최선의 행동으로 구성되지는 않는다. 쉬운 초기 상태를 받은 궤적이 높게 평가되고 어려운 상태에서 잘 대처한 궤적이 버려질 수도 있다.

성공 궤적만 남기면 오히려 나빠질 수도 있나? 가능하다. 남은 자료가 매우 작아지거나 특정 초기 상태만 대표하면 배포 때 다른 상태를 다루지 못한다. 상태 난도와 운의 영향까지 고려하려면 같은 상태에서 행동의 상대적 품질을 추정해야 한다.

쉬운 시작 상태에서 반환 100을 얻은 궤적과 어려운 상태에서 복구해 80을 얻은 궤적이라면, 상위 반환만 남길 때 두 번째 궤적의 복구 행동이 사라질 수 있다. 선별 비율을 낮출수록 행동 라벨은 깨끗해지지만 상태 coverage도 줄어든다. **상태별 기준을 뺀 advantage**로 각 행동의 상대적 품질을 평가하는 다음 방법과 연결된다.

10.28 슬라이드 28 · AWR의 연속적 가중치

AWR은 각 기록 행동의 상대적 우수성을 가중치로 만든다. $R_t = \sum_{k=t}^{T-1} \gamma^{k-t} r_k$는 시점 t 이후의 기록 궤적 반환, $\widehat{V}(s_t)$는 그 반환에 회귀한 상태가치 추정치다. 둘의 차이 $\widehat{A}_t = R_t - \widehat{V}(s_t)$를 advantage 추정치라 한다. π_θ는 학습할 정책, $\eta > 0$는 가중치의 온도다. 기록 전이 $(s_t, a_t) \sim \mathcal{D}$에 대해

$$\max_{\theta} E_{(s_t,a_t)\sim\mathcal{D}}\left[e^{\widehat{A}_t/\eta}\log\pi_\theta(a_t\mid s_t)\right]$$

로 가중 BC를 한다. η가 작으면 높은 advantage에 가중치가 몰리고 커지면 거의 BC가 된다. 보통 폭주를 막기 위해 가중치를 clip한다. 역온도 $\kappa = 1/\eta$로 쓰면 같은 가중치가 $e^{\kappa\widehat{A}}$다. AWR 논문

$\eta = 1$에서 advantage가 2인 행동의 가중치는 $e^2 \approx 7.39$, advantage가 -1인 행동은 $e^{-1} \approx 0.37$이다. 두 행동의 가중치 비가 약 20배가 된다. η가 작아지면 이 차이가 더 커져 소수의 표본만 학습을 지배할 수 있으므로 가중치 clipping이나 정규화가 필요하다.

10.29 슬라이드 29 · KL 제약에서 지수 가중치가 나오는 직관

한 상태에서 기록 행동의 분포를 $\beta(a\mid s)$, 개선할 분포를 $\pi(a\mid s)$라 하자. $Q(s,a)$는 행동가치, $V(s)$는 같은 상태의 기준 가치, $A(s,a) = Q(s,a) - V(s)$는 advantage다. $D_{\mathrm{KL}}(\pi\|\beta) = E_{a\sim\pi}[\log\{\pi(a\mid s)/\beta(a\mid s)\}]$는 새 분포가 기록 분포에서 벗어난 정도다. $\eta > 0$을 이 이탈의 벌점 계수로 두고 $E_\pi Q - \eta D_{\mathrm{KL}}$를 최대화하면

$$\pi_{\mathrm{new}}(a\mid s) \propto \beta(a\mid s)\exp(Q(s,a)/\eta) \propto \beta(a\mid s)\exp(A(s,a)/\eta)$$

가 된다. $V(s)$는 같은 상태의 모든 행동에 공통이라 정규화 상수로 흡수된다. 그래서 데이터 행동을 advantage 지수로 재가중한 지도학습이 이 개선을 근사한다. 실제 AWR은 근사 가치와 제한된 함수 계열을 쓰므로 이 이상적인 식과 완전히 같지 않다.

같은 상태의 행동 A와 B가 기록 정책에서 각각 0.5 확률이고 Q가 4와 6이라면 $\eta = 1$의 이상적 개선 분포는 $e^4 : e^6 = 1 : e^2$ 비율로 B 쪽으로 기운다. 기록 확률 $\beta(a\mid s) = 0$인 행동은 해의 support 밖에 남는다.

지수 가중치의 출처를 한 단계씩 계산해 보자. 상태 s를 고정하고 $\pi_a = \pi(a\mid s)$, $\beta_a = \beta(a\mid s)$, $Q_a = Q(s,a)$라 줄여 쓰자. $F(\pi) = \sum_a \pi_a Q_a - \eta\sum_a \pi_a \log(\pi_a/\beta_a)$를 최대화하고 $\sum_a \pi_a = 1$을 승수 c로 강제한다. π_a에 대한 정지 조건은 $Q_a - \eta[\log(\pi_a/\beta_a) + 1] + c = 0$이다. 이를 풀면 $\pi_a = \beta_a \exp(Q_a/\eta)\exp(c/\eta - 1)$이고, $\sum_a \pi_a = 1$을 대입하면 공통인자는 $1/\sum_{a'} \beta_{a'}\exp(Q_{a'}/\eta)$다. 상태마다 공통인 $V(s)$를 Q에서 빼도 행동 간 비율은 같아져 $e^{A/\eta}$를 사용할 수 있다.

왜 KL의 방향을 $D_{\mathrm{KL}}(\pi\|\beta)$로 쓰는가? 이 방향의 KL은 $\beta(a\mid s) = 0$인 행동에 새 정책이 양의 확률을 주면 무한대가 된다. 따라서 이상적인 해의 support가 기록 정책 안에 남는다. 반대 방향 $D_{\mathrm{KL}}(\beta\|\pi)$는 같은 최적화 해와 같은 support 제약을 주지 않는다.

10.30 슬라이드 30 · AWR의 장점과 경계

Actor는 **데이터에 있는 행동**만 가중해 모방한다. 그러나 AWR의 Monte Carlo 반환 R_t는 현재 행동 뒤에도 수집 정책이 실제로 했던 행동의 결과다. 좋은 현재 행동 다음에 수집 정책이 실패했다면 그 행동에 낮은 가중치가 붙을 수 있다.

두 단계 예를 보자. 현재 행동 A는 보상 0을 주고 다음 상태로 가며, 그 상태의 데이터에는 보상 10인 좋은 행동과 보상 0인 나쁜 행동이 모두 있다. 수집 정책이 좋은 행동을 10%만 골랐다면 $\gamma = 1$에서 A의 수집 정책 기준 기대 반환은 $0.1\cdot 10 + 0.9\cdot 0 = 1$이다. 현재 행동 B는 즉시 2를 받고 종료한다고 하자. A를 실행한 자료의 90%는 반환 0을 기록하므로 AWR에서는 이 사례들이 낮게 가중된다. 드문 반환 10 사례에는 큰 지수 가중치가 붙으므로 A와 B의 **최종 가중치 순위는 온도와 표본 수에 따라 달라진다.** 그러나 다음 상태에서 좋은 기록 행동을 선택하며 이어 갈 수 있다면 A 다음의 반환은 10이 될 수 있다. AWR의 Monte Carlo 가중치와 개선 정책의 미래 가치가 다른 이유이며, IQL은 데이터 안에 있는 좋은 후속 행동을 가치 부트스트랩에 반영하려 한다. 좋은 행동의 데이터가 전혀 없다면 이 개선도 불가능하다.

10.31 슬라이드 31 · SARSA식 데이터 안 부트스트랩

일반 Q 목표의 $\max_{a'} Q(s', a')$는 미관측 행동을 고른다. 연속 데이터 궤적에 실제 다음 행동 $a'_{\mathcal{D}}$가 있다면 $y = r + \gamma Q_{\bar{\phi}}(s', a'_{\mathcal{D}})$로 바꿀 수 있다. 이는 데이터 밖 행동을 평가하지 않는 SARSA식 목표다. 그러나 다음 행동이 나쁜 수집 정책에서 왔다면 그 나쁜 행동의 가치까지 그대로 반영한다. 데이터 안에서 **좋은 행동 쪽으로** 개선할 장치가 필요하다.

다음 상태에 보상 10인 행동과 보상 0인 행동이 모두 기록돼 있는데 수집된 궤적은 0을 택했다고 하자. SARSA식 목표는 그 궤적의 0을 따른다. IQL의 높은 expectile은 기록된 행동들 중 **상대적으로 좋은 후속 행동의 가치**를 부트스트랩에 반영한다.

10.32 슬라이드 32 · 데이터 행동 중 좋은 쪽을 고르기

이론적으로는 데이터 행동 분포 $\beta(a \mid s)$가 지지하는 행동 중 $\max Q(s, a)$를 택하고 싶다. 하지만 연속 상태에서 같은 s의 여러 행동을 충분히 관측하기 어렵고, 표본 최대값은 다시 오차에 민감하다. IQL은 데이터에서 관측한 $Q(s, a)$들의 상태별 **상위 방향**을 추정하는 상태 가치 $V(s)$를 학습한다. 그것이 명시적 OOD 행동 최대화의 대리 역할을 한다.

같은 상태에서 관측된 행동 Q가 $[1, 2, 9]$라면 평균은 4, 표본 최대는 9다. Expectile은 높은 Q를 놓치는 오차에 더 큰 가중치를 주어 두 값 사이의 상위 방향 가치를 얻는다. 높은 Q가 실제로 관측된 행동에서 나왔다는 점이 핵심이다.

10.33 슬라이드 33 · Expectile 가치의 의미

한 상태 s에서 기록 행동 $a \sim \mathcal{D}(\cdot \mid s)$의 $Q(s, a)$를 확률변수로 보자. 일반 제곱오차 $\min_v E[(Q - v)^2]$의 최적값은 이 Q들의 평균이다. IQL은 상태가치 $V(s)$를 평균보다 높은 쪽에 놓기 위해 잔차 $u = Q(s, a) - V(s)$에 비대칭 가중치를 준다. $\mathbf{1}\{u < 0\}$는 $u < 0$이면 1, 아니면 0인 지시함수이고, $\tau \in (0, 1)$는 expectile 수준이다.

$$\mathcal{L}_V = E_{(s,a)\sim\mathcal{D}}\left[|\tau - \mathbf{1}\{u < 0\}|\, u^2\right]$$

를 최소화한다. $\tau > 0.5$이면 V보다 높은 Q에 더 큰 벌점을 주어 V가 데이터 행동 Q분포의 위쪽으로 올라간다. $\tau = 0.9$**가 정확한 상위 10% 분위수라는 뜻은 아니다.** Expectile은 가중 제곱오차의 해이고 quantile은 절댓값 기반의 다른 통계량이다. IQL 논문

잔차 $u \geq 0$에는 τ, $u < 0$에는 $1 - \tau$가 곱해진다. 다음 정지 조건이 비대칭 가중치의 효과를 정량화한다.

한 상태에서 $V(s) = v$ 하나만 최적화한다고 보자. $(x)_+ = \max(x, 0)$는 양의 부분이다. $u = Q - v > 0$일 때 손실의 미분은 $-2\tau(Q - v)$이고 $u < 0$일 때는 $2(1 - \tau)(v - Q)$다. 기록 행동의 조건부분포 $a \sim \mathcal{D}(\cdot \mid s)$로 두 값을 평균낸 뒤 0으로 놓으면

$$\tau E_{a\sim\mathcal{D}(\cdot|s)}[(Q(s, a) - v)_+] = (1 - \tau)E_{a\sim\mathcal{D}(\cdot|s)}[(v - Q(s, a))_+]$$

이다. 왼쪽은 v보다 높은 Q를 **얼마나 놓쳤는가**, 오른쪽은 낮은 Q보다 v를 **얼마나 높게 잡았는가**의 균형이다. $\tau = 0.5$이면 두 편차의 균형점이 평균이 되고, $\tau > 0.5$이면 높은 Q를 놓친 비용이 커져 균형점이 위로 간다. 이 식은 상위 몇 퍼센트를 단순히 골라 평균내는 절차와 다르다.

$\tau = 0.9$**이면 행동의 상위 10%만 선택하거나 90th percentile을 구하는가?** 둘 다 아니다. Expectile은 높은 Q를 낮게 예측할 때 0.9, 낮은 Q를 높게 예측할 때 0.1로 **제곱오차**를 가중해 하나의 상태가치를 맞춘다. Quantile은 잔차의 크기에 선형인 pinball loss를 최소화한다. 같은 0과 10의 반반 분포에서 0.9 expectile은 9지만 0.9 quantile은 10이다. IQL의 τ는 선택할 표본의 비율도 아니다.

10.34 슬라이드 34 · 비대칭 제곱손실 예

$\tau = 0.9$라면 $u > 0$, 즉 예측 V가 관측 Q보다 낮을 때 손실 가중치가 0.9다. $u < 0$이면 가중치는 0.1이다. 높은 Q값을 놓친 경우 더 크게 벌하므로 최적 V가 올라간다. 예를 들어 같은 상태 행동 Q가 0과 10을 같은 확률로 갖는다면 평균은 5지만 $\tau = 0.9$ expectile은 $0.1v^2 + 0.9(10 - v)^2$의 최소점 $v = 9$다. 계산은 $0.2v - 1.8(10 - v) = 0$에서 나온다.

앞 슬라이드의 정지 조건에 같은 두 점 분포를 대입하면 $\tau(10 - v) = (1 - \tau)v$가 되어 $v = 10\tau$다. $\tau = 0.5$이면 평균 5, $\tau = 0.9$이면 9다. 0.9 quantile은 10이므로 expectile과 quantile의 차이도 숫자로 드러난다.

10.35 슬라이드 35 · IQL의 세 단계

IQL의 가치 학습은 세 단계다. 먼저 데이터 행동의 $Q_{\bar{\phi}}(s, a)$에 상위 expectile 회귀를 적용해 상태가치 $V_\psi(s)$를 맞춘다. 다음으로 기록 전이 (s, a, r, s', d)에서 $y = r + \gamma(1 - d)V_\psi(s')$를 만들어 $Q_\phi(s, a)$를 회귀한다. $d = 1$은 종료, $\bar{\phi}$는 target critic 파라미터, ψ는 상태가치망 파라미터다. 마지막으로 $A(s, a) = Q_\phi(s, a) - V_\psi(s)$를 계산하고 $e^{\kappa A(s,a)}$를 가중치로 기록 행동을 모방한다. $\kappa > 0$는 좋은 행동에 가중치를 몰아주는 역온도다. IQL 논문

Q 목표에는 다음 상태의 임의 actor 행동이 들어가지 않는다. 높은 expectile $V(s')$가 **데이터 안에서 좋은 행동이 이어질 때의 가치**를 대리하고, actor도 기록 행동의 가중 BC로 추출한다. τ는 가치의 상위 지향 정도, κ는 정책 추출의 선택 강도를 조절한다. 실제 배포에서는 새 상태를 방문할 수 있으므로 독립적인 rollout 평가가 필요하다.

10.36 슬라이드 36 · 방법들의 공통 기준

D4RL에서 IQL을 비롯한 offline RL 방법을 비교한 결과 표다. 과제와 데이터 유형을 나누어 읽어야 하며 한 숫자로 모든 방법의 우열을 정할 수 없다.

10.37 슬라이드 37 · 고정 자료 학습의 응용

Offline RL은 온라인 시행착오가 비싸거나 위험하고 기존 상호작용 기록이 충분한 분야에서 유용하다.

10.38 슬라이드 38 · 로봇에서 offline 사전학습과 online 미세조정

로봇의 기존 시연과 실행 로그로 정책을 미리 학습한 뒤, 허용되는 범위에서 실제 상호작용으로 조정하는 사례다.

10.39 슬라이드 39 · 핵융합 플라스마 제어

핵융합 플라스마 제어의 사례다. 실제 시스템에서 무작위 탐색을 하기 어렵기 때문에 기록된 운용 데이터를 활용한다.

10.40 슬라이드 40 · 건물 냉각 제어

건물 냉각 제어의 사례다. 기존 운용 기록에서 에너지 사용과 실내 조건의 관계를 학습한다.

10.41 슬라이드 41 · 칩 회로 설계와 PrefixRL

칩 회로 설계에 순차적 의사결정을 적용한 사례다. 기존 설계 기록이 있다면 정책 학습에 활용할 수 있다.

10.42 슬라이드 42 · 전체 정리

고정 데이터에서 정책을 개선하려면 데이터 밖 행동의 가치를 과신하지 않아야 한다. TD3+BC는 행동을 기록 근처에, CQL은 가치 평가를 보수적으로, Filtered BC는 좋은 궤적에, AWR은 높은 advantage 행동에, IQL은 데이터 안 행동의 높은 expectile에 초점을 둔다.

데이터 밖 행동의 결과는 기록에서 직접 확인할 수 없다. 성능을 판단할 때는 수집 정책의 범위, 학습 중 새 자료 유무, 독립 평가 절차를 함께 명시한다. 이 기준은 로봇·산업 제어와 다른 로그 기반 의사결정에도 적용된다.

마지막으로 새로운 고정 데이터 문제를 만나면 네 가지를 먼저 묻자. **누가 자료를 모았는가, 어떤 행동이 빠졌는가, 보상을 어떻게 측정했는가, 새 정책을 어떻게 독립 검증할 수 있는가?** 이 답이 있어야 BC와 가치 기반 방법의 차이, OOD 위험, 개선의 실질적 근거를 판단할 수 있다.

11 07-1. Model-based RL의 기초

Model-based RL은 행동 뒤의 상태와 보상을 예측하는 환경 모델을 의사결정에 이용한다. 모델을 알고 있다면 여러 행동열의 결과를 미리 비교할 수 있고, 모델을 모르면 실제 경험으로 학습해야 한다. 이 장은 모델의 뜻에서 출발해 행동열 최적화, Cross-Entropy Method(CEM), Model-Predictive Control(MPC), 짧은 가상 rollout으로 정책을 학습하는 방법까지 연결한다.

11.1 슬라이드 01 · Model-based RL

Model-based RL은 상태 전이를 예측하는 환경 모델을 계획이나 정책 학습에 사용한다.

11.2 슬라이드 02 · 세 가지 질문

모델의 뜻을 정의하고, 모델을 이용한 planning과 가상 데이터 기반 정책 최적화를 구분한다.

11.3 슬라이드 03 · 환경 모델은 무엇인가

시점 t의 상태 s_t, 행동 a_t, 다음 상태 s_{t+1}, 보상 r_t를 구분하자. 환경 모델은 조건부 분포 $P(s_{t+1}, r_t \mid s_t, a_t)$ 다. 상태 전이 $P(s_{t+1} \mid s_t, a_t)$와 보상 분포 $P(r_t \mid s_t, a_t)$를 따로 모델링할 수도 있다. 정책 $\pi(a_t \mid s_t)$는 **행동을 고르는 법칙**, 환경 모델은 **그 행동의 결과 법칙**이다.

바나나를 통에 던지는 로봇을 생각하자. s_t에는 팔과 바나나의 위치·속도, a_t에는 관절 명령, s_{t+1}에는 움직인 뒤의 위치, r_t에는 통에 들어갔는지가 담긴다. 물리 법칙이 이 전이를 정한다. 신경망 P_ϕ는 그 법칙을 데이터로 근사하는 한 표현일 뿐이다.

영상 한 장을 상태 s_t로 놓아도 될까? 현재 영상에 가려진 물체나 속도가 없다면 다음 영상의 분포가 과거 영상에도 의존할 수 있다. Markov 상태를 얻으려면 최근 관측 이력이나 기억 변수를 포함해야 한다. 모델을 확률적으로 만들어 불확실성을 표현할 수도 있지만, 빠진 정보를 언제나 복원할 수 있는 것은 아니다.

11.4 슬라이드 04 · 모델을 모르는 환경

실제 로봇이나 Gym 환경에서 에이전트는 행동을 넣고 다음 상태를 **관측**할 수 있어도 내부 전이 함수를 읽지는 못할 수 있다. 환경의 step 함수는 결과를 돌려주는 인터페이스이지, 임의의 가상 상태와 행동을 무한히 질의할 수 있는 공개 수학식이 아니다.

예를 들어 로봇이 바나나를 약하게 던졌을 때의 실제 결과 하나만 얻었다면 더 강하게 던졌을 때의 결과는 아직 알 수 없다. Model-based RL은 관측 전이로 모델을 학습하거나, 정확한 시뮬레이터를 확보해 이 반사실적 질문에 답한다.

11.5 슬라이드 05 · 모델을 안다면 가능한 일

정확한 모델이 있으면 현재 상태에서 후보 행동을 가상으로 적용해 여러 미래 궤적을 만들 수 있다. 그중 예측 반환이 높은 행동을 고르는 것이 planning이다. 같은 가상 궤적을 정책 학습용 경험으로 쓰는 것도 가능하다.

예를 들어 현재 위치에서 왼쪽·오른쪽으로 움직인 뒤의 상태를 각각 예측하면 두 선택의 장기 보상을 비교할 수 있다. 예측 모델을 데이터로 배운 경우에는 실제 실행 결과를 새 자료로 삼아 다음 계획의 출발점을 고친다.

11.6 슬라이드 06 · 던지기의 반사실적 비교

로봇이 힘 2와 힘 3으로 던지는 두 행동을 고려한다고 하자. 모델이 착지 지점을 각각 통 앞 0.5m와 통 안쪽 0.1m로 예측한다면 힘 3을 선택할 수 있다. 이 비교는 두 행동을 현실에서 모두 시도하지 않고 수행한다.

힘을 연속값으로 조절할 수 있다면 후보 두 개에서 멈출 필요가 없다. 모델 속에서 던지는 힘과 방향을 바꾸어 착지 오차가 가장 작은 입력을 찾을 수 있다. 이 최적화가 다음의 행동열 계획으로 확장된다.

11.7 슬라이드 07 · 모델의 두 사용법

World model을 $P_\phi(s', r \mid s, a)$라 쓰자. ϕ는 데이터로 학습하는 파라미터이고, s', r은 현재 s, a 뒤의 다음 상태와 보상이다. **Planning**은 이 모델 안에서 행동 후보를 평가한다. **가상 경험 생성**은 모델이 만든 $(s, a, \hat{r}, \hat{s}')$를 정책이나 가치함수의 학습 자료로 이용한다. 모자 기호는 실제 관측이 아닌 예측값을 뜻한다.

앞 방식은 의사결정 시 계산이 들 수 있고, 뒤 방식은 학습 결과를 정책에 저장한다. 같은 모델이라도 어디에 계산을 쓰는지에 따라 장점과 실패 양상이 달라진다.

11.8 슬라이드 08 · Planning으로 전환

먼저 모델 안에서 여러 행동열의 결과를 비교해 현재 행동을 고르는 planning을 살펴본다.

11.9 슬라이드 09 · 행동열의 목적함수

계획 지평 H를 앞으로 고려할 행동 수, $A = (a_t, \dots, a_{t+H-1})$를 후보 행동열이라 하자. 모델로 예측한 상태를 $\hat{s}_t = s_t$, $\hat{s}_{k+1} = f_\phi(\hat{s}_k, a_k)$라 쓰고, 예측 보상을 $\hat{r}_k = r_\phi(\hat{s}_k, a_k)$라 하자. 할인율 $\gamma \in [0, 1]$에서 한 후보의 점수는

$$\widehat{J}_H(A \mid s_t) = \sum_{k=t}^{t+H-1} \gamma^{k-t} \hat{r}_k.$$

바나나를 던질 때는 힘과 방향의 여러 시점 명령이 A가 된다. Go에서는 앞으로 둘 수를 가정해 결과를 평가한다. 두 문제 모두 모델 속에서 행동열을 비교하지만, 행동공간과 모델의 형태는 다르다.

11.10 슬라이드 10 · 전수 검색의 비용

매 시점 가능한 행동이 N개라면 길이 H의 행동열은 N^H개다. $N = 10$, $H = 6$만 해도 백만 후보가 되어 각 후보의 상태 전이를 계산해야 한다. 연속 행동에서는 후보가 무한하므로 전수 검색 자체가 불가능하다.

행동 선택지가 열 개씩만 있어도 H가 한 단계 증가할 때 후보 수는 열 배가 된다. 이 지수적 증가를 피하는 두 접근이 **모델의 기울기로 후보를 수정하는 방법**과 **일부 후보를 표본으로 비교하는 방법**이다.

11.11 슬라이드 11 · 미분 가능한 모델에서 행동을 개선하기

전이 f_ϕ와 보상 r_ϕ가 행동에 대해 미분 가능하면 $\widehat{J}_H$를 행동열 A로 미분해 경사 상승할 수 있다. 두 단계의 단순한 경우 $s_{t+1} = f_\phi(s_t, a_t)$, $J = r_\phi(s_t, a_t) + \gamma r_\phi(s_{t+1}, a_{t+1})$라 놓자. 다음 행동 a_{t+1}을 고정하면 연쇄법칙은

$$\nabla_{a_t} J = \nabla_a r_\phi(s_t, a_t) + \gamma \big(D_a f_\phi(s_t, a_t)\big)^{\mathsf{T}} \nabla_s r_\phi(s_{t+1}, a_{t+1}).$$

$D_a f_\phi$는 행동 변화가 다음 상태를 얼마나 바꾸는지 나타내는 Jacobian이다. 첫 항은 즉시 보상, 둘째 항은 바뀐 다음 상태가 미래 보상에 주는 영향이다. 더 긴 지평에서는 이 경로가 여러 전이를 통과한다.

모델이 미분 가능하면 행동 계획도 항상 잘될까? 접촉이나 충돌에서 예측 함수가 부드럽지 않을 수 있고, 학습 모델의 기울기가 실제 환경의 기울기와 다를 수 있다. 경사는 현재 후보 주변의 국소 정보를 주므로 초기 행동열에 따라 다른 해로 갈 수도 있다.

11.12 슬라이드 12 · 샘플링으로 행동열 비교하기

Random shooting은 M개의 행동열 $A^{(1)}, \dots, A^{(M)}$을 후보분포에서 뽑고, 각 행동열의 $\widehat{J}_H$를 계산해 가장 높은 것을 고른다. 전이 모델의 기울기가 없어도 되며 후보 평가를 병렬화하기 쉽다.

샘플 수 M이 작으면 좋은 행동열을 놓치고, 크면 모델 rollout 계산이 비싸진다. 예를 들어 두 시점의 좌·우 조향은 네 후보뿐이지만, 관절 12개의 연속 토크를 20단계 정하면 균일 표본이 쓸 만한 좁은 영역에 들어갈 확률이 작다.

11.13 슬라이드 13 · 기울기와 샘플링의 절충

기울기 기반 계획은 한 후보에서 어떤 방향이 좋아지는지 이용하므로 고차원 행동열을 다룰 여지가 있다. 대신 미분 가능한 모델과 믿을 만한 기울기가 필요하다. 샘플링은 불연속 보상이나 black-box 모델에도 쓸 수 있고 병렬 계산이 쉽지만 차원이 높아지면 후보가 희박해진다.

행동열 길이 H와 행동 차원 d_a를 곱한 Hd_a가 실제 최적화 변수의 수다. 같은 로봇이라도 지평을 두 배로 늘리면 검색 차원이 두 배가 된다. 연속 제어에서는 이 차원이 커질수록 균일 표본이 유용한 궤적을 찾기 어려워져, 다음 CEM처럼 후보분포를 적응적으로 옮긴다.

11.14 슬라이드 14 · Cross-Entropy Method

CEM은 random shooting의 후보분포를 반복 개선한다. 처음에는 행동열 $A \in \mathbb{R}^{Hd_a}$을 Gaussian $q_0(A)$에서 M개 뽑고, 모델 점수 상위 K개의 **elite**를 남긴다. 반복 j의 elite를 $A_{\mathrm{e}}^{(1)}, \dots, A_{\mathrm{e}}^{(K)}$라 하면 대각 Gaussian의 평균과 분산을

$$\mu_{j+1} = \frac{1}{K} \sum_{i=1}^{K} A_{\mathrm{e}}^{(i)}, \qquad \sigma_{j+1}^2 = \frac{1}{K} \sum_{i=1}^{K} \big(A_{\mathrm{e}}^{(i)} - \mu_{j+1}\big)^{\odot 2}$$

로 다시 맞춘다. $^{\odot 2}$는 좌표별 제곱이다. 새 분포에서 다시 후보를 뽑으면 이전 elite 부근을 더 집중적으로 탐색한다. 이는 미분이 아니라 **좋은 표본의 통계량**으로 분포를 바꾸는 절차다.

CEM의 elite가 한쪽 국소해로 몰리면? 분산이 너무 빨리 줄면 다른 좋은 경로를 더 이상 샘플링하지 못한다. 최소 분산을 두거나 이전 분포와 새 분포를 섞고, 여러 초기화로 비교할 수 있다. 행동 경계 밖 표본은 잘라내거나 적절히 변환해야 한다.

11.15 슬라이드 15 · CEM을 어디에 쓰는가

한 번의 CEM 반복은 후보 M개를 모델로 평가하고 상위 K개로 분포를 다시 맞춘다. 이를 J번 반복하면 대략 JM개 행동열의 모델 rollout이 필요하다. 후보별 평가는 병렬화할 수 있지만, 분포를 갱신하는 반복은 순서대로 진행된다.

고차원에서는 독립 좌표 Gaussian이 좌표 간 협력을 충분히 표현하지 못할 수도 있다. 예를 들어 두 관절을 동시에 맞춰야 하는 접촉 행동은 각 관절을 따로 흔드는 표본보다 상관된 움직임이 중요하다. 계획의 품질과 온라인 지연을 함께 평가한다.

11.16 슬라이드 16 · 정확한 모델이라는 가정

지금까지 후보 행동열의 점수는 모델이 계산했다. 실제 환경의 P와 모델 P_ϕ가 다르면 그 점수는 참 반환이 아니다. 특히 최적화기는 모델이 **잘못 높게 예측한 후보**를 적극적으로 찾을 수 있다.

모델을 모르는 일반적인 상황에서는 실제 전이 데이터를 모아 P_ϕ를 학습한다. 이후 계획으로 방문한 새 상태를 다시 데이터에 넣어 모델의 범위를 넓히는 폐회로가 필요하다.

11.17 슬라이드 17 · 전이 모델을 데이터로 학습하기

실제 경험 집합을 $\mathcal{D} = \{(s_i, a_i, s'_i)\}_{i=1}^{N}$이라 하자. 결정론적 예측 $f_\phi(s,a)$는 다음 상태와의 제곱오차 $\sum_i \|f_\phi(s_i,a_i) - s'_i\|_2^2$를 줄여 학습할 수 있다. 확률 모델 $P_\phi(s' \mid s,a)$는 $\sum_i \log P_\phi(s'_i \mid s_i, a_i)$를 높여 학습한다.

제곱오차가 왜 자연스러운지 유도해 보자. 다음 상태의 차원을 d_s라 하고 아래와 같이 가정하자.

$$P_\phi(s' \mid s,a) = \mathcal{N}\big(f_\phi(s,a), \sigma^2 I\big)\,.$$

σ^2는 모든 좌표에 공통인 고정 잡음 분산이고 I는 단위행렬이다. 이 분포의 음의 로그우도는

$$-\log P_\phi(s' \mid s,a) = \frac{d_s}{2}\log(2\pi\sigma^2) + \frac{\|s' - f_\phi(s,a)\|_2^2}{2\sigma^2}.$$

첫 항과 분모의 σ^2가 고정되어 있으므로 우도 최대화는 제곱오차 최소화와 같다. 실제 전이가 여러 결과를 낳는데 평균 하나만 예측하면 어느 결과도 제대로 나타내지 못할 수 있다. 계획에서 중요한 것은 훈련 전이의 평균 오차뿐 아니라 **계획이 방문할 상태·행동에서의 예측**이다.

11.18 슬라이드 18 · 모델 오류가 계획에 들어오는 경로

모델은 $\mathcal{D}$의 상태·행동에서 학습하지만 계획은 새 후보를 탐색한다. 훈련 분포를 $d_{\mathcal{D}}$, 계획으로 방문할 분포를 d_{plan}이라 하면 두 분포가 다를 수 있다. 그림의 절벽에서는 데이터가 거의 없는 방향의 높이를 모델이 과대평가하자, 계획기가 바로 그 방향을 선택한다.

오류 누적도 보자. 참 결정론적 전이를 f, 학습 모델을 $\hat{f}$라 하고, 각 지점의 한 단계 모델 오차가 $\|\hat{f}(s,a) - f(s,a)\| \le \varepsilon_m$라고 가정하자. 참 전이가 상태에 대해 L-Lipschitz이면 같은 행동열의 k단계 상태 오차 e_k는 $e_0 = 0$, $e_{k+1} \le L e_k + \varepsilon_m$을 만족한다. 따라서

$$e_H \le \varepsilon_m \sum_{j=0}^{H-1} L^j.$$

이 경계는 방문하는 영역 전체에서 오차 가정이 성립할 때만 유효하다. 데이터 밖에서는 ε_m 자체가 클 수 있어 긴 rollout의 예측을 더 조심해야 한다.

11.19 슬라이드 19 · 계획으로 수집한 데이터를 다시 넣기

한 번의 계획 뒤 첫 행동을 실제 환경에서 실행해 (s,a,r,s')를 관측하고 $\mathcal{D}$에 더한다. 그런 다음 전이와 보상 모델을 갱신한다. 계획이 자주 선택하는 상태·행동에 실제 표본이 쌓이므로 학습 분포도

그쪽으로 이동한다.

로봇에서 시험 자체가 위험하면 불확실성이 큰 후보에 벌점을 주거나 안전 제약을 둔다. 이렇게 하면 탐색 가능 범위를 통제하면서 실제 전이가 알려 준 구간에서 모델을 고칠 수 있다.

11.20 슬라이드 20 · 절벽 사례의 교정

처음에는 오른쪽으로 가면 위로 올라갈 수 있다고 모델이 잘못 예측했다고 하자. 계획기가 오른쪽을 선택하고 실제로는 절벽에서 멈춘 기록을 얻으면, 그 전이를 추가한 뒤 모델은 해당 경로의 점수를 낮출 수 있다. 최종 정책이 안전한 위쪽 경로를 선택하는 변화는 **실제 전이로 모델을 고친 결과**다.

절벽 그림의 변화는 **계획 → 실행 → 전이 기록 → 모델 갱신 → 재계획**의 폐회로를 보여 준다. 계획기가 높은 점수를 준 경로에서 실제로 낮은 결과가 관측되면, 다음 반복의 행동열 순위도 바뀐다.

11.21 슬라이드 21 · Open-loop와 closed-loop

Open-loop 계획은 현재 상태에서 행동열 $(a_t, \dots, a_{t+H-1})$을 정하고 관측이 달라져도 그대로 실행한다. Closed-loop 제어는 다음 관측을 받아 남은 계획을 고친다. 모델이 완벽하지 않거나 외부 교란이 있으면 두 방식은 크게 달라진다.

자율주행차가 3초치 조향을 한꺼번에 고정하면 앞차가 갑자기 멈춰도 반응하지 못한다. 매 순간 새 관측에 맞춰 계획을 다시 만들면 늦게 드러난 장애물에 대응할 수 있다.

11.22 슬라이드 22 · Model-Predictive Control

MPC는 현재 상태에서 길이 H의 최적 행동열을 구하되 **첫 행동만 실행**한다. 다음 실제 상태 s_{t+1}을 관측하면 다시 길이 H를 계획한다. 매 시점 전이 (s_t, a_t, r_t, s_{t+1})을 데이터에 추가하고 모델을 갱신할 수도 있다.

재계획은 앞서 예측한 s_{t+1}과 실제 s_{t+1}의 차이를 바로 반영한다. 대신 매 제어 시점에 CEM 같은 최적화를 반복하므로 계산이 비싸다. 제어 주기 안에 답을 내는지와 모델이 얼마나 앞까지 맞는지를 함께 정해야 한다.

재계획하면 모델 오차가 모두 사라질까? 매번 첫 행동의 결과를 새 관측으로 교정하지만, 그 첫 행동을 고르는 데 사용한 미래 예측은 여전히 틀릴 수 있다. 특히 실제로 한 번 실행하면 되돌릴 수 없는 위험한 행동은 다음 시점의 재계획으로 복구할 수 없다.

11.23 슬라이드 23 · Model-based planning의 경계

MPC는 전이 모델과 보상 모델을 이용해 현재 행동을 정한다. 목표 위치가 바뀌어 보상 함수가 달라져도 같은 전이 모델로 새 목적함수를 계획할 수 있다. 이는 환경의 움직임과 과업의 목표를 분리해 둔 이점이다.

계획 지평이 길면 계산량과 예측 오차가 커진다. 다음 장에서는 짧은 지평 뒤의 장기 성과를 가치함수로 요약해 이 문제를 다룬다.

11.24 슬라이드 24 · 정책 학습으로 전환

다음은 모델로 만든 가상 경험을 정책 학습에 사용하는 방법이다.

11.25 슬라이드 25 · 계획 결과를 정책으로 압축하기

계획기가 상태 s에서 고른 첫 행동을 $a_{\text{plan}}(s)$라 하자. 정책 π_θ를 $\mathcal{L}(\theta) = E_s[\|\pi_\theta(s) - a_{\text{plan}}(s)\|^2]$로 학습하면 실행 시 비싼 계획을 한 번의 정책 계산으로 대체할 수 있다. 이는 planner의 행동을 지도학습으로 **distillation**하는 방식이다.

Distillation은 계획에 쓰던 계산을 **학습 시점으로 옮긴다**. 실행은 빨라지지만 정책이 따라 하는 목표의 품질은 계획 지평과 모델의 예측에 의해 정해진다. 긴 과업에서는 끝 상태의 가치를 넣어 planner의 행동부터 개선할 수 있다.

11.26 슬라이드 26 · 짧은 가상 rollout을 어디서 시작할까

모델에서 긴 궤적을 한 번에 만들면 예측 오차가 쌓인다. 초기 상태에서만 짧게 시작하면 실제 정책이 나중에 방문하는 상태를 충분히 다루지 못한다. 실제 replay의 **여러 시점 상태**를 시작점으로 뽑아 각각 짧은 rollout을 만들면 두 문제를 절충할 수 있다.

예를 들어 실제 50단계 궤적의 10, 25, 40번째 상태에서 각각 5단계만 가상으로 뻗는다. 초기 상태 하나에서 50단계를 예측하는 경우보다 각 가상 궤적이 실제 관측에 가까운 곳에서 출발한다. 학습기는 접근·접촉·완료처럼 서로 다른 진행 단계의 후속 결과를 보게 된다.

11.27 슬라이드 27 · 환경 자료와 모델 자료의 순환

실제 전이 집합을 $\mathcal{D}_{\text{env}}$, 모델이 만든 전이 집합을 $\mathcal{D}_{\text{model}}$이라 하자. 현재 정책 π_θ로 현실에서 데이터를 모아 전자를 갱신하고, 그 데이터로 모델 P_ϕ를 학습한다. 실제 상태를 시작점으로 모델 속에서 π_θ를 실행해 후자를 만든 뒤, actor와 critic을 가상 자료로 갱신한다.

가상 전이는 실제 환경을 다시 호출하지 않고 업데이트용 표본을 늘린다. 그러나 critic의 목표 $r + \gamma V(s')$에 예측 $\hat{r}, \hat{s}'$를 넣으면 보상 오차와 상태 오차가 함께 전파된다. 실제 자료를 모델 교정의 기준으로 유지하고, 모델 자료의 지평과 비중을 조절하는 이유다.

11.28 슬라이드 28 · 언제 모델이 유리한가

상태 전이가 비교적 규칙적이고 실제 상호작용이 비싼 로봇 제어에서는 모델 학습의 비용을 여러 가상 시도에 나누어 쓸 수 있다. 전이 모델은 보상 라벨 없이도 연속 관측으로 학습할 수 있고, 목표 보상이 달라져도 일부 재사용할 수 있다.

모델 학습의 비용은 예측 대상에 달려 있다. 픽셀 전체를 복원하면 배경·조명까지 맞혀야 하지만, 제어에는 물체 위치와 접촉 같은 일부 특징만 중요할 수 있다. 뒤의 TD-MPC는 이 차이에 맞춰 보상과 가치에 필요한 잠재 동역학을 배운다.

11.29 슬라이드 29 · 두 경로의 정리

Model-based RL은 참 모델 또는 학습 모델로 후보 행동의 미래를 평가한다. Planning은 행동열을 경사나 CEM으로 최적화하고 MPC로 매 시점 재계획한다. Policy optimization은 모델이 만든 짧은 가상 경험으로 actor와 critic을 갱신한다.

계산의 위치가 두 경로를 구분한다. Dreamer는 잠재공간의 가상 경험으로 실행할 정책을 학습하고, TD-MPC는 짧은 잠재 계획에 terminal value를 붙여 현재 행동을 고른다. 두 알고리즘을 이 기준으로 다음 장에서 비교한다.

11.30 슬라이드 30 · 다음 단계

다음 장에서는 잠재 세계모델의 Dreamer와 가치 기반 계획을 결합한 TD-MPC를 살펴본다.

12 07-2. Dreamer와 TD-MPC

환경 모델을 배우는 목적은 미래 영상을 완벽히 복사하는 데 있지 않다. 정책을 개선하거나 현재 행동을 고르는 데 충분한 미래 정보를 얻는 데 있다. Dreamer는 관측을 잠재 상태로 압축해 그 안에서 actor와 critic을 학습한다. TD-MPC는 과업에 중요한 잠재 동역학을 배우고 짧은 계획의 끝을 가치함수로 평가한다. 두 방법의 모델 학습 목표, 가상 자료의 쓰임, 실행 시 계산을 나누어 비교한다.

12.1 슬라이드 01 · Advanced model-based RL

Dreamer와 TD-MPC는 학습한 모델을 각각 가상 경험과 계획에 사용하는 대표적 방법이다.

12.2 슬라이드 02 · 두 알고리즘을 읽는 순서

Model-based 정책 최적화를 복습한 뒤 Dreamer의 잠재 상상, TD-MPC의 가치 기반 계획으로 이어진다.

12.3 슬라이드 03 · Model-based policy optimization 복습

모델로 가상 궤적을 만들고 그 자료로 정책과 가치를 학습하는 흐름을 다시 확인한다.

12.4 슬라이드 04 · 어디에서 가상 궤적을 시작하는가

초기 상태 하나에서 긴 rollout을 만들면 오류가 누적되고, 초기 상태 주변만 학습한다. 실제 replay에서 중간 상태 s_t를 뽑아 길이 H만 모델을 전개하면 더 다양한 과업 단계에 자료를 보탤 수 있다.

예를 들어 로봇 집기의 접근·접촉·들기 단계에서 각각 시작한 5단계 예측을 만들 수 있다. 접촉 뒤의 상태를 실제로 한 번도 보지 않았다면 그 지점의 모델 예측은 여전히 불확실하다. **시작점의 coverage**와 **가상 지평**을 함께 정해야 한다.

12.5 슬라이드 05 · 가상 자료를 쓰는 학습 고리

정책 π_θ가 실제 환경에서 얻은 집합을 $\mathcal{D}_{\text{env}}$, 모델 P_ϕ가 만든 집합을 $\mathcal{D}_{\text{model}}$이라 하자. 실제 전이로 P_ϕ를 갱신하고, $\mathcal{D}_{\text{env}}$의 상태에서 π_θ를 모델 속에 실행해 가상 전이를 만든다. 그 뒤 actor와 critic을 갱신하고 새 정책으로 다시 현실 자료를 모은다.

모델을 학습하는 단계와 모델을 이용하는 단계를 구분하는 것이 중요하다. 후자는 실제 환경의 새 관측을 만들지 않는다. Dreamer는 이 고리를 픽셀 관측과 잠재 상태에 맞게 구성한다.

12.6 슬라이드 06 · Dreamer

Dreamer는 영상에서 잠재 세계모델을 학습하고 그 안의 가상 궤적으로 행동 정책을 개선한다.

12.7 슬라이드 07 · Dyna의 실제 경험과 모의 경험

Dyna는 실제 전이로 Q함수와 모델을 배우고, 모델이 만든 전이로 Q를 추가 갱신한다. 실제 자료를 $\mathcal{D}_{\text{env}}$라 하고, 학습 모델을 $\hat{P}_\phi(s', r \mid s, a)$라 하자. 모의 업데이트에서 (s, a)를 골라 $(\hat{s}', \hat{r}) \sim \hat{P}_\phi$를 얻으면 tabular Q-learning의 한 표본 갱신은

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[\hat{r} + \gamma \max_{a'} Q(\hat{s}', a') - Q(s,a) \right].$$

α는 학습률, γ는 할인율, a'는 다음 상태에서 가능한 행동이다. 실제 갱신에서는 모자 쓴 예측 대신 관측한 (s', r)를 쓴다. 두 자료의 품질 차이를 유지한 채 업데이트를 더하는 구조가 Dyna의 출발점이다. Dyna 원논문

12.8 슬라이드 08 · Dyna를 더 일반적으로 읽기

Q-learning 이외의 model-free 학습기도 모델 경험을 사용할 수 있다. 실제 자료에서 시작 상태 s를 뽑고, 행동을 정책이나 기록 자료에서 선택한 뒤, 모델로 $\hat{s}', \hat{r}$을 만든다. 짧은 가상 전이를 모아 actor–critic 또는 가치 학습에 넣는다.

모의 업데이트 수를 늘리면 현실의 전이 하나당 더 많은 학습 계산을 쓸 수 있다. 그 계산이 유익하려면 보상과 다음 상태의 예측이 정책이 방문할 영역에서 맞아야 한다. Dyna는 실제 표본 수와 모델 표본 수를 분리해 조절할 수 있는 구조다.

12.9 슬라이드 09 · 이미지에서 작동하는 Dreamer

Dreamer는 world model 학습, 잠재공간에서 가치·정책 학습, 실제 환경 상호작용을 순환한다. 카메라 영상은 크고 불필요한 픽셀 변동이 많으므로, 압축된 상태에서 다음 상태와 보상을 예측하는 편이 행동 학습에 유리하다.

이전의 PlaNet도 픽셀을 잠재 상태로 압축했지만 실행 시 온라인 planning을 중심으로 사용했다. Dreamer는 잠재 상상으로 **정책을 학습**해 행동을 낸다는 점이 중요하다. PlaNet 논문, Dreamer 논문

12.10 슬라이드 10 · World model의 잠재 상태

관측 이미지를 o_t, 잠재 상태를 z_t, 행동을 a_t라 하자. Encoder는 실제 o_t에서 현재 잠재 상태를 추론하고, dynamics prior는 이전 잠재 상태와 행동으로 다음 잠재 상태를 예측한다. Decoder는 잠재 상태에서 관측 $\hat{o}_t$를 재구성하고 reward head는 $\hat{r}_t$를 예측한다.

학습 목표를 단순화하면 **관측 재구성 손실 + 보상 예측 손실 + posterior와 prior의 KL 벌점**이다. Dreamer의 recurrent state를 h_t라 하고, 관측을 본 뒤의 잠재 posterior를 $q_\phi(z_t \mid h_t, o_t)$, 관측 없이 미래를 예측하는 prior를 $p_\phi(z_t \mid h_t)$라 하자. $p_\phi(o_t \mid h_t, z_t)$와 $p_\phi(r_t \mid h_t, z_t)$는 각각 관측·보상 예측 분포다. 한 시점의 대표적인 손실은

$$\begin{aligned} \mathcal{L}_t^{\text{world}} = & - \mathbb{E}_{z_t \sim q_\phi} \log p_\phi(o_t \mid h_t, z_t) \\ & - \mathbb{E}_{z_t \sim q_\phi} \log p_\phi(r_t \mid h_t, z_t) \\ & + \beta D_{\text{KL}}(q_\phi(z_t \mid h_t, o_t) \,\|\, p_\phi(z_t \mid h_t)) \,. \end{aligned}$$

$\beta \geq 0$는 KL 항의 가중치이고 D_{KL}은 두 분포가 얼마나 다른지를 재는 KL divergence다. 첫 두 항은 실제 관측과 보상을 잘 설명하도록, 마지막 항은 관측을 보지 않고 상상한 상태와 관측 후 상태가 어긋나지 않도록 학습시킨다. 그러므로 실행 중 미래 관측이 없어도 모델을 전개할 수 있다. 실제 버전별 구현에서는 손실 가중치와 잠재분포의 형태가 달라진다.

재구성은 언제 필요하고, 전이는 어떻게 배우나? 기록한 궤적의 실제 관측을 encoder에 넣어 관측 후 잠재 상태를 만든다. 이전 잠재 상태와 행동만 받은 dynamics prior는 그 잠재 상태를 예측하고, decoder와 reward head는 각각 관측과 보상을 맞힌다. 따라서 전이도 실제 궤적에서 얻은 목표를 이용해 학습한다. Actor·critic을 갱신할 때는 학습된 prior를 잠재공간에서 전개해 보상과 가치를 계산하므로, 가상 궤적의 각 프레임을 이미지로 복원할 필요가 없다.

12.11 슬라이드 11 · 긴 영상 예측을 어떻게 읽는가

Dreamer의 잠재 모델이 여러 단계의 미래 영상을 예측한 예시다. 예측이 길어질수록 실제 관측과 차이가 누적될 수 있다.

12.12 슬라이드 12 · 잠재 상태 위의 actor와 critic

잠재 상태 위에서 actor가 행동을 만들고 critic이 그 상태의 가치를 평가한다. 두 모듈은 실제 픽셀 전체가 아닌 학습된 잠재 표현을 사용한다.

12.13 슬라이드 13 · 잠재 상상에서 반환 만들기

실제 관측으로 얻은 z_t에서 시작해 actor가 $\hat{a}_t \sim \pi_\theta(\cdot \mid z_t)$를 내고, world model이 $\hat{z}_{t+1}, \hat{r}_t$를 예측한다. 이를 H번 반복한다. 끝에서 $V_\psi(\hat{z}_{t+H})$로 이후를 부트스트랩한다.

여러 지평을 섞는 λ-return의 한 재귀 표현은 아래와 같다. γ는 보상의 시간 할인, $\lambda \in [0,1]$는 더 먼 상상을 얼마나 사용할지 정하는 가중치다.

$$\widehat{G}_k^\lambda = \hat{r}_k + \gamma\left[(1-\lambda)V_\psi(\hat{z}_{k+1}) + \lambda \widehat{G}_{k+1}^\lambda\right], \qquad \widehat{G}_{t+H}^\lambda = V_\psi(\hat{z}_{t+H}).$$

$\lambda = 0$이면 한 단계 뒤의 가치 추정에 주로 의존하고, $\lambda = 1$이면 지평 끝까지 예측 보상을 더한다. 예를 들어 두 단계만 전개하면 $\widehat{G}_t^\lambda = \hat{r}_t + \gamma(1-\lambda)V_\psi(\hat{z}_{t+1}) + \gamma\lambda\hat{r}_{t+1} + \gamma^2\lambda V_\psi(\hat{z}_{t+2})$다. 여기서 γ는 **보상이 늦게 도착하는 시간**에 따른 할인이고, λ는 **상상한 전이를 몇 단계 신뢰할지** 정하는 혼합 계수다. 긴 지평은 모델의 작은 전이 오차를 더 멀리 전달한다.

12.14 슬라이드 14 · 보상 기울기가 초기 행동까지 가는 길

연속 행동의 단순한 결정론적 예에서 $a_t = \mu_\theta(z_t)$, $z_{t+1} = f_\phi(z_t, a_t)$라 하자. θ는 actor, ϕ는 학습된 dynamics의 파라미터다. 한 단계 보상 함수 r_ϕ와 critic V_ψ를 사용한 목표 $F(\theta) = r_\phi(z_t, \mu_\theta(z_t)) + \gamma V_\psi(f_\phi(z_t, \mu_\theta(z_t)))$를 생각하자. 시작 상태 z_t를 고정하면 연쇄법칙으로

$$\nabla_\theta F = \left[\frac{\partial r_\phi}{\partial a_t} + \gamma \frac{\partial V_\psi}{\partial z_{t+1}}\frac{\partial f_\phi}{\partial a_t}\right]\frac{\partial \mu_\theta(z_t)}{\partial \theta}.$$

각 $\partial/\partial a_t$는 행동에 대한 국소 변화율이고, $\partial\mu_\theta/\partial\theta$는 actor 파라미터를 바꿀 때 행동이 얼마나 변하는지 나타낸다. 대괄호의 첫 항은 현재 행동이 즉시 보상을 바꾸는 효과, 두 번째 항은 다음 잠재 상태를 거쳐 미래 가치를 바꾸는 효과다. 두 단계 이상 전개하면 다음 행동·보상·끝 가치의 미분이 같은 방식으로 앞 단계에 이어진다. 학습된 모델을 통과하는 기울기이므로, 모델이 틀린 영역에서는 방향도 부정확할 수 있다. Dreamer 원논문

모든 Dreamer 버전이 같은 actor 기울기를 쓰나? 연속 행동과 미분 가능한 잠재 전이에서는 경로를 통한 gradient가 핵심이다. 이산 행동·잠재변수에서는 경로 미분이 불가능한 부분에 score-function 추정이나 다른 surrogate가 필요할 수 있다. 버전과 행동공간의 학습식을 확인해야 한다.

12.15 슬라이드 15 · Dreamer의 네 단계 고리

실제 경험 수집 → 세계모델 갱신 → 잠재 궤적 생성 → actor·critic 학습의 반복이다.

12.16 슬라이드 16 · DreamerV1의 과업 범위

DreamerV1의 실험 과제를 보여 주는 그림이다. 결과 수치는 다음 비교 그래프의 평가 조건과 함께 읽는다.

12.17 슬라이드 17 · DreamerV1의 비교 그래프

연속 제어 과제에서 DreamerV1과 다른 방법의 성능을 비교한 막대그래프다. 과제별 점수를 나누어 읽어야 한다.

12.18 슬라이드 18 · DreamerV2와 이산 세계모델

DreamerV2의 이산 행동 환경 결과다. 잠재 상태의 표현 방식과 평가 과제의 차이를 구분해야 한다.

12.19 슬라이드 19 · DreamerV3의 범용성

DreamerV3가 여러 영역에서 학습한 결과를 보여 준다. 하나의 설정으로 다양한 과제를 다루려는 방향의 사례다.

12.20 슬라이드 20 · DayDreamer의 실제 로봇

DayDreamer는 실제 로봇에서 세계모델을 학습하고 조작·이동 정책을 개선한 사례다.

12.21 슬라이드 21 · Dreamer에서 얻을 설계 원칙

관측 재구성은 잠재 상태에 풍부한 정보를 담도록 돕고, 잠재공간 상상은 actor–critic 학습을 빠르게 만든다. 이 구조는 이미지처럼 고차원 관측에 강점이 있지만 decoder를 포함한 world model 학습에는 메모리와 계산이 든다.

실행 시 매번 CEM을 오래 돌리지 않고 정책이 행동을 제시한다는 점도 중요하다. 반대로 특정 현재 상태에서 계산을 더 써서 행동열을 고르고 싶다면 다음 TD-MPC가 다른 절충을 제공한다.

12.22 슬라이드 22 · TD-MPC로 전환

TD-MPC는 짧은 지평의 MPC 계획 끝에 가치함수를 붙여 긴 미래를 반영한다.

12.23 슬라이드 23 · MPC의 짧은 지평 복습

MPC는 현재 상태에서 길이 H의 행동열을 찾고 첫 행동만 실행한 뒤 새 관측으로 재계획한다. 이는 이전 계획의 예측과 실제 다음 상태가 달라졌을 때 바로 고칠 수 있다.

지평이 너무 길면 후보 행동열의 최적화와 모델 예측이 모두 어려워진다. 지평이 너무 짧으면 지금의 작은 보상 때문에 장기 목표에 필요한 행동을 놓칠 수 있다. TD-MPC는 이 두 번째 문제를 terminal value로 다룬다.

12.24 슬라이드 24 · 먼 미래를 다루는 두 길

첫 길은 모델의 짧은 계획 끝에 가치함수를 붙이는 것이다. 둘째는 모델이 만든 자료로 정책과 critic을 갱신해 장기 판단을 정책에 넣는 것이다. Dreamer는 둘째를 중심으로 사용하고, TD-MPC는 첫째를 실행 시 계획의 핵심으로 사용한다.

둘은 완전히 분리되지 않는다. TD-MPC도 정책과 Q를 학습하고, Dreamer도 imagined rollout 끝에서 critic 값을 사용한다. 차이는 **실행 시 새 행동열을 검색하는가**와 모델 학습의 목적에 있다.

12.25 슬라이드 25 · 계획 끝에 가치함수 붙이기

시점 t에서 길이 H의 모델 계획만 쓰면 그 이후 보상은 목표에 들어가지 않는다. 참 반환을 처음 H 단계와 나머지로 나누면

$$G_t = \sum_{k=0}^{H-1} \gamma^k r_{t+k} + \gamma^H G_{t+H}.$$

$\gamma \in [0,1]$는 할인율이고 G_{t+H}는 계획 끝에서 계속 행동해 얻을 반환이다. 이를 학습된 terminal value $V_\psi(\hat{s}_{t+H})$로 바꾸면, H 단계 계획의 점수는 $\sum_{k=0}^{H-1} \gamma^k \hat{r}_{t+k} + \gamma^H V_\psi(\hat{s}_{t+H})$가 된다. 모자 기호는 모델 예측을 나타낸다. 긴 미래의 모든 전이를 모델로 예측하는 대신 가치함수에 요약을 맡기는 유도다.

끝 가치가 정확하면 짧은 계획만으로 충분한가? 모델의 첫 H 단계 전이와 보상도 정확해야 하고, 후보 행동열을 충분히 탐색해야 한다. 끝 가치가 좋아도 중간 경로의 위험을 모델이 놓치거나 CEM이 좋은 후보를 찾지 못하면 첫 행동은 잘못될 수 있다.

12.26 슬라이드 26 · TD-MPC의 계획 점수

실제 관측 s_t를 encoder h_θ로 잠재 상태 z_t에 옮긴다. 후보 행동열 $A = (a_t, \dots, a_{t+H-1})$에 대해 잠재 전이 $z_{k+1} = d_\theta(z_k, a_k)$와 보상 예측 $R_\theta(z_k, a_k)$을 반복한다. π_θ를 계획 끝의 후속 행동을 제안하는 정책이라 하면 점수는

$$\widehat{J}_H(A \mid z_t) = \sum_{k=t}^{t+H-1} \gamma^{k-t} R_\theta(z_k, a_k) + \gamma^H Q_\theta(z_{t+H}, \pi_\theta(z_{t+H})).$$

Q_θ는 계획 끝 뒤의 기대 반환을 추정한다. CEM은 여러 A를 이 식으로 비교하고 첫 행동을 실행한다. Q_θ가 편향되면 계획도 그 오류를 이용할 수 있으므로 현실 자료에서의 TD 학습과 평가가 필요하다.

12.27 슬라이드 27 · 과업 지향 잠재 동역학

픽셀 이미지 전체를 다음 이미지로 복원하려면 배경 질감이나 조명까지 맞혀야 한다. TD-MPC는 관측 s를 $z = h_\theta(s)$로 압축하고, $z' = d_\theta(z, a)$가 보상과 Q 예측에 필요한 정보를 담도록 배운다. **Task-oriented**는 이미지의 모든 세부보다 제어 성과와 관련된 부분을 우선한다는 뜻이다.

로봇이 망치를 잡는 과업에서는 손끝·도구·목표의 상대 위치가 중요하고 벽 무늬는 덜 중요할 수 있다. 그러나 관측 재구성 신호를 빼면 표현이 상수로 무너질 위험도 있다. 보상·가치 목표와 잠재 전이의 일관성 목표가 표현 학습을 이끈다.

12.28 슬라이드 28 · 느리게 움직이는 target encoder

현재 encoder $h_\theta(s_{t+1})$를 그대로 전이 예측의 표적으로 쓰면 표적이 모델과 동시에 빠르게 움직인다. TD-MPC는 느리게 갱신되는 target encoder $h_{\bar\theta}$를 두고, $d_\theta(z_t, a_t)$가 $h_{\bar\theta}(s_{t+1})$에 가까워지도록 학습한다. $\bar\theta$는 현재 파라미터 θ를 천천히 따라가는 값이다.

온라인 encoder h_θ는 보상·Q·전이 예측 손실에서 온 기울기로 직접 갱신된다. 반면 target encoder는 이를 천천히 따라가며 다음 관측의 잠재 목표를 제공한다. 모든 관측을 같은 벡터로 보내면 잠재 일관성 손실만은 작아질 수 있지만, 상태에 따라 달라지는 보상과 Q를 맞히기 어려워진다. 이 두 목표의 결합이 표현 붕괴를 막는 학습 압력이다.

12.29 슬라이드 29 · TD-MPC의 세 손실과 정책 손실

실제 전이 $(s_i, a_i, r_i, s_{i+1}, d_i)$에서 $d_i = 1$은 종료를 뜻한다. $z_i = h_\theta(s_i)$, $\bar z_{i+1} = h_{\bar\theta}(s_{i+1})$라 하자. d_θ는 잠재 전이, R_θ는 보상 예측, Q_θ는 행동가치, π_θ는 정책이다. 위 첨자 π가 해당 정책을 따라갈 때의 가치를 뜻한다면 Bellman 관계는

$$Q^\pi(z_i, a_i) = \mathbb{E}[r_i + \gamma(1-d_i)Q^\pi(z_{i+1}, \pi(z_{i+1})) \mid z_i, a_i]\,.$$

기대값을 실제 표본 하나와 느리게 갱신하는 target 망으로 근사하면 다음 TD 표적을 얻는다. 막대 파라미터 $\bar\theta$는 target 망을 뜻한다.

$$y_i = r_i + \gamma(1-d_i)Q_{\bar\theta}(\bar z_{i+1}, \pi_{\bar\theta}(\bar z_{i+1}))\,.$$

이 표적을 이용한 기본 손실의 구조는

$$\begin{aligned}\mathcal{L}_i = c_r\big(R_\theta(z_i, a_i) - r_i\big)^2 + c_Q\big(Q_\theta(z_i, a_i) - y_i\big)^2 \\ + c_z\big\|d_\theta(z_i, a_i) - \bar z_{i+1}\big\|_2^2.\end{aligned}$$

$c_r, c_Q, c_z \ge 0$는 세 항의 상대 가중치다. 첫 항은 보상, 둘째는 장기 가치, 셋째는 잠재 전이를 맞힌다. 길이 H의 학습 구간에서는 $J_{\text{model}} = \sum_{i=t}^{t+H-1} \lambda^{i-t}\mathcal{L}_i$로 뒤쪽 시점의 손실을 가중할 수 있다. 이 $\lambda \in (0,1]$는 멀리 전개한 잠재 상태가 덜 정확할 수 있음을 반영하는 **학습 가중치**다. 슬라이드 13의 Dreamer λ-return과 기호는 같지만, 여기서는 손실 항의 시점별 비중을 정한다. 보상 할인율 γ와도 역할이 다르다.

정책은 예를 들어 $J_\pi(\theta) = -\mathbb{E}_z[Q_\theta(z, \pi_\theta(z))]$를 줄여 Q가 높다고 예측한 행동을 제안한다. $\mathbb{E}_z$는 학습 중 표본으로 뽑은 잠재 상태에 대한 평균이다. Actor 손실의 음수 부호는 Q를 **최대화**하려는 뜻이다. 이 목적을 여러 예측 잠재 상태에 적용하면 rollout 중간의 행동에도 학습 신호를 줄 수 있다. 위 식은 손실의 역할을 보여 주는 단순화이며 실제 구현은 target 업데이트, 손실 계수와 gradient stop을 사용한다. TD-MPC 원논문

왜 이미지 재구성 없이 잠재 전이를 학습할 수 있나? 목표 잠재 상태와의 일관성만으로는 상수 표현이 가능하지만, 같은 표현이 보상과 Q를 예측해야 하므로 과업에서 서로 다른 결과를 내는 상태들을 구분할 압력을 받는다. 느리게 갱신하는 target encoder도 움직이는 표적의 불안정을 줄인다. 그래도 관측에서 중요한 보상 단서가 빠지면 성능이 떨어질 수 있다.

12.30 슬라이드 30 · MPC에 정책을 함께 배우는 두 이유

첫째, CEM의 초기 후보를 무작위 행동열만으로 만들면 높은 차원에서 좋은 영역을 찾기 어렵다. 현재 정책의 행동열을 후보에 섞으면 계획기가 이미 괜찮다고 배운 경로 주변을 탐색할 수 있다. 정책은 계획의 **제안분포** 역할을 한다.

둘째, Q-learning의 다음 행동을 찾을 때마다 CEM을 다시 돌리면 TD 표적을 만드는 비용이 커진다. 정책 $\pi_\theta(z')$를 사용하면 다음 행동의 가치 $Q(z', \pi_\theta(z'))$를 빠르게 계산할 수 있다. 따라서 정책은 실행 시 계획기와 학습 시 critic을 모두 돕는다.

12.31 슬라이드 31 · TD-MPC의 실행 절차

현재 관측을 잠재 상태 z_t로 인코딩하고, 정책이 제안한 행동열을 포함해 CEM 후보를 만든다. 각 후보를 잠재 전이로 H 단계 전개해 예측 보상과 terminal Q를 더한다. 가장 높은 점수의 행동열에서 첫 행동만 현실에 적용하고 다음 관측으로 재계획한다.

정책 제안은 CEM의 초기 행동열을 좋은 영역으로 옮기고, terminal Q는 H 단계 이후의 결과를 점수에 담는다. 그래서 TD-MPC의 첫 행동은 현재 보상, 짧은 예측 보상, 끝 가치가 함께 결정한다. TD-MPC 논문

12.32 슬라이드 32 · 수집 · 학습 · 계획의 순환

실제 데이터 수집, 잠재 동역학 · 가치 · 정책 갱신, CEM 계획의 반복을 나타낸다.

12.33 슬라이드 33 · 고차원 연속 제어 사례

TD-MPC가 개 달리기, 휴머노이드 달리기, 망치 조작 등 연속 제어 과제를 수행한 예시다.

12.34 슬라이드 34 · 움직임 영상으로 보는 비교

SAC와 TD-MPC의 동작을 나란히 보여 주는 영상이다. 정량적 비교는 별도의 학습 곡선과 시험 조건에서 확인한다.

12.35 슬라이드 35 · 복잡한 보행에서의 차이

복잡한 보행 과제의 실행 영상이다. 동작의 성공 여부와 학습에 사용한 환경 경험량을 함께 비교해야 한다.

12.36 슬라이드 36 · 학습 곡선 읽기

TD-MPC의 학습 곡선이다. 최종 점수뿐 아니라 같은 환경 상호작용 수에서의 점수를 읽는다.

12.37 슬라이드 37 · 계획 예산을 늘릴 때

시험 시 계획 예산을 늘리면 성능이 어떻게 변하는지 보여 주는 ablation이다. 추가 계산 비용도 함께 고려한다.

12.38 슬라이드 38 · TD-MPC2로 확장하기

TD-MPC2는 TD-MPC를 더 큰 모델과 여러 과제로 확장한 사례다.

12.39 슬라이드 39 · Dreamer와 TD-MPC의 선택

Dreamer는 관측 재구성으로 풍부한 잠재 표현을 배우고, 모델 속 imagined rollout으로 actor–critic을 갱신한다. TD-MPC는 과업에 필요한 잠재 전이·보상·Q를 배우고, 실행 시 CEM으로 짧은 행동열을 검색한다. 픽셀 관측 처리, 모델 학습 비용, 실행 지연, 행동 차원을 함께 보아야 어느 방식이 맞는지 판단할 수 있다.

계획 지평과 실행 계산을 기준으로 고르면 차이가 선명하다. 빠른 정책 실행이 필요한 경우 Dreamer의 학습된 actor가, 현재 상태에서 추가 계산으로 행동을 조정할 수 있는 경우 TD-MPC의 CEM 계획이 적합하다. 관측 재구성의 필요성과 과업별 잠재 표현도 선택에 영향을 준다.

Dreamer의 시각 제어 결과는 **고차원 관측**에서 잠재 표현 학습의 이점을, TD-MPC의 Dog Run 결과는 **고차원 연속 행동**에서 짧은 계획의 이점을 보여 준다. 이는 각 방법을 한 종류의 입력에만 제한한다는 뜻은 아니다. TD-MPC 원논문도 상태와 이미지 관측의 연속 제어를 모두 평가한다. Dreamer 원논문, TD-MPC 원논문

모델이 완벽하지 않아도 왜 계획이 도움이 되나? 행동열 전체의 먼 미래를 정확히 예측할 필요는 없다. TD-MPC는 가까운 H 단계에서 후보의 상대적 차이를 모델로 비교하고, 이후 반환은 학습된 Q로 요약한다. 실제 다음 관측이 오면 계획을 다시 시작해 앞선 전이 오차를 교정한다. 이 설명도 첫 H 단계의 모델·보상 예측과 terminal Q가 정책 선택에 필요한 정도로 정확하다는 조건에 의존한다.

12.40 슬라이드 40 · 더 큰 탐색 문제로

다음 주제는 AlphaGo다. 탐색과 학습된 가치·정책을 결합하는 방식으로 이어진다.

13 과제 가이드

13.1 과제 1 · 모방학습

이 과제는 전문가 시연에서 행동을 복제하고, 실제 rollout에서 성능을 측정하며, 정책이 방문한 상태를 전문가가 교정하는 DAgger로 확장한다. 구현의 중심은 **상태–행동 데이터가 어디서 오고 어떻게 다시 모이는가**다. 실험 결과에는 직접 얻은 수치와 실행 설정을 기록한다.

13.1.1 먼저 연결할 개념

- 행동 복제의 목적함수: 전문가 데이터에서 정책의 로그우도를 높인다.
- 오류 누적과 상태 분포 이동: 훈련 정확도와 rollout 성공률이 다를 수 있다.
- DAgger의 데이터 집계: 현재 정책이 방문한 상태에도 전문가 행동을 붙인다.

13.1.2 1. 환경 인터페이스를 먼저 확인하기

환경을 초기화하면 시작 관측이 나오고, 한 단계 진행 함수에는 행동을 넣어 다음 관측·보상·종료 여부를 받는다. 과제에서 묻는 이산 행동과 연속 행동의 차이는 정책 출력이 무엇을 뜻하는지 결정한다. 이산 행동은 후보 중 하나를 고르고, 연속 행동은 구간 안의 실수 벡터를 낸다. Ant와 다른 MuJoCo 보행 환경은 연속 관절 제어다.

13.1.3 환경 튜토리얼의 세 질문

1. **step(action)의 역할과 반환값은?** 행동을 환경에 적용해 한 시점 진행한다. Gymnasium 인터페이스에서는 (observation, reward, terminated, truncated, info)를 반환한다. terminated는 과업의 실제 종료, truncated는 시간 제한 등 외부 중단이다. 다음 실행 여부는 보통 두 flag의 논리합으로 판단한다.
2. **reset()의 역할과 반환값은?** 새 episode를 시작하고 내부 상태를 초기화한다. Gymnasium에서는 (initial_observation, info)를 반환한다. 이전 episode의 마지막 관측을 그대로 이어 쓰지 않는다.
3. **이산·연속 행동공간은 어떻게 확인하나?** env.action_space를 출력하거나 형식을 확인한다. Discrete(n)은 정수 행동 $0, \ldots, n-1$, Box(low, high, shape)는 범위와 차원을 가진 연속 벡터다. env.action_space.sample()로 실제 표본의 모양을 확인할 수도 있다.

오래된 Gym 버전은 reset()이 관측 하나만, step()이 (observation, reward, done, info) 네 항목을 반환할 수 있다. 따라서 설명할 때는 **실제로 실행한 라이브러리 버전과 starter code의 unpacking 방식**을 함께 확인한다. info는 보조 진단 정보이고 보상이나 관측을 대신하지 않는다.

한 rollout은 초기화 → 관측으로 행동 선택 → 환경 진행 → 전이 저장 → 종료까지 반복으로 만든다. 환경 API의 버전에 따라 종료와 시간 제한이 따로 반환될 수 있으므로, 스타터 코드가 실제로 어떤 형식을 기대하는지 먼저 확인한다. 관측·행동·보상 배열의 첫 차원은 시간이며, 하나의 궤적 안에서는 길이가 맞아야 한다.

한 단계 진행에서 받은 종료 신호를 왜 따로 보관할까? 실제 실패·성공으로 끝난 episode와 시간 제한 때문에 끊긴 rollout은 가치함수 학습에서 다르게 처리될 수 있다. 이 과제는 BC 중심이어도 평가 episode 수와 길이를 정확히 세려면 종료 원인을 기록하는 편이 좋다.

실제로 첫 episode를 수집할 때는 관측 o_t를 저장한 뒤 그 관측으로 고른 a_t를 같은 인덱스에 저장한다. 다음 단계에서 나온 o_{t+1}를 실수로 a_t와 짝지으면 정책은 한 단계 늦은 영상을 보고 행동하는 규칙을 배운다. 이 오류는 배열의 크기가 모두 맞아도 생기므로 몇 개의 전이를 시간 순서대로 직접 읽어 보는 편이 좋다.

13.1.4 2. 행동 복제의 데이터 흐름

전문가 시연의 모든 시점을 모아 상태–행동 쌍 N 개를 만든다. s_i 는 i 번째 쌍의 상태, a_i^* 는 그때 전문가가 고른 행동이다. $\pi_\theta(a_i^* \mid s_i)$ 는 파라미터 θ 인 정책이 그 행동에 주는 확률 또는 밀도다. 행동 복제는 평균 음의 로그우도

$$\mathcal{L}_{\mathrm{BC}}(\theta) = -\frac{1}{N}\sum_{i=1}^{N} \log \pi_\theta(a_i^* \mid s_i)$$

를 최소화한다. 연속 행동에서는 d_a 를 행동 벡터의 차원, $\mu_\theta(s)$ 를 정책의 평균 행동이라 하자. 정책 분포를 평균 $\mu_\theta(s)$, 고정 공분산 $\sigma^2 I$ 인 Gaussian으로 두면 한 쌍의 음의 로그밀도는

$$-\log \pi_\theta(a_i^* \mid s_i) = \frac{\|a_i^* - \mu_\theta(s_i)\|_2^2}{2\sigma^2} + \frac{d_a}{2}\log(2\pi\sigma^2).$$

I 는 d_a 차원 항등행렬, $\|\cdot\|_2$ 는 유클리드 길이, $\log(2\pi\sigma^2)$ 의 π 는 원주율이다. 오른쪽 두 번째 항은 θ 에 무관하므로 평균 행동의 제곱오차를 최소화하는 것과 같은 최적점이다. 실제 실행에서는 정책이 자기 행동으로 훈련과 다른 상태를 만들 수 있어 rollout 반환도 평가한다.

구현은 공통 MLP 생성 → 정책의 행동·학습 함수 → 궤적 수집 유틸리티 → replay buffer → BC 에이전트 → trainer 순서로 연결하면 의존성을 따라 검증하기 쉽다. B 를 한 batch의 시연 수, d_s 와 d_a 를 상태·행동 벡터 차원이라 하자. 관측 batch의 모양이 $B \times d_s$ 라면 정책 평균은 $B \times d_a$ 이어야 하고, 행동과 전문가 목표의 단위·범위가 같아야 한다.

13.1.5 점검할 연결

1. 환경에서 얻은 관측이 정책 입력과 같은 차원인가?
2. 정책에서 나온 행동이 환경의 행동 범위를 만족하는가?
3. replay buffer가 상태와 그때의 전문가 행동을 같은 인덱스로 보관하는가?
4. 평가할 때 탐색·훈련용 잡음을 실수로 더하지 않는가?
5. 훈련·평가의 반환을 같은 종료 조건으로 계산하는가?

연속 행동의 작은 예를 보자. 시연에서 팔의 목표 위치가 $a^* = 0.6$ 인데 정책 평균이 $\mu_\theta(s) = 0.2$ 라면 오차는 -0.4 다. 제곱오차를 줄이면 평균은 0.6 쪽으로 이동한다. 다만 환경의 허용 범위가 $[-1, 1]$ 인지 $[0, 1]$ 인지에 따라 같은 출력의 물리적 의미가 달라진다. 데이터의 행동 정규화와 환경에 넘기는 행동 복원 과정을 반드시 한 쌍으로 확인한다.

훈련 손실이 낮은데 행동이 늘 경계값에 붙는다면? 정책 출력의 tanh와 환경의 clip을 중복 적용했거나, 정규화된 전문가 행동과 원래 단위의 정책 출력을 비교했을 수 있다. 평균 행동과 실제 환경에 전달된 행동을 각각 기록하면 어디서 포화되는지 찾을 수 있다.

13.1.6 3. BC 실험과 숫자의 의미

첫 실험은 **Ant-v4** 와 **Walker2d-v4·HalfCheetah-v4·Hopper-v4 중 하나**에서 BC 정책을 평가한다. Ant에서는 전문가 성과의 30% 이상이 구현 점검용 목표다. 전문가 반환은 실행 로그의 Initial_DataCollection_AverageReturn에서 확인한다. 보고할 값은 각 정책 rollout 반환의 평균과 표준편차다. K 번의 episode 반환을 $R_1, \ldots, R_K$ 라 하면 $\bar{R} = K^{-1}\sum_k R_k$ 이고 표본 표준편차는 $\sqrt{\sum_k (R_k - \bar{R})^2/(K-1)}$ 다. 한 episode만 수집해 놓고 표준편차를 보고할 수는 없다.

평가 표본 예산이 episode 길이보다 커야 여러 rollout을 얻는다. 예컨대 최대 길이가 1000단계이고 평가 예산이 5000단계이면 대략 다섯 episode를 얻는다. 표에는 환경, 전문가 성과, BC 평균±표준편차, 평가 episode 수, 훈련 데이터 양, 네트워크 크기와 업데이트 수를 함께 적으면 비교가 공정해진다.

평가 rollout의 화면 한 장과 설명을 보고서에 넣는다. 실행 영상을 확인하는 이유는 반환 숫자가 포착하지 못하는 이상 행동을 찾기 위해서다. 로봇이 제자리에서 높은 보상을 받거나 초기 몇 단계만 걷고 넘어지는 모습을 보면 보상 함수와 종료 조건을 다시 볼 수 있다.

계산 예로 세 episode의 반환이 90, 100, 110이면 평균은 100이다. s_R을 이 반환들의 표본 표준편차라 쓰면 앞서 정의한 $K-1$ 분모의 식에서

$$s_R = \sqrt{\frac{(90-100)^2 + (100-100)^2 + (110-100)^2}{3-1}} = 10.$$

`100 ± 10`이라고만 쓰면 세 번의 실행 결과인지 여러 시드 평균인지 알기 어려우므로 episode 수와 시드 수도 함께 적는다. 전문가와 BC를 비교할 때 평가 환경의 초기 상태 범위를 같게 맞추는 것도 중요하다.

13.1.7 4. 하이퍼파라미터 실험

하나의 요인에 대해 기본 설정을 포함한 **네 가지 이상의 값**을 비교한다. 예를 들어 시연량을 바꾸면 다른 조건인 네트워크 크기·훈련 단계·평가 episode를 고정한다. 여러 조건을 동시에 바꾸면 어느 요인이 성능 차이를 만들었는지 알기 어렵다.

고를 수 있는 변수에는 정책 학습 gradient step 수(num_agent_train_steps_per_iter, 기본 1000), 수집 데이터 양(batch_size, 기본 1000), 지도학습 minibatch 크기(train_batch_size, 기본 100), MLP 층 수(n_layers, 기본 2), 폭(size, 기본 64), 학습률(learning_rate, 기본 5×10^{-3})이 있다. 예를 들어 데이터 양을 고르면 **기본값 1000을 포함한 네 값**을 같은 환경에서 시험하고, 선택 이유·각 값·평가 평균 반환을 그래프와 설명에 함께 적는다. 여기에 나열한 값은 starter 설정의 기준값이지 모든 환경에서 최적인 값은 아니다.

가로축은 바꾼 변수, 세로축은 평가 반환의 평균으로 그리고 가능하면 표준편차나 여러 시드 결과를 표시한다. 시연량을 늘렸는데 성과가 멈춘다면 모델 용량보다 시연의 다양성이나 상태 분포 이동이 병목일 수 있다. 학습 단계를 늘렸는데 훈련 손실만 내려간다면 과적합 또는 배포 분포 차이를 의심할 수 있다.

예를 들어 시연 수를 100, 250, 500, 1000개로 늘리는 실험에서는 나머지 학습 설정을 고정해야 데이터 양의 효과를 읽을 수 있다. 대신 데이터 100개 조건에 업데이트 1000번, 데이터 1000개 조건에 업데이트 100번을 주면 두 조건은 데이터 양뿐 아니라 각 표본을 본 횟수도 달라진다. **환경 단계 수, gradient update 수, 전문가 질의 수**를 서로 다른 예산으로 기록하자.

13.1.8 5. DAgger의 반복

BC가 만든 초기 정책으로 환경을 실행해 현재 정책이 방문한 상태를 모은다. 반복 번호를 k, 현재 정책을 π_k, 그 방문 상태의 분포를 d_{π_k}라 쓰자. 전문가 정책 π^*에 상태 s를 입력해 교정 행동 $\pi^*(s)$를 얻고, 기존 시연 집합 $\mathcal{D}_k$에 합친다.

$$\mathcal{D}_{k+1} = \mathcal{D}_k \cup \{(s, \pi^*(s)) : s \sim d_{\pi_k}\}.$$

구현할 재라벨링 함수는 **정책이 방문한 상태**를 전문가에게 물어야 한다. 기존 전문가 데이터의 행동을 다시 읽어 붙이는 것만으로는 DAgger의 핵심인 분포 이동 교정이 생기지 않는다. 각 반복에서 추가된 시연 수와 정책 평가 반환을 함께 기록하자.

DAgger 곡선이 매 반복 항상 올라가야 할까? 그렇지 않다. 유한한 시연, 정책 최적화 오차, 환경의 무작위성 때문에 평가 반환은 출렁일 수 있다. 여러 반복과 가능한 경우 여러 시드의 경향을 보며 BC와 같은 조건에서 비교한다.

장난감 경로에서 BC가 중앙의 s_0만 보았고 실제 실행 중 왼편으로 치우친 s_1을 방문했다고 하자. s_1에서 정책 행동을 그대로 라벨로 저장하면 틀린 조향을 다시 학습할 수 있다. DAgger는 그때 전문가에게 $\pi^*(s_1)$을 물어 **복구 행동**을 붙인다. 데이터 집계 후 같은 실패가 줄어드는지 rollout에서 확인해야 한다.

전문가 라벨을 매 단계 받을 수 없으면? 질의 비용이 크면 위험하거나 불확실한 상태를 우선 선택해 사람에게 묻는 방식을 고려할 수 있다. 다만 그 경우 모든 방문 상태에 전문가 라벨을 붙이는 표준 DAgger와 데이터 분포가 달라지므로, 질의 기준과 횟수를 실험에 기록해야 한다.

13.1.9 6. BC와 DAgger 비교

앞서 BC를 시험한 **같은 두 환경**에서 DAgger 반복 횟수에 따른 평균 반환 곡선을 그린다. 비교 기준선으로 BC와 전문가의 성과를 함께 표시하면 분포 이동 교정의 효과를 해석하기 쉽다. DAgger가 개선되어도 전문가까지 닿지 못한다면 전문가 라벨의 부족, 현재 정책이 방문한 상태의 다양성, 정책 표현력이나 최적화 설정을 살핀다. 두 환경의 그래프에는 환경 이름, 네트워크 구조, 학습 데이터 양과 반복 수를 명시한다.

곡선의 가로축이 반복 횟수이면 각 반복에서 추가한 데이터 양도 설명해야 한다. 반복 수가 같은 두 실험이라도 한 번에 수집한 상태가 다르면 사용한 전문가 피드백 예산이 다르기 때문이다.

13.1.10 결과를 정리하는 형식

항목	기록할 내용	해석할 질문
BC 성능	환경별 평균 반환과 표준편차	시연 행동을 실제 실행에서 재현했나?
행동 영상	평가 rollout의 대표 장면	어느 단계에서 정책이 이탈하나?
변수 실험	네 가지 이상 설정과 동일한 평가 조건	데이터·모델·업데이트 중 무엇이 병목인가?
DAgger 곡선	두 과업의 반복별 평균 반환	교정 데이터가 분포 이동을 줄였나?
비교 설명	전문가·BC·DAgger, 데이터 예산	차이를 한 요인으로 과장하지 않았나?

보고서에는 실제 실행한 설정과 결과를 쓴다. 보고서 PDF와 필요한 코드·노트북을 하나의 압축 파일로 묶고, 영상과 TensorBoard 로그는 제외한다. 과제의 압축 파일 크기 기준은 **20 MB 미만**이다.

13.1.11 질문에 답을 쓸 때 구분할 것

환경 API의 반환값, BC와 DAgger의 데이터 흐름은 위의 개념과 코드에서 직접 확인할 수 있다. 반면 **두 환경의 실제 평균 반환, 네 가지 설정의 그래프, 실행 화면, DAgger의 개선 폭**은 구현과 실행에서 나온 결과여야 한다. 실험 전 예상과 실행 후 관찰을 분리하고, 점수만 보고 "DAgger가 항상 우월하다"처럼 일반화하지 않는다.

13.2 과제 2 · 정책경사와 PPO

이 과제는 하나의 정책경사 구현을 단계별로 개선한다. 전체 궤적 반환에서 reward-to-go로 옮기고, 상태 가치 baseline을 학습한 뒤, GAE로 advantage를 추정하고, PPO-Clip으로 같은 rollout을 여러 번 조심스럽게 사용한다. 모든 비교에서 **평가 반환과 환경 상호작용 단계 수**를 함께 본다.

13.2.1 연결되는 개념

- 로그미분을 이용한 정책경사: 환경 전이를 미분하지 않고 기대 반환의 기울기를 구한다.
- Reward-to-go와 baseline: 과거 보상을 빼고 상태 평균 성과를 기준으로 삼는다.
- GAE: 여러 길이의 TD 추정량을 결합한다.
- PPO의 정책비: 이전 정책에서 모은 데이터를 여러 번 사용할 때 갱신을 제한한다.

13.2.2 1. 전체 반환과 reward-to-go

정책 파라미터를 θ, 정책이 만든 궤적을 $\tau = (s_0, a_0, r_0, s_1, \ldots, a_{T-1}, r_{T-1}, s_T)$라 하자. T는 행동 횟수, r_k는 시점 k의 보상, $\gamma \in [0,1]$는 할인율이다. 전체 할인 반환은 $R(\tau) = \sum_{k=0}^{T-1} \gamma^k r_k$, 시점 t부터의 reward-to-go는 $G_t = \sum_{k=t}^{T-1} \gamma^{k-t} r_k$다. N개의 현재 정책 궤적에서 $R_i = R(\tau_i)$라 할 때, 각 행동의 로그확률 기울기 $\nabla_\theta \log \pi_\theta(a_t^i \mid s_t^i)$를 어느 성과로 가중하느냐에 따라 두 추정량이 갈린다. $\widehat{g}$는 표본 정책경사 추정치다.

$$\widehat{g}_{\text{full}} = \frac{1}{N} \sum_i \sum_t \nabla_\theta \log \pi_\theta(a_t^i \mid s_t^i) R_i, \qquad \widehat{g}_{\text{rtg}} = \frac{1}{N} \sum_i \sum_t \gamma^t \nabla_\theta \log \pi_\theta(a_t^i \mid s_t^i) G_t^i.$$

현재 행동은 과거 보상을 바꿀 수 없으므로 reward-to-go는 불필요한 잡음을 덜 포함한다. 위의 두 식은 **같은 초기 상태 기준 목적** $J(\theta) = E[\sum_{t=0}^{T-1} \gamma^t r_t]$에 대한 추정량이다. 두 번째 식에 γ^t가 있는 이유는 $\gamma^t G_t = \sum_{k=t}^{T-1} \gamma^k r_k$이기 때문이다. 일부 구현은 이 바깥 계수를 생략하고 G_t만 쓴다. $\gamma = 1$이면 동일하지만, $\gamma < 1$이면 원래 J의 정확한 기울기와 달라진다. 스타터 코드의 할인 관례를 확인하고 시간 인덱스를 일치시켜야 한다.

이 과제의 할인 reward-to-go 식은 바깥 γ^t 없이 G_t만 정책 항에 곱한다. 위의 엄밀한 목적함수 미분과 과제에서 구현하는 추정량을 혼동하지 않도록 구분했다. 실제 코드를 점검할 때는 과제의 시점별 할인 G_t와 전체 궤적 반환 계산부터 맞춘다.

세 단계의 보상이 $[2, -1, 4]$이고 $\gamma = 1$이면 전체 반환은 모든 시점에서 5로 쓰이지만 reward-to-go는 차례로 $[5, 3, 4]$다. 두 번째 행동에 첫 보상 2를 붙이지 않는 차이가 드러난다. 계산한 배열을 짧은 손계산과 맞춰 본 뒤 긴 rollout에 적용하면 시간 인덱스 오류를 줄일 수 있다.

13.2.3 2. CartPole 비교 실험

이산 행동 **CartPole-v0**에서 전체 반환과 reward-to-go, advantage 정규화의 유무를 비교한다. 작은 batch와 큰 batch의 학습 곡선을 **각각** 그리면 표본량이 분산을 얼마나 줄이는지 볼 수 있다. 과제의 두 batch 설정은 1000과 4000 환경 단계이며 각각 100 iteration을 실행한다. 네 가지 설정을 두 batch에 적용하므로 총 **8개 실행**이다.

네 비교쌍은 다음과 같다. 각 쌍의 앞은 batch 1000, 뒤는 batch 4000이다.

1. 전체 궤적 반환 · 정규화 없음: cartpole / cartpole_lb.
2. Reward-to-go · 정규화 없음: cartpole_rtg / cartpole_lb_rtg.
3. 전체 궤적 반환 · 정규화 있음: cartpole_na / cartpole_lb_na.

4. Reward-to-go · 정규화 있음: cartpole_rtg_na / cartpole_lb_rtg_na.

모든 실행의 공통 설정은 CartPole-v0와 100 iteration이다. 작은 batch는 1000, 큰 batch는 4000 환경 단계이며 reward-to-go는 -rtg, advantage 정규화는 -na flag로 켠다. 첫 그래프에는 작은 batch 네 곡선, 두 번째에는 큰 batch 네 곡선을 넣는다. 과제의 점검 목표는 가장 좋은 설정이 평균 반환 약 200에 도달하는 것이다. 이는 실제 결과를 대신하는 수치가 아니다.

정규화는 한 batch의 advantage에서 평균을 빼고 표준편차로 나누는 연산이다. 값의 상대적 순서는 유지하면서 업데이트의 크기를 바꾸지만, 모든 환경에서 항상 이득이라는 보장은 없다. 비교할 때 학습률, 네트워크 구조와 평가 방법은 고정하고, 보고서에는 실제 실행한 명령 옵션을 남긴다.

모든 학습 곡선의 가로축은 **정책 업데이트 횟수보다 누적 환경 단계 수**다. batch가 큰 방법은 한 번 업데이트 전에 더 많은 상호작용을 사용하므로 업데이트 횟수만 비교하면 표본 효율을 오해하기 쉽다.

큰 batch의 곡선이 매끄러우면 알고리즘이 더 좋은 것일까? 큰 batch는 기울기 표본 분산을 줄일 수 있지만 업데이트당 환경 비용이 더 든다. 같은 환경 단계 수에서의 최종 성과와 여러 시드의 변동을 비교해야 공정하다.

예를 들어 1000단계 batch로 20번 업데이트한 방법은 2만 환경 단계를 썼고, 4000단계 batch로 20번 업데이트한 방법은 8만 단계를 썼다. 같은 "20번 업데이트"가 같은 데이터 예산이 아닌 셈이다. 반환 그래프의 가로축을 환경 단계로 맞춘 뒤, 곡선의 흔들림과 목표 점수 도달 시간을 함께 해석해야 한다.

13.2.4 CartPole의 네 질문에 답하는 법

1. **정규화가 없을 때 어느 반환 추정이 나았나?** 같은 batch 안에서 전체 반환과 reward-to-go를 직접 비교한다. reward-to-go는 행동 이전 보상이라는 잡음을 없애므로 분산이 작아질 수 있지만, 실제 성능의 우열은 두 그래프가 정한다. 관찰한 도달 속도와 최종 반환을 함께 적는다.

2. **Advantage 정규화가 도움이 되었나?** 같은 batch · 같은 반환 추정에서 -na 유무만 비교한다. 정규화는 batch 평균을 빼고 표준편차로 나누어 gradient 규모를 바꾼다. 개선이 없거나 성능이 낮아져도 그 결과를 그대로 보고하고, 작은 batch의 불안정한 통계나 learning rate와의 상호작용을 논의한다.

3. **Batch 크기가 영향을 주었나?** 동일한 반환 추정 · 정규화 조건에서 1000과 4000을 비교한다. 큰 batch는 표본 gradient 변동을 줄일 수 있지만 업데이트당 상호작용을 네 배 사용한다. 반드시 같은 **환경 단계 수**에서의 반환을 비교한다.

4. **정확한 실행 명령은?** 보고서에는 실행한 8개 명령의 환경명, -n, -b, -rtg, -na, 실험명과 기본값에서 바꾼 다른 flag를 그대로 남긴다. 위 표의 실험명만 적으면 변경한 learning rate나 네트워크 설정을 재현할 수 없다.

13.2.5 3. 상태 가치 baseline

파라미터 ϕ인 가치망 $V_\phi(s_t)$는 상태 s_t에서 현재 정책을 계속 따를 때의 기대 미래 반환을 추정한다. 정책 업데이트에는 추정 advantage $A_t = G_t - V_\phi(s_t)$를 사용한다. 그 상태에서 평소보다 좋은 결과가 나온 행동은 양의 advantage, 나쁜 결과가 나온 행동은 음의 advantage를 받는다.

행동에 의존하지 않는 baseline의 기대 기여는

$$E_{a\sim\pi_\theta(\cdot|s)}[\nabla_\theta \log \pi_\theta(a \mid s) V_\phi(s)] = V_\phi(s) \nabla_\theta \sum_a \pi_\theta(a \mid s) = 0$$

이므로 정확한 기대 정책경사는 바뀌지 않는다. 식의 $\sum_a$는 이산 행동을 가정했으며 연속 행동에서는 행동공간에 대한 적분으로 바꾼다. 두 경우 모두 정책 확률 또는 밀도의 전체 합이 1이라는 정규화 성질을 쓴다. 다만 학습한 가치함수는 근사이고, 실제 구현에서 데이터 재사용 · 정규화 · 학습률이 분산과 편향에 영향을 줄 수 있다. 가치함수 손실의 하락과 정책 반환의 상승을 함께 봐야 한다.

상태 s의 보통 미래 반환이 12인데 한 행동 뒤 실제 반환이 10이면 advantage는 -2다. 보상이 양수여도 그 상태에서는 평균보다 나빴다는 뜻이다. 가치망이 $V_\phi(s) = 20$으로 크게 잘못 예측했다면 advantage는 -10이 되어 actor 갱신도 과장된다. 정책 손실과 가치 손실을 따로 관찰해야 하는 이유다.

13.2.6 4. HalfCheetah에서 baseline 평가

연속 제어 HalfCheetah에서는 reward-to-go 정책경사와 가치 baseline을 붙인 버전을 비교한다. 원본 실험 설정은 할인율 0.95, batch 5000, baseline 학습 단계 수와 학습률을 명시한다. baseline이 더 정확하면 advantage의 잡음이 줄 수 있지만, baseline을 지나치게 적게 또는 과하게 학습하면 정책 갱신이 흔들릴 수 있다.

보고서에는 baseline 손실 곡선과 평가 반환 곡선을 함께 그린다. baseline gradient step 수나 학습률을 줄인 추가 실험에서는 다른 설정을 고정해야 원인을 비교할 수 있다. 손실이 내려갔다고 성과가 반드시 오르지는 않으므로 두 그래프를 나란히 해석한다.

기본 비교는 HalfCheetah-v4, 100 iteration, batch 5000, reward-to-go, 할인율 0.95, actor 학습률 0.01을 공통으로 둔다. baseline 버전은 baseline 학습률 0.01과 매 정책 갱신당 baseline gradient step 5를 추가한다. 과제의 구현 점검 목표는 baseline 버전의 평균 반환 300 초과다.

추가 질문: baseline 학습 step 수 또는 학습률을 낮추면? 가치망이 같은 rollout을 덜 학습해 baseline 손실이 느리게 감소하거나 높게 남을 수 있다. 그러면 $G_t - V_\phi(s_t)$의 오차가 커져 actor gradient가 더 흔들릴 수 있다. 그러나 지나친 baseline 갱신을 줄여 오히려 좋아지는 경우도 있으므로 두 곡선의 실제 변화와 다른 설정의 고정 여부를 근거로 답한다. baseline 손실만 낮다는 이유로 정책 성능 향상을 단정하지 않는다.

13.2.7 5. GAE의 계산

한 단계 TD 잔차를 $\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$라고 두자. $\lambda \in [0, 1]$는 긴 반환의 반영 정도, T는 이번 rollout이 끝나는 시점이다. $\widehat{A}_t$는 시점 t의 GAE 추정치이고 경계에서는 $\widehat{A}_T = 0$으로 둔다. 여러 길이의 TD 잔차를 합치면

$$\widehat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \delta_{t+l} = \delta_t + \gamma\lambda \widehat{A}_{t+1}.$$

마지막 등식은 뒤에서 앞으로 반복 계산할 수 있게 한다. 진짜 종료라면 다음 상태 가치를 0으로 처리하고, 시간 제한으로 잘린 궤적이라면 부트스트랩 가능 여부를 환경 계약에 맞춰 결정한다. $\lambda = 0$은 한 단계 TD에 가깝고 $\lambda = 1$은 적절한 끝 처리에서 긴 보상열에 가까워진다. 두 극단은 가치 추정 오차가 주는 편향과 표본 반환의 변동 사이를 비교하는 기준이다.

두 TD 잔차가 $\delta_t = 2$, $\delta_{t+1} = -1$이고 $\gamma = 0.9$, $\lambda = 0.95$라면 뒤에서부터 계산한 $\widehat{A}_t = 2 + 0.9 \cdot 0.95 \cdot (-1) = 1.145$다. 시간 제한 없이 진짜 종료된 뒤의 잔차는 이어 붙이지 않는다. 먼저 2~3단계짜리 인공 rollout로 이 계산을 재현하면 실제 환경의 긴 배열에서 생기는 off-by-one 오류를 찾기 쉽다.

13.2.8 6. HumanoidStandup의 λ 비교

HumanoidStandup에서는 $\lambda = 0, 0.95, 1$을 같은 환경 · 네트워크 · batch · 학습률로 비교한다. 과제의 명시적인 과업 길이와 할인율을 유지해야 곡선의 차이를 λ에 연결할 수 있다. 한 그래프에 평가 반환과 누적 환경 단계를 표시하고, 어떤 값이 더 빨리 좋아졌으며 최종 성과와 변동이 어떠했는지 설명한다.

실험의 공통 조건은 HumanoidStandup-v5, episode 길이 100, 할인율 0.99, 50 iteration, 3개 hidden layer, 폭 128, batch 2000, actor 학습률 0.001, reward-to-go와 baseline 사용이다. 과제의 점검 기준은 가장 좋은 실행에서 평균 점수 약 1.15×10^4다.

$\lambda = 0$과 1은 각각 무엇인가? 0이면 $\hat{A}_t = \delta_t$로 한 단계 TD 오차만 쓴다. 1이면 $\hat{A}_t = \sum_l \gamma^l \delta_{t+l}$로 관측 가능한 긴 구간을 합치며, 완결된 episode에서는 Monte Carlo 반환에서 현재 V를 뺀 형태로

망원합된다. 시간 제한으로 잘렸다면 끝에서 bootstrap한 가치가 남는다. 짧은 쪽은 가치망 편향에, 긴 쪽은 반환 분산에 더 민감할 수 있다. 세 곡선의 도달 속도와 최종 점수로 이 절충을 해석한다.

단일 실행에서 특정 λ가 더 좋아 보인다고 일반적인 우위를 주장할 수는 없다. 가치함수의 품질, 보상의 지연, 에피소드 길이에 따라 유리한 편향–분산 절충이 달라진다.

$\lambda = 1$이면 가치함수를 전혀 쓰지 않나? 완전한 episode의 끝까지 모두 관측하고 끝에서 부트스트랩하지 않으면 GAE의 망원합이 Monte Carlo 반환에서 현재 가치 추정치를 뺀 형태가 된다. 궤적이 중간에 잘리면 끝의 가치 추정이 남을 수 있다. 종료 처리까지 함께 봐야 한다.

13.2.9 7. PPO-Clip의 목적함수

수집 때의 정책을 π_{old}로 고정하고 현재 정책 π_θ와의 행동 확률비를 $\rho_t(\theta) = \pi_\theta(a_t \mid s_t)/\pi_{\text{old}}(a_t \mid s_t)$라 하자. 이는 보상 r_t와 다른 양이다. $\widehat{A}_t$는 수집한 rollout에서 계산해 고정한 advantage 추정치, E_t는 그 rollout의 시점 표본 평균이다. $\epsilon > 0$은 허용할 확률비 범위의 반폭이고 $\text{clip}(x, l, u)$는 x를 $[l, u]$ 안으로 자른다. PPO-Clip은 다음 surrogate 목적을 최대화한다.

$$L^{\text{clip}}(\theta) = E_t\Big[\min\Big(\rho_t(\theta)\widehat{A}_t,\ \text{clip}(\rho_t(\theta), 1-\epsilon, 1+\epsilon)\widehat{A}_t\Big)\Big].$$

$\widehat{A}_t > 0$이면 그 행동의 확률을 과도하게 높이는 이득을 제한하고, $\widehat{A}_t < 0$이면 확률을 과도하게 낮추는 이득을 제한한다. 구현에서는 rollout 때의 이전 로그확률을 보존하고 현재 로그확률과의 차이를 지수화해 비를 계산하는 편이 안정적이다. clip은 엄격한 KL 신뢰 영역 보장이 아니므로 실제 정책 변화도 진단해야 한다.

수치로 clipping을 확인하자. $\widehat{A} = 2$, $\rho = 1.3$, $\epsilon = 0.2$라면 원래 항은 2.6, clipped 항은 2.4이므로 목적은 2.4를 쓴다. $\widehat{A} = -2$, $\rho = 0.5$라면 두 항은 -1과 -1.6이므로 -1.6을 쓴다. 두 부호에서 모두 `min`을 적용해야 한다. 구현 결과를 이 작은 예와 비교하면 부호 실수를 빨리 찾을 수 있다.

clipping을 적용했는데도 KL이 커질 수 있나? 그렇다. 목적함수는 관측된 표본의 유리한 추가 이득을 제한할 뿐 모든 상태의 정책 거리를 강제로 묶지 않는다. 학습 로그에 평균 KL과 clip fraction을 함께 기록하고, 너무 큰 이동이 반복되면 epoch 수나 학습률을 조정해야 한다.

13.2.10 8. Reacher에서 여러 번의 갱신

Reacher 과제는 GAE와 baseline을 공통으로 쓰는 정책경사 버전과 PPO 버전을 비교한다. PPO는 같은 on-policy batch를 여러 epoch와 minibatch에 걸쳐 사용한다. 원본 설정은 PPO 4 epoch, 4 minibatch를 사용한다. 데이터를 여러 번 보되 확률비와 clipping으로 정책 변화가 지나치게 커졌을 때의 이득을 제한하려는 것이다.

비교의 공통 조건은 Reacher-v4, episode 길이 1000, 할인율 0.99, 100 iteration, batch 5000, 학습률 0.003, advantage 정규화, reward-to-go, baseline, GAE $\lambda = 0.97$이다. PPO에만 4 epoch$\cdot$4 minibatch를 더한다. 과제의 점검 기준은 PPO 평균 점수가 약 -10이며 적어도 -15 수준에 도달하는 것이다. 최종 정책 두 개의 반환을 같은 환경 단계 예산에서 비교한다.

clip 없이 단순 정책경사 손실을 같은 batch에서 여러 번 최적화하면 첫 갱신 뒤 데이터가 이미 이전 정책의 데이터가 된다. 이후 갱신은 현재 정책 분포를 반영하지 못하고 과도한 확률 변화로 불안정해질 수 있다. 실험 결과를 설명할 때 단지 PPO의 점수뿐 아니라 같은 환경 단계 수에서의 곡선과 변동을 함께 비교한다.

질문의 핵심은 surrogate 목적을 제거한 채 여러 batch 업데이트를 할 때의 결과와 이유다. 첫 업데이트 뒤 현재 정책과 데이터 수집 정책이 달라졌는데도 옛 행동을 on-policy처럼 계속 가중한다. 그러면 유리해 보이는 표본 행동의 확률을 지나치게 바꿀 수 있다. 실제 실험에서 반드시 점수가 하락한다는 법칙은 아니지만, 업데이트의 불안정성과 표본 분포의 어긋남을 근거로 관찰한 결과를 설명할 수 있다.

13.2.11 구현 점검표

구성	확인할 항목	흔한 오류
궤적 수집	상태·행동·보상·종료 길이의 정렬	마지막 상태를 행동 배열과 같은 길이로 취급
반환 계산	전체 반환과 reward-to-go의 시간 인덱스	과거 보상이나 할인 지수를 잘못 포함
가치함수	예측과 목표의 batch 차원	정책 손실로 가치망이 우연히 역전파
GAE	역방향 재귀와 종료 마스크	진짜 종료 후 다음 가치를 더함
PPO	이전 로그확률의 고정, 비와clip	업데이트 중 이전 정책 값을 새로 계산
평가	누적 환경 단계와 여러 episode	업데이트 횟수만 가로축으로 사용

보고서에는 CartPole의 두 batch 그래프, HalfCheetah의 baseline 손실과 반환, HumanoidStandup의 λ 비교, Reacher의 PPO 비교와 각 질문의 해석을 담는다. 직접 실행한 명령과 설정을 기록하고, 실제 결과를 사용한다. 보고서 PDF와 필요한 코드를 묶어 제출하며 영상과 TensorBoard 로그는 제외한다. 과제의 압축 파일 크기 기준은 **20 MB 미만**이다.

13.3 과제 3 · DQN과 Double DQN

과제 3은 Q-learning을 신경망으로 구현하고, CartPole의 상태 벡터와 Atari의 화면 입력에서 학습을 관찰한다. 핵심은 **온라인 Q망과 target Q망의 역할**, replay에서 뽑은 전이의 TD 목표, 탐색 정책, 그리고 Double DQN의 선택·평가 분리다. 실험 결과를 읽을 때는 예측 Q값, critic 오차와 실제 평가 반환을 함께 본다.

13.3.1 먼저 확인하는 DQN T/F

1. **"Q-learning은 off-policy sample을 활용할 수 없어서 표본 효율이 낮다"** — **F.** 행동을 수집한 정책과 Bellman 목표의 greedy 정책이 달라도 transition으로 학습할 수 있다. Replay를 이용한 재사용도 가능하다.
2. **"Actor가 없으면 연속 행동의 모든 후보 Q값을 평가해 최대 행동을 고르기 어렵다"** — **T.** 이산 행동의 유한 목록과 달리 연속 행동은 무한히 많아 전수 열거할 수 없다. 다만 최적화 자체가 수학적으로 불가능하다는 뜻은 아니며, DDPG 등은 actor로 최대화를 근사한다.
3. **"같은 신경망으로 현재 Q와 목표 Q를 모두 만들면 moving target 문제가 생기므로, 안쪽 학습 반복에서는 고정된 target network를 쓸 수 있다"** — **T.** 온라인 망을 갱신하는 동안 target 망을 한동안 고정해 회귀 목표의 급격한 변화를 늦춘다.
4. **"학습이 진행될수록 보통 ϵ을 키워 탐색을 늘린다"** — **F.** 대개 초기에 크게 탐색하고 이후 ϵ을 낮춰 학습한 행동을 더 많이 활용한다.

13.3.2 1. DQN의 한 번의 업데이트

전이 $(s_t, a_t, r_t, s_{t+1}, d_t)$에서 s_t는 현재 상태, a_t는 행동, r_t는 즉시 보상, s_{t+1}은 다음 상태, $d_t \in \{0, 1\}$은 **환경의 실제 종료** 표시다. 온라인 Q망의 매개변수를 θ, 느리게 갱신되는 target 망의 매개변수를 $\bar{\theta}$, 할인율을 γ라 두자. 이산 행동에서 DQN의 목표와 제곱 TD 손실은

$$y_t = r_t + \gamma(1 - d_t) \max_{a'} Q_{\bar{\theta}}(s_{t+1}, a'), \qquad L(\theta) = \mathbb{E}_{(s,a,r,s',d)\sim\mathcal{D}}\big[(Q_\theta(s, a) - y_t)^2\big].$$

$\mathcal{D}$는 replay buffer의 전이 분포다. 계산한 y_t는 현재 온라인 망의 gradient가 흘러들지 않도록 고정된 target으로 취급한다. 실제로 종료한 상태라면 $d_t = 1$이므로 다음 상태의 가치는 더하지 않는다. 예를 들어 $r_t = 2$, $\gamma = 0.9$, 다음 상태의 최대 target Q가 4라면 비종료 전이의 목표는 5.6이고, 종료 전이의 목표는 2다.

시간 제한으로 잘린 trajectory는 실제 MDP 종료와 다르다. 환경을 reset하더라도 TD 목표에는 다음 상태에서의 가치가 남아야 하므로 bootstrap용 d_t는 0으로 저장한다. 반대로 실패나 과업 완료처럼 실제로 종료했다면 $d_t = 1$이다. 두 경우를 모두 단순한 "episode 끝"으로 합치면 시간 제한 근처의 Q값을 낮게 왜곡할 수 있다.

구현할 함수의 연결은 다음과 같다.

1. **DQNAgent.update_critic:** replay minibatch의 TD target과 critic 손실을 계산하고 최적화한다. 현재 $Q_\theta(s, a)$에서 실제 실행 행동 a의 값만 모으고, 목표에는 gradient를 흘리지 않는다.
2. **DQNAgent.update:** critic을 학습하고 조건에 맞춰 target critic을 갱신한다. 매 minibatch마다 무조건 target을 복사하면 고정 목표의 장점이 줄어든다.
3. **DQNAgent.get_action:** 이산 행동에서 ϵ-greedy를 구현한다. 무작위 행동과 greedy 행동의 확률, 평가 때의 탐색 설정을 구분한다.
4. **run_hw3.py의 학습 반복:** 수집, replay 삽입, 학습, 평가, 로그를 연결한다. 환경 단계와 gradient step은 별개로 센다.

5. **utils.py의 sample_trajectory:** 상태·행동·보상·다음 상태·종료 정보를 같은 시점에 저장한다. terminated와 truncated를 구분해 bootstrap용 done을 저장한다.

CartPole의 설정은 dqn_basic_config.py, Atari Asterix의 설정은 dqn_atari_config.py에서 만든다. replay_buffer.py의 MemoryEfficientReplayBuffer는 화면 stack을 매 transition마다 통째로 복제하지 않고 frame을 효율적으로 저장한다. 새 episode 첫 관측을 넣는 on_reset과 뒤따르는 frame을 넣는 insert의 호출 시점을 살펴야 stack이 episode 경계를 넘어 섞이지 않는다.

13.3.3 2. Replay, target 망과 탐색

연속해서 수집한 transition은 유사하다. replay buffer에서 무작위 minibatch를 뽑으면 샘플 상관을 줄이고 과거 경험을 재사용한다. 온라인 망을 매 단계 갱신하되 target 망을 주기적으로 복사하거나 소프트 갱신하면 회귀 목표가 온라인 망을 즉시 따라 움직이지 않는다. 두 장치는 서로 다른 불안정성을 해결한다.

행동을 고를 때 ϵ-greedy는 확률 ϵ로 무작위 행동을, 나머지 확률로 $\arg\max_a Q_\theta(s,a)$를 선택한다. 행동이 m개라면 greedy 행동도 무작위 탐색에 포함되므로 그 행동의 전체 선택확률은 $1-\epsilon+\epsilon/m$이다. 학습 초반에는 ϵ을 크게 두어 다양한 전이를 수집하고, 뒤로 갈수록 낮춰 학습한 행동을 더 자주 사용한다. 평가할 때의 탐색 설정은 학습 중의 설정과 구별해 명시한다.

Atari 화면에서는 한 프레임이 이동 방향을 알려주지 않을 수 있다. grayscale 변환은 색 차원을 줄이고, frame skip은 몇 단계 동안 같은 행동을 유지하며, frame stack은 연속 프레임을 묶어 시간 정보를 일부 되살린다. 화면 입력과 CartPole의 낮은 차원 상태 벡터는 네트워크 입력층이 다르지만 TD 목표의 수학은 같다.

Atari 설정은 $84\times84\times3$ RGB를 84×84 grayscale로 바꾸고, 같은 행동을 4 frame 유지하는 frame skip과 최근 4 frame을 쌓는 frame stack을 사용한다. 스타터 구현에는 이와 함께 학습률 감소 schedule과 gradient norm clipping도 들어 있다. Clipping은 gradient norm이 임계값을 넘을 때 전체 gradient를 같은 방향으로 축소한다. 이런 장치는 알고리즘의 Bellman 목표와 별도로 **최적화의 안정성**을 위한 것이다.

13.3.4 3. Double DQN의 선택·평가 분리

Q 추정에 잡음이 있을 때 $\max_a \hat{Q}(s',a)$는 우연히 크게 나온 행동을 고른다. 모든 행동의 참값이 0인 one-step 문제에서도 추정치들의 최댓값의 기댓값은 보통 0보다 크다. 행동 후보가 많아지면 큰 양의 잡음이 선택될 기회도 늘어난다.

Double DQN은 다음 행동을 온라인 망으로 고르고 그 행동을 target 망으로 평가한다.

$$a^* = \arg\max_{a'} Q_\theta(s_{t+1},a'), \qquad y_t^{\text{Double}} = r_t + \gamma(1-d_t)Q_{\bar\theta}(s_{t+1},a^*).$$

선택과 평가에 쓰인 오차의 결합을 약하게 해 과대평가를 완화한다. 두 망이 학습 과정에서 관련되어 있으므로 편향을 완전히 없앤다는 뜻은 아니다. DQN target이 target 망에서 **최대 행동 선택과 값 평가를 모두 한다**는 점과 대조하면 식을 기억하기 쉽다.

13.3.5 Toy 실험의 두 질문

Toy 환경은 상태 하나와 행동 m개를 가진 한 단계 문제다. 모든 행동의 실제 기대 보상은 $\mu=0$이고 관측 보상은 $r\sim\mathcal{N}(0,1)$이다. 따라서 실제 $Q(s,a)$는 모든 a에서 0이다. 그런데 각 행동의 noisy 추정치를 독립인 $X_1,\dots,X_m$이라 하면, 하나만 골라 평가하는 값의 기댓값은 0이어도 $\mathbb{E}[\max_i X_i]>0$이다. m이 커질수록 양의 잡음 중 더 큰 것이 선택될 기회가 늘어난다. 표준정규 잡음의 이상화에서는 최댓값의 대표적인 크기가 큰 m에 대해 대략 $\sqrt{2\log m}$로 자란다.

Double 추정의 직관은 **한 추정치로 선택하고 독립적인 다른 추정치로 평가**하는 데 있다. 두 번째 추정치를 Y_i라 하고 $I=\arg\max_i X_i$라 하면, 각 Y_i가 평균 0이고 선택에 쓰인 X들과 독립인 이상화에서

$$\mathbb{E}[Y_I] = \sum_{i=1}^{m} P(I=i)\mathbb{E}[Y_i \mid I=i] = \sum_{i=1}^{m} P(I=i)\cdot 0 = 0.$$

실습에서는 $m = 2, 4, 8, \ldots, 1024$로 늘리며 **single-Q와 double-Q의 추정 오차를 같은 그래프에서 비교**한다. 실제 Double DQN의 두 망은 완전히 독립이지 않으므로 toy의 0 기대 오차를 그대로 보장하지는 않는다. 그래프에서 행동 수에 따른 오차 추세와 두 추정의 차이를 설명하면 된다.

13.3.6 4. 실험 결과를 해석하는 순서

CartPole-v1에서는 DQN을 **seed 1·2·3**으로 실행해 평가 반환을 세 시드의 평균과 표준편차로 그린다. 세로축은 eval_return, 가로축은 학습 환경 단계 수로 맞춘다. 원래 설정의 점검 목표는 약 30만 단계 부근에 반환 500에 가까워지는 것이다. 이는 관찰 결과를 대신하지 않는다.

학습률 비교에서는 기본 **0.001**과 변경 설정 **0.05**를 CartPole-v1에서 비교한다. 예측 Q값, critic error, 평가 반환의 **세 그래프**를 각각 두 학습률의 곡선으로 그린다. 예측 Q가 커져도 실제 반환이 좋아졌다는 증거는 아니다. 지나친 학습률은 움직이는 bootstrap 목표와 함수 근사의 오차를 증폭할 수 있다. 설명할 때는 "학습률을 키우면 무조건 발산한다"라고 쓰기보다 실제 세 곡선의 시간 순서를 근거로 원인을 분석한다.

Atari **Asterix-v5**에서는 Double DQN을 seed 1·2·3으로 학습한다. 세 시드의 평가 반환 평균과 표준편차를 단계 수에 대해 그린다. 과제의 점검 목표는 각 실행에서 평가 반환이 학습 중 한 번 이상 1200을 넘는 것이다. 일시적인 최고점과 전체 학습 곡선의 수준을 구분한다.

마지막 CartPole hyperparameter 실험은 하나의 변수를 골라 **기본값을 포함한 네 값**을 같은 그래프에서 비교한다. learning rate, network architecture, exploration schedule 등이 후보이며 네 설정 파일과 변수 선택 이유를 남긴다. 다른 변수와 평가 방식을 고정해야 네 곡선의 차이를 고른 변수와 연결할 수 있다.

보고서의 결과물은 다음 다섯 묶음으로 확인한다.

1. **CartPole DQN:** 세 seed의 평가 반환 평균·표준편차와 환경 단계 x축.
2. **Learning rate 비교:** Q 예측, critic error, 평가 반환의 세 그래프에서 각각 0.001과 0.05를 비교.
3. **Double Q toy:** 행동 수가 2에서 1024로 커질 때 single·double 추정 오차의 막대그래프.
4. **Asterix Double DQN:** 세 seed의 평균·표준편차 곡선과 각 실행의 최고점.
5. **Hyperparameter:** 기본값 포함 네 값의 평가 반환 곡선, 동일 환경·평가 조건과 변수 선택 이유.

Q값이 높아지는데 실제 성적이 나빠질 수 있나? 그렇다. Q는 bootstrap으로 학습된 예측이고 평가 반환은 실제 환경에서 관찰한 결과다. max 연산의 선택 편향, 큰 학습률이나 드문 상태의 추정 오류는 예측만 높일 수 있다. 두 지표의 간격 자체가 중요한 진단 신호다.

왜 Double DQN에서도 target network가 필요한가? Double DQN의 target 망은 값 평가를 맡고, 온라인 망은 다음 행동을 고른다. target 망은 원래 DQN에서처럼 목표의 급격한 이동을 늦추는 역할도 계속한다. 선택·평가 분리와 목표 안정화가 동시에 작동한다.

실험 결과의 숫자와 그래프는 실제 실행에서 얻은 것을 사용한다. 개념 설명만으로는 seed별 성과, 큰 학습률의 실제 발산 여부나 네 hyperparameter 설정의 우열을 확정할 수 없다. 보고서 PDF와 코드를 한 압축 파일에 넣고, 영상과 로그는 제외한다. 과제의 압축 파일 크기 기준은 **20 MB 미만**이다.